Phát triển ứng dụng phân tán

Nguyễn Toàn Thắng

Thái Nguyên, năm 2026

MỤC LỤC

Danh mục từ viết tắt v

Lời nói đầu vii

1 Tổng quan hệ thống và các mô hình trao đổi thông điệp (MEP) 1

1.1 Khái niệm chung về hệ thống ứng dụng phân tán . . . 1
1.1.1 Khái niệm ứng dụng phân tán . . . 1
1.1.2 Kiến trúc máy khách–máy chủ . . . 3
1.1.3 Các mô hình trao đổi thông điệp (MEP) . . . 4
1.1.4 Ánh xạ MEP với công nghệ hiện đại . . . 8
1.2 Kỹ thuật lập trình cơ bản với ASP.NET Core Web API . . . 9
1.2.1 Cấu trúc solution và project ASP.NET Core Web API . . . 9
1.2.2 Hosting model và tổ chức mã nguồn trong `Program.cs` . . . 10
1.2.3 Tạo endpoint với Minimal API . . . 11
1.2.4 Tiếp nhận request và tạo response với `Results` . . . 12
1.2.5 Chạy và kiểm thử Web API . . . 14
1.3 [Ứng dụng] Tạo cấu trúc ứng dụng phân tán cơ bản . . . 14
1.3.1 Khởi tạo dự án API . . . 16
1.3.2 Tạo điểm cuối đơn giản . . . 17
1.3.3 Tạo máy khách gọi thử API . . . 19
Ôn tập cuối chương . . . 21

2 Xây dựng RESTful API cho tương tác máy khách–máy chủ 24

2.1 Cơ chế truyền thông đồng bộ với RESTful API . . . 24
2.1.1 HTTP và yêu cầu–phản hồi . . . 24
2.1.2 Nguyên tắc REST: tài nguyên, phương thức, không lưu trạng thái phiên . . . 27
2.1.3 Thiết kế API theo use case . . . 28
2.1.4 DTO và mô hình dữ liệu . . . 29
2.1.5 Khai thác RESTful API . . . 30
2.2 Kỹ thuật lập trình cơ bản với RESTful API . . . 31
2.2.1 Ánh xạ tài nguyên vào URI và phương thức HTTP . . . 31
2.2.2 Tiếp nhận dữ liệu từ route, query string và request body . . . 33
2.2.3 Sử dụng DTO và kiểm tra tính hợp lệ của dữ liệu đầu vào . . . 33
2.2.4 Trả về mã trạng thái và nội dung phản hồi phù hợp . . . 34
2.2.5 Gọi RESTful API và xử lý phản hồi bằng `HttpClient` . . . 35
2.3 [Ứng dụng] Xây dựng hệ thống máy khách–máy chủ cơ bản sử dụng RESTful API . . . 37
2.3.1 Xây dựng API: assignment, submission . . . 37
2.3.2 Máy khách gọi API và xử lý kết quả . . . 40
Ôn tập cuối chương . . . 43

3 Giao tiếp thời gian thực với SignalR 45

3.1 Cơ chế và mô hình truyền thông thời gian thực . . . 45
3.1.1 MEP: truyền luồng, đẩy dữ liệu và yêu cầu–phản hồi . . . 45
3.1.2 WebSocket và SignalR . . . 46
3.1.3 Ca sử dụng thời gian thực trong hệ thống . . . 47
3.2 Kỹ thuật lập trình cơ bản với SignalR . . . 48
3.2.1 Đăng ký và ánh xạ SignalR hub trong ASP.NET Core . . . 48
3.2.2 Xây dựng hub và gửi thông điệp đến client . . . 49
3.2.3 Kết nối và đăng ký nhận thông điệp bằng `HubConnection` . . . 50
3.2.4 Gửi thông điệp theo client, group và toàn bộ kết nối . . . 52
3.2.5 Quản lý vòng đời kết nối và kết nối lại . . . 52
3.3 [Ứng dụng] Sử dụng SignalR để truyền thông thời gian thực giữa máy khách và máy chủ . . . 53
3.3.1 Thiết lập SignalR hub . . . 54
3.3.2 Máy khách nhận thông báo thời gian thực . . . 58
Ôn tập cuối chương . . . 62

4 Các kiểu API phía máy khách: REST và GraphQL 65
4.1 Khái niệm và cơ chế hoạt động của GraphQL . . . 65
4.1.1 Hạn chế của REST trong truy xuất dữ liệu . . . 65
4.1.2 GraphQL và truy vấn linh hoạt . . . 66
4.1.3 So sánh REST và GraphQL . . . 68
4.1.4 Lựa chọn API theo nhu cầu máy khách . . . 69
4.2 Kỹ thuật lập trình cơ bản với GraphQL . . . 70
4.2.1 Khai báo schema và các kiểu dữ liệu GraphQL . . . 70
4.2.2 Xây dựng query, mutation và resolver . . . 71
4.2.3 Sử dụng argument, variable và lựa chọn field . . . 72
4.2.4 Tiếp nhận request và trả về response GraphQL . . . 73
4.2.5 Xử lý dữ liệu và lỗi ở phía client GraphQL . . . 76
4.3 [Ứng dụng] Xây dựng máy khách đơn giản với GraphQL . . . 78
4.3.1 Tạo điểm cuối GraphQL đơn giản . . . 78
4.3.2 Máy khách truy vấn dữ liệu theo trường . . . 84
Ôn tập cuối chương . . . 87

5 Giao tiếp giữa các dịch vụ với gRPC 90
5.1 Khái niệm và cơ chế hoạt động của RPC . . . 90
5.1.1 MEP: yêu cầu–phản hồi trong giao tiếp dịch vụ–dịch vụ . . . 90
5.1.2 RPC và gRPC . . . 91
5.1.3 Protocol Buffers và giao tiếp dựa trên hợp đồng . . . 93
5.1.4 Sự khác biệt giữa API phía máy khách và API nội bộ . . . 94
5.2 Kỹ thuật lập trình cơ bản với gRPC . . . 96
5.2.1 Khai báo service và message trong tệp `.proto` . . . 96
5.2.2 Sinh mã nguồn client stub và server stub . . . 97
5.2.3 Hiện thực và đăng ký gRPC service . . . 98
5.2.4 Tạo channel và gọi phương thức gRPC từ client . . . 99
5.2.5 Xử lý lỗi, deadline và hủy yêu cầu . . . 101
5.3 [Ứng dụng] Xây dựng hệ thống dịch vụ phân tán đơn giản với gRPC . . . 102
5.3.1 Tạo dịch vụ gRPC . . . 103

5.3.2 Gọi gRPC giữa các thành phần . . . 106
Ôn tập cuối chương . . . 108
6 Xử lý bất đồng bộ với hàng đợi và dịch vụ xử lý nền 111
6.1 Xử lý bất đồng bộ với hàng đợi . . . 111
6.1.1 MEP: bất đồng bộ dựa trên hàng đợi . . . 111
6.1.2 Hạn chế của xử lý đồng bộ . . . 112
6.1.3 Hàng đợi và mẫu dịch vụ xử lý nền . . . 113
6.1.4 Tổng quan RabbitMQ . . . 115
6.2 Kỹ thuật lập trình cơ bản với RabbitMQ và Worker Service . . . 116
6.2.1 Thiết lập connection và channel đến RabbitMQ . . . 116
6.2.2 Khai báo queue và cấu hình cơ bản . . . 117
6.2.3 Tuần tự hóa và publish message . . . 117
6.2.4 Consume message trong Worker Service . . . 119
6.2.5 Xác nhận hoặc từ chối message với ack/nack . . . 119
6.2.6 Quản lý cấu hình và vòng đời kết nối . . . 120
6.3 [Ứng dụng] Xây dựng mô hình truyền thông bất đồng bộ giữa các dịch vụ với RabbitMQ . . . 121
6.3.1 API gửi công việc vào hàng đợi . . . 122
6.3.2 Dịch vụ xử lý nền nhận và xử lý công việc . . . 125
6.3.3 Trả trạng thái “processing” . . . 127
Ôn tập cuối chương . . . 128
7 Hoàn thiện luồng hệ thống với kiến trúc hướng sự kiện và pub/sub 131
7.1 Mô hình truyền thông điệp pub/sub và dựa trên sự kiện . . . 131
7.1.1 MEP: phát hành–đăng ký . . . 131
7.1.2 Sự kiện và yêu cầu . . . 133
7.1.3 Kiến trúc hướng sự kiện . . . 134
7.1.4 Giảm phụ thuộc giữa các dịch vụ . . . 135
7.2 Kỹ thuật lập trình cơ bản với event-driven và pub/sub . . . 137
7.2.1 Thiết kế event contract và dữ liệu sự kiện . . . 137
7.2.2 Khai báo exchange, queue, binding và routing key . . . 137
7.2.3 Publish sự kiện lên topic exchange . . . 137
7.2.4 Đăng ký và xử lý sự kiện ở subscriber . . . 137
7.2.5 Quản lý phiên bản sự kiện và xử lý lặp idempotent . . . 137
7.2.6 Kết nối subscriber với kênh thông báo thời gian thực . . . 137
7.3 [Ứng dụng] Áp dụng cơ chế truyền thông điệp pub/sub vào tương tác dịch vụ–máy khách . . . 137
7.3.1 Dịch vụ xử lý nền phát sự kiện khi xử lý xong . . . 138
7.3.2 Dịch vụ nhận sự kiện và xử lý . . . 141
7.3.3 Kết hợp SignalR để thông báo cho máy khách . . . 144
Ôn tập cuối chương . . . 147
8 Vận hành hệ thống phân tán: đa tiến trình, container, quan sát và xử lý lỗi 151
8.1 Các khái niệm cơ bản liên quan đến vận hành hệ thống phân tán . . . 151
8.1.1 Hệ thống và ứng dụng . . . 151
8.1.2 Đa tiến trình và giao tiếp giữa các dịch vụ . . . 152

8.1.3 Triển khai cơ bản với Docker . . . 153
8.1.4 Lỗi trong hệ thống phân tán . . . 155
8.1.5 Gửi lại và quá thời gian chờ . . . 155
8.2 Kỹ thuật vận hành cơ bản với .NET và Docker . . . 157
8.2.1 Chạy nhiều tiến trình .NET với cổng và cấu hình khác nhau . . . 157
8.2.2 Đọc cấu hình từ `appsettings.json` và biến môi trường . . . 157
8.2.3 Đóng gói service bằng Dockerfile nhiều giai đoạn . . . 157
8.2.4 Kết nối container qua network và service name . . . 157
8.2.5 Khởi chạy hệ thống bằng Docker Compose . . . 158
8.2.6 Thiết lập health check và structured logging . . . 158
8.2.7 Thiết lập timeout, retry có giới hạn và graceful shutdown . . . 158
8.2.8 Quan sát container, log và trạng thái dịch vụ . . . 158
8.3 [Ứng dụng] Triển khai và kiểm thử khả năng phục hồi của EduSubmit . . . 158
8.3.1 Chạy API, dịch vụ xử lý nền, RabbitMQ . . . 159
8.3.2 Đóng gói hệ thống bằng Docker . . . 160
8.3.3 Mô phỏng lỗi và quan sát . . . 162
Ôn tập cuối chương . . . 163

9 Tổng quan kiến trúc hệ thống dịch vụ phân tán 167
9.1 Kiến trúc hệ thống phần mềm phân tán tổng thể gắn với dự án đã xây dựng 167
9.1.1 Tổng thể kiến trúc hệ thống phân tán . . . 168
9.1.2 Tổng kết các MEP và cách áp dụng . . . 169
9.1.3 Các thành phần của hệ thống phân tán . . . 170
9.1.4 Nguyên tắc lựa chọn kiến trúc . . . 171
9.2 [Thực hành] Trực quan hóa kiến trúc hệ thống phân tán đã xây dựng . . . 173
9.2.1 Phân tích và chú giải kiến trúc hệ thống . . . 173
9.2.2 Thảo luận các kịch bản thiết kế . . . 175
Ôn tập cuối chương . . . 177

Kết luận và định hướng mở rộng 191
Nhìn lại hành trình xây dựng EduSubmit . . . 191
Bản đồ các mô hình trao đổi thông điệp . . . 192
Từ code chạy được đến hệ thống vận hành được . . . 192
Nguyên tắc chọn công nghệ . . . 193
Những hướng mở rộng sau học phần . . . 193
Lời kết . . . 194

Tài liệu tham khảo 195

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Cụm từ đầy đủ	Diễn giải ngắn
API	Application Programming Interface	Giao diện lập trình ứng dụng, thường là hợp đồng giao tiếp giữa máy khách và máy chủ hoặc giữa các dịch vụ.
ASP.NET Core	Active Server Pages .NET Core	Khung phần mềm của Microsoft dùng để xây dựng ứng dụng web, RESTful API, hub thời gian thực và dịch vụ phía sau.
CDN	Content Delivery Network	Mạng phân phối nội dung, hệ thống máy chủ phân tán giúp tăng tốc truyền tải dữ liệu tới người dùng cuối.
CLI	Command Line Interface	Giao diện dòng lệnh, giao diện người dùng dựa trên văn bản được sử dụng để chạy các chương trình và lệnh.
DTO	Data Transfer Object	Đối tượng truyền dữ liệu qua ranh giới API, thường dùng cho thân yêu cầu hoặc thân phản hồi.
GraphQL	Graph Query Language	Ngôn ngữ truy vấn API cho phép máy khách mô tả chính xác dữ liệu cần lấy.
gRPC	gRPC Remote Procedure Call	Khung phần mềm gọi thủ tục từ xa, thường dùng cho giao tiếp dịch vụ–dịch vụ với hợp đồng rõ ràng.
HTTP	Hypertext Transfer Protocol	Giao thức tầng ứng dụng phổ biến cho giao tiếp yêu cầu–phản hồi trên web.
IPC	Inter-Process Communication	Giao tiếp giữa các tiến trình, cơ chế cho phép các chương trình chia sẻ dữ liệu và phối hợp hoạt động.
JSON	JavaScript Object Notation	Định dạng dữ liệu văn bản nhẹ, thường dùng trong yêu cầu và phản hồi của API.
MEP	Message Exchange Pattern	Mẫu trao đổi thông điệp giữa các thành phần, ví dụ yêu cầu–phản hồi, dựa trên hàng đợi hoặc phát hành–đăng ký.
REST	Representational State Transfer	Phong cách kiến trúc tổ chức API quanh tài nguyên, biểu diễn tài nguyên và các ràng buộc như không lưu trạng thái phiên.
RPC	Remote Procedure Call	Mô hình gọi thủ tục từ xa, trong đó máy khách gọi một thao tác được thực thi ở tiến trình hoặc máy khác.

Từ viết tắt	Cụm từ đầy đủ	Diễn giải ngắn
SDL	Schema Definition Language	Ngôn ngữ mô tả lược đồ trong GraphQL, dùng để định nghĩa kiểu, trường và quan hệ giữa các đối tượng.
SignalR	SignalR	Thư viện của ASP.NET Core dùng để xây dựng giao tiếp thời gian thực giữa máy chủ và máy khách.
SSE	Server-Sent Events	Cơ chế cho phép máy chủ đẩy sự kiện một chiều tới máy khách qua HTTP.
TCP/IP	Transmission Control Protocol / Internet Protocol	Bộ giao thức nền tảng cho truyền thông mạng, cung cấp định tuyến và truyền dữ liệu giữa các máy.
URI	Uniform Resource Identifier	Định danh tài nguyên, ví dụ đường dẫn tới một tài nguyên trong RESTful API.
URL	Uniform Resource Locator	Một dạng URI chỉ rõ vị trí truy cập tài nguyên, thường gồm lược đồ, máy chủ, đường dẫn và chuỗi truy vấn.
WebSocket	WebSocket	Giao thức duy trì kết nối hai chiều lâu dài giữa máy khách và máy chủ.

LỜI NÓI ĐẦU

Giáo trình *Phát triển ứng dụng phân tán* được biên soạn nhằm phục vụ học phần cùng tên trong chương trình đào tạo ngành Công nghệ thông tin. Tài liệu cung cấp cho người học những kiến thức và kỹ năng cần thiết để phân tích, thiết kế, xây dựng và vận hành các ứng dụng phân tán ở mức cơ bản, trên cơ sở kết hợp giữa tư duy hệ thống, mô hình trao đổi thông điệp và các công nghệ phát triển phần mềm hiện đại.

Nội dung giáo trình được xây dựng theo định hướng từ bài toán thực tế đến giải pháp kỹ thuật. Người học được tiếp cận các khái niệm nền tảng về hệ thống phân tán, kiến trúc máy khách–máy chủ và các mô hình trao đổi thông điệp như yêu cầu–phản hồi, truyền luồng, truyền thông qua hàng đợi và phát hành–đăng ký. Trên nền tảng đó, giáo trình lần lượt giới thiệu các cơ chế giao tiếp và triển khai phổ biến như RESTful API, SignalR, GraphQL, gRPC, hàng đợi thông điệp, dịch vụ xử lý nền, kiến trúc hướng sự kiện và triển khai bằng container.

Giáo trình hướng tới đối tượng chính là sinh viên ngành Công nghệ thông tin, đồng thời có thể dùng làm tài liệu tham khảo cho giảng viên, người học và những người quan tâm đến phát triển ứng dụng phân tán. Cách trình bày ưu tiên sự liên kết giữa lý thuyết và thực hành: mỗi nhóm kiến thức được gắn với ví dụ, hình minh họa, mã nguồn và các bài thực hành theo một hệ thống mẫu xuyên suốt. Qua đó, người học không chỉ nắm được khái niệm mà còn từng bước hình thành năng lực triển khai hệ thống có nhiều thành phần giao tiếp với nhau.

Về cấu trúc, giáo trình được tổ chức theo trình tự từ cơ bản đến nâng cao. Các chương đầu tập trung vào nền tảng giao tiếp máy khách–máy chủ, API và tương tác thời gian thực; các chương tiếp theo mở rộng sang giao tiếp giữa các dịch vụ, xử lý bất đồng bộ, kiến trúc hướng sự kiện và vận hành hệ thống. Cấu trúc này cho phép người học đọc tuần tự toàn bộ giáo trình hoặc sử dụng từng chương như một tài liệu tham khảo độc lập khi cần ôn tập hoặc triển khai một kỹ thuật cụ thể.

Trong quá trình biên soạn, tác giả đã tham khảo các tài liệu nền tảng về hệ thống phân tán, kiến trúc phần mềm và kinh nghiệm triển khai ứng dụng hiện đại. Mặc dù đã cố gắng rà soát, giáo trình khó tránh khỏi những thiếu sót. Tác giả mong nhận được ý kiến đóng góp của quý thầy cô, đồng nghiệp và người học để tài liệu tiếp tục được hoàn thiện trong các lần cập nhật sau.

Thái Nguyên, năm 2026
Nhóm tác giả

CHƯƠNG 1

TỔNG QUAN HỆ THỐNG VÀ CÁC MÔ HÌNH TRAO ĐỔI THÔNG ĐIỆP (MEP)

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Nhận diện bản chất của ứng dụng phân tán và ranh giới giao tiếp giữa các thành phần.
- Phân biệt vai trò máy khách và máy chủ trong một tương tác qua mạng.
- Giải thích các mô hình trao đổi thông điệp phổ biến và tình huống sử dụng phù hợp.
- Xây dựng lát cắt đầu tiên của dự án EduSubmit gồm API và máy khách dòng lệnh giao tiếp qua HTTP.

1.1 Khái niệm chung về hệ thống ứng dụng phân tán

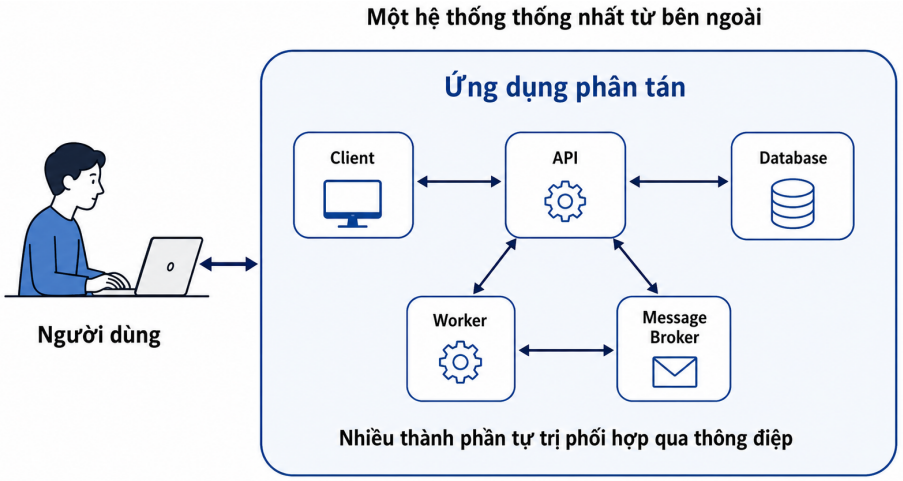
1.1.1 Khái niệm ứng dụng phân tán

Theo cách tiếp cận kinh điển của Tanenbaum và van Steen, một hệ thống phân tán có thể được hiểu là một tập hợp các phần tử tính toán tự trị phối hợp với nhau và xuất hiện trước người dùng như một hệ thống thống nhất. Điểm quan trọng trong định nghĩa này là hệ thống vừa có nhiều thành phần độc lập, vừa che bớt sự độc lập đó để người dùng hoặc chương trình phía ngoài có thể tương tác với nó như một chỉnh thể.

Từ góc nhìn phát triển phần mềm, một ứng dụng phân tán là ứng dụng được xây dựng trên tinh thần đó: các thành phần xử lý không nhất thiết nằm trong cùng một tiến trình hoặc cùng một máy tính, nhưng phối hợp với nhau qua mạng để thực hiện một mục tiêu chung. Người dùng có thể chỉ nhìn thấy một hệ thống duy nhất, trong khi bên trong hệ thống có nhiều thành phần như giao diện người dùng, API, cơ sở dữ liệu, dịch vụ xử lý nền, hàng đợi thông điệp và các dịch vụ tích hợp bên ngoài.

Ứng dụng đơn khối (monolithic application) là kiểu ứng dụng trong đó phần lớn chức năng được đóng gói, biên dịch, triển khai và vận hành như một đơn vị thống nhất. Các phần như giao diện, xử lý nghiệp vụ và truy cập dữ liệu có thể vẫn được chia thành nhiều lớp hoặc nhiều mô-đun trong mã nguồn, nhưng khi chạy chúng thường nằm trong cùng một tiến trình hoặc cùng một gói triển khai. Vì vậy, thay đổi ở một phần của hệ thống thường kéo theo việc kiểm thử, đóng gói và triển khai lại toàn bộ ứng dụng.

Điểm khác biệt quan trọng giữa ứng dụng đơn khối và ứng dụng phân tán nằm ở ranh giới giao tiếp. Trong ứng dụng đơn khối, các mô-đun thường gọi hàm trực tiếp trong cùng một tiến trình. Lời gọi này nhanh, đồng bộ và ít chịu ảnh hưởng của mạng.



Hình 1.1: Ứng dụng phân tán nhìn từ bên ngoài và bên trong

Trong ứng dụng phân tán, một thành phần muốn sử dụng chức năng của thành phần khác phải gửi thông điệp qua mạng. Khi đó, lập trình viên phải quan tâm thêm đến độ trễ, lỗi kết nối, quá thời gian chờ, gửi lại, định dạng dữ liệu, xác thực, bảo mật và khả năng quan sát hệ thống.

Trong thực tế, sinh viên đã sử dụng nhiều ứng dụng phân tán, dù khi sử dụng có thể không nhận thấy các thành phần vận hành phía sau. Xét ở mức khái quát, có thể nhận diện một số ví dụ quen thuộc như sau:

- **Ứng dụng gọi xe hoặc đặt đồ ăn:** giao diện trên điện thoại phối hợp với các dịch vụ quản lý tài khoản, định vị, tìm tài xế, xử lý đơn hàng, thanh toán và gửi thông báo. Người dùng chỉ nhìn thấy một ứng dụng, nhưng kết quả của một thao tác có thể phụ thuộc vào nhiều thành phần và dịch vụ bên ngoài.
- **Sàn thương mại điện tử:** chức năng tìm kiếm sản phẩm, quản lý giỏ hàng, kiểm tra tồn kho, thanh toán và theo dõi vận chuyển có thể được xử lý bởi các thành phần khác nhau. Các thành phần này phải trao đổi dữ liệu để duy trì trạng thái nhất quán của đơn hàng.
- **Hệ thống học trực tuyến:** trình duyệt hoặc ứng dụng di động giao tiếp với các dịch vụ quản lý khóa học, tài liệu, bài tập, điểm số và thông báo. Một tác vụ như nộp bài có thể cần lưu tệp, ghi nhận thông tin nộp bài và gửi thông báo cho giảng viên.
- **Ứng dụng nhắn tin:** máy khách gửi thông điệp tới máy chủ, máy chủ lưu trữ và chuyển tiếp thông điệp, đồng thời một dịch vụ thông báo đẩy có thể đánh thức thiết bị của người nhận khi ứng dụng không hoạt động.

Các ví dụ trên mô tả những thành phần thường gặp của từng loại hệ thống, không khẳng định kiến trúc nội bộ của một sản phẩm hoặc thương hiệu cụ thể. Cách phân chia thành phần trong thực tế phụ thuộc vào quy mô, yêu cầu nghiệp vụ và quyết định thiết kế của từng tổ chức.

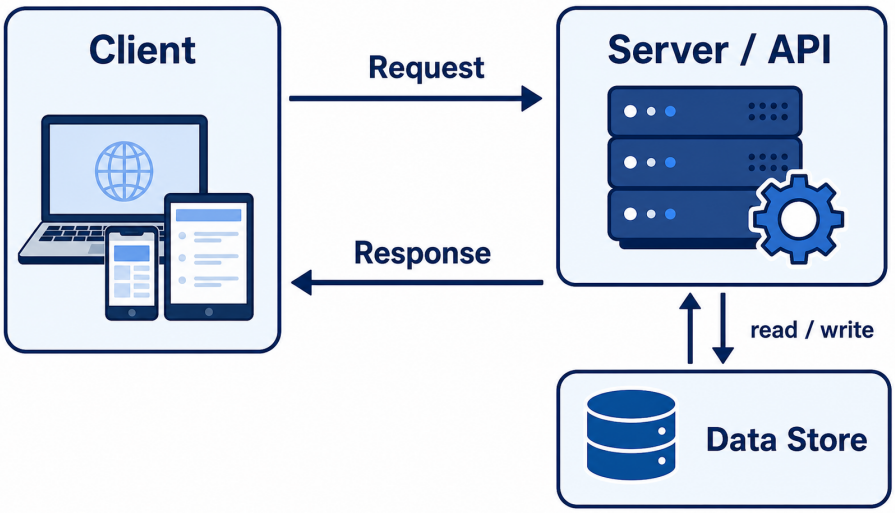
Điểm chung của các ví dụ là người dùng tương tác với một hệ thống có vẻ thống nhất, trong khi bên trong có nhiều thành phần phối hợp qua mạng. Các thành phần có thể chạy trên nhiều máy chủ, nhiều container hoặc nhiều vùng triển khai khác nhau. Ngay

cả khi được triển khai trên cùng một máy trong môi trường học tập, chúng vẫn tạo thành một ứng dụng phân tán nếu được tách thành các tiến trình độc lập và giao tiếp qua giao thức mạng hoặc cơ chế truyền thông điệp.

Không phải cứ chạy trên nhiều máy là đã có một ứng dụng phân tán được thiết kế tốt. Một hệ thống phân tán cần có cách phối hợp rõ ràng giữa các thành phần, có ranh giới giao tiếp ổn định và có cơ chế xử lý khi một phần của hệ thống chậm, lỗi hoặc tạm thời không sẵn sàng.

1.1.2 Kiến trúc máy khách–máy chủ

Kiến trúc máy khách–máy chủ, thường gặp trong tài liệu kỹ thuật dưới tên client–server, là mô hình cơ bản nhất để hiểu ứng dụng phân tán. Trong mô hình này, máy khách là thành phần khởi tạo yêu cầu, còn máy chủ là thành phần tiếp nhận yêu cầu, xử lý nghiệp vụ và trả kết quả. Máy khách có thể là trình duyệt web, ứng dụng di động, ứng dụng máy tính để bàn, công cụ dòng lệnh hoặc một dịch vụ khác. Máy chủ có thể là REST API, GraphQL API, dịch vụ gRPC, hub thời gian thực hoặc một dịch vụ xử lý nội bộ.



Hình 1.2: Mô hình máy khách–máy chủ cơ bản trong ứng dụng phân tán

Trong giáo trình này, thuật ngữ máy khách không chỉ dùng cho giao diện người dùng. Một dịch vụ gọi sang dịch vụ khác cũng có thể đóng vai trò máy khách trong một tương tác cụ thể. Vai trò máy khách và máy chủ vì vậy phụ thuộc vào chiều của lời gọi, không phụ thuộc cố định vào tên thành phần.

Một tương tác máy khách–máy chủ đơn giản thường gồm bốn bước. Thứ nhất, máy khách tạo yêu cầu chứa địa chỉ đích, phương thức hoặc tên thao tác, dữ liệu đầu vào và thông tin xác thực nếu cần. Thứ hai, yêu cầu được truyền qua mạng tới máy chủ. Thứ ba, máy chủ kiểm tra yêu cầu, thực thi logic nghiệp vụ và truy cập dữ liệu cần thiết. Thứ tư, máy chủ trả phản hồi để máy khách hiển thị, lưu lại hoặc tiếp tục một luồng xử lý khác.

Mô hình này đơn giản, dễ triển khai và phù hợp với nhiều nghiệp vụ. Tuy nhiên,

khi hệ thống lớn hơn, không phải mọi bài toán đều nên xử lý bằng một lời gọi đồng bộ từ máy khách tới máy chủ. Một số tác vụ cần phản hồi ngay, một số tác vụ cần chạy nền, một số dữ liệu cần được đẩy chủ động về máy khách, và một số sự kiện cần được phát cho nhiều dịch vụ cùng lúc. Vì vậy, ngoài khái niệm máy khách–máy chủ, người học cần nắm được các mô hình trao đổi thông điệp.

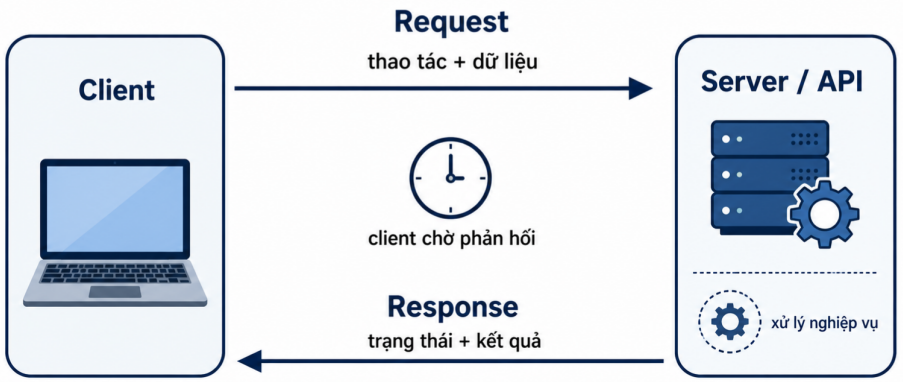
1.1.3 Các mô hình trao đổi thông điệp (MEP)

Mô hình trao đổi thông điệp, hay Message Exchange Pattern (MEP), mô tả cách các thành phần trong hệ thống gửi, nhận và phản hồi thông điệp. MEP không gắn cứng với một công nghệ cụ thể. Cùng một MEP có thể được hiện thực bằng nhiều công nghệ khác nhau, và một công nghệ có thể hỗ trợ nhiều MEP tùy cách sử dụng.

Việc nhận diện đúng MEP giúp thiết kế hệ thống rõ ràng hơn. Thay vì bắt đầu bằng câu hỏi "dùng công nghệ gì", người thiết kế nên bắt đầu bằng câu hỏi "các thành phần cần trao đổi với nhau theo kiểu nào". Khi hiểu kiểu trao đổi, ta mới chọn giao thức, khung phần mềm và hạ tầng phù hợp.

a. Yêu cầu–phản hồi

Yêu cầu–phản hồi (request–response) là mô hình phổ biến nhất trong ứng dụng web và API. Một bên gửi yêu cầu, bên còn lại xử lý và trả phản hồi. Máy khách thường chờ phản hồi trước khi tiếp tục luồng xử lý. REST API, truy vấn GraphQL và nhiều lời gọi RPC đều sử dụng mô hình này.



Hình 1.3: Mô hình trao đổi thông điệp yêu cầu–phản hồi

Ưu điểm của yêu cầu–phản hồi là dễ hiểu, dễ gỡ lỗi và phù hợp với các thao tác cần kết quả ngay, chẳng hạn đăng nhập, lấy danh sách bài tập, xem chi tiết một lượt nộp hoặc cập nhật thông tin cá nhân. Nhược điểm là máy khách và máy chủ bị ràng buộc theo thời điểm gọi. Nếu máy chủ xử lý lâu, máy khách phải chờ hoặc cần cơ chế quá thời gian chờ. Nếu máy chủ tạm thời không sẵn sàng, yêu cầu có thể thất bại.

Trong thiết kế hệ thống phân tán, yêu cầu–phản hồi nên dùng cho các thao tác ngắn, có ranh giới rõ và cần phản hồi trực tiếp. Với tác vụ dài như chấm bài tự động hoặc xuất báo cáo lớn, yêu cầu–phản hồi thường chỉ nên dùng để ghi nhận yêu cầu ban đầu, sau đó chuyển phần xử lý chính sang mô hình bất đồng bộ.

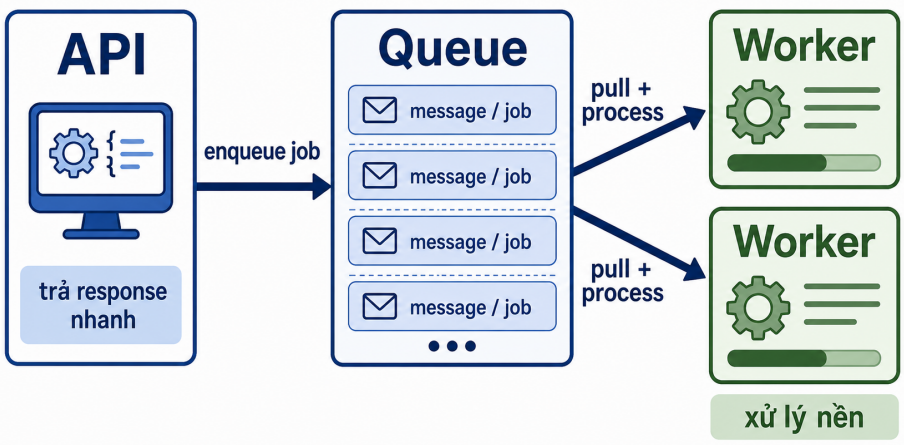
b. Thông điệp một chiều

Thông điệp một chiều (one-way message) là mô hình trong đó một thành phần gửi thông điệp đi mà không cần chờ phản hồi nghiệp vụ tương ứng. Bên gửi chỉ cần biết thông điệp đã được chấp nhận để gửi hoặc đã được đưa vào kênh truyền. Mô hình này phù hợp với nhật ký, dữ liệu đo lường, sự kiện kiểm toán hoặc các thông báo không cần trả kết quả ngay.

Thông điệp một chiều giúp giảm phụ thuộc thời gian giữa các thành phần. Tuy nhiên, vì bên gửi không nhận kết quả xử lý trực tiếp, hệ thống cần có cơ chế theo dõi trạng thái, ghi nhật ký hoặc xử lý lỗi ở phía nhận. Nếu không, lỗi có thể bị che khuất và khó truy vết.

c. Truyền thông qua hàng đợi

Truyền thông qua hàng đợi (queue-based messaging) sử dụng hàng đợi để tách bên gửi công việc và bên xử lý công việc. Thành phần gửi đưa thông điệp vào hàng đợi, sau đó dịch vụ xử lý nền lấy thông điệp ra và xử lý. Bên gửi không cần chờ dịch vụ xử lý nền hoàn tất. Đây là nền tảng của nhiều luồng xử lý bất đồng bộ.



Hình 1.4: Mô hình truyền thông qua hàng đợi

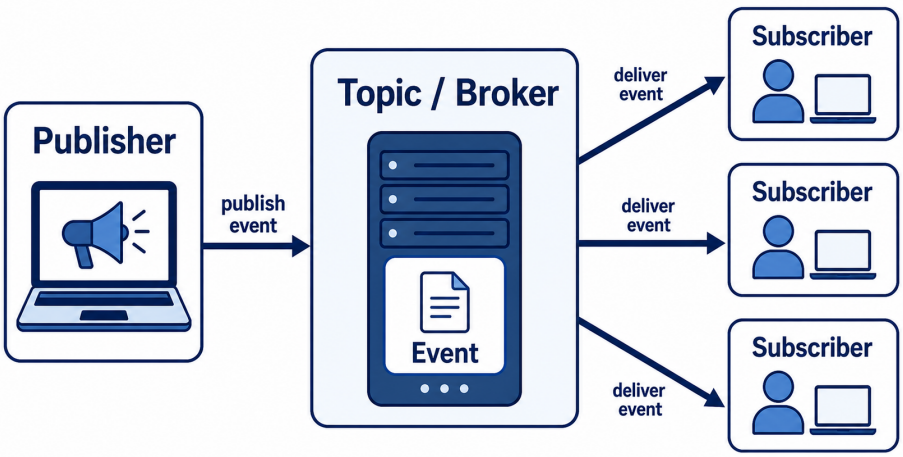
Mô hình này phù hợp với tác vụ lâu, tác vụ có thể xử lý sau, hoặc tác vụ cần điều tiết tải. Trong EduSubmit, API có thể nhận tệp bài làm, lưu thông tin lượt nộp, đưa một công việc vào hàng đợi và trả trạng thái ban đầu là `processing`. Dịch vụ xử lý nền sau đó lấy công việc từ hàng đợi để chấm bài. Cách làm này giúp API phản hồi nhanh hơn và tránh việc người dùng phải chờ toàn bộ quá trình xử lý.

Khi dùng hàng đợi, người thiết kế cần quan tâm đến độ bền của thông điệp, số lần gửi lại, xử lý thông điệp lỗi, tính lũy đẳng của dịch vụ xử lý nền và thứ tự xử lý. Một dịch vụ xử lý nền tốt cần có khả năng xử lý lại cùng một thông điệp mà không gây sai lệch dữ liệu nghiêm trọng.

Cần phân biệt nhanh truyền thông qua hàng đợi với phát hành–đăng ký. Với hàng đợi, một thông điệp thường biểu diễn một công việc và được phân phối cho một dịch vụ xử lý nền trong nhóm xử lý. Với phát hành–đăng ký, một thông điệp thường biểu diễn một sự kiện đã xảy ra và có thể được phát cho nhiều bên đăng ký độc lập, mỗi bên đăng ký tự quyết định cách phản ứng với sự kiện đó.

d. Publish–subscribe

Phát hành–đăng ký (publish–subscribe), thường viết tắt là pub/sub, là mô hình trong đó một thành phần phát sự kiện, còn nhiều thành phần khác có thể đăng ký nhận sự kiện đó. Bên phát không cần biết cụ thể có bao nhiêu bên nhận và các bên nhận sẽ làm gì. Điều này giúp giảm phụ thuộc giữa các dịch vụ.



Hình 1.5: Mô hình publish–subscribe

Ví dụ, khi chấm xong một bài nộp, dịch vụ xử lý nền có thể phát một sự kiện tên `SubmissionGraded`. Dịch vụ cập nhật điểm nhận sự kiện này để lưu kết quả. Dịch vụ thông báo nhận cùng sự kiện để gửi thông báo thời gian thực tới máy khách. Dịch vụ thống kê cũng có thể nhận sự kiện để cập nhật bảng điều khiển. Dịch vụ xử lý nền không cần gọi trực tiếp từng dịch vụ. Kiến trúc hướng sự kiện và pub/sub sẽ được học kỹ hơn ở các chương sau; tại đây chỉ cần nắm vai trò của chúng trong bản đồ MEP.

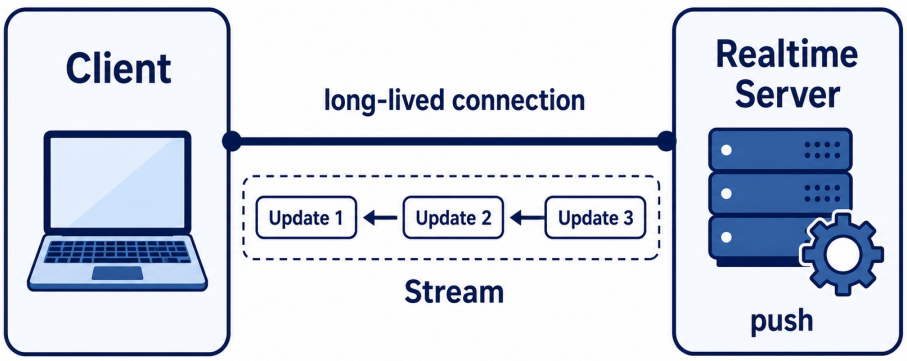
Pub/sub phù hợp với kiến trúc hướng sự kiện, nơi hệ thống được tổ chức quanh các sự kiện nghiệp vụ. Tuy nhiên, mô hình này cũng làm luồng xử lý khó nhìn hơn nếu thiếu ghi nhật ký, truy vết và quy ước đặt tên sự kiện. Khi dùng pub/sub, cần mô tả rõ sự kiện mang ý nghĩa gì, dữ liệu nào có trong phần tải dữ liệu và dịch vụ nào chịu trách nhiệm xử lý.

e. Truyền luồng và đẩy dữ liệu

Truyền luồng (streaming) và đẩy dữ liệu (push) là nhóm mô hình trong đó dữ liệu được gửi liên tục hoặc được máy chủ chủ động đẩy về máy khách khi có thay đổi. Khác với yêu cầu–phản hồi, máy khách không nhất thiết phải gửi một yêu cầu mới cho mỗi lần nhận dữ liệu.

Trong ứng dụng web hiện đại, WebSocket và SignalR thường được dùng để triển khai giao tiếp thời gian thực. Ví dụ, sau khi sinh viên nộp bài, máy khách có thể mở kết nối thời gian thực để nhận trạng thái `processing`, `graded` hoặc `failed` mà không cần liên tục gọi API kiểm tra trạng thái.

Truyền luồng phù hợp với dữ liệu cập nhật thường xuyên như nhật ký, tiến độ xử lý, thông báo, trạng thái thiết bị hoặc dữ liệu cảm biến. Đổi lại, hệ thống phải quản lý kết nối dài, xử lý mất kết nối, phân quyền theo kênh dữ liệu và tránh gửi quá nhiều thông điệp gây quá tải máy khách.



Hình 1.6: Mô hình truyền luồng và máy chủ đẩy dữ liệu

1.1.4 Ánh xạ MEP với công nghệ hiện đại

Sau khi hiểu các MEP chính, ta có thể ánh xạ chúng với các công nghệ thường gặp trong phát triển ứng dụng phân tán. Việc ánh xạ này không phải là quy tắc tuyệt đối, nhưng giúp người học có một bản đồ ban đầu để lựa chọn công nghệ theo nhu cầu giao tiếp.

Bảng 1.1: Ánh xạ MEP với công nghệ thường gặp

MEP	Công nghệ thường gặp	Tình huống sử dụng phù hợp
Yêu cầu–phản hồi	REST API, GraphQL, gRPC unary	Lấy dữ liệu hoặc gửi thao tác cần phản hồi trực tiếp
Thông điệp một chiều	Đường ống nhật ký, máy khách đo lường, bộ môi giới thông điệp	Gửi nhật ký, kiểm toán hoặc chỉ số đo lường không cần kết quả nghiệp vụ ngay
Qua hàng đợi	Hàng đợi RabbitMQ, hàng đợi Azure Service Bus, nhóm tiêu thụ Redis Streams	Xử lý nền, điều tiết tải, gửi lại công việc
Phát hành–đăng ký	RabbitMQ exchange, Kafka topic, Redis pub/sub	Phát sự kiện cho nhiều dịch vụ độc lập
Truyền luồng / đẩy dữ liệu	WebSocket, SignalR, gRPC stream, SSE	Cập nhật thời gian thực hoặc luồng dữ liệu liên tục

RESTful API thường là lựa chọn đầu tiên cho giao tiếp máy khách–máy chủ vì đơn giản, phổ biến và dễ kiểm thử. REST tổ chức API quanh tài nguyên, sử dụng các phương thức HTTP như `GET`, `POST`, `PUT` và `DELETE`. Trong giáo trình này, REST được dùng để xây dựng các điểm cuối cơ bản cho nghiệp vụ giao bài, nộp bài và truy vấn dữ liệu.

GraphQL cũng dùng yêu cầu–phản hồi, nhưng cho phép máy khách mô tả chính xác dữ liệu cần lấy. Thay vì phải gọi nhiều điểm cuối hoặc nhận dư dữ liệu, máy khách gửi một truy vấn để chọn các trường cần thiết. GraphQL phù hợp khi giao diện có nhu cầu truy vấn linh hoạt, nhiều màn hình cần các tổ hợp dữ liệu khác nhau hoặc khi muốn giảm số lượt đi-về giữa máy khách và máy chủ.

gRPC thường được dùng cho giao tiếp dịch vụ–dịch vụ. Công nghệ này dựa trên hợp đồng được mô tả bằng Protocol Buffers, hỗ trợ sinh mã máy khách/máy chủ và truyền dữ liệu hiệu quả. Trong hệ thống phân tán, gRPC phù hợp với các API nội bộ có hợp đồng rõ ràng, cần hiệu năng tốt và được kiểm soát bởi cùng một nhóm phát triển

hoặc cùng một tổ chức.

RabbitMQ và các bộ môi giới thông điệp tương tự thường được dùng cho truyền thông qua hàng đợi và pub/sub. Với hàng đợi, bộ môi giới giúp lưu thông điệp trung gian để dịch vụ xử lý nền xử lý sau. Với pub/sub, bộ môi giới giúp phát một sự kiện đến nhiều bên tiêu thụ. Bộ môi giới thông điệp vì vậy đóng vai trò quan trọng trong việc tách rời các thành phần và xây dựng luồng xử lý bất đồng bộ.

SignalR là khung phần mềm giúp triển khai giao tiếp thời gian thực trên nền .NET. SignalR che bớt chi tiết của WebSocket và có thể chuyển sang cơ chế dự phòng khi cần. Trong giáo trình này, SignalR được dùng để gửi thông báo trạng thái từ máy chủ về máy khách, đặc biệt khi kết hợp với luồng xử lý bất đồng bộ: máy khách nộp bài qua REST API, dịch vụ xử lý nền xử lý qua hàng đợi, sau đó kết quả được thông báo lại qua kênh thời gian thực.

Một hệ thống thực tế thường không chỉ dùng một MEP. Cùng một nghiệp vụ có thể đi qua nhiều kiểu trao đổi thông điệp. Ví dụ, luồng nộp bài có thể bắt đầu bằng yêu cầu–phản hồi khi máy khách gọi API, chuyển sang hàng đợi khi API đưa công việc vào hàng đợi, dùng pub/sub khi dịch vụ xử lý nền phát sự kiện hoàn tất, và kết thúc bằng đẩy dữ liệu khi máy chủ thông báo trạng thái mới về máy khách.

> Không nên chọn công nghệ chỉ vì công nghệ đó phổ biến. Hãy xác định trước đặc điểm của tương tác: cần phản hồi ngay hay có thể xử lý sau, một bên nhận hay nhiều bên nhận, dữ liệu gửi một lần hay liên tục, và lỗi cần được xử lý ở đâu. Câu trả lời cho các câu hỏi này sẽ dẫn tới MEP phù hợp.

Các chương tiếp theo của giáo trình sẽ lần lượt triển khai những MEP này bằng các công nghệ cụ thể. RESTful API và GraphQL đại diện cho nhóm API phía máy khách. SignalR đại diện cho giao tiếp thời gian thực. gRPC đại diện cho giao tiếp dịch vụ–dịch vụ. RabbitMQ, hàng đợi và pub/sub đại diện cho xử lý bất đồng bộ và kiến trúc hướng sự kiện. Khi ghép các phần này lại, người học sẽ xây dựng được một hệ thống phân tán cơ bản nhưng có đủ các luồng giao tiếp quan trọng.

1.2 Kỹ thuật lập trình cơ bản với ASP.NET Core Web API

Phần trước đã giới thiệu ranh giới máy khách–máy chủ và mô hình yêu cầu–phản hồi. Để biến ranh giới đó thành một chương trình chạy được, người học cần nắm một lớp kỹ thuật trung gian: cách tạo project Web API, cách ASP.NET Core khởi tạo ứng dụng, cách ánh xạ một địa chỉ HTTP tới đoạn mã xử lý, và cách tạo phản hồi. Phần này dùng một API lời chào tối giản, chưa đưa nghiệp vụ EduSubmit vào, nhờ đó người học có thể tập trung vào cấu trúc và API lập trình của framework trước khi tích hợp chúng trong phần ứng dụng 1.3.

Mã nguồn hoàn chỉnh nằm tại `content/examples/chapter01/section02`. Ví dụ sử dụng .NET 10 và ASP.NET Core Minimal API, không cần package ngoài. API cung cấp một endpoint kiểm tra trạng thái, một endpoint nhận tên từ URL, và một endpoint nhận dữ liệu JSON để tạo lời chào.

1.2.1 Cấu trúc solution và project ASP.NET Core Web API

Trong .NET, *project* là đơn vị biên dịch: file `.csproj` khai báo SDK, target framework và các tùy chọn biên dịch của một ứng dụng hoặc thư viện. *Solution* là lớp tổ chức phía trên, dùng để nhóm một hay nhiều project liên quan và cho phép build chúng bằng một lệnh. Một hệ thống phân tán thường có nhiều project, nhưng ví dụ đầu tiên chỉ cần một project Web API trong solution để người học nhìn rõ vai trò của từng lớp.

Cấu trúc nguồn của ví dụ như sau:

```
examples/chapter01/section02/
  BasicWebApi.slnx
  NuGet.config
  README.md
  src/
    BasicWebApi/
      BasicWebApi.csproj
      Program.cs
```

File `BasicWebApi.slnx` mô tả solution, còn thư mục `src/BasicWebApi` chứa project thực thi. Cách đặt project trong thư mục `src` giúp tách mã nguồn khỏi tài liệu hướng dẫn và các file ở cấp solution. Khi hệ thống có thêm project kiểm thử, thư viện dùng chung hoặc máy khách, các project đó có thể được bổ sung mà không làm thay đổi vai trò của project hiện tại.

Có thể tạo cấu trúc ban đầu bằng .NET CLI:

```
dotnet new sln -n BasicWebApi --format slnx
dotnet new webapi -n BasicWebApi -o src/BasicWebApi --no-openapi
dotnet sln BasicWebApi.slnx add src/BasicWebApi/BasicWebApi.csproj
```

Template `webapi` có thể tạo thêm các file cấu hình phục vụ môi trường phát triển. Ví dụ đi kèm giáo trình giữ lại tập file tối thiểu cần cho mục tiêu của section để người học dễ theo dõi; việc lược bớt file tùy chọn không làm thay đổi hosting model hoặc cú pháp Minimal API.

Mã nguồn 1.1: Khai báo project ASP.NET Core Web API trên .NET 10

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

</Project>
```

Thuộc tính `Sdk="Microsoft.NET.Sdk.Web"` phân biệt project web với project console hoặc thư viện lớp. `TargetFramework` chọn .NET 10, `Nullable` bật phân tích kiểu tham chiếu có thể nhận `null`, còn `ImplicitUsings` tự nhập các namespace thông dụng. Project không khai báo package ngoài vì các thành phần cơ bản của ASP.NET Core đã được cung cấp bởi shared framework đi kèm .NET SDK và runtime.

Solution không phải là một tiến trình chạy. Nó chỉ là cấu trúc tổ chức cho công cụ phát triển và lệnh build. Mỗi project thực thi sau này có thể trở thành một tiến trình độc lập, và chính ranh giới tiến trình mới tạo ra ranh giới giao tiếp của ứng dụng phân tán.

1.2.2 *Hosting model và tổ chức mã nguồn trong* `Program.cs`

Một Web API cần môi trường lưu trữ để quản lý vòng đời ứng dụng, cấu hình, ghi nhật ký, cơ chế tiêm phụ thuộc (dependency injection) và máy chủ HTTP. Trong hosting model hiện đại của ASP.NET Core, các công việc đó được biểu diễn bằng hai đối tượng trung tâm: `WebApplicationBuilder` trong giai đoạn cấu hình và `WebApplication` trong giai đoạn chạy.

Mã nguồn 1.2: Tạo builder, đăng ký dịch vụ và build ứng dụng

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<GreetingService>();

var app = builder.Build();
```

Lệnh `WebApplication.CreateBuilder(args)` tạo builder và đọc các nguồn cấu hình theo quy ước của ASP.NET Core, bao gồm tham số dòng lệnh và biến môi trường. Thuộc tính `builder.Services` là nơi đăng ký các dịch vụ mà endpoint có thể yêu cầu. Trong ví dụ, `GreetingService` được đăng ký dạng singleton để cùng một instance phục vụ trong suốt vòng đời tiến trình. Dịch vụ này không lưu một collection dùng chung; nó chỉ sinh mã tăng dần bằng `Interlocked`, vì vậy ví dụ vẫn an toàn khi có nhiều yêu cầu đồng thời ở mức minh họa.

Lệnh `builder.Build()` kết thúc giai đoạn khai báo và tạo đối tượng `app`. Từ thời điểm này, chương trình cấu hình đường ống xử lý và ánh xạ các endpoint lên `app`. Cuối file, `app.Run()` khởi động máy chủ HTTP và giữ tiến trình hoạt động cho đến khi ứng dụng được yêu cầu dừng. Có thể hình dung `Program.cs` theo hai pha:

1. **Chuẩn bị ứng dụng:** tạo builder, đọc cấu hình và đăng ký dịch vụ.
2. **Tạo và chạy ứng dụng:** build đối tượng `app`, ánh xạ endpoint, rồi gọi `Run`.

Thứ tự này có ý nghĩa. Dịch vụ nên được đăng ký trước `Build`; endpoint phải được ánh xạ trước `Run`. Nếu đặt mã cấu hình sau `Run`, mã đó chỉ được thực thi khi máy chủ đã dừng và vì thế không thể tham gia xử lý các yêu cầu đang đến.

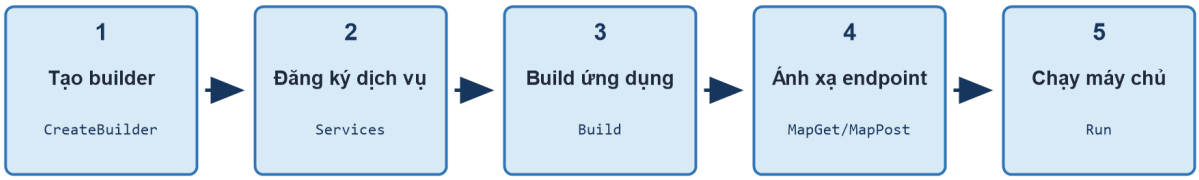
1.2.3 *Tạo endpoint với Minimal API*

Endpoint là điểm mà một yêu cầu HTTP được nối với mã xử lý trong ứng dụng. Với Minimal API, endpoint được khai báo trực tiếp bằng các phương thức như `MapGet`, `MapPost`, `MapPut` hoặc `MapDelete`. Mỗi lời gọi ánh xạ thường chứa ba thông tin: phương thức HTTP, mẫu route (route pattern) và hàm xử lý route (route handler).

Ví dụ `app.MapGet("/",handler)` nghĩa là khi máy chủ nhận yêu cầu `GET/`, ASP.NET Core sẽ gọi handler tương ứng. Cặp phương thức HTTP và route pattern xác định endpoint; cùng một đường dẫn có thể có handler khác nhau cho `GET` và `POST`. Phần 2.1 và

2.2 sẽ giải thích sâu hơn cách chọn phương thức và URI theo tư duy REST; ở đây mục tiêu chỉ là đọc và viết được cú pháp ánh xạ.

Trình tự chính khi xây dựng file `Program.cs` được tóm tắt trong Hình 1.7.



Hình 1.7: Năm bước chính khi viết một ASP.NET Core Minimal API

Mã nguồn 1.3: Ánh xạ các endpoint GET và POST bằng Minimal API

```
app.MapGet("/", () => Results.Ok(new {
    application = "BasicWebApi",
    message = "ASP.NET Core Web API is running."
}));

app.MapGet("/hello/{name}", (string name, string? language) => {
    var message = string.Equals(language, "en", StringComparison.OrdinalIgnoreCase)
        ? $"Hello, {name}!"
        : $"Xin chào, {name}!";

    return Results.Ok(new GreetingResponse(message));
});

app.MapPost("/greetings", (CreateGreetingRequest request, GreetingService service) => {
    if (string.IsNullOrWhiteSpace(request.Name)) {
        return Results.BadRequest(new ApiError("Name must not be empty."));
    }

    var greeting = service.Create(request.Name.Trim());
    return Results.Created($"/greetings/{greeting.Id}", greeting);
});

app.Run();

public sealed record CreateGreetingRequest(string Name);
```

Endpoint đầu tiên trả thông tin kiểm tra để xác nhận tiến trình API đang hoạt động. Endpoint thứ hai có route pattern `/hello/\{name\}`, trong đó phần nằm trong dấu ngoặc nhọn là tham số route. Endpoint thứ ba dùng `MapPost` để nhận một yêu cầu tạo lời chào. Mỗi handler chỉ tập trung vào một tương tác nhỏ: nhận đầu vào, thực hiện xử lý cần thiết và trả kết quả.

Minimal API giảm số lượng file và lớp cần tạo nên phù hợp cho ví dụ nhập môn, microservice nhỏ hoặc prototype. "Minimal" nói về mô hình lập trình gọn, không có nghĩa API thiếu khả năng cấu hình, dependency injection hay middleware. Khi nghiệp vụ lớn hơn, route handler nên gọi các service chuyên trách thay vì chứa toàn bộ logic trong `Program.cs`; cách tách này đã được gợi ý bằng `GreetingService` trong ví dụ.

1.2.4 *Tiếp nhận request và tạo response với* `Results`

Route handler không tự đọc chuỗi HTTP thô. ASP.NET Core thực hiện *model binding*: framework xem khai báo tham số của handler rồi lấy dữ liệu từ vị trí phù hợp trong request hoặc từ container dependency injection. Ví dụ sử dụng bốn dạng tham số cơ bản:

Bảng 1.2: Nguồn dữ liệu của các tham số route handler trong ví dụ

Tham số	Nguồn dữ liệu	Cách nhận diện trong ví dụ
`name`	Route	Trùng với placeholder `\{name\}` trong đường dẫn
`language`	Query string	Kiểu `string?` và không xuất hiện trong route
`request`	JSON request body	Kiểu phức hợp `CreateGreetingRequest` của endpoint POST
`service`	Dependency injection	Kiểu `GreetingService` đã đăng ký trong `builder.Services`

Với yêu cầu `GET/hello/An?language=vi`, giá trị `An` được gán cho tham số `name`, còn `vi` được gán cho `language`. Với yêu cầu `POST/greetings`, JSON `\{"name":"Binh"\}` được khử tuần tự hóa (deserialize) thành `CreateGreetingRequest`. Người viết handler vì vậy làm việc với giá trị C# có kiểu rõ ràng thay vì tự tách chuỗi URL hoặc tự phân tích JSON.

Đầu ra của handler được tạo bằng các factory method trong lớp `Results`. Mỗi phương thức diễn đạt một loại phản hồi HTTP thường gặp:

Bảng 1.3: Các kết quả HTTP được sử dụng trong BasicWebApi

Phương thức	Status code	Ý nghĩa trong ví dụ
`Results.Ok(value)`	200	Yêu cầu thành công; `value` được tuần tự hóa thành JSON
`Results.BadRequest(error)`	400	Dữ liệu đầu vào không hợp lệ, chẳng hạn tên rỗng
`Results.Created(uri,value)`	201	Đã tạo kết quả mới và gửi kèm header `Location`

Mã nguồn 1.4: Các kiểu dữ liệu request, response và service tối giản

```
public sealed record ApiError(string Message);

public sealed record StoredGreeting(int Id, string Name, DateTimeOffset CreatedAt);

public sealed class GreetingService {
    private int _lastId;

    public StoredGreeting Create(string name) =>
        new(Interlocked.Increment(ref _lastId), name, DateTimeOffset.UtcNow);
}
```

Các record làm cho hợp đồng dữ liệu của ví dụ rõ ràng: request có trường `Name`, phản hồi thành công chứa dữ liệu lời chào, còn phản hồi lỗi có thông điệp. `GreetingService` minh họa cách đưa xử lý ra khỏi route handler và nhận service qua dependency injection. Ví dụ chưa dùng cơ sở dữ liệu và không lưu dữ liệu bền vững; mã định danh chỉ phục vụ quan sát phản hồi `201Created`. Những vấn đề về tài nguyên, DTO và hợp đồng REST đầy đủ sẽ được phát triển ở Bài 2.

1.2.5 Chạy và kiểm thử Web API

Trước khi chạy, cần xác nhận phiên bản SDK:

```
dotnet --version
```

Ví dụ được kiểm thử với .NET SDK `10.0.301`. Từ thư mục gốc repository, có thể restore và build bằng các lệnh sau:

```
cd examples/chapter01/section02
dotnet restore BasicWebApi.slnx --configfile NuGet.config
dotnet build BasicWebApi.slnx --no-restore
```

Sau khi build thành công, chạy API trên một địa chỉ HTTP cố định để các lệnh gọi thử dễ lặp lại:

```
dotnet run --project src/BasicWebApi --urls http://localhost:5081
```

Khi terminal hiển thị thông báo ứng dụng đang lắng nghe tại `http://localhost:5081`, mở terminal thứ hai và gửi các yêu cầu:

```
curl.exe http://localhost:5081/
curl.exe "http://localhost:5081/hello/An?language=vi"
curl.exe -X POST http://localhost:5081/greetings -H "Content-Type: application/json"
↪  -d '{"name":"Binh"}'
```

Hai phản hồi đầu cho biết API đang chạy và endpoint đã nhận đúng dữ liệu từ route/query string. Yêu cầu POST trả JSON có `id`, `name` và `createdAt`; status code là `201Created`. Có thể gửi thêm body `\{"name":""\}` để quan sát phản hồi `400BadRequest`.

Một số lỗi thường gặp ở bước này gồm cổng `5081` đã được tiến trình khác sử dụng, chạy lệnh từ sai thư mục, hoặc máy có SDK không khớp với `net10.0`. Khi cổng bị trùng, có thể dừng tiến trình cũ hoặc chọn cổng khác và dùng cùng địa chỉ đó trong lệnh `curl.exe`. Khi nhận lỗi target framework, cần kiểm tra `dotnet--list-sdks` và cài đúng .NET SDK.

Kết thúc phần này, người học đã có thể đi từ một project rỗng đến API chạy được: hiểu vai trò của solution/project, tổ chức `Program.cs`, ánh xạ endpoint, nhận dữ liệu request và tạo response. Phần ứng dụng 1.3 sẽ dùng chính các kỹ thuật đó để tạo lát cắt đầu tiên của EduSubmit gồm API và máy khách chạy ở hai tiến trình độc lập.

1.3 [Ứng dụng] Tạo cấu trúc ứng dụng phân tán cơ bản

Dự án xuyên suốt: EduSubmit

Trong giáo trình này, các bài thực hành được nối với nhau quanh một dự án nhỏ có tên **EduSubmit**. Có thể hiểu EduSubmit là một cổng tiếp nhận và xử lý bài nộp trong môi trường học tập: giảng viên tạo bài tập, sinh viên xem danh sách bài tập và gửi đường dẫn bài làm, hệ thống ghi nhận lượt nộp, xử lý hoặc chấm bài ở phía sau, rồi thông báo trạng thái lại cho người dùng.

Tên EduSubmit được dùng để nhấn mạnh hai ý. Thứ nhất, đây là một bài toán giáo dục quen thuộc, nên sinh viên dễ hình dung nghiệp vụ mà không bị phân tán bởi miền ứng dụng quá xa lạ. Thứ hai, thao tác "nộp bài" không chỉ là một biểu mẫu gửi dữ liệu, mà là điểm khởi đầu cho nhiều kiểu giao tiếp trong hệ thống phân tán: gọi API đồng bộ, cập nhật trạng thái thời gian thực, truy vấn dữ liệu linh hoạt, gọi dịch vụ nội bộ, đưa công việc vào hàng đợi và phát sự kiện cho các thành phần khác.

Ở phiên bản hoàn chỉnh hơn, EduSubmit có thể gồm các thành phần sau:

- Máy khách cho sinh viên hoặc giảng viên, ban đầu là ứng dụng dòng lệnh để tập trung vào giao tiếp giữa các tiến trình.
- API phía máy khách, tiếp nhận yêu cầu đọc danh sách bài tập, tạo lượt nộp và đọc trạng thái lượt nộp.
- Cơ sở dữ liệu lưu bài tập, lượt nộp, người dùng, trạng thái xử lý và kết quả chấm.
- Kênh thời gian thực gửi trạng thái xử lý về đúng máy khách đang theo dõi một lượt nộp.
- Dịch vụ xử lý nền hoặc dịch vụ chấm bài, thực hiện các tác vụ lâu mà API không nên xử lý trực tiếp trong một yêu cầu HTTP.
- Hàng đợi và cơ chế phát hành–đăng ký để tách rời API, dịch vụ xử lý nền, dịch vụ thông báo và dịch vụ thống kê.

Giáo trình không xây dựng toàn bộ hệ thống ngay từ đầu. Thay vào đó, mỗi chương bổ sung một lát cắt nhỏ để người học thấy vì sao cần thêm một kiểu giao tiếp mới và kiểu giao tiếp đó giải quyết vấn đề gì trong cùng một nghiệp vụ. Bốn chương đầu tập trung vào ranh giới giữa máy khách và máy chủ:

Các tên dự án mã nguồn như `Submission.Api` hoặc `Submission.Client` vẫn được giữ trong ví dụ để ngắn gọn và dễ nhận ra vai trò của từng tiến trình. Khi đọc giáo trình, hãy xem các dự án đó là những lát cắt triển khai cụ thể của EduSubmit. Mục tiêu quan trọng không phải là hoàn thiện một sản phẩm nộp bài đầy đủ ngay trong chương đầu, mà là từng bước hiểu cách một hệ thống phân tán được hình thành từ các thành phần nhỏ, có ranh giới giao tiếp rõ ràng và có thể mở rộng theo nhu cầu nghiệp vụ.

Trong phần thực hành này, ta bắt đầu xây dựng lát cắt đầu tiên của EduSubmit. Bài toán đặt ra rất nhỏ: sinh viên cần xem được danh sách bài tập hiện có và gửi một lượt nộp bài gồm mã sinh viên, bài tập tương ứng và đường dẫn kho mã nguồn chứa bài làm. Ở giai đoạn này, hệ thống chưa cần đăng nhập, chưa lưu cơ sở dữ liệu và chưa chấm bài tự động. Mục tiêu là làm rõ cách hai thành phần phần mềm độc lập giao tiếp với nhau qua HTTP.

Về mặt nghiệp vụ, luồng tối thiểu cần có ba thao tác:

- Máy khách kiểm tra API có đang hoạt động hay không.

Bảng 1.4: Lộ trình mở rộng dự án EduSubmit trong bốn chương đầu

Chương	Lát cắt xây dựng	Mục tiêu học tập chính
Chương 1	API tối giản và máy khách dòng lệnh	Nhìn thấy ranh giới máy khách–máy chủ, yêu cầu–phản hồi HTTP và lỗi khi hai tiến trình giao tiếp qua mạng
Chương 2	RESTful API cho bài tập và lượt nộp	Thiết kế tài nguyên, URI, phương thức HTTP, DTO, mã trạng thái và xử lý phản hồi lỗi
Chương 3	Thông báo trạng thái bằng SignalR	Bổ sung kênh thời gian thực để máy khách nhận trạng thái xử lý mà không phải liên tục hỏi API
Chương 4	GraphQL cho truy vấn linh hoạt	So sánh REST và GraphQL khi giao diện cần lấy các tổ hợp dữ liệu khác nhau từ nhiều tài nguyên

- Máy khách lấy danh sách bài tập mà sinh viên có thể nộp.
- Máy khách gửi thông tin một lượt nộp bài và nhận lại mã lượt nộp cùng trạng thái ban đầu.

Về mặt kiến trúc, ta tách bài toán thành hai tiến trình. `Submission.Api` chịu trách nhiệm cung cấp điểm cuối HTTP và xử lý yêu cầu. `Submission.Client` chịu trách nhiệm gọi API từ phía máy khách. Sự tách biệt này là điểm quan trọng của bài thực hành: thay vì gọi hàm trực tiếp trong cùng một chương trình, máy khách phải gửi yêu cầu qua địa chỉ mạng, nhận phản hồi và xử lý lỗi nếu API không sẵn sàng.

1.3.1 Khởi tạo dự án API

Mục tiêu kỹ thuật đầu tiên là tạo một lát cắt EduSubmit nhỏ nhất có đủ hai thành phần giao tiếp qua mạng: một API đóng vai trò máy chủ và một ứng dụng dòng lệnh đóng vai trò máy khách. Hệ thống chưa cần cơ sở dữ liệu, xác thực hay hàng đợi thông điệp. Ta chỉ tập trung vào ranh giới giao tiếp: máy khách gửi yêu cầu HTTP, API xử lý và trả phản hồi HTTP. Đây là nền tảng sẽ được mở rộng ở các bài sau.

Trong ví dụ này, API có tên `Submission.Api`. Máy khách có tên `Submission.Client`. Hai dự án được đặt trong cùng một solution để sinh viên dễ mở, biên dịch và chạy, nhưng về mặt kiến trúc chúng là hai tiến trình độc lập. Khi chạy thật, máy khách không gọi hàm trực tiếp trong API; máy khách chỉ biết địa chỉ HTTP của API.

Trước khi bắt đầu, cần cài .NET 10 SDK và kiểm tra bằng lệnh:

```
dotnet --version
```

Tạo solution và hai dự án bằng các lệnh sau:

```
mkdir MiniDistributedSystem
cd MiniDistributedSystem

dotnet new sln -n MiniDistributedSystem
dotnet new web -n Submission.Api -o src/Submission.Api
```

```
dotnet new console -n Submission.Client -o src/Submission.Client

dotnet sln add src/Submission.Api/Submission.Api.csproj
dotnet sln add src/Submission.Client/Submission.Client.csproj
```

Sau bước này, cấu trúc thư mục tối thiểu nên có dạng:

```
MiniDistributedSystem/
  MiniDistributedSystem.slnx
  src/
    Submission.Api/
      Program.cs
      Submission.Api.csproj
    Submission.Client/
      Program.cs
      Submission.Client.csproj
```

Việc tách `Submission.Api` và `Submission.Client` thành hai dự án riêng có ý nghĩa quan trọng. Nếu tất cả mã nằm trong cùng một dự án và gọi nhau bằng phương thức nội bộ, ta chỉ đang xây dựng một ứng dụng đơn khối. Khi tách thành hai tiến trình và giao tiếp qua HTTP, ta bắt đầu tiếp cận bài toán phân tán: địa chỉ dịch vụ, định dạng dữ liệu, trạng thái phản hồi, lỗi mạng và khả năng thay thế từng thành phần.

Trong môi trường học tập, API và máy khách có thể chạy trên cùng một máy. Điều này không làm mất ý nghĩa phân tán của bài thực hành, vì ranh giới quan trọng ở đây là ranh giới tiến trình và giao thức giao tiếp. Khi chuyển sang môi trường triển khai, cùng một mã nguồn có thể được chạy trên hai máy hoặc hai container khác nhau.

1.3.2 Tạo điểm cuối đơn giản

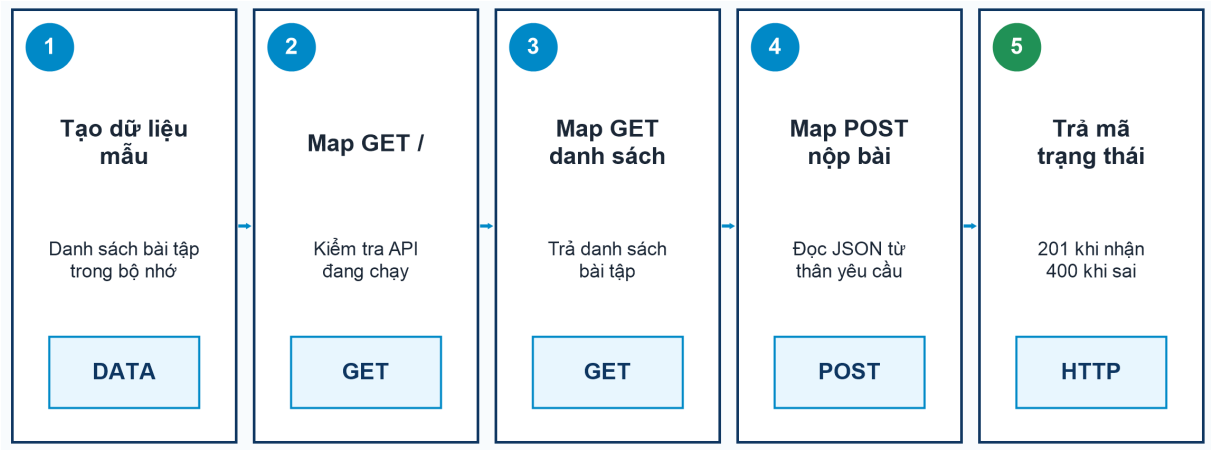
Điểm cuối (endpoint) là điểm vào mà máy khách có thể gọi để sử dụng một chức năng của máy chủ. Với HTTP API, điểm cuối thường được mô tả bằng cặp phương thức HTTP và đường dẫn, ví dụ `GET/api/assignments` hoặc `POST/api/submissions`. Ở bài này, ta chỉ tạo điểm cuối HTTP tối giản để minh họa yêu cầu–phản hồi; các nguyên tắc REST và thiết kế API đầy đủ sẽ được học kỹ hơn ở bài 2. Ta tạo ba điểm cuối nhỏ:

- `GET/`: kiểm tra API đang chạy.
- `GET/api/assignments`: lấy danh sách bài tập mẫu.
- `POST/api/submissions`: gửi một lượt nộp bài mẫu.

Mở tệp `src/Submission.Api/Program.cs` và thay nội dung bằng đoạn mã sau:

Mã nguồn 1.5: Điểm cuối tối giản trong dự án Submission.Api

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var assignments = new[]
{
    new Assignment(1, "Bai tap 1", "Tao API dau tien"),
    new Assignment(2, "Bai tap 2", "Goi API tu client")
};
```



Hình 1.8: Các bước chính khi viết mã API tối giản trong Listing 1.1

```
app.MapGet("/", () => Results.Ok(new {
    service = "Submission.Api",
    status = "running"
}));

app.MapGet("/api/assignments", () => Results.Ok(assignments));

app.MapPost("/api/submissions", (CreateSubmissionRequest request) => {
    if (request.AssignmentId <= 0 ||
        string.IsNullOrWhiteSpace(request.StudentCode) ||
        string.IsNullOrWhiteSpace(request.RepositoryUrl)) {
        return Results.BadRequest(new { message = "Du lieu nop bai khong hop le" });
    }

    var submission = new SubmissionResult(
        Guid.NewGuid(),
        request.AssignmentId,
        request.StudentCode,
        "received",
        DateTimeOffset.UtcNow);

    return Results.Created($"/api/submissions/{submission.Id}", submission);
});

app.Run();

record Assignment(int Id, string Title, string Description);

record CreateSubmissionRequest(
    int AssignmentId,
    string StudentCode,
    string RepositoryUrl);

record SubmissionResult(
    Guid Id,
    int AssignmentId,
    string StudentCode,
    string Status,
    DateTimeOffset ReceivedAt);
```

Đoạn mã trên dùng Minimal API của ASP.NET Core. Cách viết này phù hợp cho bài đầu tiên vì giảm số lượng tệp cần giải thích, nhưng vẫn giữ đúng bản chất của một máy chủ API. Biến `assignments` là dữ liệu mẫu nằm trong bộ nhớ để tránh làm bài thực hành bị nhiễu bởi cơ sở dữ liệu. Lệnh `MapGet` khai báo các điểm cuối đọc dữ liệu: điểm cuối `GET/` dùng để kiểm tra dịch vụ, còn điểm cuối `GET/api/assignments` trả danh sách bài tập cho máy khách.

Điểm cuối `POST/api/submissions` là phần quan trọng nhất của API trong ví dụ này. Tham số `CreateSubmissionRequestrequest` cho thấy ASP.NET Core tự đọc JSON trong thân yêu cầu HTTP và ánh xạ vào một đối tượng C#. Khối `if` đầu tiên kiểm tra dữ liệu đầu vào trước khi xử lý; nếu thiếu mã sinh viên, mã bài tập không hợp lệ hoặc thiếu đường dẫn kho mã nguồn, API trả `BadRequest`. Khi dữ liệu hợp lệ, API tạo `SubmissionResult`, sinh mã lượt nộp bằng `Guid.NewGuid()`, gán trạng thái ban đầu là `received` và trả `Results.Created`. Mã trạng thái HTTP `201Created` nói với máy khách rằng máy chủ đã ghi nhận một tài nguyên mới.

Các kiểu `record` ở cuối tệp đóng vai trò cấu trúc dữ liệu trao đổi qua ranh giới API. Trong ví dụ nhỏ này, chúng được đặt ngay trong `Program.cs` để dễ đọc. Ở các bài sau, khi mô hình lớn hơn, các cấu trúc này thường được tách thành DTO riêng. Điểm cuối nộp bài hiện chưa chấm bài thật; nó chỉ ghi nhận yêu cầu nộp bài trước, còn các bước xử lý dài hơn như đưa công việc vào hàng đợi, dịch vụ xử lý nền chấm bài và gửi thông báo thời gian thực sẽ được thêm ở các bài sau.

Biên dịch API để phát hiện lỗi cú pháp trước khi chạy:

```
dotnet build src/Submission.Api/Submission.Api.csproj
```

Chạy API trên một cổng cố định để máy khách dễ gọi:

```
dotnet run --project src/Submission.Api --urls http://localhost:5000
```

Khi API chạy thành công, có thể mở trình duyệt hoặc dùng công cụ HTTP để gọi `http://localhost:5000/`. Phản hồi mong đợi là một JSON nhỏ cho biết dịch vụ `Submission.Api` đang ở trạng thái `running`.

1.3.3 Tạo máy khách gọi thử API

Sau khi có API, ta cần một máy khách thật sự gọi qua HTTP để kiểm tra ranh giới máy khách–máy chủ. Trong thực tế, máy khách có thể là ứng dụng web, ứng dụng di động hoặc một dịch vụ khác. Trong bài này, máy khách dòng lệnh là lựa chọn phù hợp vì đơn giản, dễ quan sát yêu cầu–phản hồi và không làm nhiễu bài học bằng giao diện người dùng.

Mở tệp `src/Submission.Client/Program.cs` và thay nội dung bằng đoạn mã sau:

Mã nguồn 1.6: Máy khách dòng lệnh gọi API qua HTTP

```
using System.Net.Http.Json;

var apiBaseUrl = args.Length > 0 ? args[0] : "http://localhost:5000";
```

Hình 1.9: Các bước chính khi viết mã máy khách gọi API trong Listing 1.2

```
using var http = new HttpClient {
    BaseAddress = new Uri(apiBaseUrl)
};

var assignments = await http.GetFromJsonAsync<List<Assignment>>("/api/assignments");
Console.WriteLine("Danh sach bai tap:");

foreach (var assignment in assignments ?? new List<Assignment>()) {
    Console.WriteLine($"- {assignment.Id}: {assignment.Title}");
}

var request = new CreateSubmissionRequest(
    AssignmentId: 1,
    StudentCode: "SV001",
    RepositoryUrl: "https://example.com/sv001/bai-tap-1");

var response = await http.PostAsJsonAsync("/api/submissions", request);
response.EnsureSuccessStatusCode();

var result = await response.Content.ReadFromJsonAsync<SubmissionResult>();
Console.WriteLine($"Da nop bai. Ma luot nop: {result?.Id}");
Console.WriteLine($"Trang thai: {result?.Status}");

record Assignment(int Id, string Title, string Description);

record CreateSubmissionRequest(
    int AssignmentId,
    string StudentCode,
    string RepositoryUrl);

record SubmissionResult(
    Guid Id,
    int AssignmentId,
    string StudentCode,
    string Status,
    DateTimeOffset ReceivedAt);
```

Máy khách sử dụng `HttpClient` để gọi API. Biến `apiBaseUrl` lấy địa chỉ API từ tham số dòng lệnh, nếu không có thì dùng mặc định `http://localhost:5000`. Cách làm này giúp máy khách không bị gắn cứng vào một môi trường duy nhất: khi API chạy ở

cổng khác hoặc trên máy khác, ta chỉ cần truyền lại địa chỉ lúc chạy máy khách.

Luồng kiểm thử gồm hai yêu cầu:

- `GET/api/assignments`: lấy danh sách bài tập.
- `POST/api/submissions`: gửi dữ liệu nộp bài.

Hai thao tác này minh họa rõ mô hình yêu cầu–phản hồi: máy khách gửi yêu cầu, máy chủ xử lý, rồi máy khách đọc phản hồi.

Hai lệnh `GetFromJsonAsync` và `PostAsJsonAsync` là điểm cần chú ý trong máy khách. Chúng vừa gửi yêu cầu HTTP, vừa chuyển đổi JSON sang đối tượng C# hoặc ngược lại. Sau khi gửi yêu cầu nộp bài, lệnh `EnsureSuccessStatusCode` buộc máy khách phải kiểm tra mã trạng thái HTTP trước khi đọc kết quả. Nếu API trả lỗi như `400BadRequest` hoặc máy chủ không phản hồi thành công, máy khách sẽ dừng bằng ngoại lệ thay vì tiếp tục xử lý như thể yêu cầu đã thành công. Đây là thói quen quan trọng khi viết máy khách trong hệ thống phân tán.

Các `record` ở cuối tệp máy khách có cùng hình dạng dữ liệu với các đối tượng mà API gửi và nhận. Điều này không có nghĩa máy khách gọi trực tiếp mã của máy chủ; hai bên chỉ thống nhất với nhau về cấu trúc JSON trao đổi qua HTTP. Nếu sau này API đổi tên trường hoặc đổi kiểu dữ liệu mà máy khách không cập nhật tương ứng, lỗi sẽ xuất hiện tại ranh giới giao tiếp giữa hai tiến trình.

Biên dịch máy khách:

```
dotnet build src/Submission.Client/Submission.Client.csproj
```

Để chạy thử toàn bộ luồng, mở hai cửa sổ terminal. Ở cửa sổ thứ nhất, chạy API:

```
dotnet run --project src/Submission.Api --urls http://localhost:5000
```

Ở cửa sổ thứ hai, chạy máy khách:

```
dotnet run --project src/Submission.Client -- http://localhost:5000
```

Kết quả mong đợi ở máy khách là danh sách bài tập và một mã lượt nộp mới, ví dụ:

```
Danh sach bai tap:
- 1: Bai tap 1
- 2: Bai tap 2
Da nop bai. Ma luot nop: 2f6c7b7d-0d7a-4d0a-93f3-0f0e0b5a9d33
Trang thai: received
```

Mã lượt nộp thay đổi sau mỗi lần chạy vì API tạo `Guid.NewGuid()` cho từng yêu cầu nộp bài. Nếu API chưa chạy hoặc chạy ở cổng khác, máy khách sẽ lỗi kết nối. Đây là lỗi có chủ ý về mặt sư phạm: nó cho thấy trong ứng dụng phân tán, một thành phần không thể giả định thành phần còn lại luôn sẵn sàng.

Khi kiểm thử máy khách–máy chủ, nên luôn trả lời ba câu hỏi: máy khách đang gọi đúng địa chỉ chưa, máy chủ có điểm cuối tương ứng chưa, và định dạng dữ liệu hai bên mong đợi có khớp nhau không. Phần lớn lỗi ban đầu trong hệ thống phân tán nhỏ xuất phát từ ba điểm này.

Sau bài thực hành này, EduSubmit đã có cấu trúc tối thiểu gồm API và máy khách. Ở các bài tiếp theo, API sẽ được mở rộng thành RESTful API rõ ràng hơn, máy khách sẽ xử lý nhiều kết quả hơn, và các thành phần mới như hub thời gian thực, dịch vụ xử lý nền, hàng đợi thông điệp sẽ lần lượt được ghép vào để hình thành một hệ thống phân tán hoàn chỉnh hơn.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- Ứng dụng phân tán gồm nhiều thành phần tính toán độc lập phối hợp với nhau qua mạng để thực hiện một mục tiêu chung và có thể được nhìn như một hệ thống thống nhất.
- Ranh giới quan trọng của ứng dụng phân tán là ranh giới giao tiếp giữa các tiến trình hoặc dịch vụ, không chỉ là việc các thành phần có chạy trên nhiều máy hay không.
- Kiến trúc máy khách–máy chủ mô tả vai trò của bên khởi tạo yêu cầu và bên tiếp nhận, xử lý, trả phản hồi trong một tương tác cụ thể.
- Message Exchange Pattern (MEP) giúp mô tả cách các thành phần gửi, nhận và phản hồi thông điệp trước khi lựa chọn công nghệ triển khai.
- Các MEP chính trong chương gồm yêu cầu–phản hồi, thông điệp một chiều, truyền thông qua hàng đợi, phát hành–đăng ký, truyền luồng và đẩy dữ liệu.
- Một hệ thống thực tế thường kết hợp nhiều MEP trong cùng một luồng nghiệp vụ thay vì chỉ sử dụng một kiểu giao tiếp duy nhất.
- Lát cắt đầu tiên của EduSubmit gồm hai tiến trình độc lập: `Submission.Api` đóng vai trò máy chủ và `Submission.Client` đóng vai trò máy khách gọi API qua HTTP.

Câu hỏi ôn tập

1. Theo cách tiếp cận của Tanenbaum và van Steen, điểm cốt lõi của một hệ thống phân tán là gì?
2. Vì sao một ứng dụng chạy trên nhiều máy chưa chắc đã là một ứng dụng phân tán được thiết kế tốt?
3. Trong một tương tác máy khách–máy chủ, vai trò máy khách và máy chủ được xác định theo tiêu chí nào?
4. Phân biệt lời gọi hàm trực tiếp trong ứng dụng đơn khối với lời gọi qua mạng trong ứng dụng phân tán.
5. Message Exchange Pattern giúp ích gì cho người thiết kế hệ thống trước khi chọn công nghệ cụ thể?
6. So sánh yêu cầu–phản hồi và thông điệp một chiều về cách bên gửi chờ hoặc không chờ kết quả xử lý.
7. Truyền thông qua hàng đợi phù hợp với loại tác vụ nào? Vì sao mô hình này giúp

API phản hồi nhanh hơn trong một số trường hợp?
8. Phát hành–đăng ký khác truyền thông qua hàng đợi ở điểm nào về số lượng bên nhận và ý nghĩa của thông điệp?
9. Truyền luồng hoặc đẩy dữ liệu phù hợp với những loại dữ liệu nào trong hệ thống phân tán?
10. Trong luồng nộp bài trực tuyến, hãy chỉ ra ít nhất hai MEP có thể xuất hiện khi hệ thống được mở rộng ở các chương sau.

Bài tập thực hành

1. Chạy lại ví dụ `Submission.Api` và `Submission.Client`. Ghi lại các bước cần thực hiện để API chạy ở `http://localhost:5000` và máy khách gọi được API.
2. Thêm một bài tập mẫu mới vào điểm cuối `GET/api/assignments`. Chạy máy khách và kiểm tra xem danh sách bài tập in ra màn hình có thay đổi đúng hay không.
3. Thử chạy máy khách khi API chưa chạy. Ghi lại lỗi quan sát được và giải thích lỗi này liên quan như thế nào đến tính không chắc chắn của giao tiếp qua mạng.
4. Thay đổi dữ liệu gửi lên điểm cuối `POST/api/submissions` để thiếu `StudentCode` hoặc `RepositoryUrl`. Quan sát phản hồi của API và giải thích vì sao máy chủ cần kiểm tra dữ liệu đầu vào.

Câu hỏi thảo luận

1. Với thao tác "sinh viên nộp bài", trường hợp nào nên trả kết quả ngay bằng yêu cầu–phản hồi, và trường hợp nào nên chuyển sang xử lý nền bằng hàng đợi?
2. Nếu hệ thống cần thông báo cho sinh viên khi bài đã được chấm xong, thăm dò định kỳ bằng yêu cầu–phản hồi và đẩy dữ liệu thời gian thực khác nhau như thế nào về trải nghiệm người dùng và độ phức tạp triển khai?
3. Khi bắt đầu thiết kế một hệ thống phân tán nhỏ, vì sao nên xác định luồng trao đổi thông điệp trước khi quyết định dùng REST, SignalR, gRPC hay bộ môi giới thông điệp?

CHƯƠNG 2

XÂY DỰNG RESTFUL API CHO TƯƠNG TÁC MÁY KHÁCH–MÁY CHỦ

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Giải thích cấu trúc thông điệp HTTP và vai trò của mô hình yêu cầu–phản hồi trong RESTful API.
- Thiết kế URI, phương thức HTTP và mã trạng thái phù hợp với tài nguyên nghiệp vụ.
- Sử dụng DTO để tách hợp đồng dữ liệu API khỏi mô hình xử lý bên trong.
- Xây dựng và chạy thử RESTful API cùng máy khách dòng lệnh có xử lý phản hồi thành công và phản hồi lỗi.

2.1 Cơ chế truyền thông đồng bộ với RESTful API

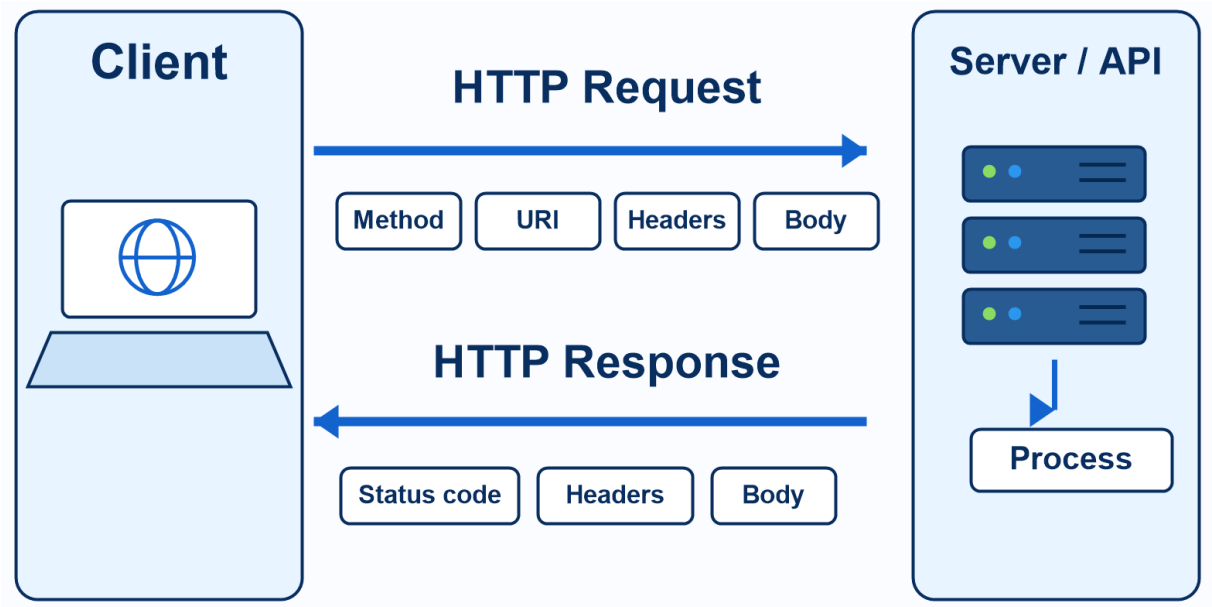
Trong chương trước, ta đã xem yêu cầu–phản hồi như một mô hình trao đổi thông điệp cơ bản: một thành phần gửi yêu cầu, thành phần còn lại xử lý và trả kết quả. RESTful API là một cách tổ chức giao tiếp máy khách–máy chủ rất phổ biến trên nền HTTP. Khi xây dựng ứng dụng phân tán, RESTful API thường được dùng ở ranh giới giữa máy khách và máy chủ, hoặc giữa các thành phần cần một giao diện đơn giản, dễ kiểm thử và dễ quan sát.

Điểm quan trọng là RESTful API không chỉ là việc tạo nhiều URL và viết mã xử lý từng URL. Một API tốt cần biểu diễn rõ tài nguyên nghiệp vụ, dùng đúng ý nghĩa của phương thức HTTP, trả về mã trạng thái phù hợp, và giữ cho mỗi yêu cầu có đủ thông tin để máy chủ xử lý mà không phụ thuộc vào trạng thái phiên ngầm. Cách thiết kế này giúp máy khách và máy chủ có thể tiến hóa tương đối độc lập, đồng thời làm cho hệ thống dễ gỡ lỗi hơn khi xảy ra lỗi qua mạng.

2.1.1 HTTP và yêu cầu–phản hồi

HTTP là giao thức tầng ứng dụng được dùng rộng rãi cho giao tiếp web. Trong một tương tác HTTP cơ bản, máy khách tạo yêu cầu gửi tới máy chủ; máy chủ đọc yêu cầu, xử lý logic cần thiết và trả phản hồi. Yêu cầu thường chứa phương thức, địa chỉ tài nguyên, phần đầu và có thể có phần thân. Phản hồi thường chứa mã trạng thái, phần đầu và có thể có phần thân biểu diễn kết quả.

Nói chính xác hơn, HTTP không chỉ là tên của một kiểu API, mà là một giao thức quy định cách máy khách và máy chủ trao đổi các *thông điệp* ở tầng ứng dụng. Trong cách nói hằng ngày, ta có thể gọi yêu cầu hoặc phản hồi là "gói tin HTTP", nhưng cần hiểu rằng đây là thông điệp HTTP ở tầng ứng dụng; khi đi qua mạng, thông điệp đó còn được chia nhỏ và đóng gói tiếp bởi TCP/IP ở các tầng thấp hơn. Với lập trình RESTful



Hình 2.1: Luồng yêu cầu–phản hồi HTTP giữa máy khách và máy chủ/API

API, thứ ta trực tiếp thiết kế và quan sát thường là thông điệp HTTP: phương thức, URI, phần đầu, phần thân, mã trạng thái và kiểu nội dung.

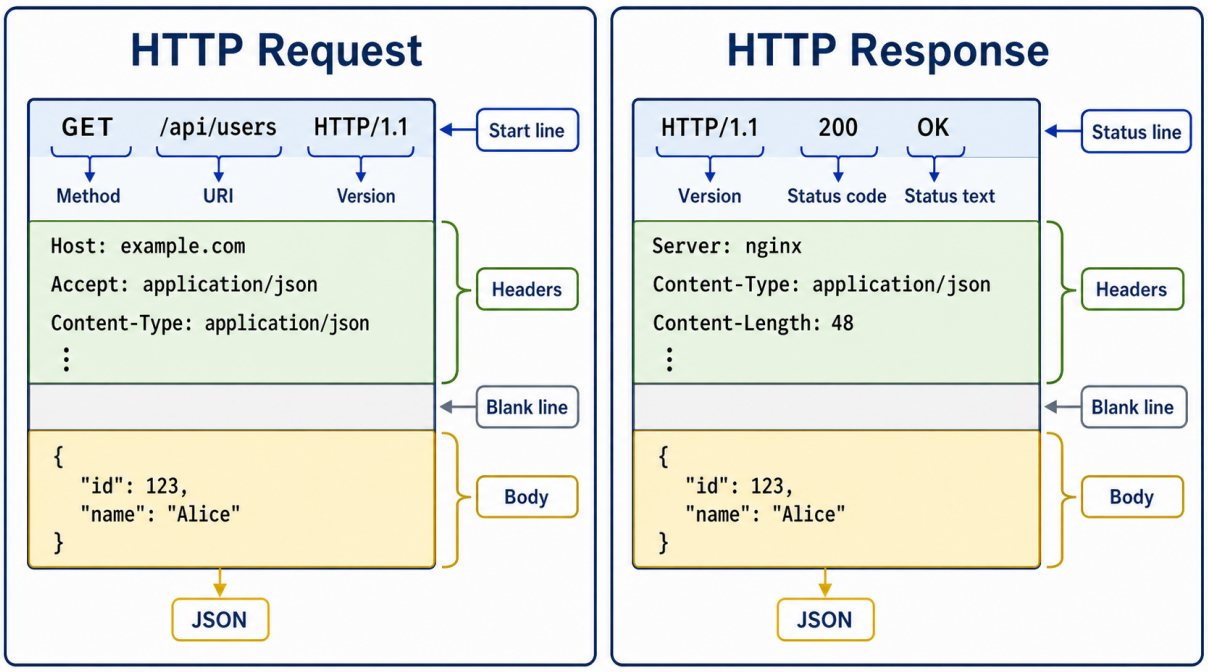
Cấu trúc tổng quát của một yêu cầu HTTP gồm ba phần chính. Phần đầu tiên là dòng yêu cầu, cho biết máy khách muốn làm gì với tài nguyên nào, ví dụ phương thức `GET` hoặc `POST` và đích yêu cầu như `/api/assignments`. Phần thứ hai là tập phần đầu, mang siêu dữ liệu cho yêu cầu: máy khách chấp nhận kiểu dữ liệu nào, phần thân đang có định dạng gì, token xác thực là gì, hoặc mã tương quan để truy vết. Phần thứ ba là phần thân, chỉ xuất hiện khi yêu cầu cần gửi dữ liệu lên máy chủ, chẳng hạn JSON tạo lượt nộp bài.

Ví dụ sau minh họa một yêu cầu tạo lượt nộp bài theo dạng gần với thông điệp HTTP/1.1 thô:

```
POST /api/assignments/1/submissions HTTP/1.1
Host: localhost:5000
Content-Type: application/json
Accept: application/json

{
  "studentCode": "SV001",
  "repositoryUrl": "https://example.com/sv001/rest-api"
}
```

Trong ví dụ này, dòng đầu tiên cho biết máy khách dùng phương thức `POST` để gửi dữ liệu tới tài nguyên lượt nộp của bài tập số 1. Phần đầu `Content-Type:application/json` nói rằng phần thân được mã hóa dưới dạng JSON. Phần đầu `Accept:application/json` nói rằng máy khách mong muốn nhận phản hồi cũng ở dạng JSON. Phần sau dòng trống là phần thân của yêu cầu, chứa dữ liệu nghiệp vụ mà máy chủ sẽ kiểm tra và xử lý.

Phản hồi HTTP cũng có cấu trúc tương tự, nhưng dòng đầu tiên là dòng trạng thái thay vì dòng yêu cầu. Dòng trạng thái cho biết phiên bản HTTP và kết quả xử lý ở dạng mã trạng thái. Phần đầu mô tả siêu dữ liệu của phản hồi, còn phần thân chứa dữ liệu trả



Hình 2.2: Cấu trúc thông điệp yêu cầu và phản hồi HTTP

về hoặc thông tin lỗi nếu có. Ví dụ, khi máy chủ tạo thành công một lượt nộp mới, phản hồi có thể có dạng:

```
HTTP/1.1 201 Created
Location: /api/submissions/8313f17c-abad-444b-b442-b80196aa5600
Content-Type: application/json

{
  "id": "8313f17c-abad-444b-b442-b80196aa5600",
  "assignmentId": 1,
  "studentCode": "SV001",
  "status": "received"
}
```

Ở đây, `201Created` cho biết máy chủ đã tạo tài nguyên mới. Phần đầu `Location` gợi ý URI để máy khách có thể đọc lại tài nguyên vừa tạo. Phần đầu `Content-Type` cho biết phần thân là JSON. Nhờ cấu trúc thống nhất này, máy khách không cần biết mã nội bộ của máy chủ; máy khách chỉ cần hiểu hợp đồng HTTP mà API công bố.

Từ góc nhìn hệ phân tán, đây là một lời gọi qua mạng. Nó khác với lời gọi hàm trong cùng tiến trình ở chỗ yêu cầu có thể chậm, thất bại, bị quá thời gian chờ hoặc trả về lỗi trung gian. Vì vậy, khi viết máy khách gọi RESTful API, lập trình viên không chỉ quan tâm tới dữ liệu trả về, mà còn phải xử lý các tình huống như không kết nối được máy chủ, máy chủ trả lỗi, yêu cầu bị lặp lại, hoặc phản hồi mất nhiều thời gian hơn dự kiến.

Mô hình yêu cầu–phản hồi có ưu điểm là rõ ràng và dễ theo dõi. Một yêu cầu thường tương ứng với một ý định cụ thể của máy khách: lấy danh sách bài tập, xem chi tiết một bài nộp, tạo một lượt nộp mới hoặc cập nhật trạng thái xử lý. Máy chủ trả về một phản hồi chứa kết quả hoặc thông tin lỗi. Nhờ đó, nhật ký, truy vết và công cụ kiểm

thử API như Swagger, Postman hoặc tệp yêu cầu HTTP đều dễ sử dụng.

Tuy nhiên, giao tiếp đồng bộ cũng tạo ra ràng buộc thời gian giữa máy khách và máy chủ. Nếu máy chủ xử lý lâu, máy khách phải chờ. Nếu tác vụ phía máy chủ thực chất là tác vụ nền, ví dụ chấm bài tự động hoặc xuất báo cáo lớn, RESTful API chỉ nên nhận yêu cầu ban đầu và trả về trạng thái như `202Accepted` hoặc một tài nguyên đại diện cho công việc. Phần xử lý dài nên được chuyển sang dịch vụ xử lý nền hoặc hàng đợi, thay vì giữ yêu cầu mở quá lâu.

RESTful API phù hợp nhất với các thao tác có ranh giới rõ: máy khách gửi một yêu cầu, máy chủ có thể kiểm tra, xử lý và trả một phản hồi trong thời gian hợp lý. Với tác vụ dài hoặc cần phát kết quả cho nhiều thành phần, REST thường chỉ là điểm khởi đầu của một luồng bất đồng bộ.

2.1.2 Nguyên tắc REST: tài nguyên, phương thức, không lưu trạng thái phiên

REST là một phong cách kiến trúc cho các hệ thống phân tán dựa trên tài nguyên và biểu diễn tài nguyên. Trong thiết kế RESTful API, ta không bắt đầu bằng câu hỏi "cần tạo hàm nào", mà bắt đầu bằng câu hỏi "hệ thống đang cho phép máy khách thao tác với tài nguyên nào". Với bài toán nộp bài trực tuyến, các tài nguyên có thể là `assignments`, `submissions`, `users` hoặc `grading-jobs`.

Mỗi tài nguyên được định danh bằng URI. Ví dụ, danh sách bài tập có thể là `/api/assignments`, một bài tập cụ thể có thể là `/api/assignments/42`, danh sách bài nộp của một bài tập có thể là `/api/assignments/42/submissions`. URI nên biểu diễn danh từ nghiệp vụ, không nên biến mọi thao tác thành động từ như `/createAssignment` hoặc `/doSubmit`. Động từ hành động đã được phương thức HTTP đảm nhận.

Trong thực tế, một URI đầy đủ có thể có dạng `https://api.example.edu/api/assignments/42/submissions?status=late\&page=1`. Phần `https` cho biết lược đồ, phần `api.example.edu` là máy chủ, phần `/api/assignments/42/submissions` là đường dẫn tới tài nguyên, còn phần sau dấu hỏi là chuỗi truy vấn. Khi thiết kế RESTful API, lập trình viên thường kiểm soát chủ yếu đường dẫn và chuỗi truy vấn: đường dẫn dùng để chỉ tài nguyên hoặc quan hệ tài nguyên; chuỗi truy vấn dùng để lọc, sắp xếp, tìm kiếm hoặc phân trang trên một tập tài nguyên.

Với EduSubmit, có thể đọc cấu trúc URI theo các lớp ý nghĩa sau:

- `/api`: tiền tố cho nhóm điểm cuối công khai của ứng dụng.
- `/api/assignments`: tập tài nguyên bài tập.
- `/api/assignments/42`: bài tập cụ thể có định danh `42`.
- `/api/assignments/42/submissions`: tập bài nộp thuộc bài tập `42`.
- `/api/submissions?studentCode=SV001`: tập bài nộp được lọc theo sinh viên.

Một quy tắc thực dụng là đặt dữ liệu định danh tài nguyên vào đường dẫn, còn điều kiện truy vấn trên một danh sách vào chuỗi truy vấn. Chẳng hạn, `/api/submissions/8313f17c` phù hợp khi máy khách muốn truy cập đúng một lượt nộp đã biết mã; `/api/submissions?studentCode=SV001` phù hợp khi máy khách muốn tìm các lượt nộp theo mã sinh viên. Cách phân chia này giúp URI dễ đọc, dễ ghi nhật ký và dễ kiểm thử, đồng thời tránh nhầm lẫn giữa "đối tượng cần thao tác" và "cách chọn một phần của danh sách".

Phương thức HTTP mô tả ý nghĩa thao tác máy khách muốn thực hiện. Bảng 2.1 tóm tắt cách dùng thường gặp trong API của giáo trình này.

Bảng 2.1: Cách dùng thường gặp của phương thức HTTP trong RESTful API

Phương thức	Ý nghĩa thường gặp	Ví dụ trong EduSubmit
GET	Lấy biểu diễn của tài nguyên	Lấy danh sách bài tập hoặc chi tiết một lượt nộp
POST	Tạo tài nguyên con hoặc yêu cầu máy chủ xử lý một lệnh mới	Tạo lượt nộp mới cho một bài tập
PUT	Thay thế toàn bộ trạng thái của tài nguyên đích	Cập nhật toàn bộ thông tin một bài tập
PATCH	Cập nhật một phần tài nguyên	Cập nhật hạn nộp hoặc trạng thái hiển thị
DELETE	Xóa hoặc hủy liên kết với tài nguyên	Xóa bản nháp bài nộp nếu nghiệp vụ cho phép

Hai thuộc tính cần đặc biệt chú ý là an toàn và lũy đẳng. Một phương thức an toàn, chẳng hạn `GET`, về mặt ý nghĩa không yêu cầu máy chủ thay đổi trạng thái nghiệp vụ. Một phương thức lũy đẳng, chẳng hạn `PUT` hoặc `DELETE`, có hiệu ứng dự kiến giống nhau dù máy khách gửi cùng yêu cầu nhiều lần. Thuộc tính lũy đẳng rất quan trọng trong hệ phân tán vì máy khách có thể phải gửi lại khi gặp quá thời gian chờ: nếu yêu cầu bị gửi lại, máy chủ không nên tạo ra kết quả sai lệch ngoài ý muốn.

Nguyên tắc không lưu trạng thái phiên yêu cầu mỗi yêu cầu phải chứa đủ thông tin để máy chủ hiểu và xử lý. Máy chủ vẫn có thể dùng cơ sở dữ liệu, bộ nhớ đệm hoặc bộ môi giới thông điệp; không lưu trạng thái phiên không có nghĩa là hệ thống không lưu dữ liệu. Điều cần tránh là để yêu cầu hiện tại phụ thuộc vào trạng thái phiên ngầm chỉ nằm trong bộ nhớ của một máy chủ cụ thể. Nếu thông tin xác thực, định danh người dùng, tham số lọc hoặc dữ liệu thao tác không được gửi rõ ràng, API sẽ khó mở rộng và khó chạy sau bộ cân bằng tải.

2.1.3 Thiết kế API theo use case

Thiết kế API nên bắt đầu từ ca sử dụng của người dùng và luồng nghiệp vụ, sau đó mới chuyển thành điểm cuối. Với EduSubmit, một ca sử dụng như "sinh viên xem danh sách bài tập" có thể dẫn tới điểm cuối `GET/api/assignments`. Ca sử dụng "sinh viên nộp bài" có thể dẫn tới `POST/api/assignments/\{assignmentId\}/submissions`. Ca sử dụng "giảng viên xem kết quả chấm" có thể dẫn tới `GET/api/submissions/\{submissionId\}/result`.

Một lỗi phổ biến là thiết kế API theo cấu trúc mã nội bộ thay vì theo ngôn ngữ nghiệp vụ. Nếu trong mã có lớp `AssignmentService`, không có nghĩa API phải có điểm cuối `/api/assignment-service/do-create`. API là hợp đồng bên ngoài giữa máy khách và máy chủ; nó nên ổn định theo nghiệp vụ hơn là theo cách ta tổ chức lớp, dịch vụ hoặc bảng trong cơ sở dữ liệu.

Khi chuyển ca sử dụng thành điểm cuối, cần trả lời một số câu hỏi:

- Tài nguyên chính của ca sử dụng là gì?
- Máy khách đang đọc, tạo, thay thế, cập nhật một phần hay xóa tài nguyên?

- Yêu cầu cần dữ liệu nào trong tuyến, chuỗi truy vấn, phần đầu và phần thân?
- Phản hồi nên trả mã trạng thái nào và phần thân nào?
- Nếu yêu cầu bị gửi lại, máy chủ có tạo ra tác dụng phụ không mong muốn không?
- Lỗi nghiệp vụ và lỗi hệ thống sẽ được biểu diễn ra sao?

Mã trạng thái cũng là một phần của thiết kế API. Một vài mã thường gặp gồm:

- `200OK`: yêu cầu thành công và có dữ liệu trả về.
- `201Created`: máy chủ đã tạo tài nguyên mới.
- `204NoContent`: thao tác thành công nhưng không cần phần thân.
- `400BadRequest`: yêu cầu sai định dạng hoặc thiếu dữ liệu.
- `401Unauthorized`: máy khách chưa xác thực.
- `403Forbidden`: máy khách đã xác thực nhưng không đủ quyền.
- `404NotFound`: tài nguyên không tồn tại hoặc không được phép tiết lộ.

Với hệ phân tán, API cũng cần được thiết kế để dễ quan sát. Mỗi yêu cầu nên có nhật ký đủ thông tin như điểm cuối, phương thức, mã trạng thái, thời gian xử lý và mã tương quan nếu có. Khi chương sau bổ sung hàng đợi, dịch vụ xử lý nền và thông báo thời gian thực, RESTful API sẽ đóng vai trò điểm tiếp nhận yêu cầu ban đầu, còn các bước xử lý dài sẽ được nối tiếp bằng các cơ chế giao tiếp khác.

2.1.4 DTO và mô hình dữ liệu

DTO, viết tắt của Data Transfer Object, là đối tượng dùng để truyền dữ liệu qua ranh giới API. Trong RESTful API, DTO thường xuất hiện dưới dạng thân yêu cầu hoặc thân phản hồi. Ví dụ, khi sinh viên nộp bài, máy khách có thể gửi một DTO tạo lượt nộp; máy chủ trả về một DTO phản hồi chứa mã lượt nộp, trạng thái ban đầu và thời điểm tiếp nhận.

Không nên để thực thể nội bộ hoặc mô hình cơ sở dữ liệu trở thành hợp đồng API một cách trực tiếp. Thực thể thường chứa nhiều chi tiết phục vụ lưu trữ, quan hệ dữ liệu, khóa ngoại hoặc trường kỹ thuật mà máy khách không cần biết. Nếu trả thực thể trực tiếp, API dễ bị phụ thuộc chặt vào lược đồ cơ sở dữ liệu; mỗi lần đổi cấu trúc lưu trữ có thể làm hỏng máy khách. DTO giúp tách hợp đồng bên ngoài khỏi mô hình nội bộ.

Một DTO tốt nên có phạm vi rõ ràng. DTO yêu cầu chỉ chứa dữ liệu máy khách được phép gửi. DTO phản hồi chỉ chứa dữ liệu máy khách cần nhận. Không nên dùng cùng một DTO cho mọi thao tác nếu các ca sử dụng khác nhau có yêu cầu khác nhau. Chẳng hạn, DTO tạo bài tập có thể cần tiêu đề, mô tả và hạn nộp; DTO trả danh sách bài tập có thể chỉ cần mã, tiêu đề và trạng thái; DTO chi tiết bài tập có thể chứa thêm nội dung mô tả đầy đủ.

Việc tách DTO cũng giúp kiểm soát bảo mật và kiểm tra hợp lệ. Máy chủ có thể kiểm tra DTO yêu cầu trước khi chuyển sang logic nghiệp vụ: trường nào bắt buộc, độ dài chuỗi có hợp lệ không, định dạng ngày giờ có đúng không, người dùng hiện tại có quyền thao tác với tài nguyên đó không. Sau khi xử lý, máy chủ ánh xạ mô hình miền sang DTO phản hồi để tránh lộ dữ liệu nhạy cảm như giá trị băm mật khẩu, cờ nội bộ hoặc thông tin vận hành.

Trong giáo trình này, hãy xem DTO như hợp đồng dữ liệu ở biên hệ thống. Thực thể và mô hình miền phục vụ logic bên trong; DTO phục vụ giao tiếp bên ngoài. Hai lớp này có thể giống nhau ở ví dụ nhỏ, nhưng nên được tách về mặt tư duy để hệ thống dễ phát triển khi nghiệp vụ lớn hơn.

2.1.5 Khai thác RESTful API

Trong phạm vi giáo trình này, "khai thác RESTful API" được hiểu là cách một lập trình viên hoặc một thành phần phần mềm sử dụng API đã được công bố: đọc tài liệu, gửi yêu cầu đúng hợp đồng, kiểm tra phản hồi, xử lý lỗi và tích hợp kết quả vào luồng nghiệp vụ. Đây không phải là "khai thác" theo nghĩa tấn công bảo mật. Một RESTful API chỉ thật sự có giá trị khi phía máy khách có thể sử dụng nó một cách rõ ràng, lặp lại được và ít phụ thuộc vào chi tiết cài đặt bên trong máy chủ.

Khi khai thác một API, điểm xuất phát không nên là mã nguồn máy chủ, mà là hợp đồng bên ngoài của API. Hợp đồng này thường gồm đường dẫn tài nguyên, phương thức HTTP, tham số đường dẫn, chuỗi truy vấn, phần đầu, DTO yêu cầu, DTO phản hồi và các mã trạng thái có thể nhận được. Ví dụ, với chức năng nộp bài, máy khách cần biết rằng thao tác tạo lượt nộp dùng `POST/api/assignments/\{assignmentId\}/submissions`, dữ liệu gửi lên nằm trong phần thân JSON, còn phản hồi thành công có mã `201Created` và có thể kèm đường dẫn đọc lại tài nguyên vừa tạo.

Một cách khai thác API có hệ thống là đi theo bốn bước. Thứ nhất, xác định tài nguyên và thao tác cần thực hiện: đọc danh sách bài tập, tạo lượt nộp, hay đọc trạng thái một lượt nộp. Thứ hai, xác định dữ liệu đầu vào: phần nào nằm trong đường dẫn, phần nào nằm trong chuỗi truy vấn, phần nào nằm trong phần thân. Thứ ba, gửi yêu cầu bằng một công cụ phù hợp như trình duyệt, Swagger UI, Postman, tệp `.http`, `curl`, hoặc mã máy khách. Thứ tư, đọc phản hồi theo cả phần thân lẫn mã trạng thái; không nên chỉ kiểm tra xem có dữ liệu JSON trả về hay không.

Ví dụ, một yêu cầu đọc danh sách bài tập thường có thể kiểm tra nhanh bằng trình duyệt hoặc công cụ dòng lệnh:

```
curl http://localhost:5000/api/assignments
```

Với thao tác tạo lượt nộp, máy khách phải gửi đúng phương thức, đúng đường dẫn, đúng kiểu nội dung và đúng DTO yêu cầu:

```
curl -i -X POST http://localhost:5000/api/assignments/1/submissions -H "Content-Type:
↪  application/json" -d "{\"studentCode\":\"SV001\",\"repositoryUrl\":\"https://examp
↪  le.com/sv001/rest-api\"}"
```

Tùy môi trường dòng lệnh, cách xuống dòng và biểu diễn chuỗi JSON có thể khác nhau. Điểm quan trọng về mặt khái niệm là yêu cầu trên có đủ ba phần: phương thức `POST`, URI định danh tài nguyên cha `/api/assignments/1/submissions`, và phần thân JSON chứa dữ liệu tạo lượt nộp. Tùy API, phản hồi thành công nên được đọc cùng mã trạng thái `201Created`; nếu API trả `400BadRequest`, máy khách cần đọc DTO lỗi thay vì cố xử lý như một lượt nộp hợp lệ.

Khai thác API tốt cũng đòi hỏi tư duy xử lý lỗi. Trong hệ phân tán, lỗi không phải là tình huống ngoại lệ hiếm gặp: máy chủ có thể tạm thời không sẵn sàng, yêu cầu có thể sai dữ liệu, người dùng có thể thiếu quyền, hoặc tài nguyên có thể không tồn tại. Vì vậy, máy khách cần phân biệt lỗi kết nối với lỗi nghiệp vụ, phân biệt `400BadRequest` với `404NotFound`, và không coi mọi phản hồi không thành công là cùng một loại lỗi. Cách xử lý này giúp giao diện người dùng hiển thị thông báo đúng hơn và giúp việc ghi nhật ký, truy vết dễ hơn khi hệ thống có nhiều thành phần.

> Khi khai thác RESTful API, hãy luôn đọc cả ba lớp thông tin: phương thức và URI cho biết thao tác đang gọi, mã trạng thái cho biết kết quả xử lý tổng quát, còn phần thân phản hồi cho biết dữ liệu hoặc chi tiết lỗi. Bỏ qua một trong ba lớp này thường dẫn tới máy khách xử lý sai tình huống.

Tóm lại, RESTful API là cách tổ chức giao tiếp đồng bộ trên nền HTTP theo hướng tài nguyên. Khi thiết kế API, ta cần kết hợp ba góc nhìn: phương thức HTTP và mã trạng thái cho đúng ngữ nghĩa, tài nguyên cho đúng ngôn ngữ nghiệp vụ, và DTO cho đúng hợp đồng dữ liệu giữa máy khách và máy chủ. Đây là nền tảng để phần thực hành tiếp theo xây dựng API cho bài tập và lượt nộp trong EduSubmit.

2.2 Kỹ thuật lập trình cơ bản với RESTful API

Phần 2.1 đã trình bày các nguyên tắc thiết kế RESTful API: tổ chức tài nguyên qua URI, dùng đúng phương thức HTTP, giữ yêu cầu không phụ thuộc trạng thái phiên, tách hợp đồng dữ liệu bằng DTO và thiết kế điểm cuối theo ca sử dụng. Phần này chuyển từ nguyên tắc sang kỹ thuật lập trình: cách hiện thực các nguyên tắc đó bằng ASP.NET Core Minimal API và cách gọi API từ máy khách bằng `HttpClient`.

Để tập trung vào kỹ thuật, phần này dùng một ví dụ tối giản: Catalog API – một API quản lý danh mục sản phẩm với ba điểm cuối. Mã nguồn đầy đủ nằm trong thư mục `content/examples/chapter02/section02/`. Sau khi nắm được các kỹ thuật cơ bản qua ví dụ này, sinh viên sẽ áp dụng chúng để xây dựng hệ thống EduSubmit hoàn chỉnh hơn trong phần 2.3.

2.2.1 Ánh xạ tài nguyên vào URI và phương thức HTTP

Trong Minimal API, mỗi điểm cuối được đăng ký bằng một lời gọi `app.MapGet`, `app.MapPost`, `app.MapPut` hoặc `app.MapDelete` trên đối tượng `WebApplication`. Tên phương thức thể hiện phương thức HTTP mà điểm cuối sẽ phản hồi. Tham số đầu tiên là mẫu URI, tham số thứ hai là hàm xử lý yêu cầu. Toàn bộ việc ánh xạ được thực hiện trực tiếp trong mã nguồn, không cần tệp cấu hình hay thuộc tính riêng.

Với Catalog API, tài nguyên `products` được ánh xạ thành ba điểm cuối:

- `GET/api/products`: lấy danh sách toàn bộ sản phẩm.
- `GET/api/products/\{id:int\}`: lấy chi tiết một sản phẩm theo mã.
- `POST/api/products`: tạo một sản phẩm mới từ dữ liệu trong phần thân yêu cầu.

Mẫu URI hỗ trợ ràng buộc kiểu qua hậu tố như `\{id:int\}`. Ràng buộc `:int` bảo đảm tham số tuyến chỉ khớp khi giá trị là số nguyên; nếu máy khách gọi `GET/api/products/abc`, yêu cầu sẽ bị từ chối ở tầng định tuyến trước khi vào hàm xử lý. Nhờ đó,

mã xử lý nhận được tham số đã được chuyển đúng kiểu `int`.

Nguyên tắc quan trọng khi ánh xạ là URI nên biểu diễn danh từ nghiệp vụ (tài nguyên), còn động từ hành động do phương thức HTTP đảm nhận. Ví dụ, thay vì tạo điểm cuối `/createProduct`, ta dùng `POST/api/products`; thay vì `/getProducts`, ta dùng `GET/api/products`. Cách đặt tên này giúp API nhất quán và dự đoán được: khi biết tài nguyên là `products`, máy khách có thể suy ra ngay cách đọc danh sách, đọc chi tiết và tạo mới.

Mã nguồn hoàn chỉnh của Catalog API được trình bày trong Listing 2.1. Các kỹ thuật được thảo luận trong các mục 2.2.2–2.2.4 đều có thể đối chiếu trực tiếp với mã nguồn này.

Mã nguồn 2.1: Catalog API – ví dụ tối giản minh họa ánh xạ URI, liên kết dữ liệu, DTO, kiểm tra hợp lệ và mã trạng thái

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var products = new List<ProductDto>
{
    new(1, "Giao trinh Lap trinh phan tan", "Giao trinh chinh thuc", 150000),
    new(2, "Tai lieu RESTful API", "Huong dan thiet ke API", 90000),
    new(3, "Bai tap thuc hanh .NET", "Bo bai tap kem loi giai", 120000)
};

app.MapGet("/", () => Results.Ok(new {
    service = "Catalog.Api",
    status = "running"
}));

app.MapGet("/api/products", () => Results.Ok(products));

app.MapGet("/api/products/{id:int}", (int id) => {
    var product = products.FirstOrDefault(p => p.Id == id);
    return product is null
        ? Results.NotFound(new ApiError("PRODUCT_NOT_FOUND", "Khong tim thay san
        ↪  pham."))
        : Results.Ok(product);
});

app.MapPost("/api/products", (CreateProductRequest request) => {
    if (string.IsNullOrWhiteSpace(request.Name) || request.Price <= 0) {
        return Results.BadRequest(new ApiError(
            "INVALID_PRODUCT",
            "Ten san pham khong duoc de trong va gia phai lon hon 0."));
    }

    var product = new ProductDto(
        Id: products.Max(p => p.Id) + 1,
        Name: request.Name.Trim(),
        Description: request.Description?.Trim() ?? "",
        Price: request.Price);

    products.Add(product);
    return Results.Created($"/api/products/{product.Id}", product);
});

app.Run();
```

```
record ProductDto(int Id, string Name, string Description, decimal Price);
record CreateProductRequest(string Name, string Description, decimal Price);
record ApiError(string Code, string Message);
```

Trong Listing 2.1, ba điểm cuối được đăng ký lần lượt sau dữ liệu mẫu. Điểm cuối `GET/` đóng vai trò kiểm tra trạng thái dịch vụ. Điểm cuối `GET/api/products` trả về toàn bộ danh sách sản phẩm. Điểm cuối `GET/api/products/\{id:int\}` tìm sản phẩm theo mã và trả về `404NotFound` nếu không tồn tại. Điểm cuối `POST/api/products` kiểm tra dữ liệu đầu vào, tạo sản phẩm mới với mã tự sinh và trả về `201Created` kèm đường dẫn đọc lại. Các record `ProductDto`, `CreateProductRequest` và `ApiError` được khai báo ở cuối tệp, đóng vai trò DTO cho đầu ra, đầu vào và thông điệp lỗi.

2.2.2 *Tiếp nhận dữ liệu từ route, query string và request body*

Minimal API tự động liên kết dữ liệu từ yêu cầu HTTP vào tham số của hàm xử lý. Cơ chế này hoạt động dựa trên vị trí của dữ liệu trong yêu cầu:

- **Tham số tuyến** (route parameter): được khai báo trong mẫu URI với cặp ngoặc nhọn, ví dụ `\{id:int\}`. Giá trị được trích xuất từ đường dẫn và tự động chuyển sang kiểu tương ứng. Trong Listing 2.1, tham số `intid` của điểm cuối `GET/api/products/\{id:int\}` nhận giá trị từ đoạn cuối cùng của đường dẫn.
- **Chuỗi truy vấn** (query string): khi hàm xử lý có tham số không xuất hiện trong mẫu URI, Minimal API tự động ánh xạ từ chuỗi truy vấn theo tên. Ví dụ, nếu thêm tham số `string?keyword` vào điểm cuối danh sách, máy khách có thể gọi `GET/api/products?keyword=.NET` để lọc sản phẩm.
- **Phần thân yêu cầu** (request body): khi tham số có kiểu phức hợp (class, record, struct), Minimal API tự động đọc phần thân và giải tuần tự từ JSON. Trong Listing 2.1, tham số `CreateProductRequestrequest` của điểm cuối `POST/api/products` được liên kết từ phần thân JSON. Điều này chỉ áp dụng cho các phương thức có phần thân như `POST`, `PUT`, `PATCH`.

Thứ tự ưu tiên khi liên kết: giá trị từ tham số tuyến được ưu tiên hơn chuỗi truy vấn nếu trùng tên. Ngoài ba nguồn trên, Minimal API còn hỗ trợ liên kết từ phần đầu HTTP, từ dịch vụ được đăng ký trong DI container, và từ các kiểu đặc biệt như `HttpContext`, `HttpRequest`, `HttpResponse`, `CancellationToken`. Những kiểu này được framework tự động phân giải, không cần khai báo trong mẫu URI.

2.2.3 *Sử dụng DTO và kiểm tra tính hợp lệ của dữ liệu đầu vào*

Trong Catalog API, DTO được khai báo dưới dạng record – một kiểu dữ liệu của C# phù hợp cho đối tượng chuyên chở dữ liệu với cú pháp ngắn gọn. Ba DTO phục vụ ba mục đích khác nhau:

- `ProductDto`: DTO phản hồi, biểu diễn một sản phẩm với các trường `Id`, `Name`, `Description` và `Price`. Đây là những gì máy khách cần biết; máy chủ không để lộ cấu trúc lưu trữ nội bộ.
- `CreateProductRequest`: DTO yêu cầu, chỉ chứa ba trường `Name`, `Description` và `Price` mà máy khách được phép gửi khi tạo sản phẩm. Trường `Id` do máy chủ tự sinh, không đến từ máy khách.

- `ApiError`: DTO lỗi, gồm hai trường `Code` (mã lỗi dạng chuỗi, để máy khách rẽ nhánh logic) và `Message` (thông điệp mô tả, để hiển thị cho người dùng hoặc ghi nhật ký). Cấu trúc này được dùng nhất quán cho mọi phản hồi lỗi.

Việc kiểm tra tính hợp lệ được thực hiện trực tiếp trong hàm xử lý trước khi thao tác với dữ liệu. Trong điểm cuối `POST/api/products`, máy chủ kiểm tra hai điều kiện: tên sản phẩm không được để trống và giá phải lớn hơn 0. Nếu không thỏa mãn, máy chủ trả về `400BadRequest` kèm `ApiError` mô tả rõ lý do. Cách kiểm tra thủ công này có ưu điểm là tường minh: sinh viên nhìn thấy chính xác điều kiện nào được kiểm tra, thứ tự kiểm tra ra sao, và thông điệp lỗi tương ứng là gì. Khi API phát triển phức tạp hơn, có thể chuyển sang dùng thư viện như FluentValidation, nhưng nguyên tắc "kiểm tra tại biên hệ thống trước khi dữ liệu vào logic nghiệp vụ" là không thay đổi.

Một điểm đáng chú ý là DTO yêu cầu và DTO phản hồi được tách biệt ngay cả khi chúng có nhiều trường giống nhau. `CreateProductRequest` không chứa `Id`, còn `ProductDto` thì có. Sự tách biệt này ngăn máy khách gửi mã sản phẩm tự chọn, đồng thời cho phép hai phía tiến hóa độc lập: nếu sau này thêm trường `Category` vào sản phẩm, DTO phản hồi có thể mở rộng mà không ảnh hưởng đến DTO tạo mới.

2.2.4 Trả về mã trạng thái và nội dung phản hồi phù hợp

Minimal API cung cấp lớp tĩnh `Results` với các phương thức tạo phản hồi HTTP tương ứng với từng mã trạng thái. Bảng 2.2 liệt kê các phương thức được dùng trong Catalog API.

Bảng 2.2: Ánh xạ giữa phương thức `Results` và mã trạng thái HTTP

Phương thức	Mã trạng thái	Ngữ cảnh sử dụng
`Results.Ok()`	200 OK	Trả về danh sách hoặc chi tiết sản phẩm
`Results.Created()`	201 Created	Tạo sản phẩm thành công, kèm URI của tài nguyên mới
`Results.BadRequest()`	400 Bad Request	Dữ liệu đầu vào không hợp lệ
`Results.NotFound()`	404 Not Found	Sản phẩm không tồn tại

Điểm cần chú ý là `Results.Created()` nhận hai tham số: URI của tài nguyên vừa tạo và đối tượng dữ liệu trả về. URI này được đặt trong phần đầu `Location` của phản hồi, cho phép máy khách đọc lại tài nguyên mà không cần tự đoán đường dẫn. Trong Listing 2.1, lời gọi `Results.Created(\$"/api/products/\{product.Id\}",product)` đồng thời trả về mã 201, cung cấp đường dẫn đọc lại sản phẩm, và trả về dữ liệu sản phẩm trong phần thân.

Một nguyên tắc quan trọng khác là sự nhất quán trong cấu trúc phản hồi. Với phản hồi thành công, phần thân chứa DTO tương ứng (`ProductDto` hoặc danh sách `ProductDto`). Với phản hồi lỗi, phần thân luôn chứa `ApiError` với mã lỗi và thông điệp. Sự nhất quán này giúp máy khách chỉ cần viết một hàm phân tích lỗi duy nhất, thay vì phải đoán cấu trúc phản hồi dựa trên mã trạng thái.

Trong thực tế, một API hoàn chỉnh còn cần thêm các mã trạng thái khác như `204NoContent` cho thao tác xóa thành công, `401Unauthorized` khi chưa xác thực, hoặc `409Conflict` khi có xung đột trạng thái. Tuy nhiên, trong phạm vi phần này, bốn mã

trạng thái trong Bảng 2.2 đã đủ để xây dựng một API có ý nghĩa nghiệp vụ và dạy được nguyên tắc trả mã trạng thái phù hợp theo tình huống.

2.2.5 *Gọi RESTful API và xử lý phản hồi bằng* `HttpClient`

Phía máy khách, .NET cung cấp lớp `HttpClient` để gửi yêu cầu HTTP và nhận phản hồi. Máy khách của Catalog API là một ứng dụng dòng lệnh, nhận địa chỉ API từ tham số dòng lệnh và thực hiện bốn bước minh họa toàn bộ vòng đời tương tác với API. Mã nguồn hoàn chỉnh được trình bày trong Listing 2.2.

Mã nguồn 2.2: Catalog Client – máy khách dòng lệnh gọi Catalog API và xử lý phản hồi

```
using System.Net;
using System.Net.Http.Json;

var apiBaseUrl = args.Length > 0 ? args[0] : "http://localhost:5100";

using var http = new HttpClient {
    BaseAddress = new Uri(apiBaseUrl)
};

// Buoc 1: lay danh sach san pham
var products = await http.GetFromJsonAsync<List<ProductDto>>("/api/products");
Console.WriteLine("Danh sach san pham:");
foreach (var p in products ?? []) {
    Console.WriteLine($"  [{p.Id}] {p.Name} - {p.Price:N0} VND");
}

// Buoc 2: tao san pham moi
var createRequest = new CreateProductRequest(
    "Nhap mon REST API",
    "Sach tham khao",
    120000);

var createResponse = await http.PostAsJsonAsync("/api/products", createRequest);

if (createResponse.StatusCode == HttpStatusCode.Created) {
    var created = await createResponse.Content
        .ReadFromJsonAsync<ProductDto>();
    Console.WriteLine($"\nDa tao san pham moi: [{created?.Id}] {created?.Name}");
} else {
    var error = await createResponse.Content
        .ReadFromJsonAsync<ApiError>();
    Console.WriteLine($"\nLoi: {error?.Code} - {error?.Message}");
    return;
}

// Buoc 3: doc lai mot san pham
var detail = await http.GetFromJsonAsync<ProductDto>("/api/products/1");
Console.WriteLine($"\nChi tiet san pham 1: {detail?.Name}");

// Buoc 4: gui yeu cau sai de quan sat loi 400
var invalidResponse = await http.PostAsJsonAsync("/api/products",
    new CreateProductRequest("", "", -1));

Console.WriteLine(
    $"\nThu gui du lieu sai: {(int)invalidResponse.StatusCode}
    ↪  {invalidResponse.StatusCode}");
```

```
var invalidError = await invalidResponse.Content
    .ReadFromJsonAsync<ApiError>();
Console.WriteLine($"{invalidError?.Code}: {invalidError?.Message}");

record ProductDto(int Id, string Name, string Description, decimal Price);
record CreateProductRequest(string Name, string Description, decimal Price);
record ApiError(string Code, string Message);
```

Listing 2.2 gồm bốn bước, mỗi bước minh họa một khía cạnh của việc gọi RESTful API từ máy khách.

Bước 1 – Đọc danh sách tài nguyên. Máy khách dùng `GetFromJsonAsync` với tham số kiểu `List<ProductDto>` để lấy danh sách sản phẩm từ `GET/api/products`. Phương thức này thực hiện đồng thời ba việc: gửi yêu cầu GET, đọc phần thân phản hồi và giải tuần tự JSON sang kiểu C#. Đây là cách gọi RESTful API ngắn gọn nhất khi ta biết chắc phản hồi thành công và có cấu trúc JSON khớp với kiểu đích.

Bước 2 – Tạo tài nguyên mới và xử lý kết quả. Máy khách tạo một đối tượng `CreateProductRequest` và gửi qua `PostAsJsonAsync` tới `POST/api/products`. Khác với bước 1, bước này *kiểm tra mã trạng thái* trước khi đọc dữ liệu: nếu mã trạng thái là `201Created`, máy khách đọc DTO sản phẩm vừa tạo từ phần thân; nếu không phải, máy khách đọc `ApiError` để biết lý do thất bại và thoát. Đây là mẫu xử lý cơ bản nhưng quan trọng: trong hệ phân tán, máy khách không được giả định mọi yêu cầu đều thành công.

Bước 3 – Đọc chi tiết một tài nguyên. Máy khách dùng `GetFromJsonAsync<ProductDto>` với đường dẫn `/api/products/1` để đọc chi tiết sản phẩm có mã 1. Bước này minh họa cách dùng tham số tuyến để truy cập một tài nguyên cụ thể.

Bước 4 – Quan sát phản hồi lỗi. Máy khách cố ý gửi một yêu cầu `POST` với tên rỗng và giá âm để kích hoạt phản hồi `400BadRequest` từ API. Sau đó, máy khách đọc `ApiError` từ phần thân phản hồi lỗi và in mã lỗi cùng thông điệp. Bước này cho thấy rằng việc xử lý lỗi không phải là tình huống ngoại lệ mà là một phần bình thường trong tương tác máy khách–máy chủ.

Một chi tiết đáng chú ý trong Listing 2.2 là `HttpClient` được khởi tạo với `BaseAddress` và dùng trong phạm vi `using` để bảo đảm giải phóng tài nguyên sau khi hoàn tất. Trong ứng dụng thực tế, `HttpClient` nên được đăng ký qua `IHttpClientFactory` để tránh vấn đề cạn kiệt socket, nhưng với ví dụ dòng lệnh chạy một lần, cách dùng trực tiếp là đủ để minh họa nguyên tắc.

Tóm lại, kỹ thuật lập trình RESTful API gồm hai phía: phía máy chủ chịu trách nhiệm ánh xạ tài nguyên vào URI và phương thức HTTP, tiếp nhận và kiểm tra dữ liệu đầu vào, rồi trả về mã trạng thái và nội dung phù hợp; phía máy khách chịu trách nhiệm gửi yêu cầu đúng hợp đồng, kiểm tra mã trạng thái và xử lý cả hai tình huống thành công lẫn thất bại. Phần tiếp theo sẽ áp dụng toàn bộ các kỹ thuật này để xây dựng hệ thống máy khách–máy chủ EduSubmit với API cho bài tập và lượt nộp.

2.3 [Ứng dụng] Xây dựng hệ thống máy khách–máy chủ cơ bản sử dụng RESTful API

Trong bài thực hành 1.3, EduSubmit đã có hai tiến trình độc lập: `Submission.Api` cung cấp một vài điểm cuối tối giản và `Submission.Client` gọi API qua HTTP. Phần đó

giúp ta nhìn thấy ranh giới máy khách–máy chủ và mô hình yêu cầu–phản hồi. Ở bài thực hành này, ta giữ lại cùng bối cảnh nộp bài, nhưng chỉnh API theo tư duy RESTful rõ hơn: tài nguyên có URI ổn định, phương thức diễn đạt đúng thao tác, DTO tách khỏi logic xử lý, và máy khách không chỉ đọc dữ liệu thành công mà còn phải xử lý phản hồi lỗi.

Mục tiêu của bài là xây dựng một lát cắt nhỏ nhưng chạy được:

- API cung cấp tài nguyên `assignments` và `submissions`.
- Máy khách lấy danh sách bài tập, tạo lượt nộp mới và đọc lại lượt nộp vừa tạo.
- Máy khách quan sát một yêu cầu sai để thấy cách API trả `400BadRequest`.

So với bài 1.3, điểm thay đổi quan trọng là điểm cuối nộp bài không còn đặt ở `POST/api/submissions` một cách chung chung. Vì lượt nộp là tài nguyên con gắn với một bài tập, API mới đặt tuyến theo quan hệ cha–con:

```
POST /api/assignments/{assignmentId}/submissions
```

Cách đặt tuyến này làm quan hệ nghiệp vụ giữa bài tập và lượt nộp rõ hơn.

2.3.1 Xây dựng API: assignment, submission

Ví dụ nguồn cho bài này nằm trong thư mục `content/examples/chapter02/section02`. Đây là một dự án mới để sinh viên có thể so sánh với bài 1.3 mà không làm thay đổi ví dụ cũ. Cấu trúc vẫn gồm hai dự án quen thuộc:

```
examples/chapter02/section03/
  RestfulClientServer.slnx
  src/
    Submission.Api/
    Submission.Client/
```

API được viết bằng ASP.NET Core Minimal API. Tệp `Program.cs` dưới đây chứa toàn bộ lát cắt máy chủ của bài thực hành:

Hình 2.3: Các bước chính khi viết mã RESTful API trong Listing 2.1

Mã nguồn 2.3: RESTful API cho bài tập và lượt nộp

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var assignments = new List<AssignmentDto>
{
    new(1, "Bai tap REST API", "Thiet ke endpoint assignment va submission", true),
    new(2, "Bai tap Client", "Viet console client goi API va xu ly ket qua", true)
};

var submissions = new List<SubmissionDto>();

app.MapGet("/", () => Results.Ok(new {
    service = "Submission.Api",
    status = "running"
}));

app.MapGet("/api/assignments", () => Results.Ok(assignments));

app.MapGet("/api/assignments/{assignmentId:int}", (int assignmentId) => {
    var assignment = assignments.FirstOrDefault(item => item.Id == assignmentId);
    return assignment is null
        ? Results.NotFound(new ApiError("ASSIGNMENT_NOT_FOUND", "Khong tim thay bai
        ↪  tap."))
        : Results.Ok(assignment);
});

app.MapPost("/api/assignments/{assignmentId:int}/submissions",
    (int assignmentId, CreateSubmissionRequest request) => {
        var assignment = assignments.FirstOrDefault(item => item.Id == assignmentId);
        if (assignment is null) {
            return Results.NotFound(new ApiError("ASSIGNMENT_NOT_FOUND", "Khong tim
            ↪  thay bai tap."));
        }

        if (!assignment.IsOpen) {
            return Results.BadRequest(new ApiError("ASSIGNMENT_CLOSED", "Bai tap da
            ↪  dong nop bai."));
        }

        if (string.IsNullOrWhiteSpace(request.StudentCode) ||
            string.IsNullOrWhiteSpace(request.RepositoryUrl)) {
            return Results.BadRequest(new ApiError("INVALID_SUBMISSION", "Thieu ma sinh
            ↪  vien hoac repository."));
        }

        var submission = new SubmissionDto(
            Id: Guid.NewGuid(),
            AssignmentId: assignmentId,
            StudentCode: request.StudentCode.Trim(),
            RepositoryUrl: request.RepositoryUrl.Trim(),
            Status: "received",
            ReceivedAt: DateTimeOffset.UtcNow);

        submissions.Add(submission);

        return Results.Created($"/api/submissions/{submission.Id}", submission);
    });
```

```
app.MapGet("/api/submissions/{submissionId:guid}", (Guid submissionId) => {
    var submission = submissions.FirstOrDefault(item => item.Id == submissionId);
    return submission is null
        ? Results.NotFound(new ApiError("SUBMISSION_NOT_FOUND", "Khong tim thay luot
        ↪  nop."))
        : Results.Ok(submission);
});

app.Run();

record AssignmentDto(
    int Id,
    string Title,
    string Description,
    bool IsOpen);

record CreateSubmissionRequest(
    string StudentCode,
    string RepositoryUrl);

record SubmissionDto(
    Guid Id,
    int AssignmentId,
    string StudentCode,
    string RepositoryUrl,
    string Status,
    DateTimeOffset ReceivedAt);

record ApiError(
    string Code,
    string Message);
```

Đoạn mã trên mở rộng trực tiếp từ ý tưởng của bài 1.3. API có hai điểm cuối đọc bài tập: một điểm cuối trả danh sách và một điểm cuối trả chi tiết theo mã bài tập. Đây là các thao tác đọc tài nguyên nên dùng `GET`. Nếu không tìm thấy bài tập, API trả `404NotFound` kèm một DTO lỗi đơn giản là `ApiError`.

Điểm cuối quan trọng nhất là điểm cuối tạo lượt nộp. Tuyến của nó cho thấy máy khách đang tạo một tài nguyên `submission` mới bên dưới một `assignment` cụ thể. Phần thân của yêu cầu chỉ chứa dữ liệu máy khách được phép gửi: `StudentCode` và `RepositoryUrl`. Phần `assignmentId` nằm trên tuyến vì nó định danh tài nguyên cha. Cách tách dữ liệu như vậy giúp điểm cuối rõ nghĩa hơn so với việc đưa mọi thứ vào phần thân.

Khi yêu cầu hợp lệ, máy chủ tạo DTO phản hồi cho lượt nộp, gán trạng thái ban đầu là `received`, lưu tạm trong bộ nhớ và trả về `201Created`. Trong lời gọi `Results.Created`, API cũng cung cấp đường dẫn đọc lại lượt nộp vừa tạo. Đây là thói quen tốt khi tạo tài nguyên mới: phản hồi không chỉ nói rằng thao tác thành công, mà còn cho máy khách biết nơi có thể đọc lại tài nguyên vừa tạo.

Trong ví dụ này, dữ liệu vẫn được lưu trong `List<T>` để bài thực hành tập trung vào thiết kế API. Điều này có nghĩa dữ liệu sẽ mất khi API dừng. Ở các bài sau, phần lưu trữ, dịch vụ xử lý nền và thông báo sẽ được bổ sung dần. Với phạm vi bài 2, điều quan trọng là nhìn thấy hợp đồng HTTP và DTO ở biên hệ thống.

Biên dịch solution để kiểm tra cả API và máy khách:

```
dotnet build examples/chapter02/section03/RestfulClientServer.slnx
```

Sau khi biên dịch thành công, chạy API trên cổng cố định:

```
dotnet run --project examples/chapter02/section03/src/Submission.Api --urls
↪  http://localhost:5000
```

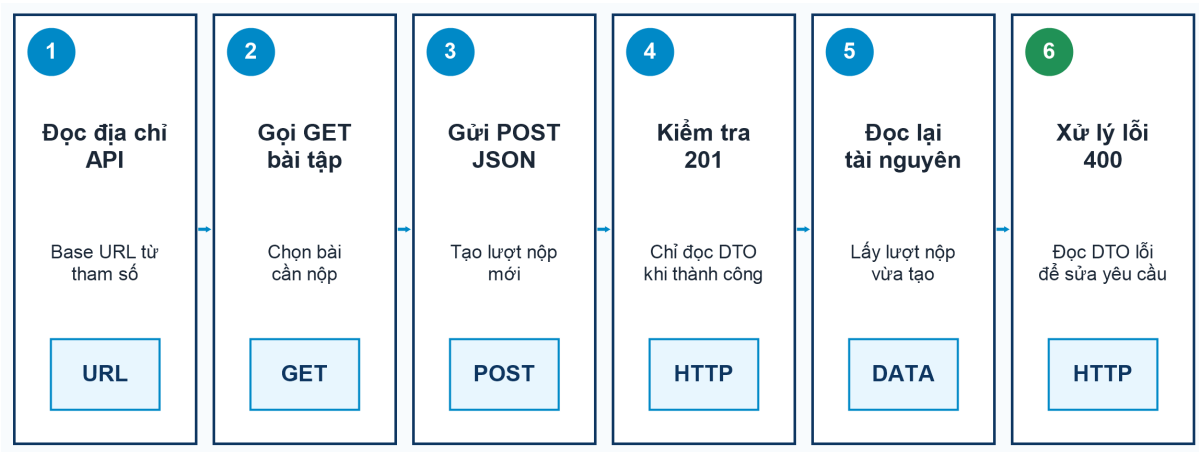
Nếu API chạy đúng, điểm cuối `http://localhost:5000/` trả về một JSON nhỏ cho biết dịch vụ đang ở trạng thái `running`. Điểm cuối này không phải nghiệp vụ chính, nhưng hữu ích khi kiểm tra nhanh máy chủ có sẵn sàng hay chưa.

Bài 1.3 tập trung vào việc "có hai tiến trình gọi nhau được". Bài 2.3 tập trung vào việc "hai tiến trình gọi nhau bằng một API có hợp đồng rõ". Khi thiết kế RESTful API, tuyến, phương thức, mã trạng thái và DTO đều là một phần của hợp đồng đó.

2.3.2 *Máy khách gọi API và xử lý kết quả*

Máy khách của bài 1.3 đã biết gọi `GET` và `POST`. Trong bài này, máy khách cần làm thêm hai việc: kiểm tra mã trạng thái trước khi đọc kết quả và xử lý phản hồi lỗi theo DTO mà API trả về. Đây là kỹ năng quan trọng trong hệ thống phân tán, vì lỗi qua mạng hoặc lỗi nghiệp vụ là tình huống bình thường chứ không phải ngoại lệ hiếm gặp.

Tệp `Program.cs` của máy khách như sau:



Hình 2.4: Các bước chính khi viết mã máy khách REST trong Listing 2.2

Mã nguồn 2.4: Máy khách dòng lệnh gọi RESTful API và xử lý lỗi

```
using System.Net;
using System.Net.Http.Json;

var apiBaseUrl = args.Length > 0 ? args[0] : "http://localhost:5000";

using var http = new HttpClient {
    BaseAddress = new Uri(apiBaseUrl)
};
```

```
var assignments = await http.GetFromJsonAsync<List<AssignmentDto>>("/api/assignments");
Console.WriteLine("Danh sach bai tap dang mo:");

foreach (var assignment in assignments ?? []) {
    Console.WriteLine($"- {assignment.Id}: {assignment.Title}");
}

var selectedAssignmentId = assignments?.FirstOrDefault()?.Id ?? 1;

var createRequest = new CreateSubmissionRequest(
    StudentCode: "SV001",
    RepositoryUrl: "https://example.com/sv001/rest-api");

var createResponse = await http.PostAsJsonAsync(
    $"/api/assignments/{selectedAssignmentId}/submissions",
    createRequest);

if (createResponse.StatusCode != HttpStatusCode.Created) {
    await PrintApiErrorAsync(createResponse);
    return;
}

var createdSubmission = await
↪  createResponse.Content.ReadFromJsonAsync<SubmissionDto>();
Console.WriteLine();
Console.WriteLine($"Da tao luot nop: {createdSubmission?.Id}");
Console.WriteLine($"Trang thai ban dau: {createdSubmission?.Status}");

var detail = await
↪  http.GetFromJsonAsync<SubmissionDto>($"/api/submissions/{createdSubmission?.Id}");
Console.WriteLine($"Doc lai luot nop: {detail?.StudentCode} -
↪  {detail?.RepositoryUrl}");

var invalidResponse = await http.PostAsJsonAsync(
    $"/api/assignments/{selectedAssignmentId}/submissions",
    new CreateSubmissionRequest(StudentCode: "", RepositoryUrl: ""));

Console.WriteLine();
Console.WriteLine($"Thu gui request loi: {(int)invalidResponse.StatusCode}
↪  {invalidResponse.StatusCode}");
await PrintApiErrorAsync(invalidResponse);

static async Task PrintApiErrorAsync(HttpResponseMessage response) {
    if (response.IsSuccessStatusCode) {
        return;
    }

    var error = await response.Content.ReadFromJsonAsync<ApiError>();
    Console.WriteLine($"{error?.Code}: {error?.Message}");
}

record AssignmentDto(
    int Id,
    string Title,
    string Description,
    bool IsOpen);

record CreateSubmissionRequest(
```

```
    string StudentCode,
    string RepositoryUrl);

record SubmissionDto(
    Guid Id,
    int AssignmentId,
    string StudentCode,
    string RepositoryUrl,
    string Status,
    DateTimeOffset ReceivedAt);

record ApiError(
    string Code,
    string Message);
```

Luồng máy khách gồm bốn bước. Đầu tiên, máy khách gọi điểm cuối danh sách bài tập để lấy các bài đang mở. Tiếp theo, máy khách chọn bài tập đầu tiên và gửi DTO tạo lượt nộp tới điểm cuối tương ứng. Nếu API trả `201Created`, máy khách đọc DTO phản hồi từ thân phản hồi và in mã lượt nộp. Sau đó, máy khách gọi điểm cuối chi tiết lượt nộp để đọc lại tài nguyên vừa tạo.

Điểm khác với bài 1.3 nằm ở đoạn kiểm tra `createResponse.StatusCode`. Máy khách không giả định mọi yêu cầu đều thành công. Nếu phản hồi không phải `201Created`, hàm `PrintApiErrorAsync` sẽ đọc `ApiError` và in mã lỗi cùng thông điệp. Ở cuối chương trình, máy khách cố ý gửi một yêu cầu sai với `StudentCode` và `RepositoryUrl` rỗng để quan sát cách API trả `400BadRequest`.

Để chạy toàn bộ luồng, mở hai cửa sổ dòng lệnh. Cửa sổ thứ nhất chạy API:

```
dotnet run --project examples/chapter02/section03/src/Submission.Api --urls
↪  http://localhost:5000
```

Cửa sổ thứ hai chạy máy khách:

```
dotnet run --project examples/chapter02/section03/src/Submission.Client --
↪  http://localhost:5000
```

Kết quả mong đợi ở máy khách có dạng:

```
Danh sach bai tap dang mo:
- 1: Bai tap REST API
- 2: Bai tap Client

Da tao luot nop: 8313f17c-abad-444b-b442-b80196aa5600
Trang thai ban dau: received
Doc lai luot nop: SV001 - https://example.com/sv001/rest-api

Thu gui yeu cau loi: 400 BadRequest
INVALID_SUBMISSION: Thieu ma sinh vien hoac kho ma nguon.
```

Mã lượt nộp sẽ khác nhau sau mỗi lần chạy vì máy chủ sinh `Guid.NewGuid()` cho từng lượt nộp. Nếu API chưa chạy, máy khách sẽ lỗi kết nối. Nếu tuyến sai, máy khách

có thể nhận `404NotFound`. Nếu phần thân sai, API trả `400BadRequest`. Các tình huống này đều có giá trị học tập: chúng buộc máy khách phải nhìn phản hồi HTTP như một hợp đồng đầy đủ, không chỉ như một cách lấy đối tượng C# từ máy chủ.

Sau bài thực hành này, EduSubmit đã có một RESTful API cơ bản hơn bài 1.3. API vẫn nhỏ, nhưng đã có tài nguyên, DTO, mã trạng thái và luồng máy khách xử lý kết quả. Đây là nền để các chương sau thêm thông báo thời gian thực bằng SignalR, xử lý nền bằng dịch vụ xử lý nền và hàng đợi, rồi tiến tới luồng hệ thống phân tán hoàn chỉnh hơn.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- RESTful API là một cách tổ chức giao tiếp đồng bộ máy khách–máy chủ trên nền HTTP, thường dùng cho các thao tác có ranh giới yêu cầu–phản hồi rõ ràng.
- Một thông điệp HTTP cần được đọc như một hợp đồng gồm phương thức, URI, phần đầu, phần thân, mã trạng thái và kiểu nội dung.
- Trong thiết kế REST, URI nên biểu diễn tài nguyên nghiệp vụ; phương thức HTTP diễn đạt thao tác mà máy khách muốn thực hiện trên tài nguyên đó.
- Đường dẫn URI thường dùng để định danh tài nguyên, còn chuỗi truy vấn phù hợp cho lọc, tìm kiếm, sắp xếp hoặc phân trang trên một tập tài nguyên.
- Nguyên tắc không lưu trạng thái phiên yêu cầu mỗi yêu cầu phải chứa đủ thông tin để máy chủ xử lý, thay vì phụ thuộc vào trạng thái phiên ngầm trong bộ nhớ của một máy chủ cụ thể.
- Mã trạng thái HTTP là một phần của hợp đồng API; máy khách cần phân biệt phản hồi thành công, lỗi dữ liệu, lỗi không tìm thấy tài nguyên và các tình huống lỗi khác.
- DTO là đối tượng truyền dữ liệu qua ranh giới API, giúp tách hợp đồng bên ngoài khỏi mô hình nội bộ hoặc mô hình lưu trữ của máy chủ.
- Bài thực hành của chương xây dựng API cho tài nguyên bài tập và lượt nộp, đồng thời viết máy khách gọi API và xử lý phản hồi lỗi.

Câu hỏi ôn tập

1. HTTP đóng vai trò gì trong giao tiếp giữa máy khách và máy chủ khi xây dựng RESTful API?
2. Một yêu cầu HTTP thường gồm những phần chính nào? Hãy nêu vai trò của từng phần.
3. Vì sao không nên chỉ đọc phần thân JSON mà bỏ qua mã trạng thái HTTP trong phản hồi?
4. Phân biệt ý nghĩa của `GET` và `POST` trong thiết kế API theo tài nguyên.
5. Trong hệ thống nộp bài, vì sao tuyến `POST/api/assignments/\{assignmentId\}/submissions` thể hiện quan hệ nghiệp vụ rõ hơn `POST/api/submissions`?
6. Khi nào nên đặt dữ liệu vào đường dẫn URI, và khi nào nên đặt vào chuỗi truy vấn?
7. Nguyên tắc không lưu trạng thái phiên khác gì với việc "máy chủ không được lưu dữ liệu"?
8. DTO giúp giảm phụ thuộc giữa máy khách và máy chủ như thế nào?
9. Trong ví dụ thực hành, API trả `400BadRequest` ở những tình huống nào? Máy

khách nên xử lý phản hồi này ra sao?

10. Vì sao thao tác tạo lượt nộp thành công nên trả `201Created` và cung cấp đường dẫn đọc lại tài nguyên vừa tạo?

Bài tập thực hành

1. Chạy solution `content/examples/chapter02/section02/RestfulClientServer.slnx`. Ghi lại lệnh biên dịch, lệnh chạy API và lệnh chạy máy khách; kiểm tra máy khách in được danh sách bài tập, mã lượt nộp và phản hồi lỗi `400BadRequest`.
2. Thêm một bài tập mẫu mới vào danh sách `assignments` trong `Submission.Api`. Chạy lại API và máy khách, sau đó giải thích vì sao máy khách không cần đổi mã để đọc được bài tập mới.
3. Bổ sung một điểm cuối `GET /api/assignments/\{assignmentId\}` vào máy khách để đọc chi tiết bài tập đã chọn trước khi tạo lượt nộp. Đầu ra cần in tiêu đề và mô tả của bài tập.
4. Thử đổi `selectedAssignmentId` trong máy khách sang một mã bài tập không tồn tại. Quan sát phản hồi `404NotFound` và điều chỉnh máy khách để in DTO lỗi thay vì tiếp tục xử lý như yêu cầu thành công.

Câu hỏi thảo luận

1. Với hệ thống nộp bài, việc đặt lượt nộp như tài nguyên con của bài tập có ưu điểm và hạn chế gì so với việc đặt tất cả lượt nộp dưới `/api/submissions`?
2. Nếu DTO phản hồi trả về toàn bộ mô hình nội bộ của máy chủ, những rủi ro nào có thể xảy ra khi hệ thống phát triển lớn hơn?
3. Khi khai thác một RESTful API do nhóm khác cung cấp, lập trình viên máy khách nên kiểm tra những phần nào của hợp đồng API trước khi viết mã tích hợp?

CHƯƠNG 3
GIAO TIẾP THỜI GIAN THỰC VỚI SIGNALR

Mục tiêu chương
Sau khi học xong chương này, người học có thể:

- Phân biệt các mô hình trao đổi thông điệp trong giao tiếp thời gian thực: yêu cầu–phản hồi, đẩy dữ liệu và truyền luồng.
- Giải thích mối quan hệ giữa WebSocket và SignalR, cũng như vai trò của hub, nhóm và kết nối lại tự động trong SignalR.
- Thiết lập SignalR hub trên máy chủ và đăng ký nhận thông báo thời gian thực từ phía máy khách.
- Kết hợp RESTful API và SignalR trong cùng một luồng nghiệp vụ, trong đó REST xử lý thao tác tài nguyên còn SignalR đẩy cập nhật trạng thái.

3.1 Cơ chế và mô hình truyền thông thời gian thực

Trong hai chương đầu, máy khách thường gửi yêu cầu và chờ máy chủ trả phản hồi. Cách giao tiếp này phù hợp với các thao tác có điểm bắt đầu và kết thúc rõ ràng, chẳng hạn lấy danh sách bài tập hoặc tạo một lượt nộp. Tuy nhiên, không phải mọi thông tin trong ứng dụng phân tán đều xuất hiện theo nhịp "hỏi rồi trả lời". Một số thông tin thay đổi theo thời gian: tiến độ xử lý, trạng thái chấm bài, thông báo mới, số lượng người đang trực tuyến hoặc dữ liệu nhật ký đang được ghi liên tục.

Truyền thông thời gian thực giải quyết nhóm bài toán đó bằng cách giảm độ trễ giữa thời điểm sự kiện xảy ra trên máy chủ và thời điểm máy khách biết về sự kiện. Trong bối cảnh web, "thời gian thực" thường nên hiểu là gần thời gian thực: cập nhật được gửi đủ nhanh để người dùng không phải chủ động tải lại hoặc kiểm tra liên tục. Thay vì bắt máy khách liên tục gửi yêu cầu để hỏi "đã xong chưa", máy chủ có thể chủ động gửi cập nhật khi trạng thái thay đổi. Trong EduSubmit, mô hình này đặc biệt hữu ích khi sinh viên nộp bài qua REST API, dịch vụ xử lý nền xử lý ở phía sau, rồi máy khách cần nhận thông báo `processing`, `graded` hoặc `failed`.

3.1.1 MEP: truyền luồng, đẩy dữ liệu và yêu cầu–phản hồi

Yêu cầu–phản hồi là mô hình một yêu cầu đi kèm một phản hồi. Máy khách thường chủ động khởi tạo tương tác và đợi kết quả trước khi tiếp tục luồng xử lý chính. Mô hình này dễ hiểu, dễ ghi nhật ký và dễ kiểm thử, nhưng không tối ưu khi dữ liệu thay đổi nhiều lần sau yêu cầu ban đầu. Nếu máy khách muốn biết tiến độ chấm bài, cách đơn giản nhất là thăm dò định kỳ (polling): cứ vài giây gọi API để hỏi trạng thái. Polling dễ triển khai nhưng tạo nhiều yêu cầu thừa, nhất là khi trạng thái chưa thay đổi.

Đẩy dữ liệu đảo chiều chủ động của tương tác. Sau khi máy khách thiết lập một kênh kết nối phù hợp, máy chủ có thể gửi thông điệp về máy khách khi có sự kiện mới.

Máy khách không cần hỏi lặp lại, còn máy chủ có thể phát thông báo đúng lúc trạng thái thay đổi. Đẩy dữ liệu phù hợp với thông báo, trạng thái xử lý, cảnh báo, tin nhắn hoặc các cập nhật có tần suất không đều.

Truyền luồng là trường hợp dữ liệu được gửi thành một chuỗi nhiều thông điệp theo thời gian. Một luồng có thể đi từ máy chủ tới máy khách, ví dụ máy chủ gửi tiến độ xử lý từng bước; từ máy khách tới máy chủ, ví dụ máy khách gửi dữ liệu đo theo thời gian; hoặc hai chiều nếu cả hai bên cùng phát dữ liệu. Điểm khác với yêu cầu–phản hồi là một tương tác logic không nhất thiết kết thúc bằng một phản hồi duy nhất. Bảng 3.1 tóm tắt đặc điểm và tình huống sử dụng của ba mô hình này.

Bảng 3.1: So sánh yêu cầu–phản hồi, đẩy dữ liệu và truyền luồng

Mô hình	Đặc điểm	Tình huống phù hợp
Yêu cầu–phản hồi	Máy khách gửi yêu cầu và nhận một phản hồi	Lấy dữ liệu, gửi thao tác ngắn, xác nhận nghiệp vụ
Đẩy dữ liệu	Máy chủ gửi thông điệp khi có sự kiện	Thông báo, cập nhật trạng thái, cảnh báo
Truyền luồng	Nhiều thông điệp được gửi theo thời gian	Tiến độ xử lý, nhật ký, dữ liệu cảm biến, luồng dữ liệu liên tục

Trong thiết kế hệ thống, ba mô hình này thường kết hợp với nhau. Ví dụ, máy khách dùng REST API để tạo lượt nộp. API trả về mã lượt nộp và trạng thái ban đầu. Sau đó dịch vụ xử lý nền xử lý bài nộp ở nền, còn máy chủ dùng kênh thời gian thực để đẩy trạng thái mới về máy khách. Nhờ vậy, yêu cầu ban đầu vẫn ngắn gọn, trong khi trải nghiệm người dùng vẫn gần với thời gian thực. Dù vậy, kênh thời gian thực không nên là nguồn sự thật duy nhất: sau khi mất kết nối hoặc mở lại trang, máy khách vẫn cần gọi API để lấy trạng thái hiện tại.

3.1.2 WebSocket và SignalR

WebSocket là một giao thức cho phép máy khách và máy chủ duy trì một kết nối hai chiều sau bước bắt tay ban đầu. Khác với yêu cầu–phản hồi HTTP thông thường, sau khi kết nối WebSocket được thiết lập, cả hai phía có thể gửi thông điệp qua lại trên cùng một kênh lâu dài. Điều này làm WebSocket phù hợp với trò chuyện, bảng điều khiển thời gian thực, thông báo trạng thái và các ứng dụng cần độ trễ thấp.

Tuy nhiên, dùng WebSocket trực tiếp nghĩa là lập trình viên phải tự quan tâm đến nhiều chi tiết: quản lý kết nối, định dạng thông điệp, định danh máy khách, xử lý mất kết nối, kết nối lại, phân quyền theo kênh, phát rộng, gửi cho một nhóm, và mở rộng khi có nhiều máy chủ. Với ứng dụng học tập nhỏ, tự dùng WebSocket có thể giúp hiểu cơ chế ở tầng thấp; với ứng dụng .NET thực tế, ta thường dùng một lớp trừu tượng cao hơn.

SignalR là thư viện của ASP.NET Core giúp xây dựng chức năng web thời gian thực. SignalR cung cấp hub như một điểm giao tiếp giữa máy chủ và máy khách. Máy chủ có thể gọi phương thức phía máy khách, máy khách có thể gọi phương thức trên hub, và ứng dụng có thể gửi thông điệp tới tất cả máy khách, một nhóm máy khách hoặc một máy khách cụ thể. Nhờ đó, mã nghiệp vụ tập trung vào sự kiện cần phát và người nhận cần thông báo, thay vì tự xử lý toàn bộ giao thức ở tầng thấp.

SignalR không đồng nghĩa với WebSocket thuần. WebSocket là cơ chế truyền tải

được ưu tiên khi khả dụng, nhưng SignalR còn có thể dùng Server-Sent Events hoặc Long Polling tùy khả năng của máy chủ và máy khách. Server-Sent Events (SSE) là cơ chế máy chủ đẩy dữ liệu một chiều qua HTTP, phù hợp khi chỉ cần gửi cập nhật từ máy chủ về máy khách mà không cần giao tiếp hai chiều. Long Polling là kỹ thuật máy khách gửi yêu cầu HTTP và giữ kết nối mở trong thời gian dài; máy chủ chỉ trả phản hồi khi có dữ liệu mới hoặc khi quá thời gian chờ, sau đó máy khách lập tức gửi yêu cầu mới. Quá trình chọn cơ chế truyền tải phù hợp giúp ứng dụng chạy được trong nhiều môi trường hơn. Đổi lại, lập trình viên cần hiểu rằng SignalR có lớp giao thức và vòng đời kết nối riêng; khi gỡ lỗi hoặc mở rộng hệ thống, không nên xem nó chỉ là một socket đơn giản.

WebSocket là một giao thức truyền thông hai chiều. SignalR là một thư viện thời gian thực ở mức cao hơn, dùng WebSocket khi có thể và che bớt nhiều chi tiết như cơ chế truyền tải dự phòng, hub, kết nối, nhóm và phát rộng.

3.1.3 Ca sử dụng thời gian thực trong hệ thống

Trong EduSubmit, thời gian thực không phải là tính năng trang trí. Nó giải quyết một vấn đề nghiệp vụ cụ thể: người dùng cần biết trạng thái mới mà không phải tự tải lại trang hoặc bấm kiểm tra liên tục. Một luồng điển hình là sinh viên nộp bài, API ghi nhận lượt nộp, dịch vụ xử lý nền chấm bài, sau đó giao diện nhận thông báo khi kết quả sẵn sàng.

Một số ca sử dụng thời gian thực phù hợp gồm:

- Cập nhật trạng thái lượt nộp: `submitted`, `processing`, `graded`, `failed`.
- Thông báo cho sinh viên khi bài đã được chấm xong.
- Thông báo cho giảng viên khi có lượt nộp mới hoặc khi dịch vụ xử lý nền gặp lỗi.
- Hiển thị tiến độ xử lý nếu tác vụ gồm nhiều bước.
- Gửi nhật ký hoặc cảnh báo vận hành trong môi trường thực hành.

Khi thiết kế ca sử dụng thời gian thực, cần xác định rõ bốn yếu tố sau:

- Sự kiện nào trong hệ thống sẽ kích hoạt thông điệp.
- Thông điệp cần gửi cho ai: tất cả máy khách, một nhóm lớp học, một giảng viên, hay đúng sinh viên sở hữu lượt nộp.
- Phần tải dữ liệu cần chứa dữ liệu tối thiểu nào để máy khách cập nhật giao diện.
- Máy khách sẽ làm gì nếu mất kết nối hoặc bỏ lỡ thông điệp; trong nhiều trường hợp, câu trả lời đúng là đồng bộ lại trạng thái bằng một REST API thông thường.

Không phải trạng thái nào cũng cần thời gian thực. Dữ liệu ít thay đổi hoặc không yêu cầu phản hồi nhanh vẫn có thể lấy bằng REST API thông thường. Giao tiếp thời gian thực nên được dùng cho các thay đổi mà độ trễ ảnh hưởng trực tiếp đến trải nghiệm hoặc luồng thao tác. Nếu dùng quá rộng, hệ thống sẽ khó kiểm soát kết nối, khó phân quyền theo người nhận và khó quan sát khi thông điệp bị mất hoặc gửi nhầm.

Trong các chương sau, REST API, dịch vụ xử lý nền, hàng đợi và pub/sub sẽ kết hợp với SignalR để tạo luồng hoàn chỉnh hơn. REST tiếp nhận yêu cầu ban đầu, hàng đợi giúp xử lý nền, dịch vụ xử lý nền phát sự kiện khi có kết quả, còn SignalR đưa cập nhật đó về giao diện. Cách ghép này thể hiện tinh thần quan trọng của phát triển ứng dụng phân tán: mỗi cơ chế giao tiếp giải quyết một phần riêng của bài toán, và thiết kế tốt là biết đặt chúng vào đúng vị trí.

3.2 Kỹ thuật lập trình cơ bản với SignalR

Phần 3.1 đã trình bày các mô hình trao đổi thông điệp thời gian thực, cơ chế hoạt động của WebSocket và vai trò của SignalR như một lớp trừu tượng giúp đơn giản hóa giao tiếp hai chiều. Phần này chuyển từ nguyên tắc sang kỹ thuật lập trình: cách đăng ký SignalR trong ứng dụng ASP.NET Core, cách xây dựng hub để gửi và nhận thông điệp, cách máy khách kết nối và đăng ký nhận thông điệp, cách gửi thông điệp có chủ đích đến từng máy khách hoặc nhóm, và cách quản lý vòng đời kết nối.

Để tập trung vào kỹ thuật, phần này dùng một ví dụ tối giản: PingApi – một máy chủ SignalR nhận yêu cầu từ máy khách và phản hồi qua nhiều kênh (gửi riêng, gửi nhóm, phát rộng). Mã nguồn đầy đủ nằm trong thư mục `content/examples/chapter03/section02/`. Sau khi nắm được các kỹ thuật cơ bản qua ví dụ này, sinh viên sẽ áp dụng chúng để xây dựng hệ thống EduSubmit với thông báo trạng thái thời gian thực trong phần 3.3.

3.2.1 Đăng ký và ánh xạ SignalR hub trong ASP.NET Core

Để dùng SignalR trong ứng dụng ASP.NET Core, cần thực hiện hai bước. Bước thứ nhất, đăng ký các dịch vụ SignalR vào container DI bằng `builder.Services.AddSignalR()`. Bước thứ hai, ánh xạ một hub vào một tuyến HTTP bằng `app.MapHub<T>()`. Hai bước này thường nằm trong `Program.cs`, ngay trước `app.Run()`.

Trong ví dụ PingApi, việc đăng ký và ánh xạ được thực hiện như sau:

```
builder.Services.AddSignalR();                // Buoc 1: dang ky dich vu
var app = builder.Build();
app.MapHub<PingHub>("/hubs/ping");            // Buoc 2: anh xa hub vao tuyen
```

Lời gọi `AddSignalR()` đăng ký các dịch vụ cốt lõi để SignalR hoạt động, bao gồm bộ quản lý hub, bộ quản lý kết nối và cơ chế tuần tự hóa thông điệp. Nếu cần, có thể cấu hình thêm trong lời gọi này, ví dụ tăng kích thước bộ đệm thông điệp hoặc thay đổi định dạng tuần tự hóa.

Lời gọi `app.MapHub<PingHub>("/hubs/ping")` ánh xạ hub vào tuyến `/hubs/ping`. Từ thời điểm này, máy khách có thể kết nối tới `http://localhost:5200/hubs/ping` và bắt đầu giao tiếp thời gian thực. Tuyến của hub không nhất thiết phải có tiền tố `/hubs/`, nhưng đây là quy ước phổ biến để phân biệt điểm cuối SignalR với điểm cuối REST API.

Toàn bộ mã nguồn PingApi được trình bày trong Listing 3.1. Các kỹ thuật về gửi thông điệp có chủ đích và quản lý vòng đời kết nối được thảo luận trong các mục tiếp theo, và đều có thể đối chiếu trực tiếp với mã nguồn này.

Mã nguồn 3.1: PingApi – ví dụ tối giản minh họa đăng ký SignalR, xây dựng hub, gửi thông điệp có chủ đích và quản lý vòng đời kết nối

```
using Microsoft.AspNetCore.SignalR;

var builder = WebApplication.CreateBuilder(args);

// 3.2.1: Dang ky SignalR vao container dich vu
builder.Services.AddSignalR();
```

```
var app = builder.Build();

app.MapGet("/", () => Results.Ok(new { service = "PingApi", status = "running" }));

// 3.2.1: Anh xa hub vao mot tuyen
app.MapHub<PingHub>("/hubs/ping");

app.Run();

// 3.2.2: Lop hub – diem giao tiep thoi gian thuc giua may chu va may khach
sealed class PingHub : Hub
{
    // May khach goi Echo, may chu phan hoi rieng cho may khach do
    public async Task Echo(string message)
    {
        await Clients.Caller.SendAsync("EchoResponse", $"Echo: {message}");
    }

    // 3.2.4: Gui thong diep toi tat ca may khach dang ket noi
    public async Task Broadcast(string message)
    {
        await Clients.All.SendAsync("BroadcastMessage", message);
    }

    // 3.2.4: Tham gia mot nhom
    public async Task JoinGroup(string groupName)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, groupName);
        await Clients.Group(groupName).SendAsync("GroupMessage",
            $"{Context.ConnectionId[..8]} da tham gia nhom {groupName}");
    }

    // 3.2.4: Gui thong diep toi mot nhom cu the
    public async Task SendToGroup(string groupName, string message)
    {
        await Clients.Group(groupName).SendAsync("GroupMessage", message);
    }

    // 3.2.5: Vong doi ket noi – duoc goi khi may khach ket noi
    public override async Task OnConnectedAsync()
    {
        await Clients.All.SendAsync("SystemMessage",
            $"Client {Context.ConnectionId[..8]} da ket noi.");
        await base.OnConnectedAsync();
    }

    // 3.2.5: Vong doi ket noi – duoc goi khi may khach ngat ket noi
    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        await Clients.All.SendAsync("SystemMessage",
            $"Client {Context.ConnectionId[..8]} da ngat ket noi.");
        await base.OnDisconnectedAsync(exception);
    }
}
```

3.2.2 Xây dựng hub và gửi thông điệp đến client

Hub là lớp kế thừa từ `Hub` – lớp cơ sở do SignalR cung cấp. Mỗi phương thức công khai của hub là một "điểm cuối thời gian thực" mà máy khách có thể gọi thông qua kết nối SignalR. Bên trong các phương thức này, máy chủ có thể gửi thông điệp ngược về máy khách.

Trong Listing 3.1, lớp `PingHub` có bốn phương thức nghiệp vụ. Phương thức `Echo` nhận một chuỗi từ máy khách và gửi ngược lại cho chính máy khách đó thông qua `Clients.Caller.SendAsync`. Phương thức `Broadcast` gửi thông điệp tới tất cả máy khách đang kết nối thông qua `Clients.All.SendAsync`. Hai phương thức `JoinGroup` và `SendToGroup` làm việc với nhóm, sẽ được thảo luận chi tiết trong Mục 3.2.4.

Cú pháp gửi thông điệp từ hub luôn có dạng `Clients.<ĐốiTượng>.SendAsync("TênPhươngThức", thamSố)`. Trong đó:

- `Clients` là thuộc tính kế thừa từ lớp `Hub`, cung cấp quyền truy cập đến tập máy khách đang kết nối.
- `<ĐốiTượng>` xác định phạm vi người nhận: `All` (tất cả), `Caller` (người gọi), `Group` (một nhóm) hoặc `Client` (một kết nối cụ thể).
- `"TênPhươngThức"` là tên phương thức mà phía máy khách đã đăng ký để nhận thông điệp.
- `thamSố` là dữ liệu gửi kèm, có thể là kiểu nguyên thủy hoặc đối tượng phức hợp.

Điều quan trọng là tên phương thức trong `SendAsync` phải khớp chính xác với tên mà máy khách đăng ký qua `connection.On<T>("TênPhươngThức", ...)`. Nếu không khớp, máy khách sẽ không nhận được thông điệp và không có lỗi biên dịch nào báo trước. Đây là một nguồn lỗi phổ biến khi phát triển ứng dụng SignalR.

3.2.3 Kết nối và đăng ký nhận thông điệp bằng `HubConnection`

Phía máy khách, lớp `HubConnection` (từ gói NuGet `Microsoft.AspNetCore.SignalR.Client`) là điểm vào chính để làm việc với SignalR. Quy trình thiết lập kết nối gồm ba bước: xây dựng kết nối, đăng ký hàm xử lý thông điệp, và mở kết nối.

Bước 1 – Xây dựng kết nối. Dùng `HubConnectionBuilder` để cấu hình địa chỉ hub và các tùy chọn. Trong ví dụ PingClient:

```
var connection = new HubConnectionBuilder()
    .WithUrl("http://localhost:5200/hubs/ping")
    .WithAutomaticReconnect()
    .Build();
```

Phương thức `WithUrl` nhận URL đầy đủ của hub, bao gồm giao thức, máy chủ, cổng và tuyến đã ánh xạ ở phía máy chủ. Phương thức `WithAutomaticReconnect` (sẽ thảo luận trong Mục 3.2.5) bật cơ chế tự động kết nối lại khi kết nối bị gián đoạn.

Bước 2 – Đăng ký hàm xử lý. Mỗi thông điệp máy chủ gửi về được định danh bằng một tên phương thức. Máy khách đăng ký hàm xử lý cho từng tên bằng `connection.On<T>("TênPhươngThức", hàmXửLý)`. Trong ví dụ PingClient, máy khách đăng ký bốn hàm xử lý cho bốn loại thông điệp khác nhau: `EchoResponse` (phản hồi riêng), `BroadcastMessage` (phát rộng), `GroupMessage` (thông điệp nhóm) và `SystemMessage` (thông điệp hệ thống).

Bước 3 – Mở kết nối. Gọi `awaitconnection.StartAsync()`. Đây là thao tác bất

đồng bộ, thực hiện bắt tay với máy chủ (bao gồm thương lượng cơ chế truyền tải) và thiết lập kênh giao tiếp. Sau khi `StartAsync` hoàn tất, máy khách có thể gọi phương thức trên hub và nhận thông điệp từ máy chủ.

Toàn bộ mã nguồn PingClient được trình bày trong Listing 3.2.

Mã nguồn 3.2: PingClient – máy khách tối giản kết nối SignalR, đăng ký nhận thông điệp và gọi phương thức hub

```
using Microsoft.AspNetCore.SignalR.Client;

var serverUrl = args.Length > 0 ? args[0] : "http://localhost:5200";

// 3.2.3: Xay dung ket noi SignalR
await using var connection = new HubConnectionBuilder()
    .WithUrl($"{serverUrl}/hubs/ping")
    .WithAutomaticReconnect()      // 3.2.5: Tu dong ket noi lai
    .Build();

// 3.2.3: Dang ky cac ham xu ly thong diep tu may chu
connection.On<string>("EchoResponse", msg =>
    Console.WriteLine($"[Echo] {msg}"));

connection.On<string>("BroadcastMessage", msg =>
    Console.WriteLine($"[Broadcast] {msg}"));

connection.On<string>("GroupMessage", msg =>
    Console.WriteLine($"[Group] {msg}"));

connection.On<string>("SystemMessage", msg =>
    Console.WriteLine($"[System] {msg}"));

// 3.2.5: Dang ky su kien vong doi phia may khach
connection.Reconnecting += _ =>
{
    Console.WriteLine("[Lifecycle] Dang ket noi lai...");
    return Task.CompletedTask;
};

connection.Closed += _ =>
{
    Console.WriteLine("[Lifecycle] Ket noi da dong.");
    return Task.CompletedTask;
};

// 3.2.3: Mo ket noi
await connection.StartAsync();
Console.WriteLine("Da ket noi den PingApi.\n");

// 3.2.2: Goi phuong thuc Echo tren hub
await connection.InvokeAsync("Echo", "Xin chao SignalR");
await Task.Delay(500);

// 3.2.4: Tham gia nhom
await connection.InvokeAsync("JoinGroup", "lop-hoc-01");
await Task.Delay(500);

// 3.2.4: Gui thong diep toi nhom
await connection.InvokeAsync("SendToGroup", "lop-hoc-01", "Thong bao tu thanh vien");
await Task.Delay(500);
```

```
// 3.2.4: Gui broadcast toi tat ca may khach
await connection.InvokeAsync("Broadcast", "Xin chao tat ca moi nguoi");

Console.WriteLine("\nDang cho thong diep... Nhan phim bat ky de thoat.");
Console.ReadKey();
```

Sau khi kết nối được mở, mã nguồn trong Listing 3.2 lần lượt gọi các phương thức hub qua `connection.InvokeAsync("TênPhươngThức", thamSố)`. Mỗi lời gọi `InvokeAsync` gửi một thông điệp từ máy khách lên máy chủ và chờ phản hồi (nếu có). Khác với `SendAsync` (gửi một chiều, không chờ kết quả), `InvokeAsync` phù hợp khi máy khách cần biết máy chủ đã nhận và xử lý yêu cầu hay chưa.

3.2.4 Gửi thông điệp theo client, group và toàn bộ kết nối

Một trong những khả năng quan trọng nhất của SignalR là gửi thông điệp có chủ đích: không phải thông báo nào cũng cần đến tất cả máy khách. SignalR cung cấp bốn phạm vi gửi thông điệp, tương ứng với bốn thuộc tính của `Clients`:

- `Clients.All`: gửi tới tất cả máy khách đang kết nối. Phù hợp với thông báo toàn hệ thống như "máy chủ sắp bảo trì" hoặc "có bài tập mới được đăng".
- `Clients.Caller`: gửi riêng cho máy khách vừa gọi phương thức hub. Phù hợp với phản hồi cá nhân như kết quả echo hoặc xác nhận thao tác.
- `Clients.Group(groupName)`: gửi tới tất cả máy khách trong một nhóm. Đây là công cụ quan trọng để triển khai các kênh thông báo theo ngữ cảnh: nhóm lớp học, nhóm theo dõi một lượt nộp, nhóm quản trị viên.
- `Clients.Client(connectionId)`: gửi tới một kết nối cụ thể. Ít dùng trong ứng dụng thông thường, nhưng hữu ích khi cần gửi thông điệp cá nhân hóa cao hoặc xử lý một kết nối đặc biệt.

Trong Listing 3.1, `Echo` dùng `Clients.Caller`, `Broadcast` dùng `Clients.All`, và `JoinGroup` / `SendToGroup` dùng `Clients.Group`. Nhóm được quản lý qua hai phương thức của lớp cơ sở `Hub`: `Groups.AddToGroupAsync(connectionId, groupName)` để thêm một kết nối vào nhóm, và `Groups.RemoveFromGroupAsync(connectionId, groupName)` để gỡ khỏi nhóm.

Một mẫu thiết kế phổ biến là dùng nhóm để khoanh vùng thông báo theo nghiệp vụ. Ví dụ, khi sinh viên theo dõi một lượt nộp, máy chủ thêm kết nối của sinh viên vào nhóm `submission:<mã-lượt-nộp>`. Sau đó, mỗi khi trạng thái lượt nộp thay đổi, máy chủ chỉ gửi thông điệp tới nhóm đó. Cách làm này ngăn sinh viên này nhìn thấy trạng thái bài nộp của sinh viên khác, đồng thời giảm số lượng thông điệp không liên quan trên đường truyền. Phần 3.3 sẽ áp dụng trực tiếp mẫu này trong bối cảnh EduSubmit.

3.2.5 Quản lý vòng đời kết nối và kết nối lại

Kết nối SignalR không tồn tại vĩnh viễn. Máy khách có thể đóng trình duyệt, mạng có thể gián đoạn, hoặc máy chủ có thể khởi động lại. Một ứng dụng thời gian thực tốt cần xử lý các sự kiện vòng đời này một cách tường minh.

Phía máy chủ, lớp `Hub` cung cấp hai phương thức ảo để can thiệp vào vòng đời kết nối:

- `OnConnectedAsync()`: được gọi khi một máy khách thiết lập kết nối thành công. Thường dùng để ghi nhật ký, phát thông báo "người dùng vừa trực tuyến", hoặc khởi tạo trạng thái cho kết nối mới.
- `OnDisconnectedAsync(Exception?exception)`: được gọi khi một máy khách ngắt kết nối. Tham số `exception` khác `null` nếu ngắt kết nối do lỗi; bằng `null` nếu máy khách chủ động ngắt. Thường dùng để dọn dẹp tài nguyên, gỡ khỏi nhóm hoặc phát thông báo "người dùng vừa ngoại tuyến".

Trong Listing 3.1, cả hai phương thức này đều được ghi đè để phát thông điệp hệ thống tới tất cả máy khách, giúp sinh viên quan sát được thời điểm kết nối được thiết lập và hủy bỏ.

Phía máy khách, `HubConnection` cung cấp hai sự kiện:

- `connection.Reconnecting`: được kích hoạt khi thư viện bắt đầu thử kết nối lại sau khi mất kết nối. Hàm xử lý sự kiện này thường dùng để cập nhật giao diện (ví dụ hiển thị trạng thái "đang kết nối lại").
- `connection.Closed`: được kích hoạt khi kết nối đóng vĩnh viễn (không thể hoặc không được cấu hình để kết nối lại). Lúc này, máy khách cần quyết định có tạo kết nối mới hay không.

Cơ chế `WithAutomaticReconnect()` (được bật trong Listing 3.2) cho phép SignalR tự động thử kết nối lại sau khi mất kết nối. Mặc định, thư viện thử lại sau 0, 2, 10 và 30 giây, sau đó dừng nếu vẫn không thành công. Có thể tùy chỉnh khoảng thời gian thử lại bằng cách truyền một mảng `TimeSpan` vào `WithAutomaticReconnect`. Trong ứng dụng thực tế, chính sách kết nối lại nên được thiết kế cùng với cơ chế đồng bộ trạng thái: sau khi kết nối lại thành công, máy khách nên gọi REST API để lấy trạng thái mới nhất, phòng trường hợp đã bỏ lỡ thông điệp trong thời gian gián đoạn.

Tóm lại, kỹ thuật lập trình SignalR xoay quanh bốn trụ cột: đăng ký và ánh xạ hub ở phía máy chủ, xây dựng phương thức hub để xử lý yêu cầu và gửi thông điệp, thiết lập kết nối và đăng ký hàm xử lý ở phía máy khách, và gửi thông điệp có chủ đích đến đúng đối tượng nhận. Phần tiếp theo sẽ áp dụng toàn bộ các kỹ thuật này để bổ sung kênh thông báo thời gian thực vào hệ thống EduSubmit, nơi REST API xử lý thao tác tài nguyên còn SignalR đẩy cập nhật trạng thái tới người dùng.

3.3 [Ứng dụng] Sử dụng SignalR để truyền thông thời gian thực giữa máy khách và máy chủ

Ở bài thực hành 2.3, EduSubmit đã có một RESTful API cho tài nguyên bài tập và lượt nộp. Máy khách có thể lấy danh sách bài tập, tạo một lượt nộp mới và đọc lại lượt nộp bằng HTTP. Cách làm đó phù hợp với mô hình yêu cầu–phản hồi: máy khách gửi yêu cầu, máy chủ trả phản hồi, rồi tương tác kết thúc.

Trong bài thực hành này, ta giữ lại lát cắt đó nhưng bổ sung một kênh thời gian thực bằng SignalR. REST API vẫn chịu trách nhiệm tạo và đọc tài nguyên. SignalR không thay thế REST; nó chỉ bổ sung khả năng đẩy thông báo trạng thái từ máy chủ về máy khách sau khi lượt nộp đã được tạo. Nhờ vậy, máy khách không cần liên tục gọi API để hỏi trạng thái mới.

Luồng thực hành gồm bốn bước:

1. Máy khách mở kết nối SignalR tới hub của máy chủ.
2. Máy khách gọi REST API để tạo một lượt nộp mới.
3. Máy khách đăng ký theo dõi mã lượt nộp vừa tạo.
4. Máy chủ mô phỏng quá trình xử lý và đẩy các trạng thái `processing`, `graded` về máy khách.

Ví dụ nguồn của bài này nằm trong thư mục `content/examples/chapter03/section02`. Cấu trúc dự án vẫn gồm hai tiến trình độc lập:

```
examples/chapter03/section03/
  RealtimeSubmissionSystem.slnx
  src/
    Submission.RealtimeApi/
    Submission.RealtimeClient/
```

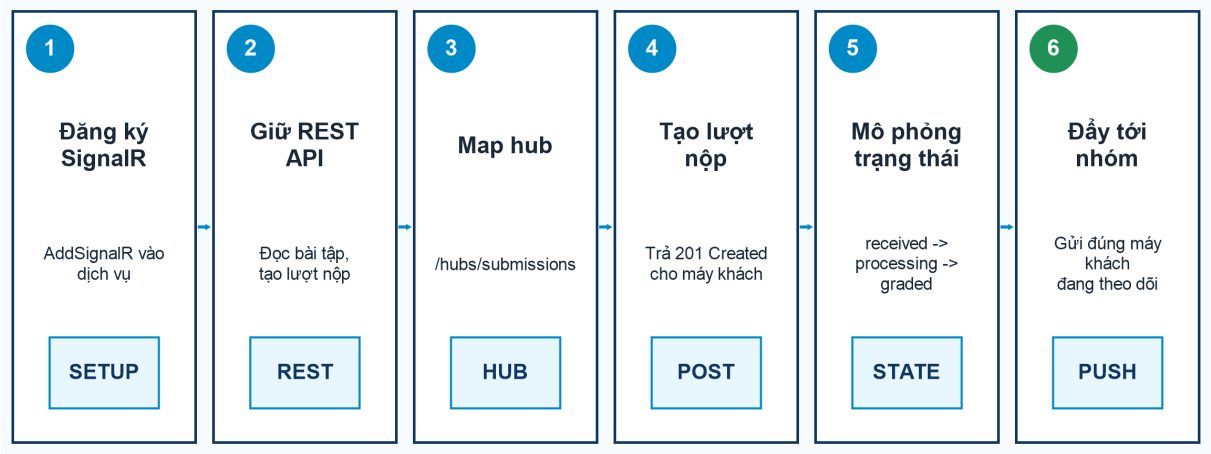
So với bài thực hành chương 2, điểm mới là API có thêm hub SignalR tại `/hubs/submissions`, còn máy khách có thêm kết nối SignalR song song với các lời gọi REST.

3.3.1 Thiết lập SignalR hub

Trong ASP.NET Core, SignalR được đăng ký bằng `AddSignalR` và được ánh xạ vào một tuyến hub bằng `MapHub`. Hub là điểm vào để máy khách thiết lập kết nối thời gian thực và gọi các phương thức phía máy chủ. Trong ví dụ này, hub có một phương thức `WatchSubmission`. Khi máy khách gọi phương thức này với mã lượt nộp, máy chủ đưa kết nối của máy khách vào một nhóm gắn với lượt nộp đó.

Tư duy nhóm rất quan trọng trong ứng dụng thời gian thực. Nếu máy chủ phát mọi thông báo cho tất cả máy khách, sinh viên này có thể nhận trạng thái bài nộp của sinh viên khác. Vì vậy, ví dụ này tạo nhóm theo mẫu `submission:\{id\}`. Khi trạng thái của một lượt nộp thay đổi, máy chủ chỉ gửi thông báo tới nhóm của lượt nộp đó.

Tệp `Program.cs` của API như sau:



Hình 3.1: Các bước chính khi viết mã API kết hợp REST và SignalR trong Listing 3.1

Mã nguồn 3.3: RESTful API kết hợp SignalR hub cho trạng thái lượt nộp

```
using Microsoft.AspNetCore.SignalR;
```

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSignalR();

var app = builder.Build();

var assignments = new List<AssignmentDto>
{
    new(1, "Bai tap SignalR", "Nhan thong bao trang thai bai nop theo thoi gian thuc",
    ↪  true),
    new(2, "Bai tap REST API", "On lai API assignment va submission", true)
};

var submissions = new List<SubmissionDto>();
var submissionsLock = new object();

app.MapGet("/", () => Results.Ok(new {
    service = "Submission.RealtimeApi",
    status = "running"
}));

app.MapGet("/api/assignments", () => Results.Ok(assignments));

app.MapGet("/api/submissions/{submissionId:guid}", (Guid submissionId) => {
    lock (submissionsLock) {
        var submission = submissions.FirstOrDefault(item => item.Id == submissionId);
        return submission is null
            ? Results.NotFound(new ApiError("SUBMISSION_NOT_FOUND", "Khong tim thay
            ↪  luot nop."))
            : Results.Ok(submission);
    }
});

app.MapPost("/api/assignments/{assignmentId:int}/submissions",
    (int assignmentId,
        CreateSubmissionRequest request,
        IHubContext<SubmissionHub> hubContext) => {
            var assignment = assignments.FirstOrDefault(item => item.Id ==
            ↪  assignmentId);
            if (assignment is null) {
                return Results.NotFound(new ApiError("ASSIGNMENT_NOT_FOUND", "Khong tim
                ↪  thay bai tap."));
            }

            if (!assignment.IsOpen) {
                return Results.BadRequest(new ApiError("ASSIGNMENT_CLOSED", "Bai tap da
                ↪  dong nop bai."));
            }

            if (string.IsNullOrWhiteSpace(request.StudentCode) ||
                string.IsNullOrWhiteSpace(request.RepositoryUrl)) {
                return Results.BadRequest(new ApiError("INVALID_SUBMISSION", "Thieu ma
                ↪  sinh vien hoac repository."));
            }

            var submission = new SubmissionDto(
                Id: Guid.NewGuid(),
                AssignmentId: assignmentId,
```

```
                StudentCode: request.StudentCode.Trim(),
                RepositoryUrl: request.RepositoryUrl.Trim(),
                Status: "received",
                ReceivedAt: DateTimeOffset.UtcNow,
                UpdatedAt: DateTimeOffset.UtcNow);

            lock (submissionsLock) {
                submissions.Add(submission);
            }

            _ = SimulateGradingAsync(submission.Id, submissions, submissionsLock,
            ↪  hubContext);

            return Results.Created($"/api/submissions/{submission.Id}", submission);
        });

app.MapHub<SubmissionHub>("/hubs/submissions");

app.Run();

static async Task SimulateGradingAsync(
    Guid submissionId,
    List<SubmissionDto> submissions,
    object submissionsLock,
    IHubContext<SubmissionHub> hubContext) {
    await Task.Delay(TimeSpan.FromSeconds(1));
    await UpdateStatusAsync(submissionId, "processing", submissions, submissionsLock,
    ↪  hubContext);

    await Task.Delay(TimeSpan.FromSeconds(2));
    await UpdateStatusAsync(submissionId, "graded", submissions, submissionsLock,
    ↪  hubContext);
}

static async Task UpdateStatusAsync(
    Guid submissionId,
    string status,
    List<SubmissionDto> submissions,
    object submissionsLock,
    IHubContext<SubmissionHub> hubContext) {
    SubmissionDto? updatedSubmission = null;

    lock (submissionsLock) {
        var index = submissions.FindIndex(item => item.Id == submissionId);
        if (index >= 0) {
            updatedSubmission = submissions[index] with {
                Status = status,
                UpdatedAt = DateTimeOffset.UtcNow
            };
            submissions[index] = updatedSubmission;
        }
    }

    if (updatedSubmission is null) {
        return;
    }

    var message = new SubmissionStatusMessage(
        updatedSubmission.Id,
```

```
        updatedSubmission.Status,
        updatedSubmission.UpdatedAt);

    await hubContext.Clients
        .Group(SubmissionGroups.ForSubmission(submissionId))
        .SendAsync("SubmissionStatusChanged", message);
}

sealed class SubmissionHub : Hub {
    public Task WatchSubmission(string submissionId) {
        return Groups.AddToGroupAsync(
            Context.ConnectionId,
            SubmissionGroups.ForSubmission(submissionId));
    }
}

static class SubmissionGroups {
    public static string ForSubmission(Guid submissionId) {
        return ForSubmission(submissionId.ToString());
    }

    public static string ForSubmission(string submissionId) {
        return $"submission:{submissionId}";
    }
}

record AssignmentDto(
    int Id,
    string Title,
    string Description,
    bool IsOpen);

record CreateSubmissionRequest(
    string StudentCode,
    string RepositoryUrl);

record SubmissionDto(
    Guid Id,
    int AssignmentId,
    string StudentCode,
    string RepositoryUrl,
    string Status,
    DateTimeOffset ReceivedAt,
    DateTimeOffset UpdatedAt);

record SubmissionStatusMessage(
    Guid SubmissionId,
    string Status,
    DateTimeOffset UpdatedAt);

record ApiError(
    string Code,
    string Message);
```

Đoạn mã trên nối tiếp trực tiếp API của chương 2. Các điểm cuối `GET /api/assignments`, `POST /api/assignments/\{assignmentId\}/submissions` và `GET /api/submissions/\{submissionId\}` vẫn giữ vai trò RESTful API. Điểm khác là khi tạo lượt nộp thành công, API không chỉ trả `201 Created`; nó còn khởi động một tác vụ mô

phỏng xử lý bằng hàm `SimulateGradingAsync`. Tác vụ này lần lượt cập nhật trạng thái thành `processing` và `graded`.

Hàm `UpdateStatusAsync` là cầu nối giữa dữ liệu REST và kênh thời gian thực. Trước hết, hàm cập nhật trạng thái trong danh sách `submissions`. Sau đó, hàm tạo `SubmissionStatusMessage` và dùng `IHubContext<SubmissionHub>` để gửi thông điệp tới đúng nhóm máy khách. Nhờ `IHubContext`, mã xử lý trong điểm cuối REST hoặc trong tác vụ nền vẫn có thể phát thông điệp SignalR mà không cần nằm trực tiếp bên trong lớp hub.

Lớp `SubmissionHub` trong ví dụ này rất nhỏ. Nó không xử lý nghiệp vụ nộp bài; nghiệp vụ vẫn nằm ở REST API. Hub chỉ quản lý việc máy khách đăng ký theo dõi một lượt nộp. Cách tách vai trò này giúp sinh viên thấy rõ: REST phù hợp để tạo và đọc tài nguyên, còn SignalR phù hợp để đẩy các thay đổi trạng thái xảy ra sau đó.

Trong ví dụ học tập, quá trình chấm bài được mô phỏng bằng `Task.Delay`. Ở hệ thống thật, phần này thường được thay bằng dịch vụ xử lý nền, hàng đợi hoặc một thành phần chấm bài riêng. Dù cơ chế xử lý phía sau thay đổi, cách máy chủ đẩy trạng thái về máy khách qua SignalR vẫn giữ cùng ý tưởng.

Biên dịch toàn bộ solution bằng lệnh:

```
dotnet build examples/chapter03/section03/RealtimeSubmissionSystem.slnx
```

Máy khách cần gói `Microsoft.AspNetCore.SignalR.Client`. Trong dự án mẫu, gói này đã được khai báo trong `Submission.RealtimeClient.csproj`. Nếu tự tạo dự án từ đầu, có thể thêm gói bằng lệnh:

```
dotnet add examples/chapter03/section03/src/Submission.RealtimeClient package
↪  Microsoft.AspNetCore.SignalR.Client --version 10.0.0
```

3.3.2 *Máy khách nhận thông báo thời gian thực*

Máy khách của bài này có hai trách nhiệm. Trách nhiệm thứ nhất giống chương 2: gọi REST API để lấy danh sách bài tập và tạo lượt nộp. Trách nhiệm thứ hai là mở kết nối SignalR, đăng ký hàm xử lý thông điệp `SubmissionStatusChanged`, rồi chờ máy chủ đẩy trạng thái mới.

Tệp `Program.cs` của máy khách như sau:

Mã nguồn 3.4: Máy khách gọi REST API và nhận thông báo trạng thái qua SignalR

```
using System.Net;
using System.Net.Http.Json;
using Microsoft.AspNetCore.SignalR.Client;

var apiBaseUrl = args.Length > 0 ? args[0].TrimEnd('/') : "http://localhost:5000";

using var http = new HttpClient {
    BaseAddress = new Uri(apiBaseUrl)
};
```

Hình 3.2: Các bước chính khi viết mã máy khách nhận thông báo SignalR trong Listing 3.2

```
await using var connection = new HubConnectionBuilder()
    .WithUrl($"{apiBaseUrl}/hubs/submissions")
    .WithAutomaticReconnect()
    .Build();

var gradedNotification = new TaskCompletionSource();

connection.On<SubmissionStatusMessage>("SubmissionStatusChanged", message => {
    Console.WriteLine(
        $"Thong bao SignalR: {message.SubmissionId} -> {message.Status} luc
        ↪  {message.UpdatedAt:HH:mm:ss}");

    if (message.Status == "graded") {
        gradedNotification.TrySetResult();
    }
});

await connection.StartAsync();
Console.WriteLine("Da ket noi SignalR.");

var assignments = await http.GetFromJsonAsync<List<AssignmentDto>>("/api/assignments");
var selectedAssignment = assignments?.FirstOrDefault();

if (selectedAssignment is null) {
    Console.WriteLine("Khong co bai tap nao de nop.");
    return;
}

Console.WriteLine($"Chon bai tap: {selectedAssignment.Id} -
↪  {selectedAssignment.Title}");

var createRequest = new CreateSubmissionRequest(
    StudentCode: "SV001",
    RepositoryUrl: "https://example.com/sv001/signalr");

var createResponse = await http.PostAsJsonAsync(
    $"/api/assignments/{selectedAssignment.Id}/submissions",
    createRequest);

if (createResponse.StatusCode != HttpStatusCode.Created) {
    await PrintApiErrorAsync(createResponse);
```

```
    return;
}

var createdSubmission = await
↪  createResponse.Content.ReadFromJsonAsync<SubmissionDto>();
if (createdSubmission is null) {
    Console.WriteLine("Khong doc duoc phan hoi tao luot nop.");
    return;
}

Console.WriteLine($"Da tao luot nop: {createdSubmission.Id}");
Console.WriteLine($"Trang thai REST ban dau: {createdSubmission.Status}");

await connection.InvokeAsync("WatchSubmission", createdSubmission.Id.ToString());
Console.WriteLine("Dang cho thong bao trang thai tu may chu...");

var completed = await Task.WhenAny(
    gradedNotification.Task,
    Task.Delay(TimeSpan.FromSeconds(10)));

if (completed != gradedNotification.Task) {
    Console.WriteLine("Qua thoi gian cho thong bao graded.");
}

var latestSubmission = await http.GetFromJsonAsync<SubmissionDto>(
    $"/api/submissions/{createdSubmission.Id}");

Console.WriteLine($"Trang thai doc lai bang REST: {latestSubmission?.Status}");

static async Task PrintApiErrorAsync(HttpResponseMessage response) {
    var error = await response.Content.ReadFromJsonAsync<ApiError>();
    Console.WriteLine($"{(int)response.StatusCode} {response.StatusCode}: {error?.Code}
    ↪  - {error?.Message}");
}

record AssignmentDto(
    int Id,
    string Title,
    string Description,
    bool IsOpen);

record CreateSubmissionRequest(
    string StudentCode,
    string RepositoryUrl);

record SubmissionDto(
    Guid Id,
    int AssignmentId,
    string StudentCode,
    string RepositoryUrl,
    string Status,
    DateTimeOffset ReceivedAt,
    DateTimeOffset UpdatedAt);

record SubmissionStatusMessage(
    Guid SubmissionId,
    string Status,
    DateTimeOffset UpdatedAt);
```

```
record ApiError(
    string Code,
    string Message);
```

Đoạn mã máy khách bắt đầu bằng `HubConnectionBuilder`. Lời gọi `WithUrl` trỏ tới hub `/hubs/submissions`; lời gọi `WithAutomaticReconnect` cho phép thư viện tự thử kết nối lại khi kết nối bị gián đoạn tạm thời. Trong ví dụ nhỏ này, cơ chế kết nối lại chỉ được bật để sinh viên nhận diện thói quen tốt khi viết máy khách thời gian thực; các chính sách kết nối lại chi tiết sẽ phụ thuộc yêu cầu vận hành của hệ thống thật.

Lời gọi `connection.On<SubmissionStatusMessage>` đăng ký hàm xử lý khi máy chủ gửi thông điệp `SubmissionStatusChanged`. Khi nhận được trạng thái mới, máy khách in mã lượt nộp, trạng thái và thời điểm cập nhật. Nếu trạng thái là `graded`, `TaskCompletionSource` được hoàn tất để chương trình biết rằng luồng mô phỏng đã kết thúc.

Sau khi kết nối SignalR được mở, máy khách vẫn dùng REST API để tạo lượt nộp. Đây là điểm nối tiếp quan trọng với chương 2: tạo tài nguyên mới vẫn là trách nhiệm của RESTful API. Chỉ sau khi có mã lượt nộp, máy khách mới gọi `WatchSubmission` trên hub để tham gia nhóm tương ứng. Từ thời điểm đó, mọi thông báo trạng thái của lượt nộp này sẽ được đẩy về máy khách qua SignalR.

Để chạy toàn bộ ví dụ, mở hai cửa sổ dòng lệnh. Cửa sổ thứ nhất chạy API:

```
dotnet run --project examples/chapter03/section03/src/Submission.RealtimeApi --urls
↪ http://localhost:5000
```

Cửa sổ thứ hai chạy máy khách:

```
dotnet run --project examples/chapter03/section03/src/Submission.RealtimeClient --
↪ http://localhost:5000
```

Kết quả mong đợi ở máy khách có dạng:

```
Da ket noi SignalR.
Chon bai tap: 1 - Bai tap SignalR
Da tao luot nop: 29ab7dd8-193e-4976-9175-50b3d4b2bca6
Trang thai REST ban dau: received
Dang cho thong bao trang thai tu may chu...
Thong bao SignalR: 29ab7dd8-193e-4976-9175-50b3d4b2bca6 -> processing luc 03:28:34
Thong bao SignalR: 29ab7dd8-193e-4976-9175-50b3d4b2bca6 -> graded luc 03:28:36
Trang thai doc lai bang REST: graded
```

Mã lượt nộp và thời điểm cập nhật sẽ khác nhau ở mỗi lần chạy. Điều cần quan sát là máy khách chỉ gọi REST để tạo lượt nộp và đọc lại trạng thái cuối, còn hai cập nhật `processing` và `graded` được máy chủ chủ động gửi qua SignalR. Đây chính là khác biệt thực hành giữa thăm dò định kỳ và đẩy dữ liệu thời gian thực.

Nếu máy khách không nhận được thông báo, hãy kiểm tra ba điểm. Thứ nhất, API có đang chạy đúng cổng mà máy khách truyền vào hay không. Thứ hai, đường dẫn hub có đúng là `/hubs/submissions` hay không. Thứ ba, máy khách đã gọi `WatchSubmission`

với đúng mã lượt nộp sau khi tạo tài nguyên hay chưa. Trong hệ thống thời gian thực, lỗi thường không nằm ở một dòng code riêng lẻ mà nằm ở thứ tự thiết lập kết nối, đăng ký nhận thông điệp và phát thông điệp.

Sau bài thực hành này, EduSubmit đã có một luồng kết hợp REST và SignalR: REST dùng cho thao tác có phản hồi trực tiếp, SignalR dùng cho thông báo trạng thái phát sinh sau yêu cầu ban đầu. Đây là nền tảng để các chương sau ghép thêm xử lý nền và hàng đợi, nơi trạng thái không còn được mô phỏng bằng `Task.Delay` mà đến từ một thành phần xử lý độc lập.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- Truyền thông thời gian thực giải quyết bài toán cập nhật trạng thái cho máy khách mà không cần máy khách liên tục gửi yêu cầu kiểm tra (thăm dò định kỳ).
- Ba mô hình trao đổi thông điệp liên quan đến thời gian thực là yêu cầu–phản hồi, đẩy dữ liệu và truyền luồng; mỗi mô hình có đặc điểm và tình huống sử dụng riêng.
- WebSocket là giao thức hai chiều lâu dài giữa máy khách và máy chủ; SignalR là thư viện mức cao của ASP.NET Core che bớt chi tiết WebSocket, cung cấp hub, nhóm, kết nối lại tự động và cơ chế truyền tải dự phòng.
- SignalR không thay thế REST; REST phù hợp với các thao tác tạo, đọc, cập nhật tài nguyên có kết quả trực tiếp, còn SignalR phù hợp với các thay đổi trạng thái xảy ra sau yêu cầu ban đầu.
- Hub là điểm kết nối để máy khách và máy chủ trao đổi thông điệp thời gian thực; `IHubContext` cho phép mã bên ngoài hub (như điểm cuối REST hoặc tác vụ nền) phát thông điệp tới máy khách.
- Nhóm trong SignalR dùng để giới hạn phạm vi nhận thông báo; ví dụ nhóm `submission:\{id\}` bảo đảm chỉ máy khách đang theo dõi một lượt nộp cụ thể mới nhận trạng thái của lượt nộp đó.
- Máy khách SignalR dùng `HubConnectionBuilder` để thiết lập kết nối, `On<T>` để đăng ký hàm xử lý thông điệp, và `WithAutomaticReconnect` để tự động kết nối lại khi mất kết nối tạm thời.

Câu hỏi ôn tập

1. Thăm dò định kỳ khác đẩy dữ liệu thời gian thực ở điểm nào về số lượng yêu cầu qua mạng và độ trễ cập nhật trạng thái?
2. Phân biệt ba mô hình yêu cầu–phản hồi, đẩy dữ liệu và truyền luồng về hướng khởi tạo và số lượng thông điệp trong một tương tác.
3. WebSocket là giao thức hay thư viện? SignalR là giao thức hay thư viện? Mối quan hệ giữa hai thành phần này là gì?
4. SignalR cung cấp những cơ chế truyền tải dự phòng nào khi WebSocket không khả dụng?
5. Trong ví dụ thực hành, vì sao cần tạo nhóm `submission:\{id\}` thay vì phát thông báo cho tất cả máy khách kết nối?
6. `IHubContext<SubmissionHub>` cho phép làm gì mà không cần viết mã trực tiếp trong lớp hub?

7. Cơ chế đăng ký hàm xử lý thông điệp bằng `On<T>` trong máy khách dùng để làm gì? Nếu bỏ bước này, điều gì sẽ xảy ra khi máy chủ gửi thông báo?
8. Trong luồng thực hành, REST được dùng cho những thao tác nào và SignalR được dùng cho những thao tác nào? Vì sao không dùng SignalR để tạo lượt nộp?
9. `WithAutomaticReconnect()` trong `HubConnectionBuilder` có tác dụng gì?
10. Nếu máy khách mất kết nối trong lúc máy chủ gửi thông báo `graded`, khi kết nối lại, máy khách nên làm gì để lấy được trạng thái cuối cùng của lượt nộp?

Bài tập thực hành

1. Chạy solution `content/examples/chapter03/section02/RealtimeSubmissionSystem.slnx`. Ghi lại lệnh biên dịch, lệnh chạy API và lệnh chạy máy khách; kiểm tra máy khách in được danh sách bài tập, mã lượt nộp, thông báo trạng thái từ SignalR và trạng thái cuối cùng đọc lại bằng REST.
2. Trong hàm `SimulateGradingAsync`, thêm trạng thái lỗi `failed`. Điều chỉnh máy khách để in cảnh báo đặc biệt khi nhận trạng thái này.
3. Sửa máy khách để theo dõi đồng thời hai lượt nộp. Kiểm tra xem thông báo của hai lượt nộp có được nhận đúng và không bị trộn lẫn không.
4. Thử chạy máy khách khi API chưa chạy. Quan sát lỗi kết nối SignalR và giải thích vì sao `WithAutomaticReconnect` chưa đủ để bảo đảm máy khách luôn nhận được thông báo.

Câu hỏi thảo luận

1. Với một hệ thống nộp bài có hàng trăm sinh viên cùng theo dõi trạng thái, việc dùng nhóm `submission:\{id\}` có ưu điểm gì so với việc phát thông báo cho tất cả máy khách và để máy khách tự lọc? Cách nào tiết kiệm băng thông hơn?
2. Khi nào nên chọn thăm dò định kỳ thay vì SignalR cho bài toán cập nhật trạng thái? Hãy nêu ít nhất hai tình huống mà thăm dò định kỳ có thể là lựa chọn hợp lý.
3. Nếu hệ thống cần mở rộng để chạy nhiều bản sao API (scale-out), việc dùng SignalR với nhóm trong bộ nhớ sẽ gặp vấn đề gì? Cần thay đổi gì để hỗ trợ kịch bản này?

CHƯƠNG 4
CÁC KIỂU API PHÍA MÁY KHÁCH: REST VÀ GRAPHQL

Mục tiêu chương
Sau khi học xong chương này, người học có thể:

- Nhận diện và phân tích ba hạn chế chính của REST trong truy xuất dữ liệu: truy xuất thừa, truy xuất thiếu và phình phiên bản điểm cuối.
- Giải thích cơ chế hoạt động của GraphQL qua ba thành phần: lược đồ, truy vấn và bộ phân giải.
- Phân biệt ưu điểm và hạn chế của REST và GraphQL, từ đó lựa chọn công nghệ phù hợp với nhu cầu máy khách cụ thể.
- Xây dựng và chạy thử một điểm cuối GraphQL tối giản, gửi truy vấn với tập trường khác nhau và quan sát sự khác biệt về dữ liệu trả về.
- Thiết kế lược đồ GraphQL cho một màn hình thực tế trong mini system EduSubmit.

4.1 Khái niệm và cơ chế hoạt động của GraphQL

Trong hai chương trước, ta đã xây dựng API theo phong cách REST: mỗi tài nguyên có URI riêng, máy khách dùng phương thức HTTP để thao tác, và máy chủ trả về biểu diễn của tài nguyên. Cách làm này đơn giản, rõ ràng và phù hợp với nhiều hệ thống. Tuy nhiên, khi ứng dụng phát triển, giao diện người dùng thường cần các tổ hợp dữ liệu khác nhau cho từng màn hình. Lúc này, REST bộc lộ một số điểm cần cân nhắc, đặc biệt liên quan đến việc truy xuất dữ liệu linh hoạt từ phía máy khách.

GraphQL là một ngôn ngữ truy vấn API, được Facebook phát triển và công bố công khai từ năm 2015. Khác với REST, nơi máy chủ quyết định cấu trúc dữ liệu trả về cho từng điểm cuối, GraphQL cho phép máy khách mô tả chính xác những trường dữ liệu mình cần. Máy chủ GraphQL thường chỉ có một điểm cuối duy nhất, tiếp nhận truy vấn dưới dạng văn bản, phân giải truy vấn đó qua lược đồ đã định nghĩa và trả về đúng những gì máy khách yêu cầu. Chương này giới thiệu GraphQL như một cách tiếp cận khác cho nhóm API phía máy khách, giúp người học có thêm lựa chọn khi thiết kế giao diện giữa máy khách và máy chủ.

4.1.1 Hạn chế của REST trong truy xuất dữ liệu

RESTful API tổ chức dữ liệu xung quanh tài nguyên. Khi máy khách cần dữ liệu từ một tài nguyên, nó gọi một điểm cuối và nhận toàn bộ biểu diễn của tài nguyên đó. Cách làm này phát sinh ba vấn đề chính khi giao diện người dùng ngày càng phức tạp.

Vấn đề thứ nhất là truy xuất thừa dữ liệu (over-fetching). Máy khách gọi `GET/api/assignments/42` và nhận toàn bộ các trường của bài tập: mã, tiêu đề, mô tả, hạn nộp, ngày tạo, trạng thái, danh sách tệp đính kèm và nhiều trường khác. Nhưng màn hình

danh sách trên điện thoại có thể chỉ cần mã và tiêu đề. Dữ liệu thừa không chỉ tốn băng thông mà còn khiến máy khách phải phân tích và bỏ qua những phần không dùng đến.

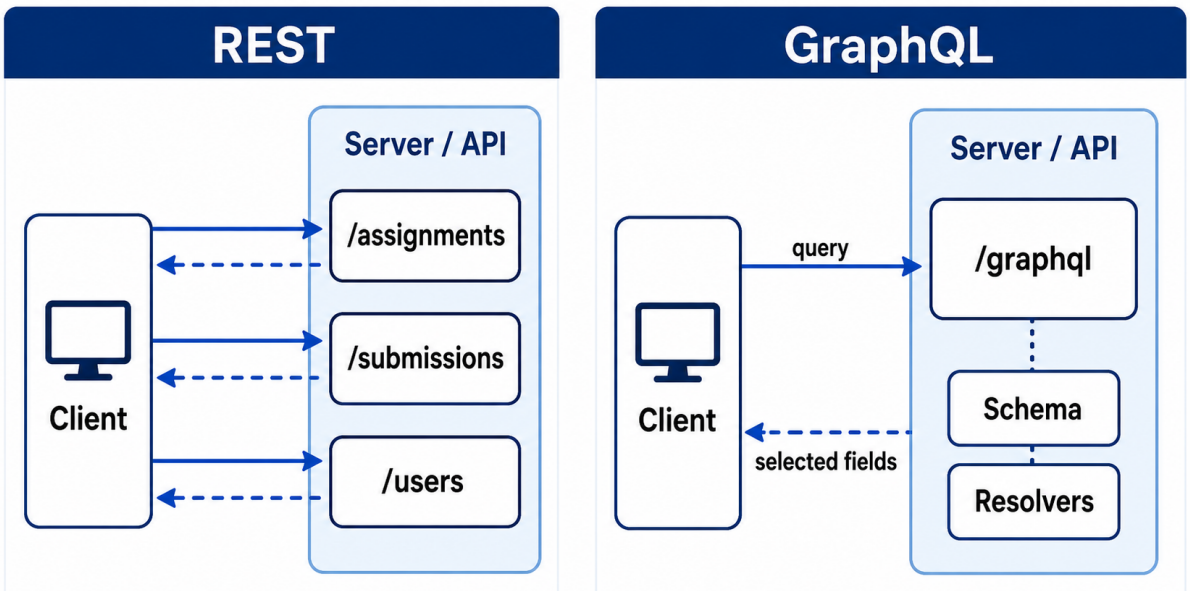
Vấn đề thứ hai là truy xuất thiếu dữ liệu (under-fetching). Một màn hình chi tiết lượt nộp có thể cần hiển thị: thông tin bài tập (từ `/api/assignments`), thông tin lượt nộp (từ `/api/submissions`), kết quả chấm (từ `/api/submissions/\{id\}/result`) và thông tin sinh viên (từ `/api/users`). Để hiển thị một màn hình, máy khách phải gọi nhiều điểm cuối, phối hợp kết quả và xử lý tình huống một trong các lời gọi thất bại. Số lượt đi–về (round trip) tăng lên, độ trễ hiển thị cũng tăng theo.

Vấn đề thứ ba là sự phình phiên bản điểm cuối. Khi giao diện cần một tổ hợp dữ liệu mới, nhóm phát triển API thường phải tạo điểm cuối mới hoặc thêm tham số truy vấn để tùy chỉnh trường trả về. Theo thời gian, số lượng điểm cuối tăng lên, tài liệu API trở nên phức tạp và việc bảo trì trở thành gánh nặng. Máy khách bị phụ thuộc vào nhịp phát triển API phía máy chủ, trong khi nhu cầu hiển thị dữ liệu thường thay đổi nhanh hơn nhu cầu thay đổi logic nghiệp vụ.

Ba vấn đề trên không có nghĩa REST là lựa chọn sai. Chúng chỉ xuất hiện rõ hơn khi ứng dụng có giao diện phức tạp, nhiều màn hình cần tổ hợp dữ liệu khác nhau, hoặc khi băng thông là tài nguyên hạn chế như trên thiết bị di động. Với API nội bộ giữa các dịch vụ, nơi hợp đồng dữ liệu ổn định và ít thay đổi, REST vẫn là lựa chọn tốt.

4.1.2 GraphQL và truy vấn linh hoạt

GraphQL giải quyết các hạn chế trên bằng cách thay đổi mô hình tương tác: thay vì máy chủ định nghĩa nhiều điểm cuối với dữ liệu trả về cố định, máy chủ định nghĩa một lược đồ (schema) mô tả tất cả kiểu dữ liệu và mối quan hệ giữa chúng, rồi cung cấp một điểm cuối duy nhất. Máy khách gửi một truy vấn (query) mô tả chính xác những trường mình cần, và máy chủ trả về đúng những trường đó, không thừa, không thiếu.



Hình 4.1: Khác biệt giữa REST và GraphQL khi máy khách cần dữ liệu từ nhiều tài nguyên

Trọng tâm của GraphQL nằm ở ba khái niệm: lược đồ, truy vấn và bộ phân giải.

Lược đồ định nghĩa các kiểu đối tượng, trường của từng kiểu, tham số truy vấn và các thao tác được phép. Truy vấn là một văn bản do máy khách gửi lên, có cấu trúc lồng phản ánh quan hệ giữa các kiểu dữ liệu. Bộ phân giải (resolver) là hàm phía máy chủ chịu trách nhiệm lấy dữ liệu cho từng trường trong lược đồ. Khi một truy vấn đến, máy chủ GraphQL duyệt qua các trường được yêu cầu và gọi bộ phân giải tương ứng.

Ví dụ, với EduSubmit, lược đồ GraphQL có thể định nghĩa kiểu `Assignment` và kiểu `Submission` như sau:

```
type Assignment {
  id: ID!
  title: String!
  description: String
  dueDate: String
  submissions: [Submission!]!
}

enum SubmissionStatus {
  received
  processing
  graded
  failed
}

type Submission {
  id: ID!
  studentCode: String!
  status: SubmissionStatus!
  submittedAt: String!
}

type Query {
  assignments: [Assignment!]!
  assignment(id: ID!): Assignment
  submissions(studentCode: String): [Submission!]!
}
```

Lược đồ trên dùng Schema Definition Language (SDL), một ngôn ngữ mô tả kiểu độc lập với ngôn ngữ lập trình. Dấu `!` nghĩa là trường không được rỗng; `[Submission!]!` nghĩa là danh sách không rỗng và mỗi phần tử trong danh sách cũng không rỗng. Kiểu `Query` là điểm vào cho các thao tác đọc dữ liệu. Khi có lược đồ, máy khách có thể viết truy vấn để lấy đúng dữ liệu cần cho một màn hình cụ thể. Ví dụ, màn hình danh sách bài tập chỉ cần mã và tiêu đề:

```
query {
  assignments {
    id
    title
  }
}
```

Màn hình chi tiết bài tập cần thêm mô tả và danh sách lượt nộp gần đây:

```
query {
```

```
  assignment(id: "42") {
    title
    description
    submissions {
      id
      studentCode
      status
    }
  }
}
```

Điểm khác biệt quan trọng là cả hai màn hình đều gọi cùng một điểm cuối, dùng cùng một lược đồ, nhưng máy khách tự quyết định lấy trường nào và lồng quan hệ nào. Máy chủ không cần tạo thêm điểm cuối hay tham số tùy chỉnh. Điều này chuyển quyền kiểm soát dữ liệu trả về từ máy chủ sang máy khách.

Về mặt kiến trúc, GraphQL vẫn hoạt động trên nền HTTP và vẫn dùng mô hình yêu cầu–phản hồi. Máy khách thường gửi yêu cầu `POST` tới điểm cuối duy nhất (thường là `/graphql`) với phần thân JSON chứa trường `query`. Máy chủ phân tích truy vấn, kiểm tra tính hợp lệ dựa trên lược đồ, thực thi từng bộ phân giải, gom kết quả thành một đối tượng JSON và trả về. Nếu truy vấn yêu cầu trường không tồn tại hoặc sai kiểu, máy chủ trả lỗi ngay từ bước kiểm tra, trước khi thực thi bộ phân giải.

GraphQL cũng hỗ trợ đột biến (mutation) cho các thao tác thay đổi dữ liệu và đăng ký (subscription) cho các cập nhật thời gian thực. Trong phạm vi chương này, ta tập trung vào truy vấn; đột biến và đăng ký sẽ được đề cập khi cần trong các chương sau.

4.1.3 So sánh REST và GraphQL

REST và GraphQL đều là công cụ xây dựng API phía máy khách. Chúng không loại trừ nhau, và việc chọn công nghệ nào phụ thuộc vào đặc điểm của ứng dụng. Bảng 4.1 so sánh hai cách tiếp cận trên một số tiêu chí chính.

Về truy xuất dữ liệu, GraphQL giải quyết tốt bài toán over-fetching và under-fetching. Một truy vấn GraphQL có thể lấy đúng dữ liệu cho một màn hình, không thừa trường và không cần nhiều lượt đi–về. Tuy nhiên, sự linh hoạt này đi kèm với chi phí: máy chủ GraphQL phải phân tích và kiểm tra truy vấn, xây dựng kế hoạch thực thi, gọi nhiều bộ phân giải và gom kết quả. Với truy vấn lồng sâu hoặc quan hệ phức tạp, việc thực thi có thể tạo ra nhiều truy vấn cơ sở dữ liệu con, dẫn đến vấn đề N+1: một truy vấn cha trả về N đối tượng, sau đó mỗi đối tượng lại kích hoạt một truy vấn con riêng, tổng cộng N+1 lượt truy vấn cơ sở dữ liệu. Cần có công cụ như DataLoader để gom nhiều yêu cầu dữ liệu cùng loại thành một truy vấn cơ sở dữ liệu duy nhất.

Về bộ nhớ đệm, REST có lợi thế nhờ tính chất tài nguyên và phương thức an toàn của `GET`. Một yêu cầu `GET /api/assignments` có thể được lưu đệm ở nhiều tầng: trình duyệt, mạng phân phối nội dung (Content Delivery Network – CDN), proxy ngược hoặc bộ nhớ đệm phía máy chủ. GraphQL thường dùng `POST` để gửi truy vấn, khiến các cơ chế bộ nhớ đệm HTTP chuẩn khó áp dụng. Bù lại, các thư viện máy khách GraphQL như Apollo Client hoặc Relay có cơ chế bộ nhớ đệm riêng, hoạt động ở cấp đối tượng và trường thay vì cấp URL.

Về tổ chức nhóm phát triển, REST thường phù hợp khi ranh giới giữa máy khách và máy chủ rõ ràng, hai bên phát triển tương đối độc lập và hợp đồng API ít thay đổi.

Bảng 4.1: So sánh REST và GraphQL trong API phía máy khách

Tiêu chí	REST	GraphQL
Điểm cuối	Nhiều điểm cuối, mỗi điểm cuối gắn với một tài nguyên hoặc tập tài nguyên	Một điểm cuối duy nhất cho mọi truy vấn
Cấu trúc dữ liệu trả về	Do máy chủ quyết định cho từng điểm cuối	Do máy khách chỉ định trong truy vấn
Quan hệ giữa các tài nguyên	Cần gọi nhiều điểm cuối hoặc nhúng tài nguyên con bằng tham số	Lồng trực tiếp trong truy vấn
Phiên bản API	Thường dùng tiền tố phiên bản như `/v2/` hoặc tham số phiên bản	Không cần phiên bản URI; lược đồ có thể thêm trường mới và đánh dấu trường cũ là không dùng nữa
Bộ nhớ đệm HTTP	Tận dụng được cơ chế bộ nhớ đệm chuẩn của HTTP (`ETag`, `Cache-Control`)	Hầu hết yêu cầu dùng `POST` nên khó tận dụng bộ nhớ đệm HTTP; cần cơ chế riêng ở tầng máy khách
Công cụ kiểm thử	Postman, curl, tệp `.http`, Swagger UI	GraphiQL, GraphQL Playground, Apollo Studio
Đường cong học tập	Thấp hơn với người mới làm quen API web	Cần học SDL, cú pháp truy vấn và tư duy lược đồ

GraphQL phù hợp khi nhóm máy khách cần truy vấn linh hoạt, giao diện thay đổi thường xuyên hoặc khi một API phục vụ nhiều loại máy khách khác nhau (web, di động, ứng dụng bên thứ ba).

Không có câu trả lời tuyệt đối cho câu hỏi "REST hay GraphQL". Hãy bắt đầu bằng việc liệt kê các màn hình hoặc ca sử dụng mà máy khách cần, xem chúng cần những tổ hợp dữ liệu nào, tần suất thay đổi ra sao, và máy khách có đủ năng lực xử lý bộ nhớ đệm phía mình hay không. Câu trả lời sẽ khác nhau giữa một ứng dụng quản lý nội bộ, một ứng dụng di động cho người dùng cuối và một API công khai cho đối tác.

4.1.4 Lựa chọn API theo nhu cầu máy khách

Sau khi đã hiểu cả REST và GraphQL, câu hỏi tiếp theo là khi nào nên chọn công nghệ nào. Việc lựa chọn không nên dựa trên xu hướng hay sở thích cá nhân, mà nên dựa trên phân tích nhu cầu thực tế của phía máy khách.

Nên ưu tiên REST khi:

- Tài nguyên nghiệp vụ có ranh giới rõ, ít thay đổi theo thời gian.
- Máy khách chỉ cần một tập dữ liệu cố định cho mỗi điểm cuối, ít khi cần tổ hợp dữ liệu khác thường.
- Hệ thống cần tận dụng bộ nhớ đệm HTTP ở nhiều tầng (CDN, proxy ngược).
- API được dùng bởi bên thứ ba và cần một hợp đồng đơn giản, dễ viết tài liệu.
- Nhóm phát triển đã quen với REST và chưa có nhu cầu truy vấn linh hoạt.

Nên ưu tiên GraphQL khi:

- Giao diện có nhiều màn hình, mỗi màn hình cần tổ hợp dữ liệu khác nhau từ nhiều

tài nguyên.
- Ứng dụng chạy trên thiết bị di động với băng thông hạn chế, cần giảm thiểu dữ liệu thừa và số lượt đi–về.
- Nhiều loại máy khách (web, iOS, Android) cùng dùng một API nhưng có nhu cầu dữ liệu khác nhau.
- Dữ liệu có nhiều quan hệ và máy khách thường cần dữ liệu lồng.
- Nhóm phát triển sẵn sàng đầu tư vào việc học lược đồ, bộ phân giải và công cụ hệ sinh thái GraphQL.

Trong thực tế, hai cách tiếp cận không nhất thiết loại trừ nhau. Một hệ thống có thể dùng REST cho các điểm cuối công khai, đơn giản, cần bộ nhớ đệm tốt (như danh sách bài tập, thông tin khóa học), đồng thời dùng GraphQL cho các màn hình phức tạp cần tổ hợp dữ liệu linh hoạt (như bảng điều khiển giảng viên, trang chi tiết sinh viên). Cả hai có thể cùng chạy trên một ứng dụng ASP.NET Core: REST thông qua bộ điều khiển hoặc minimal API, GraphQL thông qua thư viện như Hot Chocolate hoặc GraphQL.NET.

Với EduSubmit, REST vẫn là lựa chọn chính cho các thao tác cốt lõi: tạo bài tập, nộp bài, đọc trạng thái lượt nộp. Các thao tác này có hợp đồng rõ, ít thay đổi và cần bộ nhớ đệm ở một số điểm cuối đọc. GraphQL có thể được xem xét khi mở rộng hệ thống với bảng điều khiển giảng viên, nơi một màn hình cần hiển thị đồng thời: số lượt nộp theo bài tập, tỉ lệ hoàn thành, danh sách sinh viên trễ hạn và biểu đồ phân bố điểm.

Khi lựa chọn, cũng cần cân nhắc đến chi phí vận hành. Một máy chủ GraphQL cần thêm thời gian tính toán để phân tích và thực thi truy vấn so với một điểm cuối REST đơn giản. Với ứng dụng nhỏ, sự khác biệt này không đáng kể. Với ứng dụng lớn, cần có chiến lược giới hạn độ sâu truy vấn, giới hạn độ phức tạp và bộ nhớ đệm ở tầng bộ phân giải để tránh truy vấn quá tốn kém làm chậm máy chủ.

Tóm lại, mục tiêu của chương này không phải là khẳng định công nghệ nào tốt hơn, mà là trang bị cho người học khả năng phân tích nhu cầu máy khách, nhận diện ưu điểm và hạn chế của từng cách tiếp cận, từ đó đưa ra lựa chọn phù hợp với bài toán cụ thể. Phần thực hành tiếp theo sẽ hướng dẫn xây dựng một điểm cuối GraphQL đơn giản để người học trải nghiệm trực tiếp sự khác biệt giữa hai mô hình.

4.2 Kỹ thuật lập trình cơ bản với GraphQL

Phần 4.1 đã trình bày các hạn chế của REST trong truy xuất dữ liệu, mô hình truy vấn linh hoạt của GraphQL và các thành phần cốt lõi: lược đồ, truy vấn, bộ phân giải. Phần này chuyển từ nguyên tắc sang kỹ thuật lập trình: cách khai báo lược đồ và kiểu dữ liệu bằng SDL, cách xây dựng truy vấn và bộ phân giải, cách sử dụng tham số và lựa chọn trường, cách tiếp nhận yêu cầu và trả về phản hồi GraphQL, và cách máy khách gửi truy vấn và xử lý lỗi.

Để tập trung vào kỹ thuật, phần này dùng một ví dụ tối giản: BookCatalog – một máy chủ GraphQL nhận truy vấn từ máy khách và trả về dữ liệu sách với đúng các trường được yêu cầu. Mã nguồn đầy đủ nằm trong thư mục `content/examples/chapter04/section02/`. Sau khi nắm được các kỹ thuật cơ bản qua ví dụ này, sinh viên sẽ áp dụng chúng để xây dựng điểm cuối GraphQL cho EduSubmit trong phần 4.3.

4.2.1 Khai báo schema và các kiểu dữ liệu GraphQL

Trong hệ sinh thái GraphQL, lược đồ (schema) đóng vai trò là hợp đồng dữ liệu giữa máy khách và máy chủ. Lược đồ được viết bằng Schema Definition Language (SDL), một ngôn ngữ mô tả kiểu độc lập với ngôn ngữ lập trình. Mọi điểm cuối GraphQL đều bắt đầu từ một lược đồ: nó định nghĩa những kiểu đối tượng nào tồn tại, mỗi kiểu có những trường gì, kiểu dữ liệu của từng trường là gì, và những thao tác nào (query, mutation) được phép thực hiện.

Với ví dụ BookCatalog, lược đồ GraphQL có thể được mô tả bằng SDL như sau:

```
type Book {
  id: ID!
  title: String!
  author: String!
  year: Int
  genre: String
}

type Query {
  books: [Book!]!
}
```

Lược đồ trên định nghĩa hai thứ. Thứ nhất, kiểu `Book` với năm trường: `id` (kiểu `ID!`), `title` (`String!`), `author` (`String!`), `year` (`Int`) và `genre` (`String`). Dấu `!` biểu thị trường không được phép rỗng (non-nullable). Thứ hai, kiểu `Query` định nghĩa điểm vào cho thao tác đọc: trường `books` trả về một danh sách các đối tượng `Book`, trong đó cả danh sách lẫn từng phần tử đều không được rỗng (`[Book!]!`).

Trong mã nguồn C#, kiểu `Book` được khai báo dưới dạng `record` – một cấu trúc dữ liệu bất biến, phù hợp để biểu diễn đối tượng miền:

```
record Book(int Id, string Title, string Author, int Year, string Genre);
```

Đây là cầu nối giữa lược đồ GraphQL (mô tả bằng SDL) và mã nguồn phía máy chủ (cài đặt bằng C#). Trong thực tế, các thư viện như Hot Chocolate có thể tự động sinh lược đồ GraphQL từ các kiểu C# thông qua cơ chế phản chiếu (reflection), giúp tránh việc phải khai báo hai lần. Tuy nhiên, trong phạm vi học tập của chương này, ta tường minh cả hai để sinh viên hiểu rõ mối quan hệ giữa SDL và mã nguồn.

4.2.2 Xây dựng query, mutation và resolver

Khi lược đồ đã được định nghĩa, bước tiếp theo là cài đặt bộ phân giải (resolver) – hàm chịu trách nhiệm lấy dữ liệu cho từng trường. Trong một máy chủ GraphQL hoàn chỉnh, mỗi trường trong lược đồ có thể có bộ phân giải riêng. Trong ví dụ tối giản BookCatalog, bộ phân giải được đơn giản hóa thành hai hàm: một hàm trích xuất danh sách trường từ truy vấn, và một hàm định hình đối tượng kết quả chỉ với các trường được yêu cầu.

Hàm `ShapeBook` đóng vai trò bộ phân giải ở mức đối tượng: với mỗi `Book` trong kho dữ liệu, hàm tạo một từ điển chỉ chứa các trường máy khách yêu cầu. Cơ chế `switch` ánh xạ tên trường (dạng chuỗi trong truy vấn GraphQL) sang giá trị thực tế của đối tượng:

```
static Dictionary<string, object?> ShapeBook(Book book, List<string> fields)
{
    var shaped = new Dictionary<string, object?>();
    foreach (var field in fields)
    {
        shaped[field] = field switch
        {
            "id" => book.Id,
            "title" => book.Title,
            "author" => book.Author,
            "year" => book.Year,
            "genre" => book.Genre,
            _ => null
        };
    }
    return shaped;
}
```

Hàm `ExecuteQuery` điều phối toàn bộ quá trình: kiểm tra truy vấn, trích xuất trường, kiểm tra tính hợp lệ và gọi `ShapeBook` cho từng cuốn sách. Đây là mô hình thu nhỏ của những gì một công cụ GraphQL thực thụ làm khi thực thi truy vấn. Trong thư viện Hot Chocolate hoặc GraphQL.NET, các bước này được tự động hóa: bộ phân tích cú pháp đọc truy vấn, bộ kiểm tra đối chiếu với lược đồ, và bộ thực thi gọi lần lượt từng bộ phân giải theo cây truy vấn.

Về mutation, trong phạm vi chương này ta chưa cài đặt mutation vì mục tiêu là minh họa cơ chế truy vấn linh hoạt – điểm khác biệt chính giữa GraphQL và REST. Mutation trong GraphQL cũng tuân theo mô hình tương tự: khai báo trong lược đồ dưới kiểu `Mutation`, nhận tham số đầu vào, gọi bộ phân giải tương ứng và trả về kiểu dữ liệu đã định nghĩa. Sinh viên có thể tham khảo thêm về mutation trong tài liệu chính thức của GraphQL hoặc các thư viện như Hot Chocolate.

4.2.3 Sử dụng argument, variable và lựa chọn field

Một trong những khả năng quan trọng nhất của GraphQL là máy khách được quyền chỉ định chính xác những trường mình cần – không thừa, không thiếu. Cơ chế này được gọi là field selection. Khi máy khách gửi truy vấn `\{books\{idtitle\}\}`, máy chủ chỉ trả về `id` và `title` cho mỗi cuốn sách, mặc dù đối tượng `Book` trong bộ nhớ còn có `author`, `year` và `genre`.

Trong ví dụ BookCatalog, hàm `ParseBookFields` chịu trách nhiệm trích xuất danh sách trường từ văn bản truy vấn:

```
static List<string> ParseBookFields(string query)
{
    var normalized = Regex.Replace(query, @"\s+", " ");
    var match = Regex.Match(
        normalized,
        @"books\s*\{\s*(?<fields>[A-Za-z\s]+?)\s*\}",
        RegexOptions.IgnoreCase);

    if (!match.Success)
        return [];
```

```
    return match.Groups["fields"].Value
        .Split(' ', StringSplitOptions.RemoveEmptyEntries |
        ↪  StringSplitOptions.TrimEntries)
        .Distinct(StringComparer.Ordinal)
        .ToList();
}
```

Hàm này dùng biểu thức chính quy để bắt phần nằm trong cặp ngoặc nhọn sau từ khóa `books`, sau đó tách thành danh sách tên trường và loại bỏ trùng lặp. Đây là cách làm tối giản để minh họa nguyên lý; máy chủ GraphQL thực tế dùng bộ phân tích cú pháp (parser) đúng chuẩn, có khả năng xử lý truy vấn lồng, bí danh (alias), phân mảnh (fragment) và nhiều cấu trúc phức tạp khác.

GraphQL còn hỗ trợ tham số (argument) để lọc hoặc tùy chỉnh dữ liệu ngay trong truy vấn. Ví dụ, nếu lược đồ định nghĩa `book(id:ID!):Book`, máy khách có thể truy vấn một cuốn sách cụ thể:

```
query {
  book(id: "3") {
    title
    author
  }
}
```

Tham số `id` được truyền trực tiếp trong truy vấn. Ngoài ra, GraphQL còn hỗ trợ biến (variable) để tách tham số khỏi truy vấn, giúp truy vấn tái sử dụng được và tránh phải nối chuỗi để tạo truy vấn động. Biến được khai báo sau tên thao tác và truyền giá trị trong một đối tượng JSON riêng có tên `variables`. Ví dụ BookCatalog trong phần này chưa dùng biến để giữ mã nguồn ngắn gọn, nhưng đây là kỹ thuật quan trọng khi xây dựng ứng dụng thực tế.

4.2.4 Tiếp nhận request và trả về response GraphQL

Khác với REST nơi mỗi tài nguyên có URI riêng, máy chủ GraphQL thường chỉ có một điểm cuối duy nhất – thường là `POST/graphql`. Mọi truy vấn, dù cần dữ liệu gì, đều gửi tới cùng một địa chỉ này. Máy chủ đọc trường `query` trong thân yêu cầu, thực thi và trả về kết quả.

Trong ví dụ BookCatalog, toàn bộ mã nguồn máy chủ được đặt trong một tệp `Program.cs` duy nhất để sinh viên dễ theo dõi toàn bộ luồng xử lý. Tệp này được trình bày trong Listing 4.1.

Mã nguồn 4.1: BookGraphqlApi – ví dụ tối giản minh họa khai báo kiểu dữ liệu, phân giải truy vấn, định hình phản hồi và xử lý lỗi trong GraphQL

```
using System.Text.RegularExpressions;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// 4.2.1: Du lieu mau - kho sach trong bo nho
var books = new List<Book>
```

```
{
    new(1, "C# in Depth", "Jon Skeet", 2019, "Programming"),
    new(2, "Designing Data-Intensive Applications", "Martin Kleppmann", 2017,
    ↪  "Architecture"),
    new(3, "The Pragmatic Programmer", "David Thomas", 2019, "Engineering"),
    new(4, "Clean Code", "Robert C. Martin", 2008, "Engineering"),
    new(5, "Learning GraphQL", "Eve Porcello", 2018, "Web")
};

app.MapGet("/", () => Results.Ok(new { service = "BookGraphqlApi", graphqlEndpoint =
↪  "/graphql" }));

// 4.2.4: Diem cuoi GraphQL duy nhat - nhan truy van, phan giai, tra ket qua
app.MapPost("/graphql", (GraphqlRequest request) =>
{
    var response = ExecuteQuery(request.Query, books);
    return response.HasErrors
        ? Results.BadRequest(response)
        : Results.Ok(response);
});

app.Run();

// 4.2.2 & 4.2.3: Phan giai truy van - doc danh sach truong, kiem tra hop le, tao du
↪  lieu dinh hinh
static GraphqlResponse ExecuteQuery(string? query, List<Book> books)
{
    if (string.IsNullOrWhiteSpace(query))
        return GraphqlResponse.FromError("Truy van GraphQL khong duoc rong.");

    var requestedFields = ParseBookFields(query);
    if (requestedFields.Count == 0)
        return GraphqlResponse.FromError("Truy van can co dang: query { books { id
        ↪  title } }.");

    var supportedFields = new HashSet<string>(StringComparer.Ordinal)
        { "id", "title", "author", "year", "genre" };

    var unsupported = requestedFields
        .Where(f => !supportedFields.Contains(f))
        .ToArray();

    if (unsupported.Length > 0)
        return GraphqlResponse.FromError(
            $"Truong khong ho tro: {string.Join(", ", unsupported)}. " +
            $"Cac truong ho tro: {string.Join(", ", supportedFields)}");

    var shaped = books
        .Select(b => ShapeBook(b, requestedFields))
        .ToArray();

    return GraphqlResponse.FromData(new Dictionary<string, object?>
    {
        ["books"] = shaped
    });
}

// 4.2.2 & 4.2.3: Trich xuat danh sach truong tu truy van dang query { books { ... } }
static List<string> ParseBookFields(string query)
```

```
{
    var normalized = Regex.Replace(query, @"\s+", " ");
    var match = Regex.Match(
        normalized,
        @"books\s*\{\s*(?<fields>[A-Za-z\s]+?)\s*\}",
        RegexOptions.IgnoreCase);

    if (!match.Success)
        return [];

    return match.Groups["fields"].Value
        .Split(' ', StringSplitOptions.RemoveEmptyEntries |
        ↪  StringSplitOptions.TrimEntries)
        .Distinct(StringComparer.Ordinal)
        .ToList();
}

// 4.2.2: Dinh hinh doi tuong ket qua – chi giu lai cac truong may khach yeu cau
static Dictionary<string, object?> ShapeBook(Book book, List<string> fields)
{
    var shaped = new Dictionary<string, object?>();
    foreach (var field in fields)
    {
        shaped[field] = field switch
        {
            "id" => book.Id,
            "title" => book.Title,
            "author" => book.Author,
            "year" => book.Year,
            "genre" => book.Genre,
            _ => null
        };
    }
    return shaped;
}

// 4.2.1: Kieu du lieu mien – tuong ung voi type Book trong luoc do GraphQL
record Book(int Id, string Title, string Author, int Year, string Genre);

// 4.2.4: Kieu du lieu nhan truy van tu may khach
record GraphqlRequest(string Query);

// 4.2.4 & 4.2.5: Kieu phan hoi GraphQL – chua data khi thanh cong, errors khi that bai
sealed record GraphqlResponse(
    Dictionary<string, object?>? Data,
    string[]? Errors)
{
    public bool HasErrors => Errors is { Length: > 0 };

    public static GraphqlResponse FromData(Dictionary<string, object?> data)
        => new(data, null);

    public static GraphqlResponse FromError(string message)
        => new(null, [message]);
}
```

Luồng xử lý trong Listing 4.1 gồm bốn bước. Bước thứ nhất, `MapPost("/graphql", ...)` tiếp nhận yêu cầu HTTP POST với phần thân được deserialize thành `GraphqlRequest`

– một `record` chỉ chứa trường `Query`. Bước thứ hai, `ExecuteQuery` kiểm tra truy vấn không được rỗng, gọi `ParseBookFields` để trích xuất danh sách trường và đối chiếu với tập trường được hỗ trợ. Bước thứ ba, nếu có trường không hợp lệ, máy chủ trả về `GraphqlResponse` chứa mảng `Errors` và HTTP `400BadRequest`. Bước thứ tư, nếu truy vấn hợp lệ, máy chủ gọi `ShapeBook` cho từng cuốn sách, gom kết quả vào đối tượng `Data` với khóa `books` và trả về HTTP `200OK`.

Cấu trúc phản hồi GraphQL được mô hình hóa bởi kiểu `GraphqlResponse`:

```
sealed record GraphqlResponse(
    Dictionary<string, object?>? Data,
    string[]? Errors)
{
    public bool HasErrors => Errors is { Length: > 0 };
    // ...
}
```

Đây là quy ước quan trọng của GraphQL: phản hồi luôn có một trong hai – `data` khi thành công, hoặc `errors` khi thất bại. Trong thực tế, một phản hồi có thể chứa đồng thời cả `data` và `errors`: ví dụ khi truy vấn yêu cầu nhiều trường và một vài trường bị lỗi trong lúc phân giải, các trường còn lại vẫn có thể có dữ liệu. Quy ước này giúp máy khách xử lý nhất quán mọi phản hồi từ máy chủ GraphQL.

4.2.5 Xử lý dữ liệu và lỗi ở phía client GraphQL

Phía máy khách, việc giao tiếp với máy chủ GraphQL bắt đầu bằng `HttpClient` – không khác biệt so với REST. Điểm khác biệt nằm ở nội dung gửi đi: thay vì một đối tượng DTO cụ thể, máy khách gửi một văn bản truy vấn (query string) mô tả hình dạng dữ liệu mong muốn.

Toàn bộ mã nguồn máy khách BookCatalog được trình bày trong Listing 4.2.

Mã nguồn 4.2: BookGraphqlClient – máy khách tối giản gửi truy vấn GraphQL với các tập trường khác nhau và xử lý lỗi

```
using System.Net.Http.Json;
using System.Text.Json;

var apiBaseUrl = args.Length > 0 ? args[0].TrimEnd('/') : "http://localhost:5200";

using var http = new HttpClient { BaseAddress = new Uri(apiBaseUrl) };

// 4.2.3 & 4.2.5: Truy van 1 – danh sach sach dang rut gon, chi can id va title
var compactQuery = """
query {
  books {
    id
    title
  }
}
""";

// 4.2.3 & 4.2.5: Truy van 2 – danh sach day du, kem author, year va genre
var fullQuery = """
query {
```

```
  books {
    id
    title
    author
    year
    genre
  }
}
""";

// 4.2.5: Truy van 3 - chua truong khong duoc ho tro de kiem tra xu ly loi
var invalidQuery = """
query {
  books {
    id
    price
  }
}
""";

await SendQueryAsync(http, "Truy van rut gon - chi can id va title", compactQuery);
await SendQueryAsync(http, "Truy van day du - tat ca cac truong", fullQuery);
await SendQueryAsync(http, "Truy van loi - yeu cau truong khong ho tro", invalidQuery);

// 4.2.5: Gui truy van, doc phan hoi va xu ly loi
static async Task SendQueryAsync(HttpClient http, string title, string query)
{
    Console.WriteLine();
    Console.WriteLine(title);
    Console.WriteLine(new string('-', title.Length));
    Console.WriteLine(query);

    var response = await http.PostAsJsonAsync("/graphql", new GraphqlRequest(query));
    var body = await response.Content.ReadFromJsonAsync<GraphqlResponse>();

    if (!response.IsSuccessStatusCode)
    {
        Console.WriteLine($"Loi {(int)response.StatusCode}: {string.Join(", ",
        ↪  body?.Errors ?? [])}");
        return;
    }

    var json = JsonSerializer.Serialize(
        body?.Data,
        new JsonSerializerOptions { WriteIndented = true });

    Console.WriteLine(json);
}

record GraphqlRequest(string Query);

record GraphqlResponse(JsonElement? Data, string[]? Errors);
```

Máy khách trong Listing 4.2 gửi ba truy vấn để minh họa các tình huống khác nhau. Truy vấn thứ nhất chỉ yêu cầu `id` và `title` – phù hợp với màn hình danh sách rút gọn trên thiết bị di động, nơi không gian hiển thị hạn chế và không cần biết tác giả hay năm xuất bản. Truy vấn thứ hai yêu cầu tất cả năm trường – phù hợp với màn hình chi tiết

hoặc trang quản trị. Truy vấn thứ ba cố ý yêu cầu trường `price` – một trường không tồn tại trong lược đồ – để minh họa cách máy chủ từ chối và máy khách xử lý lỗi.

Hàm `SendQueryAsync` gói gọn logic gửi truy vấn và đọc phản hồi. Hàm dùng `PostAsJsonAsync` để gửi yêu cầu POST tới `/graphql`, sau đó đọc phản hồi dưới dạng `GraphqlResponse`. Nếu mã trạng thái không phải thành công, hàm trích xuất mảng `Errors` và hiển thị. Nếu thành công, hàm truy cập `Data` và in ra dưới dạng JSON có định dạng. Điểm đáng chú ý là `Data` được khai báo với kiểu `JsonElement?` – một kiểu linh hoạt cho phép đọc bất kỳ cấu trúc JSON nào mà không cần deserialize thành một lớp C# cụ thể. Điều này phản ánh bản chất của GraphQL: hình dạng dữ liệu trả về do máy khách quyết định, nên không thể định nghĩa trước một kiểu DTO cố định.

Khi chạy ví dụ, sinh viên sẽ quan sát thấy hai truy vấn đầu trả về các tập trường khác nhau dù cùng gọi `POST/graphql`. Đây chính là điểm khác biệt cốt lõi giữa GraphQL và REST: với REST, để có hai hình dạng dữ liệu khác nhau, thường cần hai điểm cuối khác nhau hoặc tham số truy vấn; với GraphQL, máy khách chỉ cần thay đổi danh sách trường trong truy vấn.

Tóm lại, kỹ thuật lập trình GraphQL xoay quanh năm trụ cột: khai báo lược đồ bằng SDL để định nghĩa hợp đồng dữ liệu, xây dựng bộ phân giải để cung cấp dữ liệu cho từng trường, cho phép máy khách chọn trường để tránh truy xuất thừa hoặc thiếu dữ liệu, thiết kế điểm cuối duy nhất tiếp nhận và thực thi truy vấn, và xây dựng máy khách gửi truy vấn có chọn lọc trường cùng với xử lý lỗi. Phần tiếp theo sẽ áp dụng toàn bộ các kỹ thuật này để xây dựng điểm cuối GraphQL trong hệ thống EduSubmit, nơi máy khách có thể truy vấn danh sách bài tập với các trường khác nhau tùy theo màn hình hiển thị.

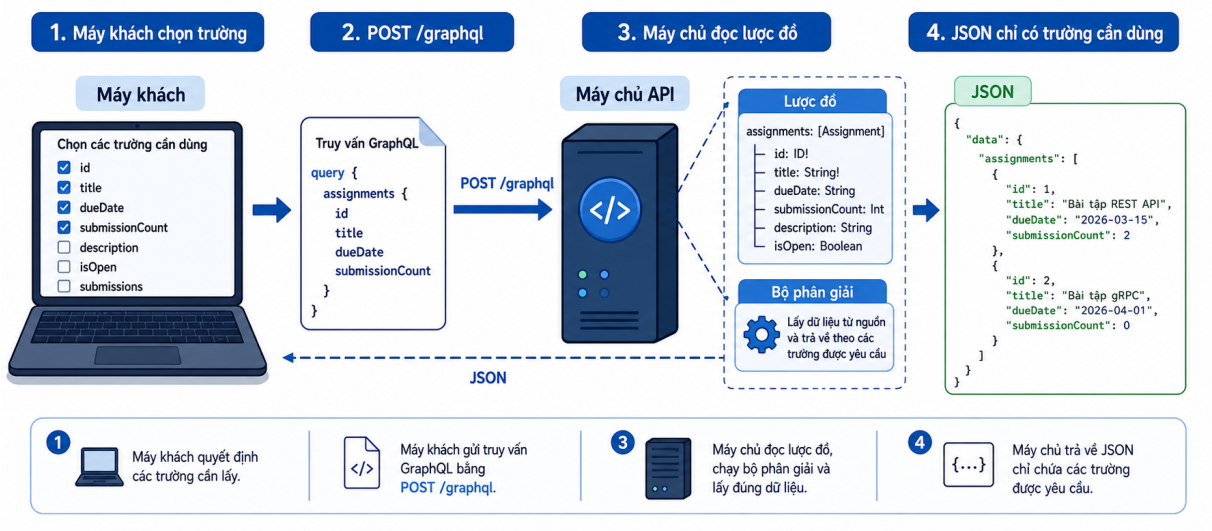
4.3 [Ứng dụng] Xây dựng máy khách đơn giản với GraphQL

Ở phần trước, ta đã thấy GraphQL hữu ích khi máy khách cần quyết định chính xác dữ liệu nào sẽ được trả về. Phần thực hành này đưa ý tưởng đó vào EduSubmit bằng một lát cắt nhỏ: máy chủ cung cấp một điểm cuối `POST/graphql`, còn máy khách gửi truy vấn để lấy danh sách bài tập với các trường khác nhau tùy theo màn hình.

Ví dụ nguồn của bài này nằm trong thư mục `content/examples/chapter04/section02`. Cấu trúc dự án gồm hai tiến trình độc lập:

```
examples/chapter04/section03/
  GraphqlSubmissionSystem.slnx
  src/
    Submission.GraphqlApi/
    Submission.GraphqlClient/
```

Để giữ bài thực hành tập trung, ví dụ chỉ cài đặt một tập con rất nhỏ của GraphQL: nhận một truy vấn dạng `assignments`, đọc danh sách trường được yêu cầu, rồi trả về JSON chỉ chứa các trường đó. Trong hệ thống thật, ta không nên tự viết bộ phân tích GraphQL như ví dụ này; nên dùng thư viện hoàn chỉnh như Hot Chocolate hoặc GraphQL.NET để có lược đồ, kiểm tra kiểu, bộ phân giải, đột biến, phân trang, bảo mật và công cụ gỡ lỗi tốt hơn. Mục tiêu ở đây là giúp sinh viên nhìn thấy rõ cơ chế "máy khách chọn trường" trước khi dùng thư viện lớn.



Hình 4.2: Luồng truy vấn GraphQL đơn giản trong EduSubmit

4.3.1 *Tạo điểm cuối GraphQL đơn giản*

Điểm cuối GraphQL của ví dụ này nhận yêu cầu HTTP `POST` với phần thân JSON có trường `query`. Máy chủ không định nghĩa nhiều điểm cuối như `/api/assignments` hay `/api/assignments/open`; thay vào đó, cùng một điểm cuối `/graphql` có thể trả về các hình dạng dữ liệu khác nhau dựa trên truy vấn của máy khách.

Tệp `Program.cs` của API thiết lập dữ liệu mẫu, tạo bộ thực thi truy vấn và ánh xạ điểm cuối `/graphql`:

Hình 4.3: Các bước chính khi viết mã điểm cuối GraphQL trong Listing 4.1

Mã nguồn 4.3: Submission.GraphqlApi – điểm cuối GraphQL đơn giản cho EduSubmit

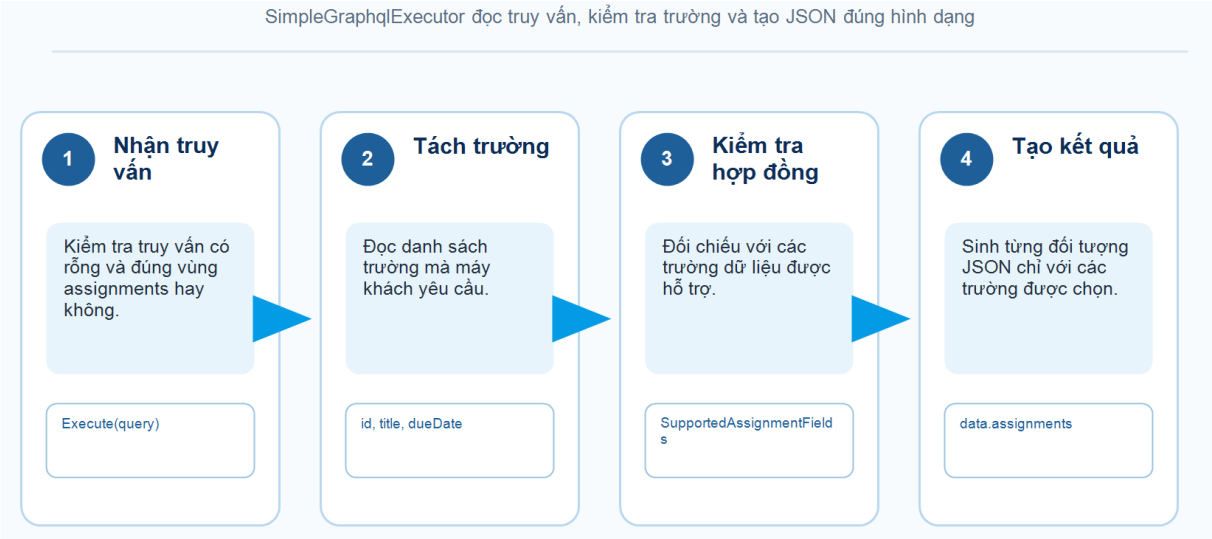
```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var assignments = new List<Assignment>
```

```
{
    new(
        Id: 1,
        Title: "Bai tap REST API",
        Description: "Xay dung API quan ly bai tap va luot nop.",
        DueDate: new DateOnly(2026, 3, 15),
        IsOpen: true,
        Submissions:
        [
            new Submission("SV001", "received"),
            new Submission("SV002", "graded")
        ]),
    new(
        Id: 2,
        Title: "Bai tap SignalR",
        Description: "Nhan thong bao trang thai bai nop theo thoi gian thuc.",
        DueDate: new DateOnly(2026, 3, 22),
        IsOpen: true,
        Submissions:
        [
            new Submission("SV001", "processing")
        ]),
    new(
        Id: 3,
        Title: "Bai tap GraphQL",
        Description: "Truy van dung cac truong ma man hinh can hien thi.",
        DueDate: new DateOnly(2026, 3, 29),
        IsOpen: false,
        Submissions: [])
};

var executor = new SimpleGraphqlExecutor(assignments);

app.MapGet("/", () => Results.Ok(new {
    service = "Submission.GraphqlApi",
    graphqlEndpoint = "/graphql"
}));

app.MapPost("/graphql", (GraphqlRequest request) => {
    var response = executor.Execute(request.Query);
    return response.HasErrors
        ? Results.BadRequest(response)
        : Results.Ok(response);
});

app.Run();

record GraphqlRequest(string Query);

record Assignment(
    int Id,
    string Title,
    string Description,
    DateOnly DueDate,
    bool IsOpen,
    List<Submission> Submissions);

record Submission(
    string StudentCode,
```

```
    string Status);
```

Đoạn mã trên có ba vai trò chính. Thứ nhất, danh sách `assignments` đóng vai trò kho dữ liệu trong bộ nhớ cho EduSubmit. Mỗi bài tập có tiêu đề, mô tả, hạn nộp, trạng thái mở nộp và danh sách lượt nộp. Thứ hai, `SimpleGraphqlExecutor` được tạo một lần và nhận danh sách bài tập để phục vụ truy vấn. Thứ ba, `MapPost("/graphql", ...)` là điểm vào duy nhất của máy khách GraphQL. Nếu truy vấn hợp lệ, API trả `200OK`; nếu truy vấn rỗng hoặc yêu cầu trường chưa hỗ trợ, API trả `400BadRequest` cùng mảng lỗi.

Phần xử lý truy vấn nằm trong tệp `SimpleGraphqlExecutor.cs`. Đây không phải là bộ máy GraphQL đầy đủ, mà chỉ là một phiên bản học tập để minh họa cách máy chủ đọc các trường được yêu cầu rồi tạo phản hồi đúng hình dạng.



Hình 4.4: Các bước chính khi viết mã bộ thực thi truy vấn chọn trường trong Listing 4.2

Mã nguồn 4.4: SimpleGraphqlExecutor – bộ thực thi truy vấn GraphQL tối giản minh họa cơ chế chọn trường

```
using System.Text.RegularExpressions;

sealed class SimpleGraphqlExecutor {
    private static readonly string[] SupportedAssignmentFields =
    [
        "id",
        "title",
        "description",
        "dueDate",
        "isOpen",
        "submissionCount"
    ];

    private readonly IReadOnlyList<Assignment> assignments;

    public SimpleGraphqlExecutor(IReadOnlyList<Assignment> assignments) {
        this.assignments = assignments;
    }

    public GraphqlResponse Execute(string? query) {
```

```
        if (string.IsNullOrWhiteSpace(query)) {
            return GraphqlResponse.FromError("Truy van GraphQL khong duoc rong.");
        }

        var requestedFields = ParseAssignmentFields(query);
        if (requestedFields.Count == 0) {
            return GraphqlResponse.FromError(
                "Truy van can co dang: query { assignments { id title } }.");
        }

        var unsupportedFields = requestedFields
            .Except(SupportedAssignmentFields, StringComparer.Ordinal)
            .ToArray();

        if (unsupportedFields.Length > 0) {
            return GraphqlResponse.FromError(
                $"Truong khong ho tro: {string.Join(", ", unsupportedFields)}.");
        }

        var result = assignments
            .Select(assignment => ShapeAssignment(assignment, requestedFields))
            .ToArray();

        return GraphqlResponse.FromData(new Dictionary<string, object?> {
            ["assignments"] = result
        });
    }

    private static IReadOnlyList<string> ParseAssignmentFields(string query) {
        var normalized = Regex.Replace(query, @"\s+", " ");
        var match = Regex.Match(
            normalized,
            @"assignments\s*\{\s*(?<fields>[A-Za-z\s]+?)\s*\}",
            RegexOptions.IgnoreCase);

        if (!match.Success) {
            return [];
        }

        return match.Groups["fields"].Value
            .Split(' ', StringSplitOptions.RemoveEmptyEntries |
            ↪  StringSplitOptions.TrimEntries)
            .Distinct(StringComparer.Ordinal)
            .ToArray();
    }

    private static Dictionary<string, object?> ShapeAssignment(
        Assignment assignment,
        IReadOnlyList<string> requestedFields) {
        var shaped = new Dictionary<string, object?>();

        foreach (var field in requestedFields) {
            shaped[field] = field switch {
                "id" => assignment.Id,
                "title" => assignment.Title,
                "description" => assignment.Description,
                "dueDate" => assignment.DueDate.ToString("yyyy-MM-dd"),
                "isOpen" => assignment.IsOpen,
                "submissionCount" => assignment.Submissions.Count,
```

```
                _ => null
            };
        }

        return shaped;
    }
}

sealed record GraphqlResponse(
    Dictionary<string, object?>? Data,
    string[]? Errors) {
    public bool HasErrors => Errors is { Length: > 0 };

    public static GraphqlResponse FromData(Dictionary<string, object?> data) {
        return new GraphqlResponse(data, null);
    }

    public static GraphqlResponse FromError(string message) {
        return new GraphqlResponse(null, [message]);
    }
}
```

Hàm `Execute` thể hiện ba bước quan trọng của một máy chủ GraphQL ở mức tối giản. Bước đầu tiên là kiểm tra truy vấn có tồn tại và có đúng dạng mà ví dụ hỗ trợ hay không. Bước thứ hai là so sánh các trường máy khách yêu cầu với danh sách trường được phép: `id`, `title`, `description`, `dueDate`, `isOpen` và `submissionCount`. Bước cuối cùng là tạo từng đối tượng kết quả chỉ với các khóa đã được yêu cầu. Nếu máy khách không yêu cầu `description`, trường này sẽ không xuất hiện trong JSON trả về.

Điểm cần chú ý là máy chủ vẫn kiểm soát hợp đồng dữ liệu. Máy khách được chọn trường, nhưng không được tùy ý truy cập mọi thứ trong bộ nhớ của API. Danh sách `SupportedAssignmentFields` đóng vai trò tương tự một phần nhỏ của lược đồ: nó nói rõ máy khách được hỏi những gì. Trong thư viện GraphQL thật, vai trò này được thực hiện bằng lược đồ có kiểu dữ liệu rõ ràng, bộ phân giải cho từng trường và cơ chế kiểm tra truy vấn trước khi thực thi.

> Ví dụ này dùng chuỗi và biểu thức chính quy để giảm số lượng phụ thuộc, nên chỉ phù hợp cho mục tiêu học tập. Không dùng cách này cho hệ thống sản xuất. Một máy chủ GraphQL thật cần phân tích cú pháp đúng chuẩn, kiểm tra kiểu, giới hạn độ sâu truy vấn, giới hạn độ phức tạp và xử lý lỗi nhất quán.

Biên dịch toàn bộ solution bằng lệnh:

```
dotnet build examples/chapter04/section03/GraphqlSubmissionSystem.slnx
```

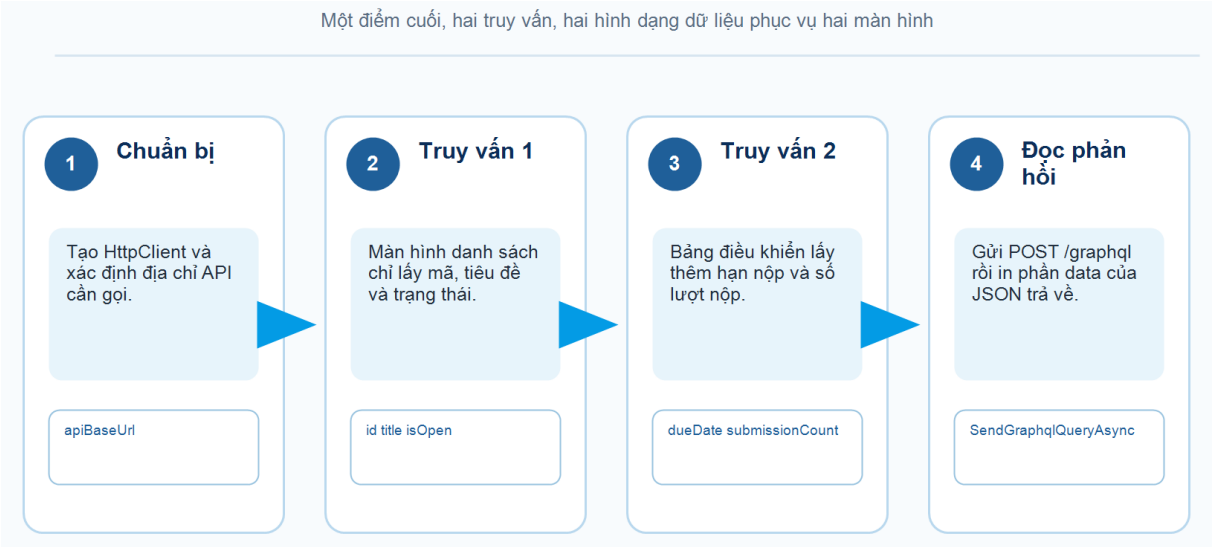
Sau đó chạy API ở cửa sổ dòng lệnh thứ nhất:

```
dotnet run --project examples/chapter04/section03/src/Submission.GraphqlApi --urls
↪  http://localhost:5000
```

4.3.2 *Máy khách truy vấn dữ liệu theo trường*

Máy khách của ví dụ này gửi hai truy vấn tới cùng một điểm cuối `/graphql`. Truy vấn thứ nhất phục vụ màn hình danh sách bài tập, nên chỉ cần mã, tiêu đề và trạng thái mở nộp. Truy vấn thứ hai phục vụ bảng điều khiển giảng viên, nên cần thêm ngày hạn nộp và số lượt nộp. Đây chính là khác biệt thực hành quan trọng so với REST: thay vì tạo thêm điểm cuối hoặc trả một DTO cố định cho mọi màn hình, máy khách mô tả hình dạng dữ liệu cần dùng ngay trong truy vấn.

Tệp `Program.cs` của máy khách như sau:



Hình 4.5: Các bước chính khi viết mã máy khách gửi truy vấn GraphQL trong Listing 4.3

Mã nguồn 4.5: Máy khách gửi hai truy vấn GraphQL với các tập trường khác nhau

```
using System.Net.Http.Json;
using System.Text.Json;

var apiBaseUrl = args.Length > 0 ? args[0].TrimEnd('/') : "http://localhost:5000";

using var http = new HttpClient {
    BaseAddress = new Uri(apiBaseUrl)
};

var compactQuery = """
query {
  assignments {
    id
    title
    isOpen
  }
}
""";

var dashboardQuery = """
query {
  assignments {
    id
    title
```

```
    dueDate
    submissionCount
  }
}
""";

await SendGraphqlQueryAsync(http, "Truy van gon cho danh sach bai tap", compactQuery);
await SendGraphqlQueryAsync(http, "Truy van cho bang dieu khien giang vien",
↪  dashboardQuery);

static async Task SendGraphqlQueryAsync(
    HttpClient http,
    string title,
    string query) {
    Console.WriteLine();
    Console.WriteLine(title);
    Console.WriteLine(new string('-', title.Length));
    Console.WriteLine(query);

    var response = await http.PostAsJsonAsync("/graphql", new GraphqlRequest(query));
    var body = await response.Content.ReadFromJsonAsync<GraphqlResponse>();

    if (!response.IsSuccessStatusCode) {
        Console.WriteLine($"Loi {(int)response.StatusCode}: {string.Join(", ",
        ↪  body?.Errors ?? [])}");
        return;
    }

    var formattedJson = JsonSerializer.Serialize(
        body?.Data,
        new JsonSerializerOptions { WriteIndented = true });

    Console.WriteLine(formattedJson);
}

record GraphqlRequest(string Query);

record GraphqlResponse(
    JsonElement? Data,
    string[]? Errors);
```

Đoạn mã máy khách dùng `HttpClient` giống các chương trước, nhưng khác ở chỗ phần thân yêu cầu chứa một văn bản truy vấn. Hàm `SendGraphqlQueryAsync` nhận tiêu đề, chuỗi truy vấn, gửi `POST/graphql`, rồi in phần `data` của phản hồi. Vì hai truy vấn dùng cùng URL nhưng chọn các trường khác nhau, sinh viên có thể quan sát trực tiếp rằng hình dạng JSON trả về thay đổi theo yêu cầu của máy khách.

Chạy máy khách ở cửa sổ dòng lệnh thứ hai:

```
dotnet run --project examples/chapter04/section03/src/Submission.GraphqlClient --
↪  http://localhost:5000
```

Kết quả mong đợi có dạng rút gọn như sau:

```
Truy van gon cho danh sach bai tap
----------------------------------
```

```
query {
  assignments {
    id
    title
    isOpen
  }
}
{
  "assignments": [
    {
      "id": 1,
      "title": "Bai tap REST API",
      "isOpen": true
    }
  ]
}

Truy van cho bang dieu khien giang vien
---------------------------------------
query {
  assignments {
    id
    title
    dueDate
    submissionCount
  }
}
{
  "assignments": [
    {
      "id": 1,
      "title": "Bai tap REST API",
      "dueDate": "2026-03-15",
      "submissionCount": 2
    }
  ]
}
```

Kết quả thực tế sẽ có đủ ba bài tập mẫu, nhưng đoạn rút gọn trên đã cho thấy điểm chính: cùng là danh sách bài tập, phản hồi thứ nhất không có trường ngày hạn nộp và số lượt nộp. Phản hồi thứ hai có hai trường này vì truy vấn đã yêu cầu `dueDate` và `submissionCount`. Máy khách không cần nhận một đối tượng lớn rồi tự bỏ bớt trường không dùng.

Khi tự thử nghiệm, hãy sửa một truy vấn để thêm hoặc bớt trường trong danh sách được hỗ trợ. Ví dụ, nếu thêm `description`, JSON trả về sẽ có thêm mô tả bài tập. Nếu yêu cầu một trường chưa có trong danh sách hỗ trợ, API trả lỗi `400BadRequest`. Cách phản hồi lỗi này giúp nhấn mạnh rằng GraphQL không phải là cơ chế "đọc tự do" dữ liệu phía máy chủ; máy khách chỉ được truy vấn trong phạm vi lược đồ mà máy chủ công bố.

Sau phần thực hành này, EduSubmit có thêm một cách đọc dữ liệu phía máy khách bên cạnh REST. REST vẫn phù hợp cho các thao tác có tài nguyên và quy ước rõ ràng, còn GraphQL hữu ích khi một màn hình cần tổ hợp dữ liệu linh hoạt theo trường. Ở các dự án lớn hơn, nhóm phát triển có thể giữ REST cho thao tác nộp bài, tải tệp và đọc tài nguyên đơn giản, đồng thời dùng GraphQL cho bảng điều khiển hoặc màn hình tổng hợp nhiều loại dữ liệu.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **REST** tổ chức API xung quanh tài nguyên với nhiều điểm cuối, mỗi điểm cuối trả về biểu diễn cố định do máy chủ quyết định. Phù hợp khi hợp đồng dữ liệu ổn định và cần tận dụng bộ nhớ đệm HTTP.
- **GraphQL** dùng một điểm cuối duy nhất, cho phép máy khách mô tả chính xác các trường dữ liệu cần lấy. Máy chủ định nghĩa lược đồ (schema), máy khách gửi truy vấn (query), bộ phân giải (resolver) thực thi và trả về JSON đúng hình dạng được yêu cầu.
- **Ba hạn chế chính của REST trong truy xuất dữ liệu**: truy xuất thừa (over-fetching) – nhận nhiều trường hơn cần thiết; truy xuất thiếu (under-fetching) – phải gọi nhiều điểm cuối cho một màn hình; phình phiên bản điểm cuối – phải tạo thêm điểm cuối khi giao diện thay đổi.
- **SDL (Schema Definition Language)** là ngôn ngữ mô tả kiểu độc lập với ngôn ngữ lập trình, dùng để định nghĩa kiểu đối tượng, trường, tham số và thao tác trong lược đồ GraphQL.
- **So sánh REST và GraphQL**: REST có lợi thế về bộ nhớ đệm HTTP, đường cong học tập thấp và công cụ kiểm thử phong phú. GraphQL có lợi thế về truy vấn linh hoạt, giảm lượt đi–về và không cần phiên bản URI.
- **Vấn đề N+1 trong GraphQL**: một truy vấn cha trả về N đối tượng, mỗi đối tượng lại kích hoạt truy vấn con riêng, tạo ra N+1 lượt truy vấn cơ sở dữ liệu. Cần công cụ như DataLoader để gom nhiều yêu cầu cùng loại.
- **Nguyên tắc lựa chọn**: dựa trên nhu cầu máy khách, không dựa trên xu hướng. REST khi tài nguyên rõ, ít thay đổi, cần bộ nhớ đệm. GraphQL khi giao diện phức tạp, nhiều màn hình cần tổ hợp dữ liệu khác nhau, hoặc băng thông hạn chế.
- **Kết hợp cả hai**: một hệ thống có thể dùng REST cho thao tác cốt lõi và GraphQL cho bảng điều khiển hoặc màn hình tổng hợp. Trong ASP.NET Core, REST qua bộ điều khiển/minimal API, GraphQL qua Hot Chocolate hoặc GraphQL.NET.

Câu hỏi ôn tập

1. Nêu định nghĩa GraphQL. GraphQL khác với REST ở điểm cốt lõi nào trong cách máy khách nhận dữ liệu?
2. Cho ví dụ về over-fetching và under-fetching trong một API REST quản lý bài tập và lượt nộp. Mỗi ví dụ cần nêu rõ máy khách cần dữ liệu gì và nhận được dữ liệu gì.
3. Vẽ sơ đồ và giải thích ba thành phần chính của kiến trúc GraphQL: lược đồ, truy vấn và bộ phân giải. Mỗi thành phần đảm nhận vai trò gì trong một yêu cầu?
4. Giải thích ý nghĩa của dấu `!` và cú pháp `[Type!]!` trong SDL của GraphQL. Cho ví dụ minh họa.
5. So sánh cách REST và GraphQL xử lý việc lấy dữ liệu từ nhiều tài nguyên liên quan (ví dụ: bài tập và danh sách lượt nộp của bài tập đó). Cách nào cần nhiều lượt đi–về hơn? Vì sao?
6. Trình bày vấn đề N+1 trong GraphQL. Nó xảy ra khi nào? Có thể giải quyết bằng cách nào?
7. REST có lợi thế gì về bộ nhớ đệm so với GraphQL? Giải thích dựa trên phương

thức HTTP và tính chất tài nguyên của REST.

8. Liệt kê 3 tình huống nên ưu tiên REST và 3 tình huống nên ưu tiên GraphQL khi thiết kế API phía máy khách. Giải thích ngắn gọn lý do cho mỗi tình huống.
9. Trong ví dụ thực hành Submission.GraphqlApi, hãy giải thích vai trò của danh sách trường được hỗ trợ và cách máy chủ quyết định trường nào được phép truy vấn.
10. Khi máy khách gửi truy vấn GraphQL yêu cầu một trường không có trong danh sách hỗ trợ của `SimpleGraphqlExecutor`, API trả về mã trạng thái nào và nội dung gì? Tại sao không nên trả về toàn bộ dữ liệu và để máy khách tự lọc?

Bài tập thực hành

1. **Mở rộng trường hỗ trợ trong GraphQL API**. Sao chép mã nguồn từ thư mục ví dụ của bài 4.3 và thêm trường `maxPoints` vào danh sách trường được hỗ trợ. Cập nhật dữ liệu mẫu trong `Program.cs` để mỗi bài tập có điểm tối đa. Viết một truy vấn mới trong máy khách để lấy `id`, `title` và `maxPoints`. Chạy API và máy khách, xác nhận JSON trả về có trường mới.
2. **Thêm quan hệ lồng trong truy vấn**. Mở rộng `SimpleGraphqlExecutor` để hỗ trợ trường con (ví dụ: `submissions\{id,status\}` lồng trong `assignments`). Máy khách gửi truy vấn lồng và kiểm tra JSON trả về có cấu trúc lồng tương ứng. *Gợi ý*: phân tích cú pháp truy vấn để phát hiện dấu ngoặc nhọn lồng, sau đó gọi đệ quy để giải quyết từng cấp.
3. **So sánh hiệu năng REST và GraphQL**. Viết một máy khách nhỏ gọi 10 lần liên tiếp cùng một yêu cầu: (a) qua REST `GET/api/assignments` (dùng lại API từ bài 2), (b) qua GraphQL với truy vấn chỉ lấy `id` và `title`. Đo tổng thời gian và kích thước dữ liệu nhận được cho mỗi cách. Báo cáo kết quả và giải thích sự khác biệt (nếu có).
4. **Thiết kế lược đồ GraphQL cho bảng điều khiển giảng viên**. Với mini system EduSubmit, hãy thiết kế một lược đồ GraphQL (bằng SDL) cho bảng điều khiển giảng viên cần hiển thị: danh sách bài tập kèm số lượt nộp, tỉ lệ đã chấm, và danh sách sinh viên trễ hạn. Không cần cài đặt bộ phân giải; chỉ cần viết lược đồ và giải thích cách máy khách sẽ dùng truy vấn nào để lấy dữ liệu cho màn hình này.

Câu hỏi thảo luận

1. Một công ty khởi nghiệp đang xây dựng ứng dụng di động cho người dùng cuối. Ứng dụng có 12 màn hình khác nhau, mỗi màn hình cần tổ hợp dữ liệu từ 3–5 tài nguyên. Băng thông di động là mối quan tâm hàng đầu. Hãy phân tích ưu và nhược điểm của REST và GraphQL cho bài toán này. Bạn sẽ chọn công nghệ nào? Vì sao?
2. Trong một hệ thống lớn, nhóm phát triển quyết định dùng đồng thời REST và GraphQL: REST cho API công khai, GraphQL cho bảng điều khiển nội bộ. Hãy thảo luận những thách thức khi vận hành cả hai trong cùng một hệ thống (tài liệu API, bảo mật, giám sát, đào tạo nhóm mới). Đề xuất cách giải quyết cho từng thách thức.
3. GraphQL cho phép máy khách tự do chọn trường dữ liệu. Điều này có thể dẫn đến truy vấn quá sâu hoặc quá phức tạp, gây quá tải máy chủ. Hãy đề xuất ít nhất ba chiến lược để bảo vệ máy chủ GraphQL khỏi các truy vấn tốn kém. Với mỗi

chiến lược, nêu ưu điểm và hạn chế.

CHƯƠNG 5
GIAO TIẾP GIỮA CÁC DỊCH VỤ VỚI GRPC

Mục tiêu chương
Sau khi học xong chương này, người học có thể:

- Giải thích cơ chế hoạt động của RPC, vai trò của client stub và server stub, và các biến thể của RPC trong giao tiếp dịch vụ–dịch vụ.
- Phân biệt RPC cổ điển và gRPC hiện đại, nhận diện ba trụ cột của gRPC: HTTP/2, Protocol Buffers và bốn kiểu streaming.
- Đọc và viết hợp đồng giao tiếp bằng Protocol Buffers (`.proto`), hiểu quy trình sinh mã tự động và lợi ích của contract-based communication.
- Phân tích sự khác biệt giữa API hướng máy khách (REST/GraphQL) và API nội bộ (gRPC), từ đó lựa chọn công nghệ phù hợp cho từng ranh giới giao tiếp.
- Xây dựng và chạy thử một hệ thống gồm dịch vụ gRPC và máy khách gRPC, quan sát luồng dữ liệu từ REST bên ngoài đến gRPC nội bộ trong mini system EduSubmit.

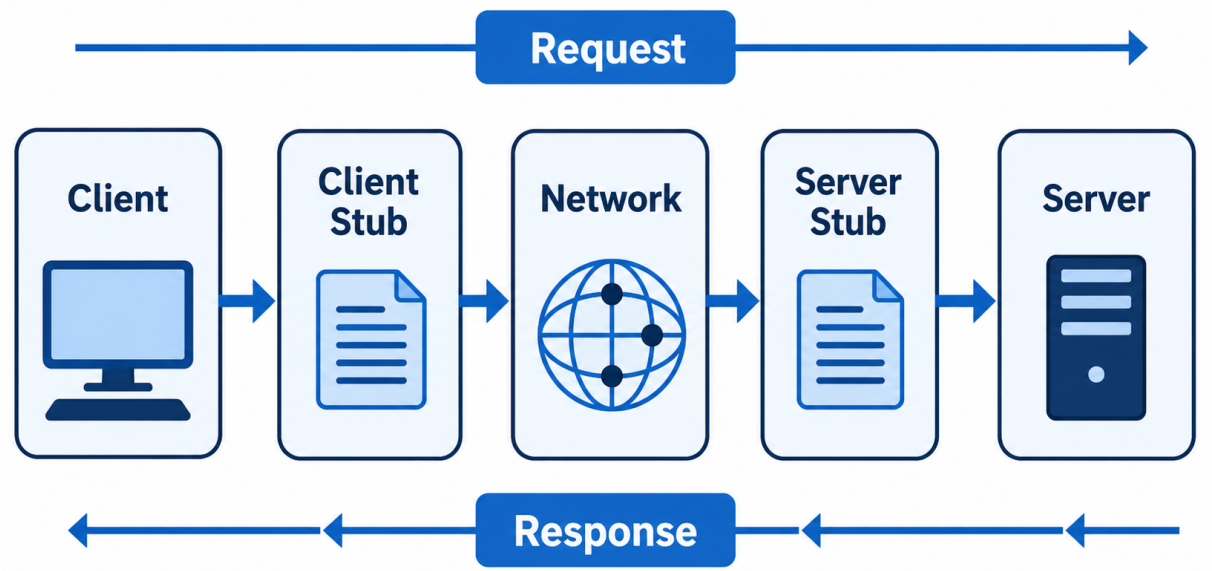
5.1 Khái niệm và cơ chế hoạt động của RPC

5.1.1 MEP: yêu cầu–phản hồi trong giao tiếp dịch vụ–dịch vụ

Trong các chương trước, chúng ta đã làm quen với mẫu trao đổi thông điệp yêu cầu–phản hồi (request–response) giữa máy khách và máy chủ qua RESTful API (bài 2) và giao tiếp thời gian thực qua SignalR (bài 3). Khi bước vào thế giới giao tiếp giữa các dịch vụ (service-to-service), mẫu yêu cầu–phản hồi vẫn đóng vai trò trung tâm, nhưng được hiện thực ở một tầng thấp hơn và với yêu cầu khắt khe hơn về hiệu năng.

RPC (Remote Procedure Call – Gọi thủ tục từ xa) là mô hình giao tiếp cơ bản nhất trong các hệ thống phân tán, được thiết kế để che giấu hoàn toàn sự phức tạp của việc truyền thông điệp qua mạng khỏi người lập trình. Ý tưởng cốt lõi của RPC rất đơn giản: khi một chương trình trên máy A cần gọi một hàm được thực thi trên máy B, lời gọi đó trông giống hệt như gọi một thủ tục cục bộ. Người lập trình không cần quan tâm đến việc thiết lập kết nối mạng, đóng gói tham số, xử lý lỗi truyền thông hay chuyển đổi kiểu dữ liệu giữa các nền tảng khác nhau.

Để hiện thực hóa ý tưởng này, mọi hệ thống RPC đều dựa trên hai thành phần then chốt: **client stub** và **server stub** (còn gọi là skeleton). Khi client muốn gọi một thủ tục từ xa, nó không gọi trực tiếp đến server mà gọi thông qua client stub – một thủ tục cục bộ có chữ ký giống hệt thủ tục từ xa. Client stub đảm nhận việc **đóng gói tham số (marshalling)** thành một định dạng chuẩn có thể truyền qua mạng, sau đó gửi thông điệp yêu cầu đến server. Phía server, server stub nhận thông điệp, **mở gói tham số (unmarshalling)**, gọi thủ tục thật với các tham số đã được khôi phục, rồi đóng gói kết quả trả về và gửi ngược lại cho client. Trong suốt quá trình này, client bị chặn (blocked)



Hình 5.1: Luồng RPC cơ bản với client stub, mạng truyền thông và server stub

– nó tạm dừng thực thi và chờ cho đến khi nhận được phản hồi từ server.

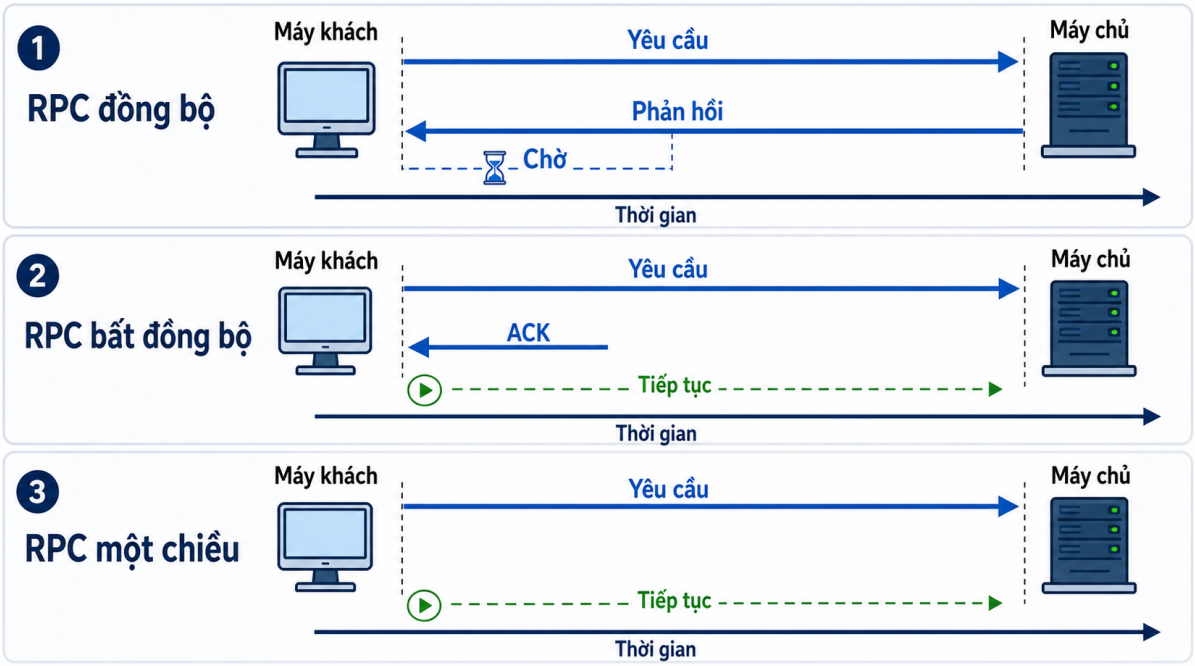
Hành vi chặn của RPC truyền thống, tuy đơn giản và dễ hiểu, không phải lúc nào cũng phù hợp với mọi tình huống thực tế. Qua thời gian, nhiều biến thể của RPC đã được phát triển để đáp ứng các nhu cầu khác nhau:

- **RPC bất đồng bộ (Asynchronous RPC)**: Server gửi ngay một tín hiệu xác nhận (ACK) khi nhận được yêu cầu, sau đó mới thực thi thủ tục được gọi. Client nhận ACK và tiếp tục xử lý mà không cần chờ kết quả. Biến thể này hữu ích khi không có giá trị trả về, hoặc khi client cần gửi yêu cầu đến nhiều server song song mà không muốn bị chặn tuần tự.
- **RPC đồng bộ trì hoãn (Deferred Synchronous RPC)**: Kết hợp RPC bất đồng bộ với cơ chế gọi lại (callback). Client gửi yêu cầu, nhận ACK và tiếp tục làm việc. Khi server hoàn tất xử lý, nó chủ động gửi kết quả về và kích hoạt hàm callback đã đăng ký trước đó phía client.
- **RPC một chiều (One-way RPC)**: Client gửi yêu cầu và tiếp tục thực thi ngay lập tức, không chờ bất kỳ phản hồi nào, kể cả ACK. Đây là dạng giao tiếp "bắn và quên" (fire-and-forget), đánh đổi độ tin cậy để lấy tốc độ.

Trong bối cảnh giao tiếp dịch vụ–dịch vụ, các biến thể này mang lại sự linh hoạt cần thiết. Ví dụ, khi dịch vụ xử lý bài nộp cần thông báo cho dịch vụ chấm điểm rằng một bài mới đã sẵn sàng, nó có thể dùng RPC một chiều hoặc RPC bất đồng bộ – không cần đợi kết quả chấm điểm ngay lúc đó. Ngược lại, khi dịch vụ cổng (gateway) cần lấy danh sách bài nộp của một sinh viên, nó sẽ dùng RPC đồng bộ truyền thống vì cần kết quả để trả về cho máy khách.

5.1.2 RPC và gRPC

RPC không phải là một ý tưởng mới. Từ những năm 1990, các hệ thống RPC chuẩn hóa như **DCE RPC (Distributed Computing Environment)** do Open Group phát triển đã đặt nền móng cho nhiều công nghệ sau này, bao gồm Microsoft DCOM và Samba. Mục tiêu của DCE RPC rất tham vọng: cho phép client gọi thủ tục trên server một cách



Hình 5.2: So sánh RPC đồng bộ, RPC bất đồng bộ và RPC một chiều

trong suốt, tự động định vị server (binding), xử lý phân mảnh thông điệp, và chuyển đổi kiểu dữ liệu giữa các kiến trúc phần cứng khác nhau (ví dụ: big-endian và little-endian). Client và server có thể được viết bằng các ngôn ngữ khác nhau, chạy trên các hệ điều hành khác nhau.

Tuy nhiên, các hệ thống RPC thế hệ đầu có những hạn chế đáng kể. Chúng phụ thuộc vào các giao thức truyền thông phức tạp, khó triển khai qua tường lửa, và thiếu khả năng tương tác giữa các hệ sinh thái khác nhau. Sự trỗi dậy của HTTP/2 và nhu cầu giao tiếp hiệu năng cao trong kiến trúc microservices đã thúc đẩy sự ra đời của **gRPC** – một khung phần mềm RPC mã nguồn mở do Google phát triển, hiện là dự án của Cloud Native Computing Foundation (CNCF).

gRPC kế thừa tinh thần của RPC cổ điển nhưng hiện đại hóa nó trên ba trụ cột chính:

1. **HTTP/2 làm tầng vận chuyển**: Thay vì dùng giao thức riêng, gRPC chạy trên HTTP/2, tận dụng các tính năng như multiplexing (nhiều yêu cầu song song trên một kết nối), nén tiêu đề (header compression), và đẩy dữ liệu từ server (server push). Điều này giúp gRPC vượt qua tường lửa dễ dàng và đạt hiệu năng cao hơn so với REST over HTTP/1.1.
2. **Protocol Buffers làm ngôn ngữ đặc tả giao diện**: gRPC dùng Protocol Buffers (protobuf) làm IDL (Interface Definition Language) và định dạng tuần tự hóa. So với JSON, protobuf cho ra thông điệp nhỏ hơn, nhanh hơn khi đóng gói/mở gói, và có kiểm tra kiểu tại thời điểm biên dịch.
3. **Hỗ trợ bốn kiểu giao tiếp**: gRPC không giới hạn ở mẫu request–response truyền thống. Nó hỗ trợ:
    - **Unary RPC**: Một yêu cầu, một phản hồi – tương tự RPC cổ điển.
    - **Server streaming RPC**: Client gửi một yêu cầu, server trả về một luồng phản hồi.

- **Client streaming RPC**: Client gửi một luồng yêu cầu, server trả về một phản hồi.
- **Bidirectional streaming RPC**: Cả hai bên gửi luồng dữ liệu độc lập.

Nhờ ba trụ cột này, gRPC đặc biệt phù hợp cho giao tiếp nội bộ giữa các dịch vụ trong hệ thống phân tán, nơi hiệu năng, kiểu dữ liệu chặt chẽ và hợp đồng rõ ràng được đặt lên hàng đầu. Phần 5.3 sẽ hướng dẫn chi tiết cách tạo và gọi dịch vụ gRPC trong môi trường .NET.

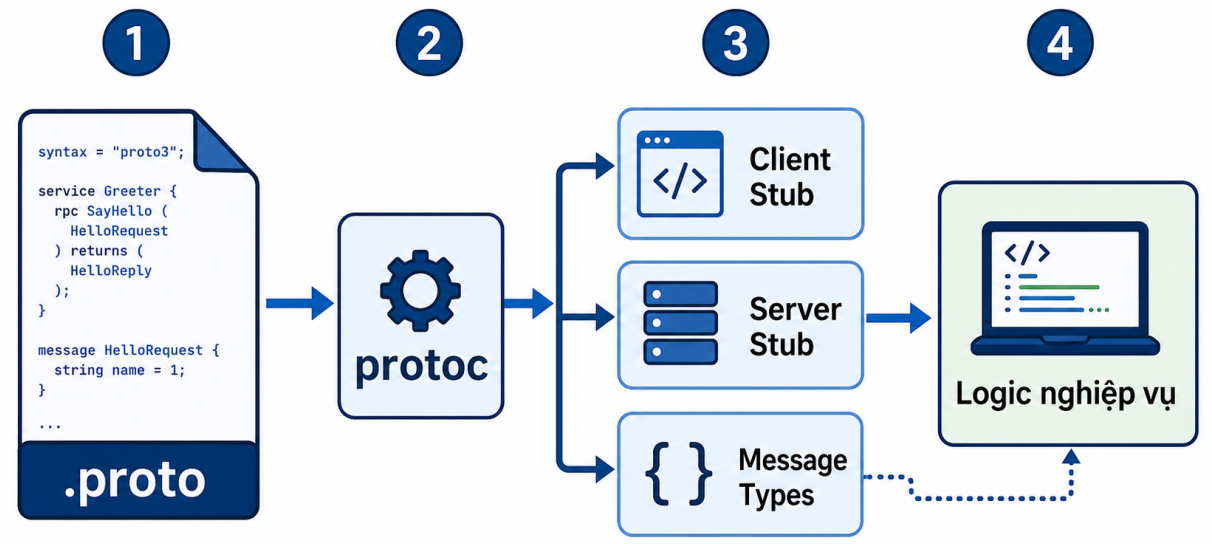
5.1.3 Protocol Buffers và giao tiếp dựa trên hợp đồng

Mọi hệ thống RPC, từ DCE RPC cổ điển cho đến gRPC hiện đại, đều xoay quanh một khái niệm then chốt: **hợp đồng giao tiếp (communication contract)**. Hợp đồng này được mô tả bằng một **Ngôn ngữ Đặc tả Giao diện (Interface Definition Language – IDL)**, định nghĩa chính xác những thủ tục nào có thể được gọi, mỗi thủ tục nhận tham số gì và trả về kết quả gì. IDL chỉ mô tả *cú pháp* (syntax) của giao tiếp – tên hàm, kiểu dữ liệu, tham số, giá trị trả về và ngoại lệ. Phần *ngữ nghĩa* (semantic) – tức ý nghĩa thực sự của từng thao tác – được ghi lại bằng ngôn ngữ tự nhiên trong tài liệu thiết kế.

Vai trò của IDL vượt xa việc chỉ làm tài liệu tham khảo. Khi một file IDL được biên dịch, trình biên dịch IDL sẽ tự động sinh ra:

- **Client stub**: Đoạn code phía client, chịu trách nhiệm đóng gói tham số, gửi yêu cầu và mở gói kết quả.
- **Server stub**: Đoạn code phía server, nhận yêu cầu, mở gói tham số và gọi hàm thực thi.
- **Các lớp dữ liệu (message types)**: Các cấu trúc dữ liệu tương ứng với thông điệp yêu cầu và phản hồi, được sinh ra bằng ngôn ngữ đích (C#, Java, Python, v.v.).

Nhờ vậy, người lập trình chỉ cần tập trung vào logic nghiệp vụ; toàn bộ phần truyền thông và tuần tự hóa được sinh tự động và đảm bảo nhất quán giữa client và server.



Hình 5.3: Quy trình sinh mã từ hợp đồng `.proto` trong gRPC

Trong hệ sinh thái gRPC, IDL được hiện thực bởi **Protocol Buffers (protobuf)**. Một file `.proto` điển hình bao gồm ba thành phần:

1. **Khai báo thông điệp (`message`)**: Định nghĩa cấu trúc dữ liệu cho yêu cầu và phản hồi. Mỗi trường có một kiểu dữ liệu (scalar hoặc composite), một tên và một số thứ tự (field number) dùng để nhận diện khi tuần tự hóa nhị phân.
2. **Khai báo dịch vụ (`service`)**: Định nghĩa tập các thủ tục RPC mà server triển khai, mỗi thủ tục nhận một kiểu yêu cầu và trả về một kiểu phản hồi (hoặc một luồng nếu dùng streaming).
3. **Tùy chọn biên dịch (`option`)**: Cấu hình ngôn ngữ đích, không gian tên, đường dẫn sinh code và các tùy chọn khác.

Giao tiếp dựa trên hợp đồng mang lại ba lợi ích quan trọng cho hệ thống phân tán:

- **Khả năng tương tác (interoperability)**: Hai triển khai khác nhau của cùng một hợp đồng có thể thay thế cho nhau, bất kể ngôn ngữ lập trình hay nền tảng. Một server viết bằng C++ có thể phục vụ client viết bằng Java, Python hoặc C# một cách trong suốt.
- **Khả năng chuyển đổi (portability)**: Client được viết dựa trên interface, không phụ thuộc vào triển khai cụ thể. Khi server thay đổi công nghệ nhưng vẫn giữ nguyên interface, client không cần sửa đổi.
- **An toàn kiểu tại thời điểm biên dịch (compile-time type safety)**: Không giống như JSON trong REST, nơi cấu trúc dữ liệu chỉ được kiểm tra khi chạy, protobuf đảm bảo client và server luôn thống nhất về kiểu dữ liệu ngay từ bước biên dịch.

5.1.4 Sự khác biệt giữa API phía máy khách và API nội bộ

Không phải mọi API đều được tạo ra như nhau. Trong một hệ thống phân tán hiện đại, chúng ta cần phân biệt rõ hai loại API phục vụ hai đối tượng hoàn toàn khác nhau: **API hướng máy khách (client-facing API)** và **API nội bộ (internal API)**.

API hướng máy khách là các API được thiết kế để phục vụ người dùng cuối hoặc ứng dụng bên thứ ba, thường thông qua Internet. Chúng được xây dựng dựa trên các chuẩn web phổ biến như REST hoặc GraphQL, ưu tiên tính dễ hiểu, khả năng khám phá (discoverability), bảo mật và khả năng tương thích ngược. Một API RESTful cho phép bất kỳ ai có kết nối HTTP đều có thể gọi được, và tài liệu API thường được công khai. Đây chính là loại API chúng ta đã xây dựng trong bài 2.

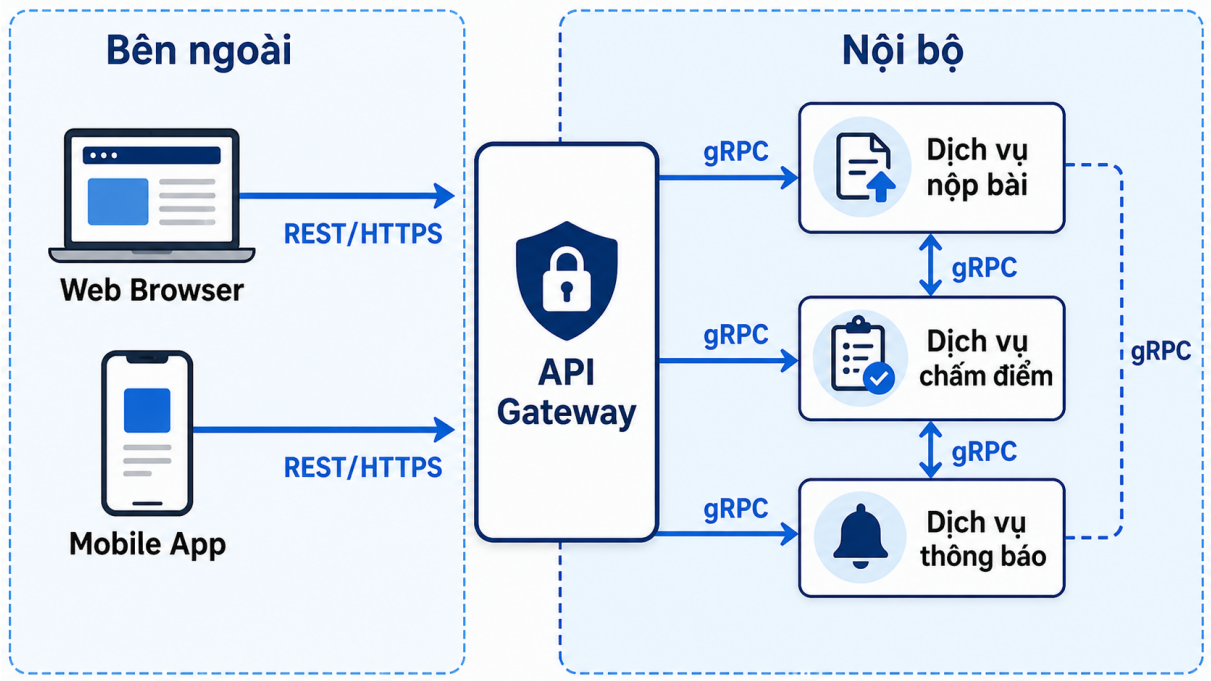
API nội bộ phục vụ giao tiếp giữa các dịch vụ trong cùng một hệ thống, thường nằm sau tường lửa hoặc trong cùng một cụm (cluster). Ở đây, các ưu tiên thay đổi: hiệu năng, độ trễ thấp, an toàn kiểu và khả năng streaming trở nên quan trọng hơn tính dễ khám phá hay khả năng gọi từ trình duyệt. gRPC được thiết kế chính xác cho kịch bản này.

Bảng 5.1 tóm tắt những khác biệt then chốt giữa hai loại API.

Việc lựa chọn công nghệ phù hợp cho từng loại giao tiếp không phải là câu hỏi "hoặc cái này hoặc cái kia". Trong thực tế, một hệ thống phân tán lành mạnh thường sử dụng **cả hai**: REST (hoặc GraphQL) cho giao tiếp với bên ngoài, và gRPC cho giao tiếp nội bộ. API Gateway đóng vai trò cầu nối, nhận yêu cầu REST từ máy khách và chuyển tiếp sang các dịch vụ nội bộ qua gRPC.

Một số nguyên tắc giúp lựa chọn đúng:

- **Dùng REST/GraphQL** khi API cần được gọi từ trình duyệt, ứng dụng di động hoặc bên thứ ba; khi ưu tiên sự đơn giản và khả năng gỡ lỗi thủ công; khi dữ liệu



Hình 5.4: Ranh giới giữa API hướng máy khách và API nội bộ trong kiến trúc microservices

Bảng 5.1: So sánh API hướng máy khách và API nội bộ

Tiêu chí	API hướng máy khách	API nội bộ
Mục đích	Phục vụ ứng dụng khách (web, di động, bên thứ ba)	Giao tiếp giữa các dịch vụ trong hệ thống
Công nghệ điển hình	REST, GraphQL, Webhook	gRPC, message queue (RabbitMQ), REST (nếu đơn giản)
Định dạng dữ liệu	JSON (văn bản, con người đọc được)	Protobuf (nhị phân, máy đọc)
Giao thức	HTTP/1.1 hoặc HTTP/2	HTTP/2 (gRPC), AMQP (RabbitMQ)
Hợp đồng	OpenAPI/Swagger, SDL (GraphQL)	File `.proto` (protobuf)
Kiểm tra kiểu	Khi chạy (runtime)	Khi biên dịch (compile-time)
Hiệu năng	Trung bình (JSON parsing)	Cao (tuần tự hóa nhị phân)
Streaming	Hạn chế (SSE, WebSocket cần riêng)	Hỗ trợ sẵn (server, client, bidirectional)
Khả năng tương thích	Ưu tiên tương thích ngược	Ưu tiên tính nhất quán giữa các phiên bản
Bảo mật	API key, OAuth2, rate limiting	mTLS, service mesh (Istio/Linkerd)

cần được caching qua CDN.

- **Dùng gRPC** khi hai dịch vụ nội bộ cần giao tiếp với tần suất cao và độ trễ thấp; khi cần streaming hai chiều; khi có nhiều service cùng tuân thủ một hợp đồng chặt chẽ; khi hiệu năng tuần tự hóa là yếu tố then chốt.
- **Dùng message queue** khi giao tiếp cần tính bất đồng bộ và bền vững; khi dịch vụ nhận có thể không hoạt động tại thời điểm gửi; khi cần phân phối công việc cho nhiều worker – đây là chủ đề của bài 6 và bài 7.

Trong phần tiếp theo của chương này, chúng ta sẽ tập trung vào gRPC – công nghệ đã trở thành lựa chọn hàng đầu cho giao tiếp nội bộ trong các hệ thống microservices hiện đại.

5.2 Kỹ thuật lập trình cơ bản với gRPC

Phần 5.1 đã trình bày mô hình RPC, ba trụ cột của gRPC (HTTP/2, Protocol Buffers, bốn kiểu giao tiếp), vai trò của IDL trong giao tiếp dựa trên hợp đồng, và sự khác biệt giữa API hướng máy khách và API nội bộ. Phần này chuyển từ nguyên tắc sang kỹ thuật lập trình: cách khai báo service và message trong tệp `.proto`, cách cấu hình sinh mã stub cho máy khách và máy chủ, cách hiện thực và đăng ký dịch vụ gRPC, cách tạo kênh và gọi phương thức gRPC từ máy khách, và cách xử lý lỗi, deadline cùng hủy yêu cầu.

Để tập trung vào kỹ thuật, phần này dùng một ví dụ tối giản: CalcGrpc – một máy chủ gRPC cung cấp bốn phép tính số học (cộng, trừ, nhân, chia) và một máy khách dòng lệnh gọi các phép tính đó, bao gồm cả tình huống lỗi và hủy yêu cầu. Mã nguồn đầy đủ nằm trong thư mục `content/examples/chapter05/section02/`. Sau khi nắm được các kỹ thuật cơ bản qua ví dụ này, sinh viên sẽ áp dụng chúng để xây dựng hệ thống gRPC nội bộ cho EduSubmit trong phần 5.3.

5.2.1 Khai báo service và message trong tệp `.proto`

Mọi dự án gRPC đều bắt đầu từ một tệp `.proto` – nơi định nghĩa hợp đồng giao tiếp giữa máy khách và máy chủ. Tệp này được viết bằng ngôn ngữ Protocol Buffers (protobuf) và đóng ba vai trò: định nghĩa cấu trúc dữ liệu (message), định nghĩa tập thủ tục có thể gọi từ xa (service), và làm đầu vào cho trình biên dịch sinh mã.

Trong ví dụ CalcGrpc, tệp `calculator.proto` khai báo dịch vụ `Calculator` với bốn thủ tục RPC kiểu unary (một yêu cầu, một phản hồi), cùng hai kiểu thông điệp dùng chung cho cả bốn thủ tục:

Mã nguồn 5.1: Hợp đồng gRPC cho dịch vụ máy tính – khai báo service và message trong calculator.proto

```
syntax = "proto3";

option csharp_namespace = "CalcGrpc";

package calc;

// 5.2.1: Khai bao dich vu Calculator voi cac thao tac co ban
service Calculator {
  rpc Add (CalcRequest) returns (CalcReply);
  rpc Subtract (CalcRequest) returns (CalcReply);
  rpc Multiply (CalcRequest) returns (CalcReply);
```

```
  rpc Divide (CalcRequest) returns (CalcReply);
}

// 5.2.1: Thong diep yeu cau – chua hai toan hang a va b
message CalcRequest {
  double a = 1;
  double b = 2;
}

// 5.2.1: Thong diep phan hoi – chua ket qua
message CalcReply {
  double result = 1;
}
```

Tệp `.proto` trong Listing 5.1 có bốn thành phần cần chú ý. Thứ nhất, dòng `syntax="proto3"` xác nhận phiên bản protobuf được sử dụng – proto3 là phiên bản hiện tại, được khuyến nghị cho mọi dự án mới. Thứ hai, tùy chọn `csharp_namespace` chỉ định không gian tên C# mà mã sinh ra sẽ thuộc về. Thứ ba, khối `serviceCalculator` định nghĩa bốn thủ tục RPC, mỗi thủ tục khai báo bằng từ khóa `rpc`, theo sau là tên thủ tục, kiểu thông điệp yêu cầu trong ngoặc đơn và kiểu thông điệp phản hồi sau từ khóa `returns`. Thứ tư, hai khối `message` định nghĩa cấu trúc dữ liệu cho yêu cầu và phản hồi. Mỗi trường trong message có ba thành phần: kiểu dữ liệu (`double`), tên trường (`a`, `b`, `result`) và số thứ tự (1, 2) – đây là định danh nhị phân dùng khi tuần tự hóa, không phải giá trị mặc định.

Điểm đáng chú ý trong thiết kế này là cả bốn thủ tục (`Add`, `Subtract`, `Multiply`, `Divide`) đều dùng chung cặp thông điệp `CalcRequest` và `CalcReply`. Đây là một thực hành phổ biến trong thiết kế hợp đồng gRPC: khi nhiều thủ tục có cùng hình dạng dữ liệu đầu vào và đầu ra, dùng chung message giúp giảm trùng lặp và giữ cho tệp `.proto` ngắn gọn. Ngược lại, nếu hai thủ tục cần tập tham số khác nhau, nên tạo message riêng để giữ tính rõ ràng.

5.2.2 *Sinh mã nguồn client stub và server stub*

Sau khi có tệp `.proto`, bước tiếp theo là cấu hình để trình biên dịch sinh mã C# từ hợp đồng. Trong hệ sinh thái .NET, việc này được thực hiện thông qua khai báo `<Protobuf>` trong tệp `.csproj` và các gói NuGet tương ứng.

Cơ chế hoạt động như sau: khi biên dịch, gói `Grpc.Tools` đọc tất cả tệp `.proto` được khai báo trong `<Protobuf>` và gọi trình biên dịch protobuf (`protoc`) kèm plugin gRPC cho C#. Kết quả là hai tập mã nguồn được sinh tự động vào thư mục `obj/` – người lập trình không bao giờ sửa trực tiếp các tệp này. Tập thứ nhất chứa các lớp thông điệp (`CalcRequest`, `CalcReply`) và logic tuần tự hóa. Tập thứ hai chứa lớp cơ sở cho dịch vụ (`Calculator.CalculatorBase`) và lớp máy khách (`Calculator.CalculatorClient`).

Dự án máy chủ (`CalcGrpcService`) khai báo như sau trong `.csproj`:

```
<Protobuf Include="Protos\calculator.proto" GrpcServices="Server" />
```

Giá trị `GrpcServices="Server"` yêu cầu sinh mã phía máy chủ: chỉ lớp cơ sở `CalculatorBase` được tạo ra để ta kế thừa và hiện thực. Dự án máy khách (`CalcGrpcClient`) dùng cùng tệp `.proto` nhưng với cấu hình khác:

```
<Protobuf Include="..\CalcGrpcService\Protos\calculator.proto" GrpcServices="Client" />
```

Giá trị `GrpcServices="Client"` yêu cầu sinh mã phía máy khách: lớp `CalculatorClient` được tạo ra với các phương thức tương ứng với từng thủ tục RPC. Lưu ý rằng đường dẫn trong `Include` trỏ đến tệp `.proto` dùng chung nằm trong dự án máy chủ, đảm bảo máy khách và máy chủ luôn dùng cùng một hợp đồng – không có chỗ cho sai lệch.

Về gói NuGet, dự án máy chủ chỉ cần `Grpc.AspNetCore` – gói này đã bao gồm sẵn mọi thứ cần thiết để chạy dịch vụ gRPC trên ASP.NET Core. Dự án máy khách cần ba gói: `Google.Protobuf` (thư viện tuần tự hóa protobuf), `Grpc.Net.Client` (lớp `GrpcChannel` và hỗ trợ gọi RPC), và `Grpc.Tools` (trình biên dịch `.proto` sang C# khi build). Nếu dùng Visual Studio hoặc dotnet CLI, mã được sinh lại mỗi khi build, đảm bảo luôn đồng bộ với tệp `.proto`.

5.2.3 Hiện thực và đăng ký gRPC service

Khi mã đã được sinh, công việc của người lập trình phía máy chủ là hiện thực logic nghiệp vụ cho từng thủ tục RPC. Để làm điều này, ta tạo một lớp kế thừa từ lớp cơ sở được sinh tự động (`Calculator.CalculatorBase`) và ghi đè các phương thức tương ứng.

Lớp `CalculatorService` trong ví dụ CalcGrpc hiện thực cả bốn phép tính. Mỗi phương thức nhận hai tham số: một đối tượng yêu cầu (`CalcRequest`) chứa dữ liệu từ máy khách, và một đối tượng ngữ cảnh (`ServerCallContext`) chứa thông tin về cuộc gọi hiện tại (metadata, deadline, địa chỉ máy khách, v.v.).

Mã nguồn 5.2: Hiện thực dịch vụ Calculator gRPC với bốn phép tính và xử lý lỗi chia cho 0

```
using CalcGrpc;
using Grpc.Core;

namespace CalcGrpcService.Services;

// 5.2.3: Hien thuc dich vu - ke thua lop co so duoc sinh tu .proto
public class CalculatorService : Calculator.CalculatorBase
{
    public override Task<CalcReply> Add(CalcRequest request, ServerCallContext context)
    {
        return Task.FromResult(new CalcReply { Result = request.A + request.B });
    }

    public override Task<CalcReply> Subtract(CalcRequest request, ServerCallContext
    ↪  context)
    {
        return Task.FromResult(new CalcReply { Result = request.A - request.B });
    }

    public override Task<CalcReply> Multiply(CalcRequest request, ServerCallContext
    ↪  context)
    {
        return Task.FromResult(new CalcReply { Result = request.A * request.B });
    }

    // 5.2.5: Divide nem RpcException neu b == 0 - may khach se nhan loi
    public override Task<CalcReply> Divide(CalcRequest request, ServerCallContext
    ↪  context)
```

```
    {
        if (request.B == 0)
            throw new RpcException(new Status(StatusCode.InvalidArgument, "Khong the
            ↪  chia cho 0."));

        return Task.FromResult(new CalcReply { Result = request.A / request.B });
    }
}
```

Ba phương thức `Add`, `Subtract` và `Multiply` có logic đơn giản: đọc hai toán hạng từ `request`, thực hiện phép tính tương ứng và trả về kết quả trong `CalcReply`. Kết quả được gói trong `Task.FromResult` vì các phương thức này là đồng bộ nhưng chữ ký gRPC yêu cầu trả về `Task<T>`. Phương thức `Divide` kiểm tra điều kiện chia cho 0 và ném `RpcException` nếu vi phạm – đây là cơ chế báo lỗi chuẩn trong gRPC, sẽ được thảo luận chi tiết trong Mục 5.2.5.

Sau khi hiện thực dịch vụ, cần đăng ký nó vào pipeline của ứng dụng. Việc này được thực hiện trong `Program.cs` với hai lời gọi:

Mã nguồn 5.3: Đăng ký và ánh xạ dịch vụ gRPC trong CalcGrpcService

```
using CalcGrpcService.Services;

var builder = WebApplication.CreateBuilder(args);

// 5.2.3: Dang ky dich vu gRPC vao container DI
builder.Services.AddGrpc();

var app = builder.Build();

// 5.2.3: Anh xa dich vu gRPC vao tuyen HTTP/2
app.MapGrpcService<CalculatorService>();
app.MapGet("/", () => "CalcGrpcService dang chay. Dung CalcGrpcClient de goi cac phuong
↪  thuc.");

app.Run();
```

Lời gọi `builder.Services.AddGrpc()` đăng ký các dịch vụ cốt lõi để gRPC hoạt động: bộ quản lý kết nối HTTP/2, cơ chế tuần tự hóa protobuf, và middleware xử lý yêu cầu gRPC. Lời gọi `app.MapGrpcService<CalculatorService>()` ánh xạ lớp dịch vụ vào một tuyến nội bộ – khác với REST, tuyến gRPC không được định nghĩa bởi người lập trình mà được xác định bởi tên gói (`package`) và tên dịch vụ trong tệp `.proto`. Trong ví dụ này, máy khách sẽ kết nối tới `http://localhost:5200` và gọi các thủ tục thông qua định danh `calc.Calculator/Add`, `calc.Calculator/Divide`, v.v. Một điểm cuối GET `/` được thêm vào để kiểm tra nhanh dịch vụ có đang chạy hay không, nhưng đây không phải là thành phần bắt buộc.

5.2.4 Tạo channel và gọi phương thức gRPC từ client

Phía máy khách, việc gọi một thủ tục gRPC trải qua ba bước: tạo kênh giao tiếp (`GrpcChannel`), tạo máy khách từ kênh (`CalculatorClient`), và gọi phương thức tương ứng với thủ tục mong muốn.

Kênh (channel) là trừu tượng hóa của một kết nối HTTP/2 tới máy chủ gRPC. Một kênh có thể phục vụ nhiều lời gọi đồng thời nhờ tính năng multiplexing của HTTP/2. Kênh được tạo bằng `GrpcChannel.ForAddress(url)`, trong đó `url` là địa chỉ gốc của máy chủ (không kèm tên dịch vụ hay thủ tục). Kênh nên được tạo một lần và tái sử dụng trong suốt vòng đời ứng dụng – việc tạo kênh mới cho mỗi lời gọi sẽ làm mất lợi ích của multiplexing và gây lãng phí tài nguyên.

Máy khách (client) là một thể hiện của lớp được sinh tự động từ `.proto`. Máy khách được tạo bằng cách truyền kênh vào constructor: `newCalculator.CalculatorClient(channel`. Lớp này chứa các phương thức tương ứng một-một với các thủ tục RPC đã khai báo.

Toàn bộ mã nguồn máy khách CalcGrpc được trình bày trong Listing 5.4.

Mã nguồn 5.4: CalcGrpcClient – máy khách tối giản tạo kênh, gọi bốn phép tính và xử lý các tình huống lỗi

```
using CalcGrpc;
using Grpc.Core;
using Grpc.Net.Client;

var serverUrl = args.Length > 0 ? args[0] : "http://localhost:5200";

// 5.2.4: Tao kenh gRPC – mot ket noi HTTP/2 toi may chu
using var channel = GrpcChannel.ForAddress(serverUrl);

// 5.2.4: Tao client tu kenh – client stub duoc sinh tu .proto
var client = new Calculator.CalculatorClient(channel);

Console.WriteLine($"Da ket noi den CalcGrpcService tai {serverUrl}.\n");

// 5.2.4: Goi Add – unary RPC don gian
var addReply = await client.AddAsync(new CalcRequest { A = 10, B = 20 });
Console.WriteLine($"Add(10, 20) = {addReply.Result}");

// 5.2.4: Goi Multiply
var mulReply = await client.MultiplyAsync(new CalcRequest { A = 7, B = 8 });
Console.WriteLine($"Multiply(7, 8) = {mulReply.Result}");

// 5.2.5: Goi Divide – truong hop thanh cong
var divReply = await client.DivideAsync(new CalcRequest { A = 100, B = 4 });
Console.WriteLine($"Divide(100, 4) = {divReply.Result}");

// 5.2.5: Goi Divide – truong hop loi (chia cho 0)
try {
    await client.DivideAsync(new CalcRequest { A = 10, B = 0 });
} catch (RpcException ex) {
    Console.WriteLine($"Divide(10, 0) -> Loi RPC: {ex.StatusCode} –
    ↪  {ex.Status.Detail}");
}

// 5.2.5: Deadline – thiet lap thoi gian cho, huy loi goi neu vuot qua
try {
    await client.AddAsync(
        new CalcRequest { A = 1, B = 2 },
        deadline: DateTime.UtcNow.AddMilliseconds(-1)); // deadline da het han
} catch (RpcException ex) when (ex.StatusCode == StatusCode.DeadlineExceeded) {
    Console.WriteLine($"Add voi deadline qua khu -> Loi RPC: {ex.StatusCode}");
}
```

```
// 5.2.5: Huy yeu cau - dung CancellationToken
using var cts = new CancellationTokenSource();
cts.Cancel(); // huy ngay lap tuc

try {
    await client.AddAsync(
        new CalcRequest { A = 3, B = 4 },
        cancellationToken: cts.Token);
} catch (RpcException ex) when (ex.StatusCode == StatusCode.Cancelled) {
    Console.WriteLine($"Add bi huy -> Loi RPC: {ex.StatusCode}");
}

Console.WriteLine("\nHoan tat.");
```

Trong Listing 5.4, sau khi tạo kênh và máy khách, chương trình lần lượt gọi `AddAsync`, `MultiplyAsync` và `DivideAsync` – mỗi lời gọi là một unary RPC: máy khách gửi `CalcRequest`, máy chủ xử lý và trả về `CalcReply`. Tên phương thức trong C# luôn có hậu tố `Async` – đây là quy ước của gRPC cho .NET, phản ánh bản chất bất đồng bộ của mọi lời gọi mạng.

Cấu trúc của lời gọi gRPC từ máy khách luôn có dạng: `client.TenThuTucAsync(request)` cho trường hợp đơn giản nhất, hoặc `client.TenThuTucAsync(request, deadline: ..., cancellationToken: ...)` khi cần kiểm soát thời gian chờ và hủy. Không giống với REST nơi ta cần tự serialize đối tượng thành JSON và gửi qua `HttpClient`, gRPC xử lý toàn bộ việc đóng gói tham số và mở gói kết quả một cách trong suốt. Người lập trình làm việc trực tiếp với các kiểu dữ liệu C# được sinh từ `.proto` – không cần quan tâm đến định dạng trên đường truyền.

5.2.5 *Xử lý lỗi, deadline và hủy yêu cầu*

Trong hệ thống phân tán, lỗi là điều không thể tránh khỏi. Máy chủ có thể từ chối tham số không hợp lệ, mạng có thể gián đoạn, hoặc thời gian xử lý có thể vượt quá giới hạn cho phép. gRPC cung cấp ba cơ chế để đối phó với những tình huống này: mã trạng thái (status code), deadline và hủy yêu cầu (cancellation).

Mã trạng thái. Khi máy chủ gặp lỗi trong quá trình xử lý, nó ném `RpcException` với một đối tượng `Status` chứa hai thông tin: `StatusCode` (một enum với các giá trị như `InvalidArgument`, `NotFound`, `Internal`, `DeadlineExceeded`, v.v.) và `Detail` (chuỗi mô tả lỗi). Máy khách bắt `RpcException` và kiểm tra `StatusCode` để quyết định cách xử lý.

Trong Listing 5.2, phương thức `Divide` kiểm tra nếu `b==0` thì ném:

```
throw new RpcException(new Status(StatusCode.InvalidArgument, "Khong the chia cho
↪  0."));
```

Phía máy khách trong Listing 5.4, lời gọi `DivideAsync` với `B=0` được bao trong khối `try-catch`:

```
try
{
    await client.DivideAsync(new CalcRequest { A = 10, B = 0 });
}
catch (RpcException ex)
{
```

```
    Console.WriteLine($"Divide(10, 0) -> Loi RPC: {ex.StatusCode} -
    ↪  {ex.Status.Detail}");
}
```

Kết quả in ra sẽ là `InvalidArgument-Khongthechiacho0.`, cho thấy mã trạng thái và thông điệp lỗi được truyền nguyên vẹn từ máy chủ sang máy khách.

Deadline là cơ chế giới hạn tổng thời gian một lời gọi được phép tồn tại, tính từ thời điểm máy khách gửi yêu cầu. Deadline được truyền dưới dạng tham số `deadline: DateTime.UtcNow.AddMilliseconds(...)` trong lời gọi phương thức. Khi deadline bị vượt quá, gRPC tự động hủy lời gọi và ném `RpcException` với `StatusCode.DeadlineExceeded`. Deadline khác với timeout ở chỗ nó được lan truyền từ máy khách sang máy chủ: nếu máy khách đặt deadline 500ms, máy chủ cũng nhận được giá trị này và có thể chủ động dừng xử lý nếu biết rằng không thể hoàn tất kịp.

Trong Listing 5.4, lời gọi `AddAsync` với `deadline:DateTime.UtcNow.AddMilliseconds(-1)` (deadline đã hết hạn trước khi gửi đi) minh họa trường hợp deadline bị vượt quá. Trong thực tế, deadline nên được đặt đủ lớn để bao phủ thời gian xử lý bình thường, nhưng đủ nhỏ để không giữ tài nguyên quá lâu – đây là một quyết định thiết kế cần cân nhắc dựa trên đặc điểm của từng dịch vụ.

Hủy yêu cầu (cancellation) cho phép máy khách chủ động hủy một lời gọi đang thực thi thông qua `CancellationToken`. Cơ chế này hữu ích khi người dùng thoát khỏi màn hình đang chờ kết quả, hoặc khi một tác vụ phụ thuộc bị hủy và không còn cần kết quả nữa. Trong Listing 5.4, một `CancellationTokenSource` được tạo và hủy ngay lập tức trước khi gọi `AddAsync`. Lời gọi ném `RpcException` với `StatusCode.Cancelled`.

Thứ tự ưu tiên giữa deadline và cancellation rất quan trọng: nếu cả hai cùng được chỉ định, lời gọi sẽ thất bại với điều kiện nào đến trước. Ví dụ, nếu deadline là 5 giây nhưng token bị hủy sau 2 giây, lời gọi thất bại với `Cancelled` sau 2 giây. Ngược lại, nếu token chưa bị hủy nhưng 5 giây đã trôi qua, lời gọi thất bại với `DeadlineExceeded`.

Cả ba cơ chế – mã trạng thái, deadline và hủy yêu cầu – nên được sử dụng phối hợp trong ứng dụng thực tế. Một mẫu thiết kế phổ biến là: mọi lời gọi gRPC đều có deadline mặc định (ví dụ 30 giây), các lời gọi nhạy cảm với thời gian có deadline ngắn hơn (ví dụ 5 giây), và `CancellationToken` được truyền từ tầng trên (ASP.NET Core cung cấp `HttpContext.RequestAborted` cho mỗi yêu cầu HTTP đến). Phần 5.3 sẽ áp dụng trực tiếp mẫu này khi xây dựng lời gọi gRPC giữa `Submission.Api` và `Grading.GrpcService` trong hệ thống EduSubmit.

Tóm lại, kỹ thuật lập trình gRPC xoay quanh năm trụ cột: khai báo service và message trong tệp `.proto` để định nghĩa hợp đồng giao tiếp, cấu hình sinh mã stub trong `.csproj` để tự động tạo lớp máy khách và lớp cơ sở máy chủ, hiện thực dịch vụ bằng cách kế thừa lớp cơ sở và đăng ký qua `AddGrpc()`/`MapGrpcService<T>()`, tạo kênh và gọi phương thức từ máy khách qua lớp client được sinh tự động, và xử lý lỗi thông qua `RpcException`, deadline và `CancellationToken`. Phần tiếp theo sẽ áp dụng toàn bộ các kỹ thuật này để bổ sung kênh giao tiếp gRPC nội bộ vào hệ thống EduSubmit, nơi `Submission.Api` gọi `Grading.GrpcService` để chấm điểm bài nộp.

5.3 [Ứng dụng] Xây dựng hệ thống dịch vụ phân tán đơn giản với gRPC

Ở chương 4, EduSubmit đã có một điểm cuối GraphQL để máy khách chọn đúng các trường cần đọc từ danh sách bài tập. GraphQL trong bài đó thuộc nhóm API hướng máy khách: nó tối ưu cho màn hình web hoặc di động cần dữ liệu linh hoạt. Phần thực hành này chuyển sang một ranh giới khác của cùng hệ thống: giao tiếp nội bộ giữa các dịch vụ. Thay vì để máy khách gọi trực tiếp dịch vụ chấm bài, ta giữ `Submission.Api` làm API ở biên ngoài và để API này gọi `Grading.GrpcService` bằng gRPC ở phía sau.

Lát cắt thực hành của chương này vì vậy bổ sung một vai trò mới cho EduSubmit. Khi sinh viên gửi bài nộp qua REST, `Submission.Api` nhận dữ liệu, tạo một yêu cầu gRPC dựa trên hợp đồng `.proto`, gửi sang dịch vụ chấm bài, nhận điểm số và phản hồi kết quả REST cho máy khách. Cách tổ chức này giúp sinh viên nhìn rõ ranh giới đã thảo luận ở phần 5.4: REST hoặc GraphQL phù hợp với giao diện bên ngoài, còn gRPC phù hợp với lời gọi nội bộ có hợp đồng chặt chẽ.

Ví dụ nguồn của bài này nằm trong thư mục `content/examples/chapter05/section02`. Cấu trúc dự án gồm hai tiến trình độc lập:

```
examples/chapter05/section03/
  GrpcSubmissionSystem.slnx
  src/
    Grading.GrpcService/
    Submission.Api/
```

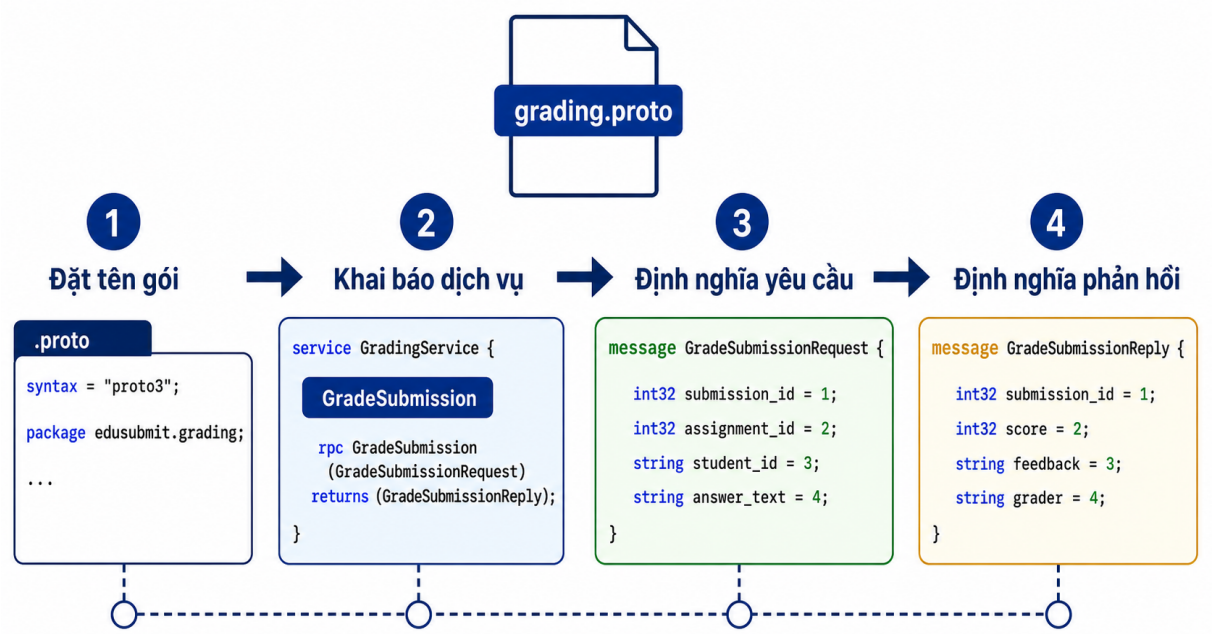
`Grading.GrpcService` là dịch vụ gRPC nội bộ, chịu trách nhiệm chấm một bài nộp đơn giản. `Submission.Api` là Minimal API nhận yêu cầu REST từ bên ngoài và đóng vai trò máy khách gRPC khi cần gọi sang dịch vụ chấm bài. Ví dụ cố ý giữ logic chấm điểm ở mức giả lập để trọng tâm của bài nằm ở hợp đồng `.proto`, mã sinh tự động và lời gọi dịch vụ–dịch vụ.

5.3.1 Tạo dịch vụ gRPC

Điểm khởi đầu của gRPC không phải là lớp C# mà là hợp đồng giao tiếp. Trong ví dụ này, hợp đồng được đặt trong tệp `grading.proto`. Tệp này khai báo một dịch vụ `GradingService` với thủ tục `GradeSubmission`. Thủ tục nhận thông tin bài nộp và trả về điểm số cùng phản hồi ngắn.

Mã nguồn 5.5: Hợp đồng gRPC cho dịch vụ chấm bài – khai báo service và message trong grading.proto

```
syntax = "proto3";

option csharp_namespace = "EduSubmit.Grpc";

package edusubmit.grading;

service GradingService {
  rpc GradeSubmission (GradeSubmissionRequest) returns (GradeSubmissionReply);
}

message GradeSubmissionRequest {
  int32 submission_id = 1;
```



Hình 5.5: Các bước chính khi viết hợp đồng gRPC trong Listing 5.1

```
  int32 assignment_id = 2;
  string student_id = 3;
  string answer_text = 4;
}

message GradeSubmissionReply {
  int32 submission_id = 1;
  int32 score = 2;
  string feedback = 3;
  string grader = 4;
}
```

Listing trên có ba phần cần chú ý. Khai báo `service` xác định các thủ tục từ xa mà dịch vụ cung cấp. Hai `message` xác định hình dạng dữ liệu đi vào và đi ra. Mỗi trường có một số thứ tự như `submission_id=1`; số này là định danh nhị phân của trường khi protobuf tuần tự hóa dữ liệu, nên không nên tùy tiện đổi sau khi hợp đồng đã được dùng bởi nhiều dịch vụ.

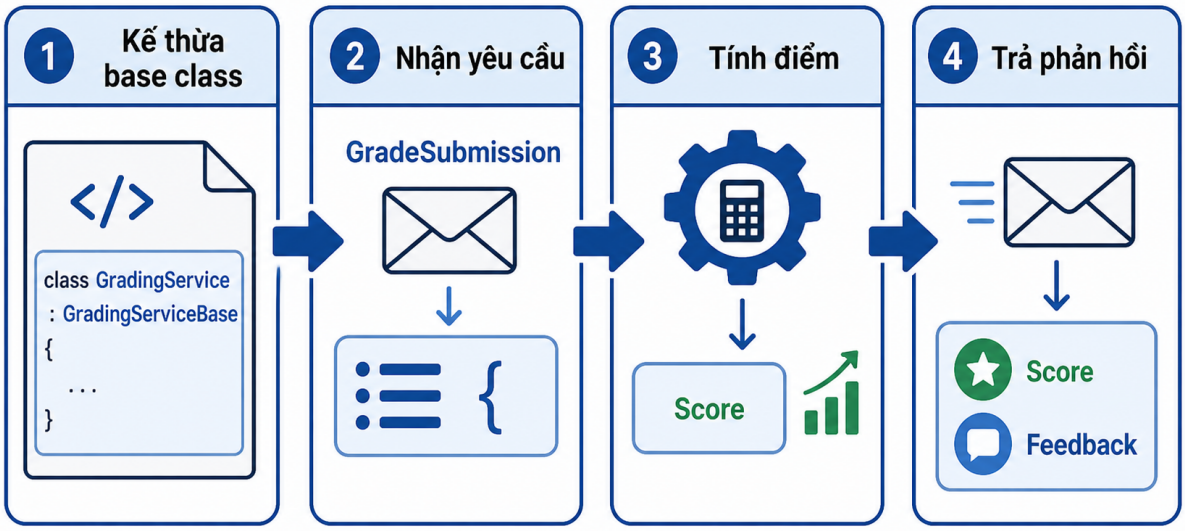
Tùy chọn `csharp_namespace` cho biết mã C# sinh ra sẽ nằm trong không gian tên nào. Khi biên dịch project, công cụ gRPC đọc tệp `.proto` và sinh ra mã cho yêu cầu, phản hồi và lớp cơ sở của dịch vụ. Người lập trình không viết các lớp này bằng tay; ta chỉ triển khai phần logic nghiệp vụ còn thiếu.

Project `Grading.GrpcService` khai báo tệp `.proto` trong `.csproj` như sau:

```
<Protobuf Include="Protos\grading.proto" GrpcServices="Server" />
```

Giá trị `GrpcServices="Server"` nói rằng project này cần mã phía máy chủ. Ở project máy khách, ta sẽ dùng cùng hợp đồng nhưng đổi sang `GrpcServices="Client"`. Đây là điểm khác biệt quan trọng so với nhiều ví dụ REST trước đó: client và server không tự diễn giải JSON theo hai cách riêng, mà cùng được sinh mã từ một hợp đồng duy nhất.

Sau khi có hợp đồng, ta triển khai dịch vụ chấm bài trong C#. Lớp dịch vụ kế thừa lớp cơ sở do gRPC sinh ra và ghi đè phương thức tương ứng với thủ tục trong `.proto`.



Hình 5.6: Các bước chính khi triển khai dịch vụ gRPC trong Listing 5.2

Mã nguồn 5.6: Triển khai dịch vụ chấm bài gRPC

```
using EduSubmit.Grpc;
using Grpc.Core;

namespace Grading.GrpcService.Services;

public sealed class GradingService : EduSubmit.Grpc.GradingService.GradingServiceBase {
    public override Task<GradeSubmissionReply> GradeSubmission(
        GradeSubmissionRequest request,
        ServerCallContext context) {
        var normalizedAnswer = request.AnswerText.Trim();
        var score = normalizedAnswer.Length switch {
            >= 80 => 10,
            >= 40 => 8,
            >= 12 => 6,
            _ => 3
        };

        var feedback = score >= 8
            ? "Bai nop co cau tra loi ro rang, du thong tin chinh."
            : "Bai nop can bo sung lap luan va vi du cu the hon.";

        return Task.FromResult(new GradeSubmissionReply {
            SubmissionId = request.SubmissionId,
            Score = score,
            Feedback = feedback,
            Grader = "Grading.GrpcService"
        });
    }
}
```

Trong listing này, phương thức `GradeSubmission` nhận `GradeSubmissionRequest` đã được mở gói từ thông điệp gRPC. Ví dụ dùng độ dài câu trả lời để tính điểm giả lập,

sau đó trả về `GradeSubmissionReply`. Phần quan trọng không nằm ở công thức tính điểm, mà nằm ở kiểu dữ liệu: cả yêu cầu lẫn phản hồi đều là các lớp sinh từ `grading.proto`. Nếu người lập trình đổi tên trường hoặc đổi kiểu dữ liệu sai hợp đồng, lỗi thường xuất hiện ngay lúc biên dịch thay vì đợi tới khi chạy.

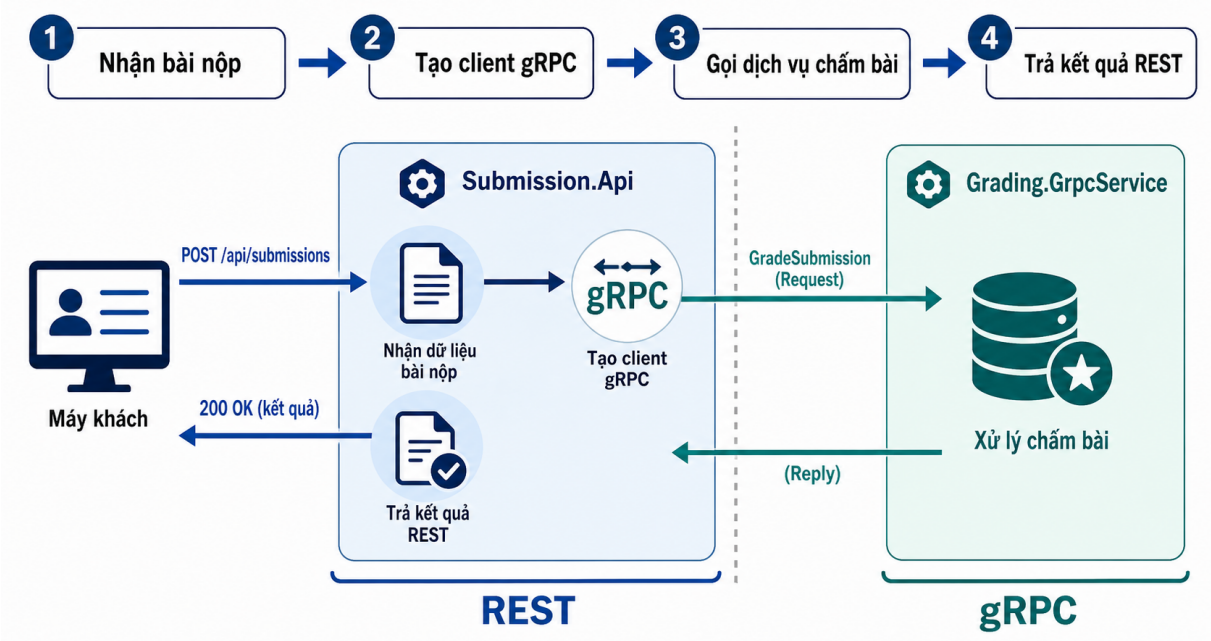
Tệp `Program.cs` của dịch vụ gRPC chỉ cần đăng ký gRPC và ánh xạ lớp dịch vụ:

```
builder.Services.AddGrpc();

var app = builder.Build();

app.MapGrpcService<GradingService>();
```

Khi chạy, `Grading.GrpcService` không được gọi bằng trình duyệt giống một điểm cuối REST thông thường. Máy khách cần dùng gRPC client sinh từ cùng hợp đồng `.proto`. Đây là lý do dịch vụ gRPC thường được đặt sau API Gateway hoặc sau các API hướng máy khách, thay vì công khai trực tiếp cho trình duyệt.

5.3.2 *Gọi gRPC giữa các thành phần*

Phần tiếp theo là `Submission.Api`. Đây vẫn là API mà máy khách bên ngoài gọi bằng REST, tương tự các chương trước. Điểm mới là API này không tự chấm bài, mà gọi nội bộ sang `Grading.GrpcService`. Cách làm này mô phỏng một kiến trúc microservices nhỏ: dịch vụ nhận bài nộp giữ trách nhiệm nhận và lưu bài, còn dịch vụ chấm bài giữ trách nhiệm tính điểm.



Hình 5.7: Các bước chính khi API nộp bài gọi dịch vụ chấm bài bằng gRPC

Mã nguồn 5.7: API nộp bài gọi dịch vụ chấm bài bằng gRPC

```
using EduSubmit.Grpc;
using Grpc.Net.Client;
```

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton(_ => GrpcChannel.ForAddress("http://localhost:5101"));
builder.Services.AddSingleton(provider =>
    new GradingService.GradingServiceClient(provider.GetRequiredService<GrpcChannel>()
    ↪  ));

var app = builder.Build();

app.MapPost("/api/submissions", async (
    SubmitAssignmentRequest request,
    GradingService.GradingServiceClient gradingClient) => {
        var submission = new StoredSubmission(
            SubmissionId: 1001,
            request.AssignmentId,
            request.StudentId,
            request.AnswerText);

        var gradingReply = await gradingClient.GradeSubmissionAsync(new
        ↪  GradeSubmissionRequest {
            SubmissionId = submission.SubmissionId,
            AssignmentId = submission.AssignmentId,
            StudentId = submission.StudentId,
            AnswerText = submission.AnswerText
        });

        return Results.Ok(new SubmissionResult(
            submission.SubmissionId,
            gradingReply.Score,
            gradingReply.Feedback,
            gradingReply.Grader));
    });

app.Run();

public sealed record SubmitAssignmentRequest(
    int AssignmentId,
    string StudentId,
    string AnswerText);

public sealed record StoredSubmission(
    int SubmissionId,
    int AssignmentId,
    string StudentId,
    string AnswerText);

public sealed record SubmissionResult(
    int SubmissionId,
    int Score,
    string Feedback,
    string Grader);
```

Đầu file, `Submission.Api` tạo `GrpcChannel` tới địa chỉ nội bộ `http://localhost:5101`, sau đó đăng ký `GradingServiceClient`. Client này không phải lớp tự viết; nó được sinh từ `grading.proto` nhờ khai báo `GrpcServices="Client"` trong project API. Khi điểm cuối `POST/api/submissions` nhận bài nộp, API tạo `GradeSubmissionRequest`, gọi `GradeSubmissionAsync`, rồi chuyển `GradeSubmissionReply` thành phản hồi REST trả về cho máy khách.

Luồng dữ liệu trong listing có thể đọc theo bốn bước. Thứ nhất, máy khách gửi JSON tới `Submission.Api`. Thứ hai, API chuyển dữ liệu đó sang thông điệp protobuf có kiểu rõ ràng. Thứ ba, API gọi gRPC tới `Grading.GrpcService`. Thứ tư, kết quả từ gRPC được đóng gói lại thành JSON REST. Như vậy, ranh giới bên ngoài vẫn dễ thử bằng công cụ HTTP quen thuộc, còn ranh giới nội bộ được hưởng lợi từ hợp đồng chặt chẽ và mã sinh tự động.

> Trong ví dụ học tập này, `GrpcChannel` được trỏ thẳng tới `localhost`. Trong hệ thống triển khai thật, địa chỉ dịch vụ thường đến từ cấu hình, biến môi trường, service discovery hoặc service mesh. Không nên ghi cứng địa chỉ nội bộ vào code sản xuất.

Biên dịch toàn bộ solution bằng lệnh:

```
dotnet build examples/chapter05/section03/GrpcSubmissionSystem.slnx
```

Sau đó chạy dịch vụ chấm bài ở cửa sổ dòng lệnh thứ nhất:

```
dotnet run --project examples/chapter05/section03/src/Grading.GrpcService --urls
↪  http://localhost:5101
```

Chạy API nộp bài ở cửa sổ dòng lệnh thứ hai:

```
dotnet run --project examples/chapter05/section03/src/Submission.Api --urls
↪  http://localhost:5100
```

Gửi một bài nộp mẫu bằng PowerShell:

```
Invoke-RestMethod `
  -Method Post `
  -Uri http://localhost:5100/api/submissions `
  -ContentType "application/json" `
  -Body '{"assignmentId":1,"studentId":"SV001","answerText":"gRPC dung hop dong .proto
  ↪  de tao client stub va server stub."}'
```

Kết quả mong đợi có dạng:

```
submissionId score feedback                                        grader
------------ ----- --------                                        ------
        1001     8 Bai nop co cau tra loi ro rang, du thong...   Grading.GrpcService
```

Trường `grader` cho thấy kết quả được tạo bởi dịch vụ chấm bài nội bộ. Nếu tắt `Grading.GrpcService` rồi gọi lại `POST/api/submissions`, `Submission.Api` sẽ không thể hoàn tất lời gọi gRPC. Đây là một điểm thực hành quan trọng: khi tách hệ thống thành nhiều dịch vụ, ta có được ranh giới trách nhiệm rõ hơn, nhưng cũng phải xử lý thêm các vấn đề của hệ thống phân tán như lỗi kết nối, hết thời gian chờ, thử lại và theo dõi nhật ký. Các chủ đề này sẽ tiếp tục xuất hiện khi EduSubmit chuyển sang truyền thông bất đồng bộ ở chương 6.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **RPC (Remote Procedure Call)** là mô hình giao tiếp che giấu sự phức tạp của truyền thông mạng, cho phép gọi thủ tục từ xa như gọi thủ tục cục bộ. Hai thành phần then chốt là client stub (đóng gói tham số – marshalling) và server stub (mở gói tham số – unmarshalling).
- **Ba biến thể RPC**: Asynchronous RPC (server gửi ACK ngay, client không chờ kết quả), Deferred Synchronous RPC (kết hợp async với callback), One-way RPC (bắn và quên, không ACK).
- **gRPC** là khung RPC hiện đại dựa trên ba trụ cột: HTTP/2 làm tầng vận chuyển (multiplexing, header compression), Protocol Buffers làm IDL và định dạng tuần tự hóa nhị phân, và hỗ trợ bốn kiểu giao tiếp (unary, server streaming, client streaming, bidirectional streaming).
- **Protocol Buffers (protobuf)** là ngôn ngữ đặc tả giao diện của gRPC. File `.proto` gồm ba thành phần: khai báo `message` (cấu trúc dữ liệu), khai báo `service` (thủ tục RPC), và `option` (cấu hình sinh mã). Trình biên dịch protobuf tự động sinh client stub, server stub và message classes.
- **Giao tiếp dựa trên hợp đồng** mang lại ba lợi ích: khả năng tương tác (interoperability) giữa các ngôn ngữ/nền tảng, khả năng chuyển đổi (portability) khi thay đổi triển khai, và an toàn kiểu tại thời điểm biên dịch (compile-time type safety).
- **Phân biệt client-facing API và internal API**: API hướng máy khách (REST, GraphQL) ưu tiên tính dễ hiểu, khám phá và bộ nhớ đệm. API nội bộ (gRPC, message queue) ưu tiên hiệu năng, độ trễ thấp, an toàn kiểu và streaming. API Gateway đóng vai trò cầu nối giữa hai thế giới.
- **Thực hành gRPC**: Hợp đồng `.proto` được khai báo trong `.csproj` với `GrpcServices="Server"` hoặc `"Client"`. Server triển khai lớp kế thừa từ lớp cơ sở sinh tự động. Client dùng `GrpcChannel` và client stub sinh từ cùng hợp đồng để gọi dịch vụ.
- **Luồng dữ liệu trong hệ thống hybrid**: Máy khách gửi JSON REST -> API chuyển sang protobuf -> gọi gRPC nội bộ -> kết quả protobuf chuyển lại thành JSON REST. Ranh giới ngoài dùng HTTP quen thuộc, ranh giới trong hưởng lợi từ hợp đồng chặt chẽ.

Câu hỏi ôn tập

1. Nêu định nghĩa RPC. Giải thích vai trò của client stub và server stub trong một lời gọi RPC. Vẽ sơ đồ minh họa luồng dữ liệu giữa các thành phần.
2. Phân biệt ba biến thể của RPC: asynchronous, deferred synchronous và one-way. Với mỗi biến thể, cho một ví dụ cụ thể trong hệ thống EduSubmit.
3. gRPC khác gì so với RPC cổ điển (DCE RPC)? Nêu và giải thích ba trụ cột chính làm nên sự khác biệt đó.
4. Liệt kê và mô tả bốn kiểu giao tiếp mà gRPC hỗ trợ. Với mỗi kiểu, cho biết nó phù hợp với tình huống nào trong hệ thống phân tán.
5. Protocol Buffers là gì? So sánh protobuf với JSON về: kích thước thông điệp, tốc độ tuần tự hóa, và kiểm tra kiểu dữ liệu.
6. Một file `.proto` điển hình gồm những thành phần nào? Giải thích ý nghĩa của field number trong khai báo `message` và tại sao không nên thay đổi nó sau khi

hợp đồng đã được triển khai.

7. Trình bày ba lợi ích của giao tiếp dựa trên hợp đồng (contract-based communication) trong hệ thống phân tán. Cho ví dụ minh họa cho mỗi lợi ích.
8. Lập bảng so sánh API hướng máy khách và API nội bộ theo ít nhất 5 tiêu chí. Từ đó giải thích tại sao không nên dùng REST cho mọi loại giao tiếp trong hệ thống microservices.
9. Trong ví dụ thực hành `Grading.GrpcService`, hãy giải thích ý nghĩa của khai báo `GrpcServices="Server"` trong `.csproj`. Nếu project `Submission.Api` cần gọi dịch vụ này, khai báo tương ứng sẽ là gì?
10. Mô tả luồng dữ liệu đầy đủ khi một máy khách gửi bài nộp qua REST và nhận kết quả chấm điểm. Chỉ rõ: đâu là giao tiếp client-facing, đâu là giao tiếp internal, và định dạng dữ liệu tại mỗi ranh giới.

Bài tập thực hành

1. **Mở rộng hợp đồng gRPC**. Sao chép mã nguồn từ `content/examples/chapter05/section02/` và thêm một thủ tục RPC mới vào `grading.proto`: `GetSubmissionHistory` trả về danh sách các lần chấm trước đó cho một sinh viên. Cập nhật server để triển khai thủ tục mới, cập nhật client để gọi thủ tục này và in kết quả. Chạy và xác nhận.
2. **Thêm server streaming**. Mở rộng `grading.proto` với thủ tục `WatchSubmissions` dùng server streaming: client gửi mã sinh viên, server trả về luồng các bài nộp đang được chấm theo thời gian thực. Triển khai phía server (dùng `await foreach` hoặc `yield return`) và phía client (dùng `ResponseStream`). Mô tả sự khác biệt khi gỡ lỗi so với unary RPC.
3. **So sánh hiệu năng REST và gRPC**. Viết một chương trình đo: gọi cùng một chức năng chấm điểm qua (a) REST endpoint mới (`POST /api/grading`) và (b) gRPC client trực tiếp. Đo thời gian phản hồi và kích thước dữ liệu cho 100 lần gọi. Báo cáo sự khác biệt và giải thích nguyên nhân.
4. **Thiết kế hợp đồng gRPC cho dịch vụ mới**. EduSubmit cần thêm dịch vụ thông báo (Notification Service). Hãy thiết kế file `notification.proto` với ít nhất hai thủ tục RPC: gửi thông báo đến một người dùng (unary) và theo dõi luồng thông báo (server streaming). Giải thích lý do chọn kiểu RPC cho từng thủ tục.

Câu hỏi thảo luận

1. Một nhóm phát triển đề xuất dùng gRPC cho toàn bộ giao tiếp trong hệ thống, bao gồm cả giao tiếp với trình duyệt. Hãy phân tích những thách thức của đề xuất này (trình duyệt hỗ trợ, gỡ lỗi, bộ nhớ đệm, công cụ kiểm thử). Trong điều kiện nào thì đề xuất này khả thi?
2. Khi một trường trong `.proto` bị thay đổi kiểu dữ liệu hoặc bị xóa, điều gì xảy ra với các client và server đang dùng phiên bản cũ của hợp đồng? Thảo luận các chiến lược quản lý phiên bản hợp đồng (versioning) trong gRPC để duy trì khả năng tương thích ngược.
3. So sánh ba cách tổ chức giao tiếp nội bộ trong hệ thống phân tán: REST nội bộ, gRPC, và message queue. Với mỗi cách, nêu ưu điểm, hạn chế và tình huống phù hợp. Nếu bạn thiết kế EduSubmit từ đầu, bạn sẽ chọn cách nào cho giao tiếp giữa API và dịch vụ chấm điểm? Vì sao?

CHƯƠNG 6

XỬ LÝ BẤT ĐỒNG BỘ VỚI HÀNG ĐỢI VÀ DỊCH VỤ XỬ LÝ NỀN

Mục tiêu chương
Sau khi học xong chương này, người học có thể:

- Nhận diện mẫu trao đổi thông điệp bất đồng bộ dựa trên hàng đợi và phân biệt với các mẫu đồng bộ đã học (REST, gRPC).
- Phân tích ba hạn chế của xử lý đồng bộ và giải thích cách hàng đợi thông điệp khắc phục từng hạn chế.
- Giải thích mẫu dịch vụ xử lý nền (worker service pattern), vai trò của producer, consumer, và cơ chế cạnh tranh giữa các consumer.
- Thiết kế và xây dựng luồng xử lý bất đồng bộ với RabbitMQ: API gửi công việc vào hàng đợi, worker nhận và xử lý, và trả trạng thái "processing" cho máy khách.
- Chạy thử và quan sát hành vi của hệ thống bất đồng bộ trong các tình huống: worker hoạt động bình thường, worker ngừng đột ngột, và nhiều worker cạnh tranh cùng lúc.

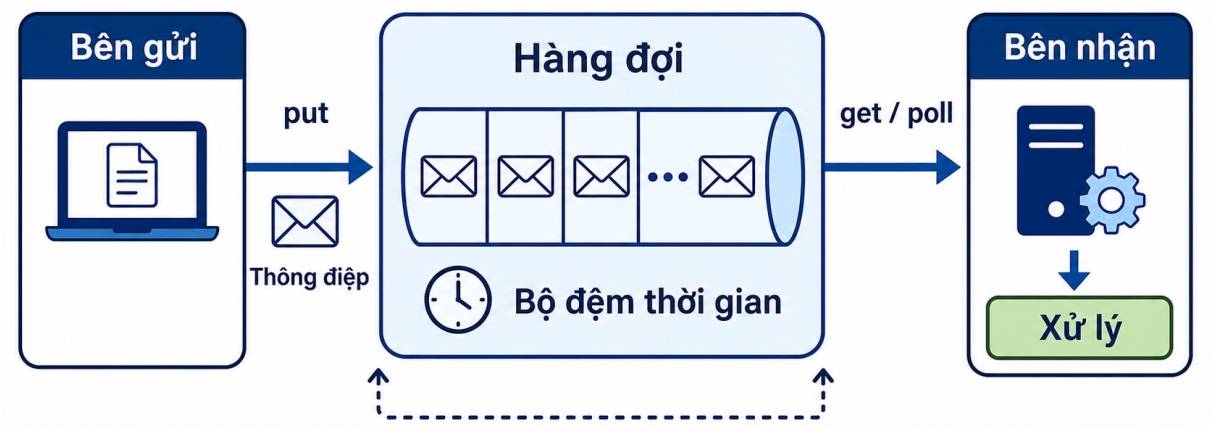
6.1 Xử lý bất đồng bộ với hàng đợi

6.1.1 MEP: bất đồng bộ dựa trên hàng đợi

Trong các chương trước, chúng ta đã làm việc với các mẫu trao đổi thông điệp đồng bộ: REST (bài 2) và gRPC (bài 5) đều dựa trên mô hình yêu cầu–phản hồi, trong đó máy khách gửi yêu cầu và chờ cho đến khi nhận được phản hồi từ máy chủ. SignalR (bài 3) mở rộng mô hình này với khả năng đẩy dữ liệu thời gian thực, nhưng vẫn duy trì kết nối trực tiếp giữa hai bên. Chương này giới thiệu một mẫu trao đổi thông điệp khác về bản chất: **bất đồng bộ dựa trên hàng đợi (asynchronous queue-based MEP)**.

Ý tưởng cốt lõi của mẫu này rất đơn giản: thay vì giao tiếp trực tiếp, bên gửi đưa thông điệp vào một **hàng đợi (queue)** – một vùng đệm trung gian do phần mềm trung gian hướng thông điệp (Message-Oriented Middleware – MOM) quản lý. Bên nhận, khi sẵn sàng, lấy thông điệp ra khỏi hàng đợi và xử lý. Hai bên không cần biết về sự tồn tại của nhau, không cần chạy đồng thời, và không cần thống nhất về tốc độ xử lý. Hàng đợi đóng vai trò như một **bộ đệm thời gian (temporal buffer)**, hấp thụ sự khác biệt giữa tốc độ gửi và tốc độ xử lý.

Khác với RPC, nơi cả bên gọi và bên được gọi phải hoạt động đồng thời (temporal coupling), giao tiếp qua hàng đợi cho phép **tách rời về mặt thời gian (temporal decoupling)**. Bên gửi có thể đưa thông điệp vào hàng đợi ngay cả khi bên nhận chưa khởi động. Bên nhận có thể lấy thông điệp ra xử lý sau hàng phút, thậm chí hàng giờ. Đây là điểm khác biệt căn bản so với socket hay RPC – những cơ chế được thiết kế để truyền



Hình 6.1: Mẫu trao đổi thông điệp bất đồng bộ dựa trên hàng đợi

dữ liệu trong mili-giây, còn hệ thống hàng đợi được thiết kế để hỗ trợ truyền thông điệp có thể kéo dài nhiều phút.

Giao diện làm việc với hàng đợi thường cung cấp bốn thao tác cơ bản:

- `put`: Đưa thông điệp vào hàng đợi. Thao tác không chặn (non-blocking) – bên gửi tiếp tục xử lý ngay sau khi thông điệp được chấp nhận.
- `get`: Lấy thông điệp khỏi hàng đợi. Thao tác chặn (blocking) – nếu hàng đợi rỗng, bên nhận tạm dừng cho đến khi có thông điệp mới.
- `poll`: Kiểm tra và lấy thông điệp không chặn. Nếu hàng đợi rỗng, trả về ngay với trạng thái "không có thông điệp".
- `notify`: Đăng ký một hàm gọi lại (callback) được kích hoạt tự động khi có thông điệp mới đến.

Tùy theo nhu cầu, hệ thống hàng đợi có thể hoạt động theo hai chế độ truyền thông:

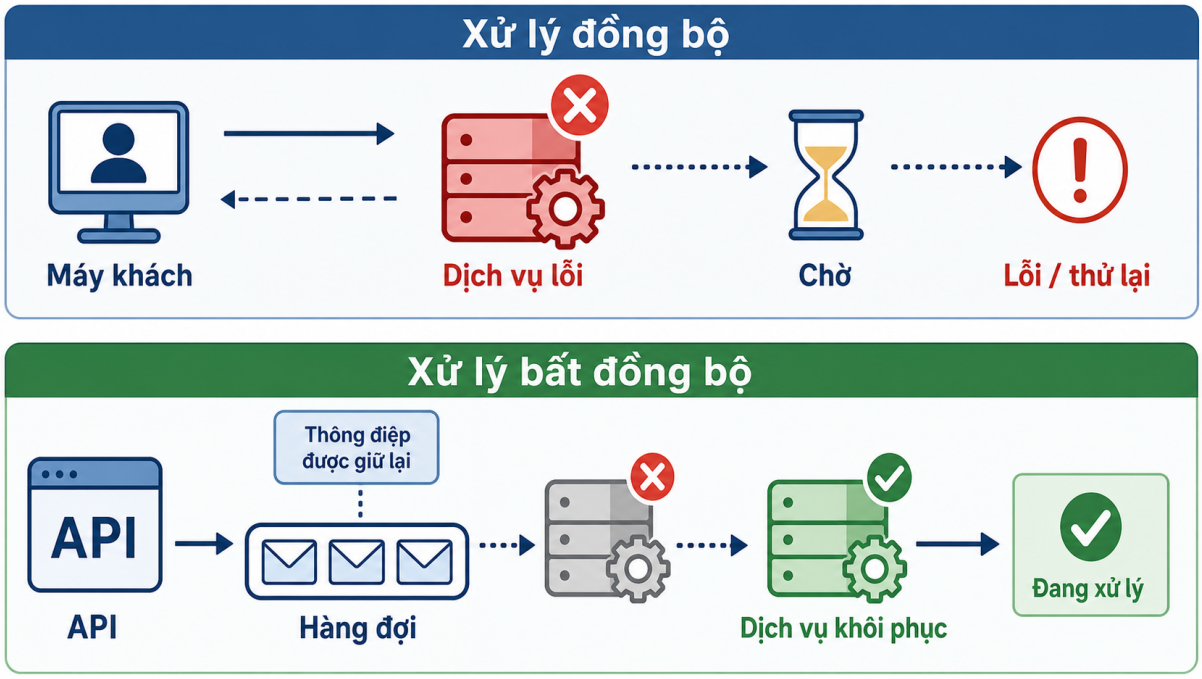
- **Tạm thời (transient)**: Thông điệp chỉ tồn tại trong bộ nhớ. Nếu bên nhận không hoạt động tại thời điểm gửi, thông điệp bị mất. Phù hợp cho dữ liệu không quan trọng hoặc có thể gửi lại dễ dàng.
- **Bền vững (persistent)**: Thông điệp được lưu vào đĩa hoặc bộ nhớ bền vững. Nếu cả bên gửi và bên nhận đều không hoạt động, thông điệp vẫn được giữ lại và sẽ được xử lý khi hệ thống khôi phục. Đây là chế độ mặc định cho các hệ thống sản xuất.

6.1.2 Hạn chế của xử lý đồng bộ

Để hiểu tại sao giao tiếp bất đồng bộ qua hàng đợi lại cần thiết, ta cần nhìn lại những hạn chế của xử lý đồng bộ. Trong kiến trúc RPC truyền thống (bao gồm REST và gRPC), client gửi yêu cầu và bị chặn cho đến khi server xử lý xong và trả về kết quả. Mô hình này tuy đơn giản và trực quan, nhưng bộc lộ ba vấn đề chính khi áp dụng vào hệ thống phân tán quy mô lớn.

Vấn đề thứ nhất: liên kết chặt về mặt thời gian (temporal coupling). Trong giao tiếp đồng bộ, cả bên gọi và bên được gọi phải đồng thời hoạt động. Nếu server ngừng hoạt động tại thời điểm client gửi yêu cầu, yêu cầu đó thất bại hoàn toàn. Client buộc phải tự triển khai cơ chế thử lại (retry), và trong lúc chờ đợi, toàn bộ luồng xử lý của client bị đình trệ. Trong một hệ thống có hàng chục dịch vụ phụ thuộc lẫn nhau, một

dịch vụ gặp sự cố có thể tạo ra hiệu ứng domino, làm tê liệt toàn bộ chuỗi xử lý.



Hình 6.2: So sánh xử lý đồng bộ và xử lý bất đồng bộ khi dịch vụ xử lý gặp lỗi

Vấn đề thứ hai: không hấp thụ được tải đột biến (load spikes). Trong mô hình đồng bộ, mỗi yêu cầu đến server đều được xử lý ngay lập tức. Khi lưu lượng tăng đột biến – ví dụ: sinh viên đồng loạt nộp bài trước hạn chót – server phải xử lý tất cả yêu cầu cùng lúc. Nếu số lượng yêu cầu vượt quá khả năng xử lý, server trở nên quá tải, thời gian phản hồi tăng vọt, và một số yêu cầu bị từ chối. Việc mở rộng server (scale up/out) cần thời gian, trong khi tải đột biến xảy ra tức thời.

Vấn đề thứ ba: xử lý kéo dài làm nghẽn luồng chính. Một số tác vụ – như chấm điểm tự động, tạo báo cáo, gửi email hàng loạt – có thể mất từ vài giây đến vài phút để hoàn thành. Nếu những tác vụ này được thực thi đồng bộ trong luồng xử lý của API, chúng giữ kết nối HTTP mở trong suốt thời gian đó, tiêu tốn tài nguyên máy chủ và làm tăng độ trễ cho các yêu cầu khác. Tệ hơn, nếu kết nối bị ngắt giữa chừng (timeout), toàn bộ kết quả xử lý bị mất.

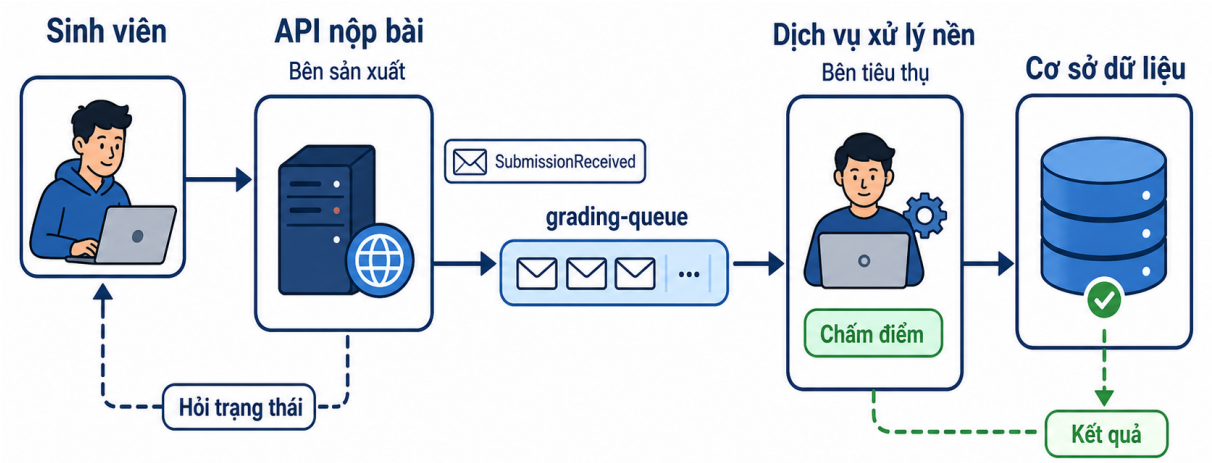
Xử lý đồng bộ không phải lúc nào cũng là lựa chọn sai. Với các thao tác nhanh, cần phản hồi ngay và ít rủi ro thất bại – như đọc danh sách bài tập hay kiểm tra trạng thái lượt nộp – REST hoặc gRPC vẫn là lựa chọn phù hợp. Hàng đợi được dùng khi: (1) tác vụ mất nhiều thời gian, (2) bên nhận có thể không hoạt động, (3) cần hấp thụ tải đột biến, hoặc (4) cần đảm bảo không mất dữ liệu khi có sự cố.

6.1.3 Hàng đợi và mẫu dịch vụ xử lý nền

Khi đã chấp nhận rằng không phải mọi tác vụ đều nên được xử lý đồng bộ, câu hỏi tiếp theo là: tổ chức bên xử lý như thế nào? Câu trả lời là **mẫu dịch vụ xử lý nền (worker service pattern)**, một trong những mẫu kiến trúc phổ biến nhất trong hệ thống phân tán.

Trong mẫu này, hệ thống được chia thành hai vai trò rõ rệt:

1. **Bên sản xuất (producer)**: Thường là API hoặc dịch vụ tiếp nhận yêu cầu từ người dùng. Khi cần thực thi một tác vụ không đồng bộ, producer không tự xử lý mà đóng gói thông tin thành một thông điệp và đưa vào hàng đợi. Ngay sau đó, producer trả về phản hồi cho người dùng – thường là trạng thái "đã tiếp nhận" hoặc "đang xử lý".
2. **Bên tiêu thụ (consumer / worker)**: Là một hoặc nhiều tiến trình chạy nền, liên tục theo dõi hàng đợi. Khi có thông điệp mới, worker lấy ra, xử lý, và – nếu cần – gửi kết quả hoặc cập nhật trạng thái.



Hình 6.3: Mẫu dịch vụ xử lý nền trong quy trình chấm điểm của EduSubmit

Hàng đợi đóng vai trò then chốt trong việc **tách rời (decouple)** producer và consumer. Producer không cần biết có bao nhiêu worker đang chạy, worker nào sẽ xử lý thông điệp của mình, hay worker đó được viết bằng ngôn ngữ gì. Ngược lại, worker không cần biết ai đã gửi thông điệp, chỉ cần biết cách xử lý nó. Sự tách rời này mang lại ba lợi ích kiến trúc quan trọng:

- **Khả năng mở rộng độc lập (independent scalability)**: Khi khối lượng công việc tăng, ta có thể tăng số lượng worker mà không cần thay đổi producer. Hàng đợi tự động phân phối thông điệp đến các worker theo cơ chế cạnh tranh (competing consumers): mỗi thông điệp chỉ được giao cho một worker, đảm bảo không xử lý trùng lặp.
- **Khả năng chịu lỗi (fault tolerance)**: Nếu một worker gặp sự cố và ngừng hoạt động, thông điệp đang xử lý dở có thể được trả lại hàng đợi (nếu cấu hình thời gian chờ – acknowledgment timeout) và được worker khác tiếp nhận. Thông điệp chưa được xử lý vẫn an toàn trong hàng đợi bền vững.
- **Khả năng cân bằng tải tự nhiên (natural load leveling)**: Khi producer gửi thông điệp nhanh hơn tốc độ xử lý của worker, thông điệp được xếp hàng (queued) thay vì bị từ chối. Khi tốc độ gửi giảm, worker dần xử lý hết tồn đọng. Hàng đợi hoạt động như một "bể chứa" hấp thụ sự chênh lệch tốc độ.

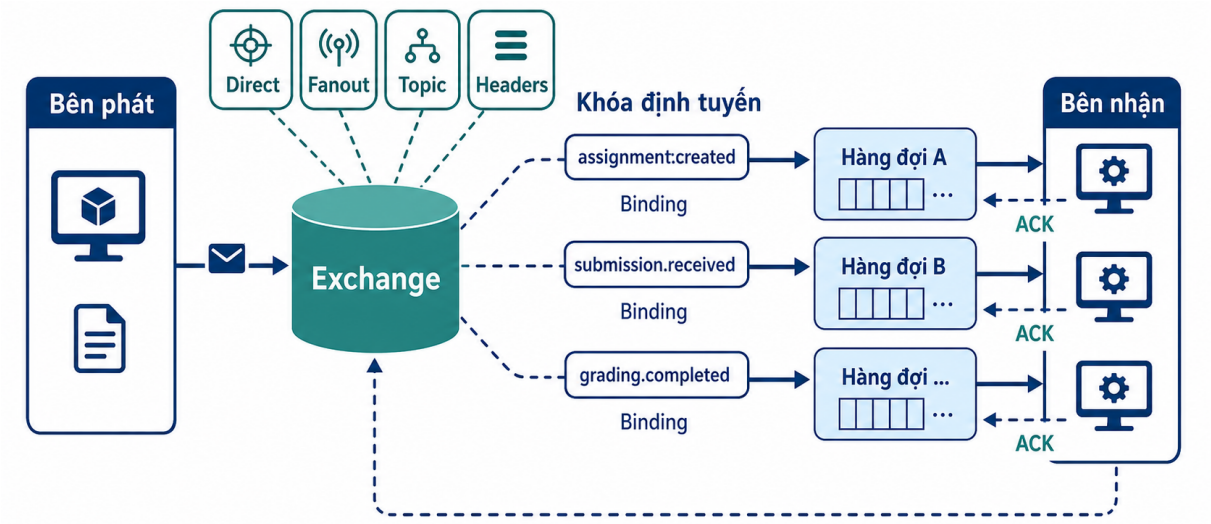
Trong EduSubmit, worker pattern được áp dụng cho quy trình chấm điểm tự động. Khi sinh viên nộp bài, API tiếp nhận bài nộp, lưu vào cơ sở dữ liệu, và gửi một thông điệp `SubmissionReceived` vào hàng đợi `grading-queue`. Worker chấm điểm (grading

worker) nhận thông điệp, thực thi quy trình chấm (biên dịch, chạy bộ kiểm thử, so sánh kết quả), và cập nhật kết quả vào cơ sở dữ liệu. Trong suốt quá trình này, API không hề bị chặn – sinh viên nhận được phản hồi "bài đã được tiếp nhận, đang chấm" gần như ngay lập tức.

6.1.4 *Tổng quan RabbitMQ*

Trong số các hệ thống hàng đợi thông điệp hiện nay, **RabbitMQ** là một trong những lựa chọn phổ biến nhất, đặc biệt trong hệ sinh thái .NET. RabbitMQ là một **message broker** mã nguồn mở, triển khai giao thức **AMQP (Advanced Message-Queuing Protocol)** – giao thức chuẩn hóa được phát triển từ năm 2006 nhằm đảm bảo khả năng tương tác giữa các hệ thống hàng đợi khác nhau.

Khác với mô hình hàng đợi đơn giản (producer gửi thẳng vào queue của consumer), RabbitMQ giới thiệu thêm một tầng trung gian: **exchange**. Trong mô hình này, producer không gửi thông điệp trực tiếp vào hàng đợi, mà gửi đến một exchange. Exchange, dựa trên **khóa định tuyến (routing key)** và kiểu của nó, quyết định chuyển thông điệp đến một hoặc nhiều hàng đợi. Cơ chế này cho phép RabbitMQ hỗ trợ nhiều mẫu định tuyến phức tạp hơn là chỉ "một hàng đợi, một bên nhận".



Hình 6.4: Kiến trúc định tuyến thông điệp trong RabbitMQ

RabbitMQ hỗ trợ bốn kiểu exchange chính:

- **Direct exchange**: Chuyển thông điệp đến hàng đợi có khóa định tuyến khớp chính xác với khóa được chỉ định. Phù hợp cho giao tiếp điểm–điểm (point-to-point), như gửi công việc chấm điểm đến đúng worker.
- **Fanout exchange**: Phát tán thông điệp đến tất cả hàng đợi đã liên kết với exchange, bỏ qua khóa định tuyến. Phù hợp cho mẫu phát hành–đăng ký (publish–subscribe), như thông báo "bài tập mới được tạo" đến tất cả dịch vụ quan tâm.
- **Topic exchange**: Định tuyến dựa trên mẫu (pattern) trong khóa định tuyến, cho phép khớp một phần. Ví dụ: khóa `assignment.created.math` khớp với mẫu `assignment.created.*`. Phù hợp khi cần định tuyến linh hoạt theo chủ đề.
- **Headers exchange**: Định tuyến dựa trên tiêu đề (header) của thông điệp thay vì khóa định tuyến. Ít phổ biến hơn, dùng khi logic định tuyến phức tạp và không thể biểu diễn bằng chuỗi đơn giản.

Về mặt đảm bảo độ tin cậy, RabbitMQ cung cấp cơ chế **xác nhận (acknowledgment)**: sau khi consumer nhận và xử lý xong một thông điệp, nó gửi tín hiệu ACK về cho broker. Chỉ khi nhận được ACK, broker mới xóa thông điệp khỏi hàng đợi. Nếu consumer ngừng hoạt động trước khi gửi ACK (trong thời gian chờ được cấu hình), broker tự động chuyển thông điệp đó sang consumer khác. Cơ chế này đảm bảo mỗi thông điệp được xử lý **ít nhất một lần (at-least-once)**.

RabbitMQ cũng hỗ trợ **thông điệp bền vững (message durability)**: thông điệp được ghi xuống đĩa trước khi broker xác nhận đã nhận. Kết hợp với **hàng đợi bền vững (durable queue)** – hàng đợi tồn tại qua các lần khởi động lại broker – RabbitMQ có thể đảm bảo không mất thông điệp ngay cả khi broker gặp sự cố. Đây là tính năng then chốt cho các hệ thống sản xuất, nơi mất dữ liệu là không thể chấp nhận.

Để làm việc với RabbitMQ trong .NET, thư viện chính thức `RabbitMQ.Client` cung cấp các lớp và phương thức để tạo kết nối, khai báo exchange và queue, gửi và nhận thông điệp. Phần ứng dụng của chương này (mục 6.3) sẽ hướng dẫn chi tiết cách cài đặt RabbitMQ, tạo producer và consumer bằng C#, và chạy thử luồng xử lý bất đồng bộ hoàn chỉnh.

Trong chương tiếp theo (bài 7), chúng ta sẽ mở rộng mẫu hàng đợi thành kiến trúc hướng sự kiện (event-driven architecture) với mẫu phát hành–đăng ký, nơi nhiều dịch vụ có thể cùng phản ứng với một sự kiện – đưa hệ thống tiến thêm một bước về phía tách rời hoàn toàn.

6.2 Kỹ thuật lập trình cơ bản với RabbitMQ và Worker Service

Phần 6.1 đã giới thiệu mẫu trao đổi thông điệp bất đồng bộ và kiến trúc của RabbitMQ. Phần này đi vào các thao tác lập trình cụ thể: từ thiết lập kết nối, khai báo hàng đợi, gửi và nhận thông điệp, đến xác nhận xử lý và quản lý vòng đời kết nối. Mỗi mục tập trung vào một kỹ thuật riêng, dùng đoạn mã ngắn để sinh viên nắm được thao tác cốt lõi. Ví dụ trong phần này dùng một hệ thống nhỏ gồm hai tiến trình: `MessageSender` gửi thông điệp công việc vào hàng đợi, và `TaskProcessor` nhận thông điệp từ hàng đợi để xử lý. Mã nguồn đầy đủ nằm trong `content/examples/chapter06/section02/`.

6.2.1 Thiết lập connection và channel đến RabbitMQ

Trong RabbitMQ, mọi giao tiếp giữa ứng dụng và broker đều đi qua hai đối tượng: `IConnection` và `IChannel`. `IConnection` đại diện cho một kết nối TCP lâu dài đến broker; `IChannel` là kênh logic bên trong kết nối đó, nơi thực sự thực thi các thao tác như khai báo hàng đợi, gửi và nhận thông điệp. Một kết nối có thể chứa nhiều kênh, mỗi kênh hoạt động độc lập.

Thư viện `RabbitMQ.Client` cung cấp `ConnectionFactory` để tạo kết nối. Điểm khởi đầu tối thiểu chỉ cần địa chỉ máy chủ RabbitMQ:

Mã nguồn 6.1: Tạo connection và channel đến RabbitMQ

```
var factory = new ConnectionFactory { HostName = "localhost" };

await using var connection = await factory.CreateConnectionAsync();
await using var channel = await connection.CreateChannelAsync();
```

```
// Su dung channel de thao tac voi hang doi
```

Hai điểm cần chú ý. Thứ nhất, phương thức `CreateConnectionAsync` và `CreateChannelAsync` là phiên bản bất đồng bộ của API cổ điển `CreateConnection` / `CreateModel`. Trong ứng dụng .NET hiện đại, phiên bản bất đồng bộ được ưu tiên vì không chặn luồng trong lúc thiết lập kết nối mạng. Thứ hai, từ khóa `await using` đảm bảo cả kết nối và kênh được giải phóng đúng cách khi thoát khỏi phạm vi, kể cả khi có ngoại lệ. Bỏ sót `Dispose` có thể khiến broker giữ kết nối mồ côi, tiêu tốn tài nguyên.

6.2.2 Khai báo queue và cấu hình cơ bản

Sau khi có kênh, thao tác đầu tiên thường là khai báo hàng đợi. Trong RabbitMQ, khai báo hàng đợi là thao tác **idempotent**: nếu hàng đợi đã tồn tại với tham số trùng khớp, lệnh không làm gì thêm; nếu chưa tồn tại, broker tạo mới. Điều này cho phép cả bên gửi và bên nhận cùng khai báo hàng đợi trong mã của mình mà không cần lo về thứ tự khởi động.

Mã nguồn 6.2: Khai báo hàng đợi với bốn tham số cấu hình

```
await channel.QueueDeclareAsync(
    queue: "demo.task.queue",
    durable: true,
    exclusive: false,
    autoDelete: false,
    arguments: null);
```

Ý nghĩa của từng tham số:

- `durable`: Khi `true`, hàng đợi tồn tại qua các lần khởi động lại broker. Hàng đợi không bền (`false`) bị xóa khi broker dừng. Hệ thống sản xuất luôn dùng `true` để tránh mất thông điệp.
- `exclusive`: Khi `true`, hàng đợi chỉ được dùng bởi kênh đã tạo ra nó và tự động bị xóa khi kênh đó đóng. Dùng cho hàng đợi tạm thời, riêng tư của một consumer.
- `autoDelete`: Khi `true`, hàng đợi tự động bị xóa khi consumer cuối cùng hủy đăng ký. Khác với `exclusive`, hàng đợi `autoDelete` có thể được dùng chung bởi nhiều consumer.
- `arguments`: Từ điển chứa tham số mở rộng như TTL (thời gian sống của thông điệp), độ dài tối đa của hàng đợi, hoặc cấu hình dead-letter exchange.

6.2.3 Tuần tự hóa và publish message

Để gửi thông điệp vào RabbitMQ, dữ liệu cần được chuyển thành mảng byte. Với .NET, `System.Text.Json` là lựa chọn mặc định: tuần tự hóa đối tượng thành JSON, rồi mã hóa chuỗi JSON thành mảng byte với UTF-8. Sau đó, thông điệp được gửi qua `BasicPublishAsync` kèm theo thuộc tính mô tả kiểu nội dung và định danh.

Đầu tiên, định nghĩa kiểu thông điệp dùng chung. Mã dưới đây được đặt trong project `Messaging.Contracts` để cả bên gửi và bên nhận cùng tham chiếu:

Mã nguồn 6.3: Kiểu dữ liệu dùng chung cho thông điệp

```
namespace Messaging.Contracts;
```

```
public static class QueueNames {
    public const string TaskQueue = "demo.task.queue";
}

public sealed record TaskMessage(
    string TaskId,
    string TaskName,
    int Priority,
    DateTimeOffset CreatedAt);
```

Đoạn mã trên định nghĩa hằng số `QueueNames.TaskQueue` để tránh lỗi đánh máy tên hàng đợi giữa các project, cùng với `TaskMessage` – đối tượng được tuần tự hóa và gửi qua RabbitMQ.

Tiếp theo, mã gửi thông điệp trong `MessageSender`:

Mã nguồn 6.4: Tuần tự hóa đối tượng và publish thông điệp vào hàng đợi

```
var message = new TaskMessage(
    TaskId: "TASK-0001",
    TaskName: taskName.Trim(),
    Priority: 1,
    CreatedAt: DateTimeOffset.UtcNow);

var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(message));

var properties = new BasicProperties
{
    ContentType = "application/json",
    MessageId = message.TaskId,
    Persistent = true
};

await channel.BasicPublishAsync(
    exchange: string.Empty,
    routingKey: QueueNames.TaskQueue,
    mandatory: false,
    basicProperties: properties,
    body: body);
```

Bốn quyết định thiết kế đáng ghi nhớ:

1. `ContentType="application/json"`: Báo cho bên nhận biết cách giải mã payload. Thiếu trường này, bên nhận không thể biết chắc dữ liệu là JSON, XML hay nhị phân.
2. `Persistent=true`: Đánh dấu thông điệp cần được ghi xuống đĩa trước khi broker xác nhận đã nhận. Kết hợp với hàng đợi `durable`, thiết lập này bảo đảm thông điệp không bị mất khi broker gặp sự cố. Đánh đổi là hiệu năng: ghi đĩa chậm hơn giữ trong bộ nhớ.
3. `exchange=string.Empty`: Exchange rỗng là "default exchange" do RabbitMQ tạo sẵn, tự động định tuyến thông điệp đến hàng đợi có tên trùng khớp với `routingKey`. Đây là cách đơn giản nhất để bắt đầu; các kiểu exchange phức tạp hơn được bàn ở bài 7.

4. `MessageId`: Định danh duy nhất cho mỗi thông điệp, giúp phát hiện trùng lặp và truy vết thông điệp trong nhật ký.

6.2.4 *Consume message trong Worker Service*

Phía bên nhận, mẫu lập trình phổ biến nhất trong .NET là `AsyncEventingBasicConsumer`: đăng ký một hàm xử lý bất đồng bộ, hàm này được gọi mỗi khi broker đẩy thông điệp đến. Đây là cơ chế đẩy (push) – worker không cần định kỳ kiểm tra hàng đợi; RabbitMQ chủ động giao thông điệp khi có.

Mã nguồn 6.5: Đăng ký consumer để nhận thông điệp từ hàng đợi

```
var consumer = new AsyncEventingBasicConsumer(channel);

consumer.ReceivedAsync += async (_, eventArgs) =>
{
    var json = Encoding.UTF8.GetString(eventArgs.Body.Span);
    var task = JsonSerializer.Deserialize<TaskMessage>(json);

    if (task is null)
    {
        await channel.BasicNackAsync(eventArgs.DeliveryTag,
            multiple: false, requeue: false);
        return;
    }

    // Xu ly task...
    Console.WriteLine($"Dang xu ly {task.TaskId}: {task.TaskName}");

    await channel.BasicAckAsync(eventArgs.DeliveryTag, multiple: false);
};

await channel.BasicConsumeAsync(
    queue: QueueNames.TaskQueue,
    autoAck: false,
    consumer: consumer);
```

Ba chi tiết trong đoạn mã trên cần đọc chậm:

- `eventArgs.Body.Span`: Thân thông điệp được truy cập qua `ReadOnlyMemory<byte>` – không cần cấp phát mảng mới. `Encoding.UTF8.GetString` trên span này chuyển trực tiếp thành chuỗi.
- **Kiểm tra** `taskisnull`: Luôn kiểm tra kết quả giải tuần tự hóa. Nếu JSON không hợp lệ hoặc không khớp kiểu, `Deserialize` trả về `null`. Khi đó, worker từ chối thông điệp với `requeue:false` – không đưa lại vào hàng đợi vì thông điệp hỏng sẽ tiếp tục gây lỗi ở lần nhận sau.
- `autoAck:false`: Tắt tự động xác nhận – đây là điểm mấu chốt của cơ chế đảm bảo độ tin cậy, sẽ được phân tích chi tiết ở mục tiếp theo.

6.2.5 *Xác nhận hoặc từ chối message với ack/nack*

Cơ chế xác nhận thông điệp (**message acknowledgment**) là nền tảng cho độ tin cậy trong RabbitMQ. Khi consumer đăng ký với `autoAck:false`, broker giữ thông điệp trong hàng đợi cho đến khi nhận được tín hiệu xác nhận từ consumer. Có hai tín hiệu:

- `BasicAckAsync`: Xác nhận đã xử lý thành công – broker xóa thông điệp khỏi hàng đợi vĩnh viễn.
- `BasicNackAsync`: Từ chối thông điệp. Tham số `requeue` quyết định: nếu `true`, thông điệp được đưa lại vào hàng đợi để thử lại; nếu `false`, thông điệp bị loại bỏ (hoặc chuyển sang dead-letter exchange nếu có cấu hình).

Mã nguồn 6.6: Ba tình huống xác nhận và từ chối thông điệp

```
// Truong hop 1: xu ly thanh cong, xoa khoi hang doi
await channel.BasicAckAsync(deliveryTag, multiple: false);

// Truong hop 2: xu ly that bai tam thoi, yeu cau thu lai
await channel.BasicNackAsync(deliveryTag, multiple: false, requeue: true);

// Truong hop 3: thong diep hong, khong thu lai
await channel.BasicNackAsync(deliveryTag, multiple: false, requeue: false);
```

Tham số `deliveryTag` là số thứ tự giao nhận do broker gán (lấy từ `eventArgs.DeliveryTag`), dùng để báo cho broker biết chính xác thông điệp nào đang được xác nhận hoặc từ chối. Tham số `multiple` khi đặt `true` sẽ xác nhận hoặc từ chối tất cả thông điệp có delivery tag nhỏ hơn hoặc bằng giá trị được truyền vào – tiết kiệm lượt gọi mạng khi xử lý hàng loạt, nhưng cần thận trọng vì có thể vô tình xác nhận thông điệp chưa được xử lý.

Một tình huống cần lưu ý: nếu worker ngừng hoạt động trước khi gửi ack hoặc nack (sập tiến trình, mất mạng), RabbitMQ coi thông điệp đó chưa được xử lý và tự động giao lại cho consumer khác sau khoảng thời gian chờ (consumer timeout). Đây chính là cơ chế **xử lý ít nhất một lần (at-least-once delivery)**. Hệ quả là trong một số tình huống, thông điệp có thể bị xử lý trùng lặp; các hệ thống sản xuất thường kết hợp thêm cơ chế khử trùng lặp (idempotency) ở tầng ứng dụng.

6.2.6 Quản lý cấu hình và vòng đời kết nối

Trong mã thực tế, địa chỉ máy chủ RabbitMQ không nên được mã cứng. Với .NET, cấu hình được đọc từ biến môi trường, `appsettings.json`, hoặc đối số dòng lệnh.

Mã nguồn 6.7: Đọc cấu hình từ biến môi trường

```
var hostName = Environment.GetEnvironmentVariable("RabbitMQ__HostName")
    ?? "localhost";

var factory = new ConnectionFactory { HostName = hostName };
```

Về vòng đời kết nối, nguyên tắc quan trọng là: **kết nối dùng lâu dài, kênh dùng ngắn hạn**. Mở kết nối TCP mới cho mỗi thao tác là thao tác đắt đỏ (bắt tay ba bước TCP, phân bổ tài nguyên ở broker). Trong ứng dụng thực tế, `IConnection` nên được tạo một lần khi khởi động và tái sử dụng toàn ứng dụng. `IChannel` có thể được tạo mới cho mỗi thao tác vì chi phí tạo kênh thấp hơn nhiều.

Với ứng dụng ASP.NET Core, kết nối thường được đăng ký làm singleton trong dependency injection:

Mã nguồn 6.8: Đăng ký connection làm singleton trong ASP.NET Core

```
builder.Services.AddSingleton<IConnection>(_ =>
{
    var factory = new ConnectionFactory
    {
        HostName = builder.Configuration["RabbitMQ:HostName"] ?? "localhost"
    };
    return factory.CreateConnectionAsync().GetAwaiter().GetResult();
});
```

Lưu ý: đoạn mã trên dùng `GetAwaiter().GetResult()` thay vì `await` vì phương thức đăng ký dịch vụ không hỗ trợ bất đồng bộ. Đây là một trong số ít tình huống chấp nhận được việc chặn luồng – nó chỉ xảy ra một lần duy nhất khi khởi động.

Thư viện `RabbitMQ.Client` từ phiên bản 7 cung cấp cơ chế tự động phục hồi kết nối (`AutomaticRecoveryEnabled`), giúp giảm đáng kể mã xử lý lỗi phía ứng dụng:

Mã nguồn 6.9: Bật tự động phục hồi kết nối

```
var factory = new ConnectionFactory
{
    HostName = hostName,
    AutomaticRecoveryEnabled = true,
    NetworkRecoveryInterval = TimeSpan.FromSeconds(10)
};
```

Khi bật tùy chọn này, thư viện tự động thử kết nối lại sau khi mất kết nối – consumer được đăng ký lại, hàng đợi được khai báo lại, và việc nhận thông điệp tiếp tục mà không cần can thiệp thủ công.

Tổng kết

Sáu kỹ thuật trên là bộ công cụ tối thiểu để lập trình với RabbitMQ trong .NET. Để xem toàn bộ mã nguồn hoàn chỉnh của ví dụ `MessageSender` và `TaskProcessor`, sinh viên có thể tham khảo:

```
examples/chapter06/section02/
  src/
    Messaging.Contracts/
    MessageSender/
    TaskProcessor/
  README.md
```

Mục 6.3 sẽ ghép tất cả kỹ thuật này vào một hệ thống phân tán hoàn chỉnh trong bối cảnh EduSubmit, với API nhận bài nộp, worker chấm điểm, và cơ chế trả trạng thái "đang xử lý" cho máy khách.

6.3 [Ứng dụng] Xây dựng mô hình truyền thông bất đồng bộ giữa các dịch vụ với RabbitMQ

Ở bài thực hành trước, `Submission.Api` gọi `Grading.GrpcService` bằng gRPC và chờ dịch vụ chấm bài trả kết quả. Cách làm đó giúp ta thấy rõ giao tiếp dịch vụ–dịch vụ

theo mô hình yêu cầu–phản hồi, nhưng nó vẫn có một điểm yếu quan trọng: nếu việc chấm bài mất nhiều thời gian hoặc dịch vụ chấm bài tạm dừng, luồng xử lý của API bị ảnh hưởng trực tiếp. Bài thực hành này thay đổi ranh giới đó. `Submission.Api` không chấm bài ngay và cũng không gọi trực tiếp dịch vụ chấm bài; API chỉ tiếp nhận bài nộp, đưa một công việc chấm bài vào hàng đợi RabbitMQ, rồi trả về trạng thái `Processing`.

Lát cắt mới của EduSubmit gồm ba project:

```
examples/chapter06/section03/
  AsyncQueueSubmissionSystem.slnx
  src/
    EduSubmit.Contracts/
    Submission.Api/
    Grading.Worker/
```

`EduSubmit.Contracts` chứa các kiểu dữ liệu dùng chung giữa API và dịch vụ xử lý nền. `Submission.Api` đóng vai trò bên sản xuất thông điệp: nhận yêu cầu nộp bài qua HTTP, tạo công việc chấm bài và gửi vào hàng đợi. `Grading.Worker` đóng vai trò bên tiêu thụ thông điệp: lắng nghe hàng đợi, lấy công việc ra, xử lý và xác nhận với RabbitMQ rằng thông điệp đã hoàn tất.

So với chương 5, sự thay đổi quan trọng không nằm ở thuật toán chấm điểm, mà nằm ở vị trí của thời gian chờ. Trong mô hình gRPC đồng bộ, thời gian chấm bài nằm trong thời gian phản hồi HTTP của API. Trong mô hình hàng đợi, API kết thúc sớm sau khi RabbitMQ đã nhận công việc; thời gian chấm bài được chuyển sang tiến trình xử lý nền.

6.3.1 API gửi công việc vào hàng đợi

Trước khi viết API, ta cần thống nhất hình dạng của thông điệp đưa vào hàng đợi. Thông điệp không nên là một chuỗi tùy tiện, vì API và dịch vụ xử lý nền phải hiểu cùng một dữ liệu. Trong ví dụ này, project `EduSubmit.Contracts` định nghĩa tên hàng đợi, yêu cầu nộp bài, thông điệp công việc chấm bài và phản hồi trạng thái.

Mã nguồn 6.10: Các kiểu thông điệp dùng chung trong luồng RabbitMQ

```
namespace EduSubmit.Contracts;

public static class MessagingNames {
    public const string GradingQueue = "edusubmit.grading.jobs";
}

public sealed record SubmitAssignmentRequest(
    int AssignmentId,
    string StudentId,
    string AnswerText);

public sealed record GradingJobMessage(
    int SubmissionId,
    int AssignmentId,
    string StudentId,
    string AnswerText,
    DateTimeOffset SubmittedAt);
```

```
public sealed record SubmissionAcceptedResponse(
    int SubmissionId,
    string Status,
    string StatusUrl);

public sealed record SubmissionStatusResponse(
    int SubmissionId,
    string Status);
```

`MessagingNames.GradingQueue` là tên logic của hàng đợi chấm bài. Đây là điểm liên kết có chủ ý giữa bên gửi và bên nhận: API không biết tiến trình xử lý nền nào sẽ xử lý công việc, nhưng cả hai bên phải thống nhất tên hàng đợi. `GradingJobMessage` chứa lượng dữ liệu tối thiểu để dịch vụ xử lý nền có thể chấm bài: mã lượt nộp, mã bài tập, mã sinh viên, nội dung câu trả lời và thời điểm nộp.

Trong `Submission.Api`, điểm cuối `POST/api/submissions` không còn gọi dịch vụ chấm bài trực tiếp. Nó tạo một mã lượt nộp mới, lưu trạng thái ban đầu là `Processing`, đóng gói công việc thành `GradingJobMessage`, rồi gửi thông điệp vào RabbitMQ.

Mã nguồn 6.11: API nhận bài nộp và phát công việc chấm bài vào RabbitMQ

```
using System.Collections.Concurrent;
using System.Globalization;
using System.Text;
using System.Text.Json;
using EduSubmit.Contracts;
using RabbitMQ.Client;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<SubmissionStore>();
builder.Services.AddSingleton<RabbitMqPublisher>();

var app = builder.Build();

app.MapPost("/api/submissions", async (
    SubmitAssignmentRequest request,
    SubmissionStore store,
    RabbitMqPublisher publisher,
    CancellationToken cancellationToken) => {
        var submissionId = store.CreateProcessingSubmission();

        var job = new GradingJobMessage(
            submissionId,
            request.AssignmentId,
            request.StudentId,
            request.AnswerText,
            DateTimeOffset.UtcNow);

        await publisher.PublishAsync(job, cancellationToken);

        return Results.Accepted(
            $"/api/submissions/{submissionId}",
            new SubmissionAcceptedResponse(
                submissionId,
                "Processing",
```

```
                $"/api/submissions/{submissionId}"));
    });

app.MapGet("/api/submissions/{submissionId:int}", (
    int submissionId,
    SubmissionStore store) => {
        var status = store.GetStatus(submissionId);

        return status is null
            ? Results.NotFound()
            : Results.Ok(new SubmissionStatusResponse(submissionId, status));
    });

app.Run();

public sealed class SubmissionStore {
    private int _nextSubmissionId = 1000;
    private readonly ConcurrentDictionary<int, string> _statuses = new();

    public int CreateProcessingSubmission() {
        var submissionId = Interlocked.Increment(ref _nextSubmissionId);
        _statuses[submissionId] = "Processing";
        return submissionId;
    }

    public string? GetStatus(int submissionId) =>
        _statuses.GetValueOrDefault(submissionId);
}

public sealed class RabbitMqPublisher(IConfiguration configuration) {
    private readonly string _hostName =
        configuration["RabbitMq:HostName"] ?? "localhost";

    public async Task PublishAsync(
        GradingJobMessage job,
        CancellationToken cancellationToken) {
        var factory = new ConnectionFactory { HostName = _hostName };

        await using var connection =
            await factory.CreateConnectionAsync(cancellationToken);
        await using var channel =
            await connection.CreateChannelAsync(cancellationToken: cancellationToken);

        await channel.QueueDeclareAsync(
            queue: MessagingNames.GradingQueue,
            durable: true,
            exclusive: false,
            autoDelete: false,
            arguments: null,
            cancellationToken: cancellationToken);

        var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(job));
        var properties = new BasicProperties {
            ContentType = "application/json",
            MessageId = job.SubmissionId.ToString(CultureInfo.InvariantCulture),
            Persistent = true
        };

        await channel.BasicPublishAsync(
```

```
            exchange: string.Empty,
            routingKey: MessagingNames.GradingQueue,
            mandatory: false,
            basicProperties: properties,
            body: body,
            cancellationToken: cancellationToken);
    }
}
```

Listing trên có ba điểm cần đọc chậm. Thứ nhất, phản hồi của API dùng `Results.Accepted` thay vì `Results.Ok`. Về mặt HTTP, điều này diễn đạt rằng hệ thống đã tiếp nhận yêu cầu nhưng kết quả cuối cùng chưa sẵn sàng. Thứ hai, `SubmissionStore` chỉ lưu trạng thái trong bộ nhớ để giữ ví dụ ngắn gọn; trong hệ thống thật, trạng thái lượt nộp sẽ được lưu trong cơ sở dữ liệu để không mất khi API khởi động lại. Thứ ba, `RabbitMqPublisher` khai báo một hàng đợi bền vững và gửi thông điệp với `Persistent=true`. Hai thiết lập này giúp thông điệp có cơ hội sống sót tốt hơn khi broker hoặc tiến trình tiêu thụ gặp sự cố.

Đoạn gửi thông điệp sử dụng exchange rỗng và `routingKey` đúng bằng tên hàng đợi. Đây là cách đơn giản nhất để làm việc với RabbitMQ: thông điệp được định tuyến trực tiếp đến một hàng đợi cụ thể. Bài 7 sẽ mở rộng ý tưởng này sang exchange phát hành–đăng ký, nơi một sự kiện có thể được chuyển đến nhiều hàng đợi khác nhau.

> Trong ví dụ học tập này, `RabbitMqPublisher` tạo kết nối và kênh mới cho mỗi lần gửi để mã dễ đọc. Trong hệ thống sản xuất, kết nối RabbitMQ thường được quản lý như tài nguyên dùng lại, có xử lý mất kết nối, thử kết nối lại và theo dõi lỗi gửi thông điệp. Mục tiêu của bài này là làm rõ luồng bất đồng bộ trước khi tối ưu vận hành.

6.3.2 *Dịch vụ xử lý nền nhận và xử lý công việc*

Phía còn lại của mô hình là `Grading.Worker`. Đây là một tiến trình console chạy độc lập với API. Khi khởi động, tiến trình kết nối đến RabbitMQ, khai báo cùng hàng đợi `edusubmit.grading.jobs`, rồi đăng ký một consumer bất đồng bộ. Mỗi khi có thông điệp mới, RabbitMQ giao thông điệp cho worker, worker giải mã JSON, xử lý công việc chấm bài và gửi xác nhận đã xử lý xong.

Mã nguồn 6.12: Dịch vụ xử lý nền nhận công việc chấm bài từ RabbitMQ

```
using System.Text;
using System.Text.Json;
using EduSubmit.Contracts;
using RabbitMQ.Client;
using RabbitMQ.Client.Events;

var rabbitMqHostName = Environment.GetEnvironmentVariable("RabbitMq__HostName")
    ?? "localhost";

var factory = new ConnectionFactory { HostName = rabbitMqHostName };

await using var connection = await factory.CreateConnectionAsync();
await using var channel = await connection.CreateChannelAsync();
```

```
await channel.QueueDeclareAsync(
    queue: MessagingNames.GradingQueue,
    durable: true,
    exclusive: false,
    autoDelete: false,
    arguments: null);

var consumer = new AsyncEventingBasicConsumer(channel);

consumer.ReceivedAsync += async (_, eventArgs) => {
    var json = Encoding.UTF8.GetString(eventArgs.Body.Span);
    var job = JsonSerializer.Deserialize<GradingJobMessage>(json);

    if (job is null) {
        await channel.BasicNackAsync(
            eventArgs.DeliveryTag,
            multiple: false,
            requeue: false);
        return;
    }

    var score = Grade(job.AnswerText);
    await Task.Delay(TimeSpan.FromSeconds(2));

    Console.WriteLine(
        $"Da cham submission {job.SubmissionId}: score={score},
        ↪  student={job.StudentId}");

    await channel.BasicAckAsync(
        eventArgs.DeliveryTag,
        multiple: false);
};

await channel.BasicConsumeAsync(
    queue: MessagingNames.GradingQueue,
    autoAck: false,
    consumer: consumer);

Console.WriteLine("Grading.Worker dang lang nghe hang doi edusubmit.grading.jobs.");
Console.WriteLine("Nhan Ctrl+C de dung worker.");

await Task.Delay(Timeout.InfiniteTimeSpan);

static int Grade(string answerText) {
    var normalized = answerText.Trim();

    return normalized.Length switch {
        >= 80 => 10,
        >= 40 => 8,
        >= 12 => 6,
        _ => 3
    };
}
```

Cơ chế quan trọng nhất trong listing này là xác nhận thủ công. Khi đăng ký consumer với tùy chọn `autoAck:false`, RabbitMQ không xóa thông điệp ngay khi giao cho worker. Chỉ sau khi xử lý xong, worker mới gọi `BasicAckAsync`. Nếu thông điệp không

đọc được, ví dụ gọi `BasicNackAsync` và không đưa lại vào hàng đợi. Trong bài toán thực tế, quyết định đưa lại hàng đợi, chuyển sang hàng đợi lỗi hay ghi nhật ký phụ thuộc vào loại lỗi và chính sách vận hành.

Đoạn `Task.Delay` mô phỏng việc chấm bài mất thời gian. Khi sinh viên gửi bài, API vẫn trả kết quả gần như ngay lập tức vì API không chờ đoạn xử lý này. Nếu chạy nhiều tiến trình `Grading.Worker`, RabbitMQ sẽ phân phối các công việc giữa các worker theo kiểu các bên tiêu thụ cạnh tranh: mỗi thông điệp chấm bài được giao cho một worker, không phải cho tất cả worker.

Biên dịch toàn bộ solution bằng lệnh:

```
dotnet build examples/chapter06/section03/AsyncQueueSubmissionSystem.slnx
```

Để chạy thử, trước hết khởi động RabbitMQ bằng Docker:

```
docker run --name edusubmit-rabbitmq --rm -p 5672:5672 -p 15672:15672
↪ rabbitmq:3-management
```

Sau đó chạy API ở cửa sổ dòng lệnh thứ hai:

```
dotnet run --project examples/chapter06/section03/src/Submission.Api --urls
↪ http://localhost:5200
```

Chạy dịch vụ xử lý nền ở cửa sổ dòng lệnh thứ ba:

```
dotnet run --project examples/chapter06/section03/src/Grading.Worker
```

Khi worker khởi động thành công, cửa sổ dòng lệnh sẽ hiển thị:

```
Grading.Worker dang lang nghe hang doi edusubmit.grading.jobs.
Nhan Ctrl+C de dung worker.
```

6.3.3 *Trả trạng thái "processing"*

Bây giờ gửi một bài nộp mẫu đến API:

```
Invoke-RestMethod `
  -Method Post `
  -Uri http://localhost:5200/api/submissions `
  -ContentType "application/json" `
  -Body '{"assignmentId":1,"studentId":"SV001","answerText":"RabbitMQ giup API tra ve
  ↪ Processing va de worker cham bai sau."}'
```

Phản hồi mong đợi của API có dạng:

```
submissionId status     statusUrl
------------ ------     ---------
        1001 Processing /api/submissions/1001
```

Điểm cần quan sát là phản hồi này xuất hiện trước khi worker chấm xong. Trạng thái `Processing` không phải là lỗi hay kết quả tạm bợ; nó là một phần của thiết kế bất đồng bộ. API nói với máy khách rằng hệ thống đã nhận trách nhiệm xử lý công việc, nhưng kết quả cuối cùng sẽ đến sau. Đây là cách các hệ thống nộp bài, xử lý video, tạo báo cáo hoặc gửi email hàng loạt thường làm để tránh giữ kết nối HTTP quá lâu.

Ở cửa sổ worker, sau khoảng hai giây, ta sẽ thấy dòng kết quả tương tự:

```
Da cham submission 1001: score=8, student=SV001
```

Có thể kiểm tra trạng thái hiện tại của lượt nộp bằng lệnh:

```
Invoke-RestMethod http://localhost:5200/api/submissions/1001
```

Trong ví dụ tối giản này, API chỉ biết trạng thái `Processing`, vì worker mới in kết quả ra màn hình chứ chưa cập nhật ngược về kho trạng thái chung. Giới hạn này là có chủ ý: bài 6 tập trung vào mẫu hàng đợi một chiều từ API sang dịch vụ xử lý nền. Ở bài 7, ta sẽ hoàn thiện vòng phản hồi bằng cách để worker phát sự kiện `SubmissionGraded`; dịch vụ khác có thể nhận sự kiện đó và đẩy thông báo thời gian thực về máy khách bằng SignalR.

Khi tự thử nghiệm, hãy dừng `Grading.Worker`, gửi thêm vài bài nộp, rồi khởi động worker lại. Nếu RabbitMQ vẫn đang chạy, các thông điệp chưa xử lý vẫn nằm trong hàng đợi và sẽ được worker nhận sau khi khởi động. Đây là minh họa trực tiếp cho tách rời thời gian: API và worker không cần chạy đồng thời tại đúng thời điểm sinh viên nộp bài.

> Không nên dùng hàng đợi để thay thế mọi lời gọi đồng bộ. Nếu người dùng cần dữ liệu ngay, ví dụ đọc danh sách bài tập hoặc kiểm tra thông tin tài khoản, REST hoặc GraphQL vẫn phù hợp hơn. Hàng đợi nên được dùng cho công việc dài, có thể xử lý sau, có thể thất bại tạm thời và cần được điều tiết tải.

Sau phần thực hành này, EduSubmit đã có một luồng xử lý bất đồng bộ cơ bản: API nhận bài nộp, RabbitMQ giữ công việc chấm bài, và dịch vụ xử lý nền tiêu thụ công việc theo tốc độ riêng. Mảnh ghép tiếp theo là biến kết quả xử lý nền thành sự kiện để nhiều thành phần khác cùng phản ứng mà không cần phụ thuộc trực tiếp vào worker.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **MEP bất đồng bộ dựa trên hàng đợi**: Thay vì giao tiếp trực tiếp (REST, gRPC), bên gửi đưa thông điệp vào hàng đợi do MOM (Message-Oriented Middleware) quản lý. Bên nhận lấy thông điệp ra khi sẵn sàng. Hàng đợi đóng vai trò bộ đệm thời gian (temporal buffer), cho phép tách rời thời gian (temporal decoupling) giữa hai bên.
- **Bốn thao tác cơ bản với hàng đợi**: `put` (gửi không chặn), `get` (nhận có chặn), `poll` (kiểm tra không chặn), `notify` (đăng ký callback khi có thông điệp mới). Hai

chế độ truyền thông: tạm thời (transient – thông điệp trong bộ nhớ) và bền vững (persistent – thông điệp lưu xuống đĩa).

- **Ba hạn chế của xử lý đồng bộ**: (1) Liên kết chặt về thời gian (temporal coupling) – server ngừng thì client thất bại; (2) Không hấp thụ được tải đột biến (load spikes) – mỗi yêu cầu được xử lý ngay, server dễ quá tải; (3) Xử lý kéo dài làm nghẽn luồng chính – tác vụ dài giữ kết nối HTTP mở, tiêu tốn tài nguyên.
- **Mẫu dịch vụ xử lý nền (worker service pattern)**: Hệ thống chia thành hai vai trò: producer (API nhận yêu cầu, gửi thông điệp vào hàng đợi) và consumer/worker (tiến trình nền nhận thông điệp, xử lý, gửi ACK). Hàng đợi tách rời producer và consumer, mang lại ba lợi ích: khả năng mở rộng độc lập, khả năng chịu lỗi, và khả năng cân bằng tải tự nhiên (competing consumers).
- **RabbitMQ và AMQP**: RabbitMQ là message broker mã nguồn mở triển khai giao thức AMQP. Kiến trúc gồm ba tầng: producer -> exchange -> queue -> consumer. Exchange hỗ trợ bốn kiểu: direct (khớp chính xác routing key), fanout (phát tán đến tất cả queue), topic (khớp mẫu routing key), headers (định tuyến theo header thông điệp).
- **Cơ chế đảm bảo tin cậy**: Acknowledgment (ACK) – consumer xác nhận sau khi xử lý xong, nếu chưa ACK mà ngừng hoạt động, broker chuyển thông điệp sang consumer khác (at-least-once delivery). Message durability và durable queue – thông điệp và hàng đợi sống sót qua các lần khởi động lại broker.
- **Thực hành luồng bất đồng bộ**: API (`Submission.Api`) nhận bài nộp, gửi `GradingJobMessage` vào hàng đợi `edusubmit.grading.jobs`, trả về `202Accepted` với trạng thái `Processing`. Worker (`Grading.Worker`) lắng nghe hàng đợi, nhận thông điệp với `autoAck: false`, xử lý chấm điểm và gọi `BasicAckAsync`. API không chờ worker – thời gian chấm bài được chuyển từ luồng HTTP sang tiến trình nền.
- **Nguyên tắc lựa chọn**: Không dùng hàng đợi để thay thế mọi lời gọi đồng bộ. Hàng đợi phù hợp khi: (1) tác vụ mất nhiều thời gian, (2) bên nhận có thể không hoạt động, (3) cần hấp thụ tải đột biến, (4) cần đảm bảo không mất dữ liệu khi có sự cố. Với thao tác nhanh, cần phản hồi ngay, REST/gRPC vẫn là lựa chọn tốt hơn.

Câu hỏi ôn tập

1. Nêu định nghĩa MEP bất đồng bộ dựa trên hàng đợi. So sánh mô hình này với RPC đồng bộ về: thời gian phản hồi, sự phụ thuộc giữa bên gửi và bên nhận, và khả năng chịu lỗi khi một bên ngừng hoạt động.
2. Liệt kê và mô tả bốn thao tác cơ bản khi làm việc với hàng đợi thông điệp (`put`, `get`, `poll`, `notify`). Phân biệt thao tác chặn (blocking) và không chặn (non-blocking) trong bốn thao tác trên.
3. Phân biệt chế độ truyền thông tạm thời (transient) và bền vững (persistent) trong hệ thống hàng đợi. Khi nào nên chọn chế độ bền vững? Cho ví dụ trong hệ thống EduSubmit.
4. Trình bày ba hạn chế của xử lý đồng bộ trong hệ thống phân tán. Với mỗi hạn chế, giải thích cách hàng đợi thông điệp giúp khắc phục vấn đề đó.
5. Vẽ sơ đồ và giải thích mẫu dịch vụ xử lý nền (worker service pattern). Nêu rõ vai trò của producer, consumer và hàng đợi. Ba lợi ích kiến trúc của mẫu này là gì?

6. Trong cơ chế cạnh tranh giữa các consumer (competing consumers), điều gì đảm bảo mỗi thông điệp chỉ được xử lý bởi đúng một worker? Nếu một worker ngừng hoạt động giữa chừng khi đang xử lý thông điệp, điều gì xảy ra với thông điệp đó?
7. Mô tả kiến trúc định tuyến thông điệp trong RabbitMQ: vai trò của exchange và routing key. Phân biệt cách hoạt động của bốn kiểu exchange: direct, fanout, topic và headers. Với mỗi kiểu, cho một tình huống sử dụng phù hợp trong hệ thống EduSubmit.
8. Giải thích cơ chế acknowledgment (ACK) trong RabbitMQ. Tại sao cần `autoAck: false` trong hệ thống sản xuất? So sánh đảm bảo "at-least-once" của RabbitMQ với đảm bảo "exactly-once" – tại sao exactly-once khó đạt được trong hệ thống phân tán?
9. Trong ví dụ thực hành, `Submission.Api` gửi `GradingJobMessage` vào hàng đợi và trả về `202Accepted`. Giải thích ý nghĩa của mã HTTP 202 trong ngữ cảnh này. Tại sao không dùng `200OK`?
10. Mô tả luồng dữ liệu đầy đủ khi một sinh viên nộp bài trong hệ thống EduSubmit sử dụng RabbitMQ: từ lúc API nhận yêu cầu HTTP đến khi worker hoàn tất chấm điểm. Chỉ rõ: đâu là giao tiếp đồng bộ, đâu là giao tiếp bất đồng bộ, và thông điệp đi qua những thành phần nào.

Bài tập thực hành

1. **Thêm trường dữ liệu vào thông điệp**. Sao chép mã nguồn từ `content/examples/chapter06/section02/` và mở rộng `GradingJobMessage` với trường `Priority` (mức ưu tiên chấm bài). Cập nhật API để nhận giá trị ưu tiên từ yêu cầu HTTP, cập nhật worker để hiển thị mức ưu tiên khi xử lý. Chạy và xác nhận luồng hoạt động.
2. **Mô phỏng nhiều worker cạnh tranh**. Khởi động ba tiến trình `Grading.Worker` cùng lúc (ba cửa sổ dòng lệnh). Gửi 10 bài nộp liên tiếp đến API. Quan sát: mỗi bài nộp được worker nào xử lý? Có worker nào nhận nhiều hơn các worker khác không? Giải thích cơ chế phân phối của RabbitMQ. Ghi lại kết quả và nhận xét.
3. **Mô phỏng lỗi worker**. Khởi động một `Grading.Worker` với `autoAck: false`. Gửi một bài nộp, quan sát worker nhận thông điệp, rồi dừng worker bằng Ctrl+C **trước khi** nó gọi `BasicAckAsync`. Chờ vài giây, khởi động lại worker. Quan sát: thông điệp có được xử lý lại không? Giải thích hành vi dựa trên cơ chế acknowledgment timeout của RabbitMQ.
4. **Thêm worker thứ hai với chức năng khác**. Tạo một project `Notification.Worker` mới, lắng nghe cùng hàng đợi `edusubmit.grading.jobs` nhưng chỉ ghi nhật ký (log) mỗi khi có bài được chấm. Chạy đồng thời `Grading.Worker` và `Notification.Worker`. Quan sát: cả hai worker có cùng nhận được thông điệp không? Nếu không, giải thích tại sao và đề xuất cách để cả hai worker cùng nhận được thông báo (gợi ý: fanout exchange trong bài 7).

Câu hỏi thảo luận

1. Trong hệ thống EduSubmit hiện tại, worker chấm điểm xong nhưng chưa cập nhật kết quả ngược về `Submission.Api`. Hãy đề xuất hai phương án để worker thông báo kết quả chấm điểm: (a) worker gọi trực tiếp API của `Submission.Api`,

(b) worker phát một sự kiện `SubmissionGraded` vào một exchange fanout. So sánh ưu điểm, hạn chế và mức độ tách rời (decoupling) của hai phương án.

2. Khi lưu lượng nộp bài tăng đột biến (sinh viên đồng loạt nộp trước hạn chót), hàng đợi có thể tích tụ hàng nghìn thông điệp chưa xử lý. Thảo luận các chiến lược để giảm tồn đọng: tăng số lượng worker, giới hạn độ dài hàng đợi (queue length limit), ưu tiên thông điệp (message priority), và cơ chế từ chối chủ động (backpressure). Chiến lược nào phù hợp nhất cho bài toán chấm điểm tự động?
3. So sánh ba mô hình giao tiếp nội bộ đã học đến thời điểm này: REST nội bộ (bài 2), gRPC (bài 5), và message queue (bài 6). Lập bảng so sánh theo các tiêu chí: kiểu MEP, độ trễ, khả năng chịu lỗi, khả năng mở rộng, mức độ tách rời, và độ phức tạp triển khai. Với mỗi tiêu chí, nêu tình huống trong EduSubmit mà một mô hình vượt trội hơn hai mô hình còn lại.

CHƯƠNG 7

HOÀN THIỆN LUỒNG HỆ THỐNG VỚI KIẾN TRÚC HƯỚNG SỰ KIỆN VÀ PUB/SUB

Mục tiêu chương
Sau khi học xong chương này, người học có thể:

- Nhận diện mẫu trao đổi thông điệp phát hành–đăng ký (pub/sub) và phân biệt với các mẫu điểm–điểm đã học (REST, gRPC, hàng đợi).
- Phân tích sự khác biệt giữa yêu cầu (request) và sự kiện (event) về hướng giao tiếp, tính chủ động, và số lượng bên nhận.
- Giải thích kiến trúc hướng sự kiện (EDA), vai trò của event bus, và cách EDA cho phép mở rộng chức năng mà không sửa đổi dịch vụ hiện có.
- Đánh giá mức độ phụ thuộc (coupling) của bốn mô hình giao tiếp – REST, gRPC, hàng đợi + worker, pub/sub + sự kiện – và lựa chọn mô hình phù hợp cho từng ranh giới trong hệ thống.
- Xây dựng và chạy thử luồng hướng sự kiện hoàn chỉnh: worker phát sự kiện sau khi xử lý, dịch vụ nhận sự kiện qua topic exchange, và kết hợp SignalR để thông báo kết quả thời gian thực đến máy khách.

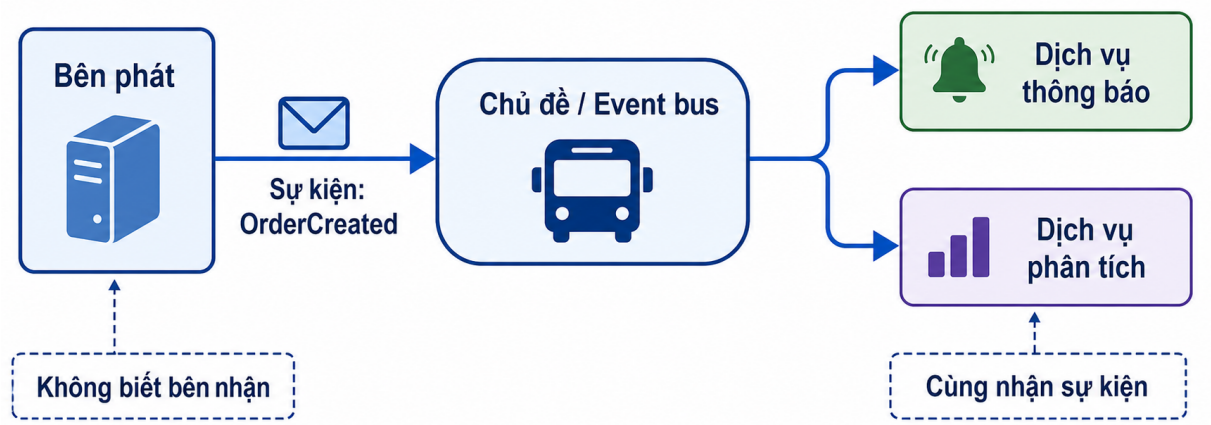
7.1 Mô hình truyền thông điệp pub/sub và dựa trên sự kiện

7.1.1 MEP: phát hành–đăng ký

Trong chương trước, chúng ta đã làm quen với mẫu trao đổi thông điệp bất đồng bộ dựa trên hàng đợi: bên sản xuất gửi thông điệp vào hàng đợi, bên tiêu thụ lấy ra và xử lý. Mẫu này thuộc loại **điểm–điểm (point-to-point)** – mỗi thông điệp được gửi đến một hàng đợi và được xử lý bởi đúng một bên tiêu thụ. Chương này giới thiệu một mẫu trao đổi thông điệp khác, mở rộng khả năng giao tiếp lên mức **một–nhiều (one-to-many)**: mẫu **phát hành–đăng ký (publish–subscribe, hay pub/sub)**.

Ý tưởng cốt lõi của pub/sub rất khác biệt so với các mẫu đã học. Thay vì gửi thông điệp đến một địa chỉ cụ thể (URL trong REST, phương thức trong gRPC, hay hàng đợi trong worker pattern), bên phát (publisher) công bố thông điệp lên một **chủ đề (topic)** – một định danh logic mô tả loại sự kiện. Bên đăng ký (subscriber) bày tỏ sự quan tâm đến một hoặc nhiều chủ đề từ trước. Khi có thông điệp mới được phát hành trên một chủ đề, tất cả bên đăng ký quan tâm đến chủ đề đó đều nhận được thông điệp. Ba bên – publisher, subscriber, và cơ chế trung gian – tạo thành tam giác giao tiếp hoàn toàn khác với mô hình gọi–đáp truyền thống.

Để hiểu vị trí của pub/sub trong bức tranh tổng thể, có thể phân loại các mô hình phối hợp (coordination models) theo hai chiều: **thời gian** (có cần các bên chạy đồng thời?) và **tham chiếu** (có cần biết định danh của nhau?). Bảng 7.1 tóm tắt bốn mô hình chính.



Hình 7.1: Mẫu phát hành–đăng ký với một sự kiện được chuyển đến nhiều bên nhận

Bảng 7.1: Bốn mô hình phối hợp trong hệ thống phân tán

Mô hình	Thời gian	Tham chiếu	Ví dụ công nghệ
Trực tiếp (Direct)	Đồng thời	Biết định danh	REST, gRPC, RPC
Hộp thư (Mailbox)	Không đồng thời	Biết định danh	Hàng đợi (RabbitMQ queue), worker pattern
Dựa trên sự kiện (Event-based)	**Đồng thời**	**Không biết định danh**	**Pub/sub (RabbitMQ fanout/topic), SignalR**
Không gian dữ liệu chung (Shared data space)	Không đồng thời	Không biết định danh	Linda tuple spaces, Redis pub/sub với persistence

Pub/sub thuộc góc phần tư thứ ba: các bên không cần biết định danh của nhau (referential decoupling), nhưng cần hoạt động đồng thời tại thời điểm sự kiện được phát ra (temporal coupling). Tuy nhiên, với message broker có khả năng lưu trữ (như RabbitMQ với durable queue), pub/sub có thể đạt được cả hai chiều tách rời – tiến gần đến mô hình shared data space.

Có hai biến thể chính của pub/sub:

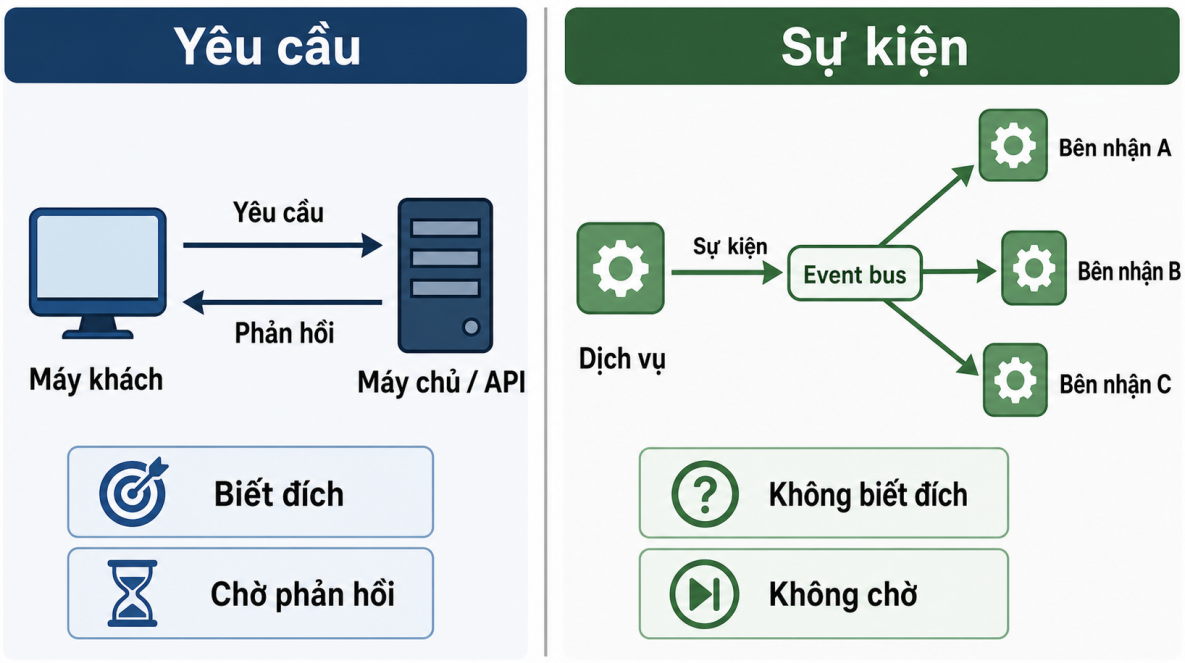
- **Dựa trên chủ đề (topic-based)**: Subscriber đăng ký theo tên chủ đề hoặc mẫu chủ đề. Ví dụ: đăng ký `assignment.*` để nhận mọi sự kiện liên quan đến bài tập. Đây là dạng phổ biến nhất, được RabbitMQ hỗ trợ qua topic exchange.
- **Dựa trên nội dung (content-based)**: Subscriber đăng ký theo điều kiện lọc trên nội dung thông điệp. Ví dụ: chỉ nhận sự kiện `SubmissionGraded` khi điểm số lớn hơn 8. Mạnh hơn về khả năng biểu diễn nhưng phức tạp hơn khi kiểm tra khớp.

7.1.2 Sự kiện và yêu cầu

Pub/sub không chỉ là một cơ chế định tuyến thông điệp khác – nó đại diện cho một sự thay đổi căn bản trong cách nghĩ về giao tiếp giữa các thành phần. Sự khác biệt cốt lõi nằm ở hai khái niệm: **yêu cầu (request)** và **sự kiện (event)**.

Yêu cầu là hành động chủ động và có định hướng. Khi client gửi `GET /api/assignments`, nó biết chính xác mình đang hỏi ai (API server), hỏi cái gì (danh sách bài tập), và mong đợi một câu trả lời. Yêu cầu mang tính **mệnh lệnh (imperative)**: "Hãy đưa cho tôi dữ liệu này."

Sự kiện là thông báo thụ động và không định hướng. Khi dịch vụ chấm điểm phát ra sự kiện `SubmissionGraded`, nó không biết và không quan tâm ai sẽ nhận. Nó chỉ đơn thuần thông báo một sự thật đã xảy ra: "Bài nộp #42 đã được chấm xong, điểm 8.5." Sự kiện mang tính **khai báo (declarative)**: "Điều này đã xảy ra."



Hình 7.2: So sánh giao tiếp bằng yêu cầu và giao tiếp bằng sự kiện

Bảng 7.2 đối chiếu hai khái niệm này trên các tiêu chí then chốt.

Bảng 7.2: So sánh yêu cầu và sự kiện

Tiêu chí	Yêu cầu (Request)	Sự kiện (Event)
Hướng giao tiếp	Điểm–điểm (one-to-one)	Một–nhiều (one-to-many)
Bên gửi biết bên nhận?	Có (biết URL/endpoint/queue)	Không (chỉ biết topic)
Đồng bộ / bất đồng bộ	Đồng bộ (chờ phản hồi)	Bất đồng bộ (bắn và quên)
Tính chủ động	Chủ động (hỏi để lấy)	Thụ động (thông báo đã xảy ra)
Thời điểm xử lý	Ngay lập tức	Khi subscriber sẵn sàng
Số bên nhận	Một (server cụ thể)	Nhiều (tất cả subscriber)
Ví dụ EduSubmit	`GET/api/assignments`	`SubmissionGraded \ {submissionId : 42 , score : 8.5\}`

Trong thực tế, một hệ thống thường sử dụng cả hai. Yêu cầu phù hợp cho các thao tác đọc dữ liệu và các tương tác cần phản hồi ngay. Sự kiện phù hợp cho việc thông báo thay đổi trạng thái và kích hoạt các phản ứng dây chuyền. Ở bài 2 và bài 5, chúng ta đã dùng yêu cầu (REST, gRPC) cho giao tiếp đồng bộ. Ở bài 6, chúng ta bắt đầu chuyển sang thông điệp bất đồng bộ với hàng đợi. Chương này hoàn thiện bức tranh với sự kiện – mảnh ghép cuối cùng trong bộ ba MEP.

7.1.3 Kiến trúc hướng sự kiện

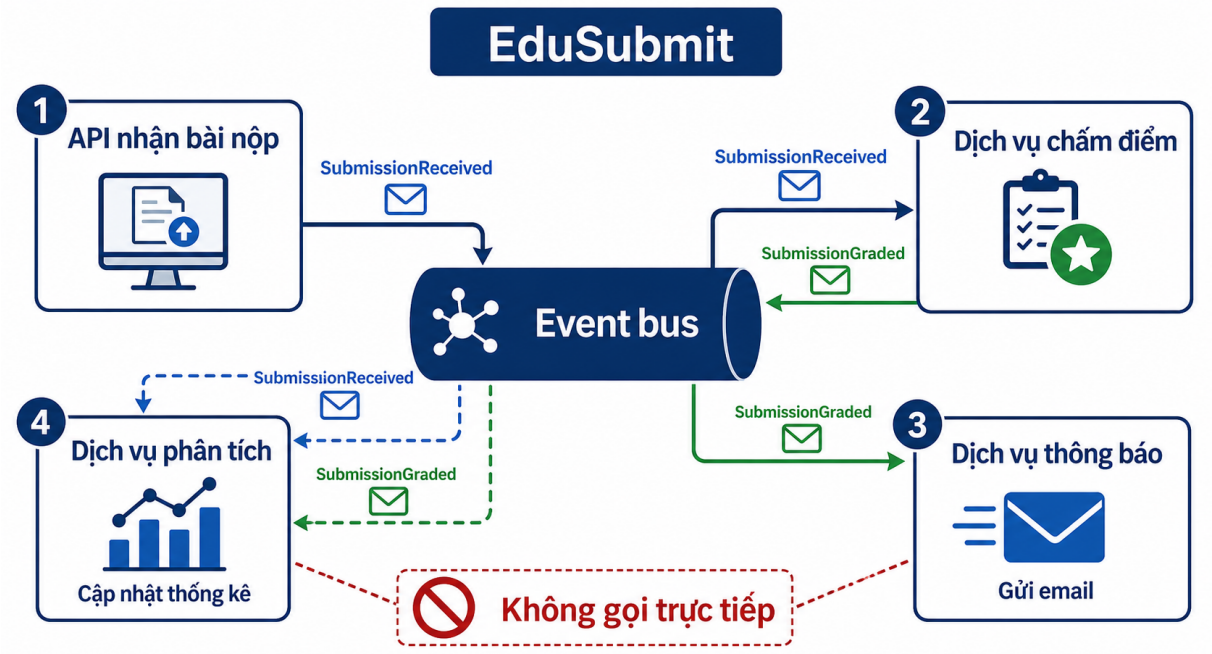
Khi các sự kiện trở thành phương tiện giao tiếp chính giữa các dịch vụ, hệ thống chuyển từ kiến trúc **hướng yêu cầu (request-driven)** sang kiến trúc **hướng sự kiện (event-driven architecture – EDA)**. Đây không chỉ là một thay đổi về công nghệ, mà là một sự thay đổi về tư duy thiết kế.

Trong kiến trúc hướng yêu cầu, một dịch vụ gọi trực tiếp dịch vụ khác khi cần điều gì đó. Dịch vụ A phải biết dịch vụ B tồn tại, biết cách gọi B, và thường phải chờ B phản hồi. Điều này tạo ra chuỗi phụ thuộc tường minh: A phụ thuộc B, B phụ thuộc C, và cứ thế. Khi một mắt xích trong chuỗi gặp sự cố, toàn bộ chuỗi bị ảnh hưởng.

Trong kiến trúc hướng sự kiện, các dịch vụ không gọi nhau trực tiếp. Thay vào đó, mỗi dịch vụ phát hành sự kiện khi trạng thái của nó thay đổi, và đăng ký lắng nghe các sự kiện mà nó quan tâm. Trung tâm của kiến trúc này là **event bus** – một cơ chế trung gian tiếp nhận sự kiện từ publisher và chuyển đến tất cả subscriber đã đăng ký.

Hình dung trong EduSubmit: khi sinh viên nộp bài, API không trực tiếp gọi dịch vụ chấm điểm, không gọi dịch vụ thông báo, không gọi dịch vụ phân tích. API chỉ làm đúng một việc: lưu bài nộp và phát hành sự kiện `SubmissionReceived`. Từ thời điểm đó, mỗi dịch vụ tự quyết định phản ứng của mình:

1. **Dịch vụ chấm điểm** nhận `SubmissionReceived`, thực thi quy trình chấm, và phát hành `SubmissionGraded`.



Hình 7.3: Kiến trúc hướng sự kiện trong quy trình xử lý bài nộp của EduSubmit

2. **Dịch vụ thông báo** nhận `SubmissionGraded`, gửi email cho sinh viên kèm điểm số.
3. **Dịch vụ phân tích** nhận cả `SubmissionReceived` và `SubmissionGraded`, cập nhật bảng thống kê: số lượt nộp theo thời gian, tỉ lệ hoàn thành, phân bố điểm.

Điểm mấu chốt là: API không hề biết đến sự tồn tại của dịch vụ thông báo hay dịch vụ phân tích. Nếu ngày mai nhóm phát triển thêm một dịch vụ phát hiện đạo văn, dịch vụ này chỉ cần đăng ký lắng nghe `SubmissionReceived` – không cần thay đổi bất kỳ dòng code nào trong API. Đây chính là sức mạnh của kiến trúc hướng sự kiện: **khả năng mở rộng chức năng mà không sửa đổi hệ thống hiện có**.

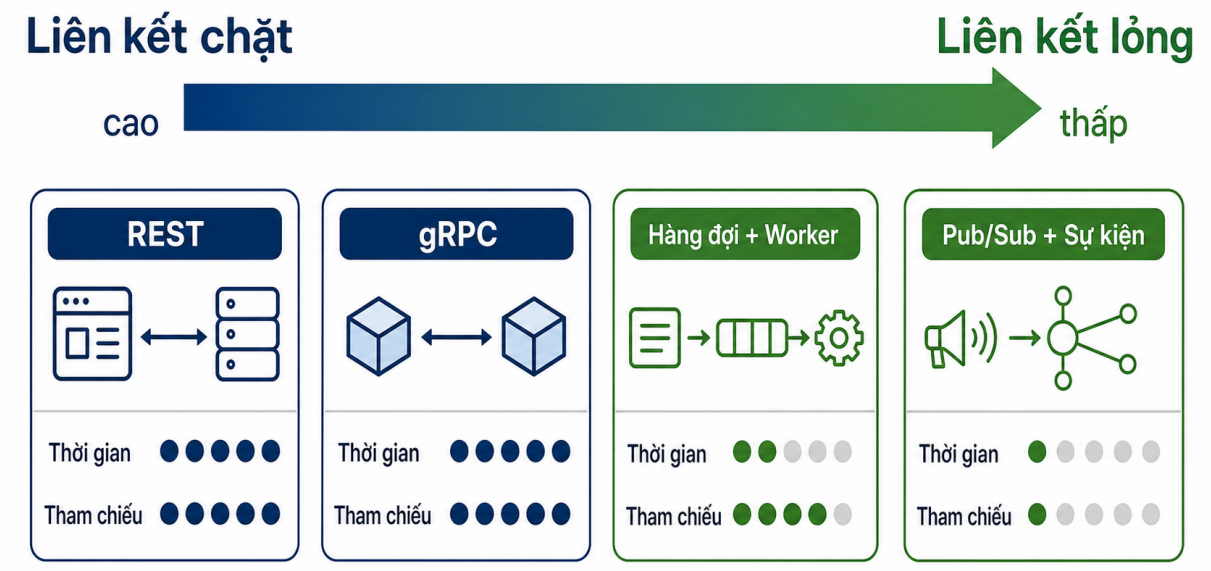
Về mặt hạ tầng, event bus có thể được hiện thực ở nhiều mức độ phức tạp khác nhau. Ở mức đơn giản nhất, một topic exchange trong RabbitMQ (với nhiều queue cùng bind đến một exchange qua routing key) có thể đóng vai trò event bus. Ở quy mô lớn hơn, các công cụ chuyên dụng như Apache Kafka, Azure Event Grid, hoặc AWS EventBridge cung cấp khả năng lưu trữ sự kiện lâu dài, phát lại sự kiện (event replay), và xử lý luồng sự kiện (event streaming).

7.1.4 Giảm phụ thuộc giữa các dịch vụ

Xuyên suốt giáo trình này, chúng ta đã đi từ kiến trúc liên kết chặt đến liên kết lỏng. Hành trình đó có thể được tóm tắt qua bốn mức độ phụ thuộc giảm dần:

1. **REST (bài 2)**: Client biết chính xác URL của server, gửi yêu cầu và chờ phản hồi. Cả hai chiều thời gian và tham chiếu đều liên kết chặt.
2. **gRPC (bài 5)**: Client biết contract (.proto) của server, gửi yêu cầu và chờ phản hồi. Hợp đồng rõ ràng hơn nhưng vẫn liên kết chặt cả hai chiều.
3. **Hàng đợi + Worker (bài 6)**: Producer gửi thông điệp vào queue, không cần consumer chạy đồng thời. Tham chiếu vẫn liên kết (biết tên queue) nhưng thời gian đã tách rời.

4. **Pub/Sub + Sự kiện (bài 7)**: Publisher phát hành sự kiện lên event bus, không biết ai sẽ nhận. Cả thời gian và tham chiếu đều có thể tách rời – đây là mức độ giảm phụ thuộc cao nhất trong bốn mô hình.



Hình 7.4: Mức độ giảm phụ thuộc khi chuyển từ gọi trực tiếp sang pub/sub hướng sự kiện

Mức độ phụ thuộc (coupling) không phải là thước đo "tốt" hay "xấu" tuyệt đối – nó là một sự đánh đổi (trade-off). Giảm phụ thuộc mang lại:

- **Khả năng phát triển độc lập**: Mỗi dịch vụ có thể được phát triển, kiểm thử và triển khai bởi các nhóm khác nhau, với chu kỳ phát hành riêng. Dịch vụ thông báo có thể nâng cấp từ email lên push notification mà không ảnh hưởng đến API.
- **Khả năng chịu lỗi tốt hơn**: Khi một dịch vụ gặp sự cố, các dịch vụ khác không bị ảnh hưởng trực tiếp. Nếu dịch vụ thông báo ngừng hoạt động, sự kiện `SubmissionGraded` vẫn được phát hành và lưu trong event bus; khi dịch vụ khôi phục, nó có thể xử lý các sự kiện tồn đọng.
- **Khả năng mở rộng chức năng**: Dịch vụ mới có thể được thêm vào hệ thống bằng cách đăng ký lắng nghe sự kiện có sẵn, không cần sửa đổi publisher hay các subscriber khác.

Tuy nhiên, giảm phụ thuộc cũng đi kèm với chi phí:

- **Tính trong suốt giảm**: Khó theo dõi luồng dữ liệu end-to-end khi các dịch vụ không gọi nhau trực tiếp. Cần công cụ distributed tracing (như OpenTelemetry) để tái tạo lại đường đi của một yêu cầu qua hệ thống.
- **Tính nhất quán cuối cùng**: Trong kiến trúc hướng sự kiện, dữ liệu giữa các dịch vụ không nhất quán tức thời. Sau khi API phát hành `SubmissionReceived`, dịch vụ phân tích có thể mất vài giây để cập nhật số liệu. Người dùng có thể thấy trạng thái cũ trong khoảng thời gian ngắn đó.
- **Độ phức tạp vận hành**: Debug một hệ thống hướng sự kiện khó hơn debug một chuỗi gọi đồng bộ. Cần công cụ giám sát chuyên dụng cho event bus, cơ chế xử lý thông điệp chết (dead letter), và chiến lược khôi phục khi mất sự kiện.

Nguyên tắc chỉ đạo là: **chỉ giảm phụ thuộc khi cần, và giảm đúng mức**. Với các thao tác cốt lõi, đơn giản và ít thay đổi, REST hoặc gRPC vẫn là lựa chọn tốt. Với các

luồng xử lý có nhiều bên quan tâm đến cùng một sự kiện, hoặc khi cần khả năng mở rộng chức năng trong tương lai, pub/sub và event-driven là lựa chọn phù hợp.

Trong phần ứng dụng của chương này (mục 7.3), chúng ta sẽ áp dụng pub/sub vào EduSubmit: worker chấm điểm phát hành sự kiện khi xử lý xong, dịch vụ nhận sự kiện và cập nhật trạng thái, kết hợp SignalR để thông báo kết quả thời gian thực cho người dùng.

7.2 Kỹ thuật lập trình cơ bản với event-driven và pub/sub

Phần 7.1 đã giới thiệu mẫu phát hành–đăng ký, sự khác biệt giữa yêu cầu và sự kiện, và kiến trúc hướng sự kiện. Phần này đi vào các thao tác lập trình cụ thể: từ thiết kế contract sự kiện, khai báo exchange và binding, publish sự kiện, đến đăng ký subscriber và xử lý các vấn đề thực tế như trùng lặp và phiên bản. Mỗi mục tập trung vào một kỹ thuật với đoạn mã tối giản, dùng ví dụ một hệ thống nhỏ gồm hai tiến trình: `EventPublisher` phát sự kiện vào topic exchange, và `EventSubscriber` đăng ký nhận sự kiện theo mẫu định tuyến. Mã nguồn đầy đủ nằm trong `content/examples/chapter07/section02/`.

7.2.1 Thiết kế event contract và dữ liệu sự kiện

Khác với mô hình hàng đợi (bài 6) nơi producer và consumer thống nhất với nhau qua tên hàng đợi, trong pub/sub các bên thống nhất với nhau qua ba yếu tố: **tên exchange**, **routing key**, và **cấu trúc dữ liệu sự kiện**. Ba yếu tố này tạo thành **hợp đồng sự kiện (event contract)** – một thỏa thuận ngầm giữa publisher và subscriber về "kênh" truyền thông và "hình dạng" thông điệp.

Trong .NET, contract được đặt trong một project dùng chung để cả bên phát và bên nhận cùng tham chiếu:

Mã nguồn 7.1: Event contract: tên exchange, routing key và cấu trúc dữ liệu sự kiện

```
namespace EventContracts;

public static class EventNames {
    public const string Exchange = "demo.events";
}

public static class RoutingKeys {
    // order.*  - moi su kien ve don hang
    public const string OrderCreated = "order.created";
    public const string OrderPaid = "order.paid";
    public const string OrderShipped = "order.shipped";

    // user.*   - moi su kien ve nguoi dung
    public const string UserRegistered = "user.registered";
    public const string UserUpdated = "user.updated";
}

public sealed record DemoEvent(
    string EventId,
    string EventType,
    string Message,
    string Version,
```

```
    DateTimeOffset Timestamp);
```

Ba thành phần của contract cần được thiết kế cẩn thận:

- `EventId`: Định danh duy nhất cho mỗi sự kiện, thường dùng GUID hoặc chuỗi có tiền tố + số thứ tự. Trường này là nền tảng cho cơ chế khử trùng lặp (idempotency) sẽ bàn ở mục 7.2.5.
- `EventType`: Chuỗi mô tả loại sự kiện, thường trùng với routing key. Trường này giúp subscriber nhanh chóng phân loại sự kiện mà không cần phân tích routing key từ metadata của RabbitMQ.
- `Version`: Số phiên bản của cấu trúc sự kiện (ví dụ `"1.0"`, `"2.0"`). Khi cần thêm trường mới vào sự kiện, tăng phiên bản giúp subscriber phân biệt định dạng cũ và mới – chi tiết ở mục 7.2.5.

Routing key trong pub/sub được thiết kế theo cấu trúc phân cấp, phân cách bằng dấu chấm. Ví dụ: `order.created`, `order.paid`, `user.registered`. Cấu trúc này cho phép subscriber dùng ký tự đại diện (wildcard) để đăng ký theo nhóm: `order.*` khớp với mọi sự kiện về đơn hàng, `*.*` khớp với mọi sự kiện hai cấp.

7.2.2 Khai báo exchange, queue, binding và routing key

Trong RabbitMQ pub/sub, exchange là điểm trung tâm tiếp nhận sự kiện từ publisher. Có bốn kiểu exchange (đã giới thiệu ở mục 6.1.4), nhưng với pub/sub, **topic exchange** là lựa chọn phù hợp nhất vì nó hỗ trợ định tuyến theo mẫu (pattern-based routing).

Mã nguồn 7.2: Khai báo topic exchange – điểm phát sự kiện trung tâm

```
await channel.ExchangeDeclareAsync(
    exchange: "demo.events",
    type: ExchangeType.Topic,
    durable: true,
    autoDelete: false,
    arguments: null);
```

Exchange là tài nguyên dùng chung, do đó cả publisher và subscriber đều nên khai báo nó (idempotent). Sau khi có exchange, publisher sẵn sàng gửi sự kiện. Về phía subscriber, cần thêm hai bước: tạo queue riêng và **bind** queue đó vào exchange với một routing pattern.

Mã nguồn 7.3: Subscriber tạo queue riêng và bind vào exchange với routing pattern

```
// Tao queue rieng cho subscriber nay (tam thoi, tu dong xoa)
var queueName = "demo.subscriber.order-star";
await channel.QueueDeclareAsync(
    queue: queueName,
    durable: false, exclusive: true, autoDelete: true,
    arguments: null);

// Bind queue vao exchange voi pattern "order.*"
await channel.QueueBindAsync(
    queue: queueName,
```

```
    exchange: "demo.events",
    routingKey: "order.*",
    arguments: null);
```

Điểm khác biệt then chốt với mô hình hàng đợi (bài 6): mỗi subscriber tạo **queue riêng** thay vì dùng chung một queue. Điều này đảm bảo mỗi subscriber nhận được **bản sao** của sự kiện, không phải cạnh tranh nhau. Tham số `exclusive:true` và `autoDelete: true` giúp queue tự động bị xóa khi subscriber ngắt kết nối – phù hợp cho subscriber tạm thời trong môi trường phát triển.

Hai ký tự đại diện trong routing pattern của RabbitMQ topic exchange:

- `*` (một từ): Khớp chính xác một cấp trong routing key. Ví dụ: `order.*` khớp `order.created` và `order.paid`, nhưng không khớp `order` hay `order.created.v2`.
- `\#` (không hoặc nhiều từ): Khớp không hoặc nhiều cấp. Ví dụ: `order.\#` khớp `order.created`, `order.created.v2`, và cả `order` (không có cấp con). `\#` đứng một mình khớp mọi routing key – subscriber nhận toàn bộ sự kiện.

7.2.3 Publish sự kiện lên topic exchange

Khác với bài 6, nơi thông điệp được gửi vào một hàng đợi cụ thể qua exchange rỗng, trong pub/sub thông điệp được gửi đến **topic exchange** kèm theo **routing key**. RabbitMQ dựa vào routing key và các binding đã thiết lập để quyết định chuyển sự kiện đến những queue nào.

Mã nguồn 7.4: Publish sự kiện vào topic exchange với routing key

```
var evt = new DemoEvent(
    EventId: "EVT-0001",
    EventType: "order.created",
    Message: "Don hang moi duoc tao",
    Version: "1.0",
    Timestamp: DateTimeOffset.UtcNow);

var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(evt));

var properties = new BasicProperties
{
    ContentType = "application/json",
    MessageId = evt.EventId,
    Type = evt.EventType,
    Persistent = true
};

await channel.BasicPublishAsync(
    exchange: "demo.events",
    routingKey: "order.created",
    mandatory: false,
    basicProperties: properties,
    body: body);
```

So với `BasicPublishAsync` trong bài 6, điểm khác biệt duy nhất là tham số `exchange`: thay vì `string.Empty` (default exchange), ta truyền tên topic exchange. Với default exchange, routing key phải trùng tên hàng đợi đích; với topic exchange, routing key mô tả **chủ đề** của sự kiện và RabbitMQ tự định tuyến đến tất cả queue có binding pattern khớp.

Thuộc tính `Type` trên `BasicProperties` là một trường tùy chọn nhưng hữu ích: nó cho phép subscriber hoặc công cụ giám sát nhanh chóng nhận diện loại sự kiện mà không cần giải mã toàn bộ payload. Đây là một thực hành tốt trong thiết kế hệ thống hướng sự kiện.

7.2.4 Đăng ký và xử lý sự kiện ở subscriber

Phía subscriber, mẫu lập trình tương tự consumer trong bài 6: dùng `AsyncEventingBasicConsum` để đăng ký hàm xử lý bất đồng bộ. Điểm khác biệt là subscriber cần thực hiện đủ ba bước trước khi bắt đầu nhận: khai báo exchange, tạo queue, và bind.

Mã nguồn 7.5: Subscriber hoàn chỉnh: bind vào exchange, nhận và xử lý sự kiện

```
using System.Collections.Concurrent;
using System.Text;
using System.Text.Json;
using EventContracts;
using RabbitMQ.Client;
using RabbitMQ.Client.Events;

var hostName = Environment.GetEnvironmentVariable("RabbitMQ__HostName") ?? "localhost";
var bindingPattern = args.Length > 0 ? args[0] : "order.*";

var factory = new ConnectionFactory { HostName = hostName };

await using var connection = await factory.CreateConnectionAsync();
await using var channel = await connection.CreateChannelAsync();

// Khai bao topic exchange (giong ben publisher)
await channel.ExchangeDeclareAsync(
    exchange: EventNames.Exchange,
    type: ExchangeType.Topic,
    durable: true,
    autoDelete: false,
    arguments: null);

// Tao queue rieng cho subscriber nay
var queueName = $"demo.subscriber.{bindingPattern.Replace("*", "star").Replace("#",
↪  "hash")}";
await channel.QueueDeclareAsync(
    queue: queueName,
    durable: false,
    exclusive: true,
    autoDelete: true,
    arguments: null);

// Bind queue vao exchange voi routing key pattern
await channel.QueueBindAsync(
    queue: queueName,
    exchange: EventNames.Exchange,
    routingKey: bindingPattern,
    arguments: null);

Console.WriteLine($"Subscriber dang lang nghe: exchange={EventNames.Exchange},
↪  pattern={bindingPattern}");
Console.WriteLine($"Queue: {queueName}");

var processedIds = new ConcurrentDictionary<string, byte>();
```

```
var consumer = new AsyncEventingBasicConsumer(channel);

consumer.ReceivedAsync += async (_, eventArgs) => {
    var json = Encoding.UTF8.GetString(eventArgs.Body.Span);
    var evt = JsonSerializer.Deserialize<DemoEvent>(json);

    if (evt is null) {
        await channel.BasicNackAsync(eventArgs.DeliveryTag, multiple: false, requeue:
        ↪  false);
        return;
    }

    // Idempotency: kiem tra trung lap bang EventId
    if (!processedIds.TryAdd(evt.EventId, 0)) {
        Console.WriteLine($"[{DateTime.Now:HH:mm:ss}] Bo qua trung lap:
        ↪  {evt.EventId}");
        await channel.BasicAckAsync(eventArgs.DeliveryTag, multiple: false);
        return;
    }

    Console.WriteLine($"[{DateTime.Now:HH:mm:ss}] Nhan: {evt.EventId} [{evt.EventType}]
    ↪  v{evt.Version} - {evt.Message}");

    // Gia lap thoi gian xu ly
    await Task.Delay(TimeSpan.FromMilliseconds(500));

    await channel.BasicAckAsync(eventArgs.DeliveryTag, multiple: false);
};

await channel.BasicConsumeAsync(
    queue: queueName,
    autoAck: false,
    consumer: consumer);

Console.WriteLine("Nhan Ctrl+C de dung.");
await Task.Delay(Timeout.InfiniteTimeSpan);
```

Ba chi tiết đáng chú ý trong listing trên:

1. **Binding pattern từ đối số dòng lệnh**: Subscriber nhận pattern đăng ký qua `args[0]`, cho phép chạy nhiều tiến trình subscriber với các pattern khác nhau mà không cần biên dịch lại. Sinh viên có thể mở ba cửa sổ dòng lệnh, chạy một subscriber với `order.*`, một với `user.*`, và một với `\#` để quan sát cách RabbitMQ định tuyến sự kiện đến từng subscriber.
2. **Khử trùng lặp bằng `ConcurrentDictionary`**: Biến `processedIds` lưu các `EventId` đã xử lý. Khi nhận được sự kiện, subscriber kiểm tra `TryAdd`: nếu `EventId` đã tồn tại, sự kiện bị bỏ qua. Đây là cách đơn giản nhất để đảm bảo **idempotency** – mỗi sự kiện chỉ được xử lý một lần, ngay cả khi RabbitMQ giao lại do lỗi mạng hoặc consumer timeout.
3. **Hàng đợi tạm thời (transient queue)**: Với `durable:false`, `exclusive:true`, `autoDelete:true`, queue chỉ tồn tại trong thời gian subscriber chạy. Khi subscriber dừng, queue tự động bị xóa – phù hợp cho subscriber không cần lưu trữ sự kiện khi ngoại tuyến. Trong hệ thống sản xuất, subscriber quan trọng thường dùng `durable:true` để đảm bảo không mất sự kiện khi tạm dừng.

7.2.5 *Quản lý phiên bản sự kiện và xử lý lặp idempotent*

Hai vấn đề thực tế phát sinh khi hệ thống hướng sự kiện vận hành lâu dài: **thay đổi cấu trúc sự kiện** và **xử lý trùng lặp**.

Phiên bản sự kiện (event versioning). Khi hệ thống phát triển, cấu trúc sự kiện có thể cần thêm trường. Ví dụ: ban đầu `DemoEvent` chỉ có `EventId` và `Message`; sau đó cần thêm `Priority`. Nếu publisher gửi sự kiện phiên bản 2.0 nhưng subscriber vẫn mong đợi phiên bản 1.0, lỗi giải tuần tự hóa sẽ xảy ra.

Ba chiến lược phổ biến để xử lý phiên bản:

- **Luôn tương thích tiến (forward-compatible)**: Chỉ thêm trường mới có giá trị mặc định. Subscriber cũ bỏ qua trường không nhận diện được (JSON.NET mặc định làm được việc này với `JsonSerializer.Deserialize`). Đây là chiến lược đơn giản nhất và nên được ưu tiên.
- **Phiên bản song song**: Publisher gửi đồng thời cả hai phiên bản trong giai đoạn chuyển tiếp. Subscriber mới đọc phiên bản mới, subscriber cũ vẫn đọc phiên bản cũ. Sau khi tất cả subscriber đã nâng cấp, ngừng gửi phiên bản cũ.
- **Đăng ký theo phiên bản**: Dùng routing key chứa số phiên bản, ví dụ `order.created.v2`. Subscriber cũ đăng ký `order.created.v1`, subscriber mới đăng ký `order.created.v2`. Cách này tận dụng chính cơ chế định tuyến của RabbitMQ.

Xử lý trùng lặp (idempotency). Như đã đề cập ở bài 6, RabbitMQ đảm bảo **at-least-once delivery**: mỗi thông điệp được giao ít nhất một lần, nhưng trong một số tình huống (mất mạng, consumer timeout, broker khởi động lại) có thể bị giao trùng. Trong pub/sub, vấn đề này càng quan trọng vì một sự kiện có thể được giao đến nhiều subscriber, mỗi subscriber cần tự bảo vệ khỏi trùng lặp.

Mã nguồn 7.6: Kiểm tra trùng lặp bằng EventId trước khi xử lý

```
var processedIds = new ConcurrentDictionary<string, byte>();

// Trong ham xu ly:
if (!processedIds.TryAdd(evt.EventId, 0))
{
    Console.WriteLine($"Bo qua trung lap: {evt.EventId}");
    await channel.BasicAckAsync(eventArgs.DeliveryTag, multiple: false);
    return;
}

// Xu ly su kien...

await channel.BasicAckAsync(eventArgs.DeliveryTag, multiple: false);
```

Trong ví dụ này, `ConcurrentDictionary` đóng vai trò kho lưu tạm các ID đã xử lý. Lưu ý rằng đây là giải pháp trong bộ nhớ – nếu subscriber khởi động lại, bộ nhớ bị xóa và các sự kiện trùng lặp có thể bị xử lý lại. Trong hệ thống sản xuất, ID đã xử lý nên được lưu trong cơ sở dữ liệu hoặc cache ngoài (Redis) với thời gian hết hạn (TTL) phù hợp.

7.2.6 Kết nối subscriber với kênh thông báo thời gian thực

Pub/sub giải quyết vấn đề giao tiếp giữa các dịch vụ phía máy chủ, nhưng người dùng cuối không kết nối trực tiếp đến RabbitMQ. Để đưa kết quả sự kiện đến máy khách, subscriber thường đóng thêm vai trò **cầu nối (bridge)**: nhận sự kiện từ event bus, sau đó đẩy thông báo đến máy khách qua một kênh thời gian thực như SignalR (bài 3) hoặc Server-Sent Events. Mẫu kiến trúc này tạo ra sự phân tách rõ ràng về trách nhiệm:

- **RabbitMQ**: Vận chuyển sự kiện giữa các dịch vụ phía máy chủ (service-to-service). Đảm bảo độ tin cậy, định tuyến linh hoạt, và khả năng mở rộng subscriber.
- **SignalR**: Vận chuyển thông báo từ dịch vụ đến máy khách (service-to-client). Đảm bảo độ trễ thấp, kết nối hai chiều, và khả năng nhóm người dùng theo ngữ cảnh (bài nộp, phòng thi, v.v.).

Trong thực hành, subscriber sau khi nhận và xử lý sự kiện từ RabbitMQ có thể gọi:

Mã nguồn 7.7: Subscriber đẩy thông báo đến máy khách qua SignalR hub

```
// Sau khi xu ly su kien thanh cong:
await hubContext.Clients
    .Group($"submission:{evt.SubmissionId}")
    .SendAsync("SubmissionGraded", new { evt.SubmissionId, evt.Score });
```

Đoạn mã trên giả định subscriber được tích hợp sẵn `IHubContext<NotificationHub>` (đã học ở bài 3.2.4). Nhóm (group) được đặt theo mã bài nộp, đảm bảo chỉ máy khách đang theo dõi bài nộp cụ thể mới nhận được thông báo – thay vì toàn bộ người dùng trong hệ thống.

Không nên nhầm lẫn vai trò của hai công nghệ: RabbitMQ không thay thế SignalR, và SignalR không thay thế RabbitMQ. Chúng hoạt động ở hai tầng khác nhau trong kiến trúc tổng thể. Bài 7.3 sẽ minh họa đầy đủ mẫu cầu nối này trong bối cảnh EduSubmit.

Tổng kết

Sáu kỹ thuật trên là bộ công cụ tối thiểu để lập trình pub/sub hướng sự kiện với RabbitMQ trong .NET. Để xem toàn bộ mã nguồn hoàn chỉnh của ví dụ `EventPublisher` và `EventSubscriber`, sinh viên có thể tham khảo:

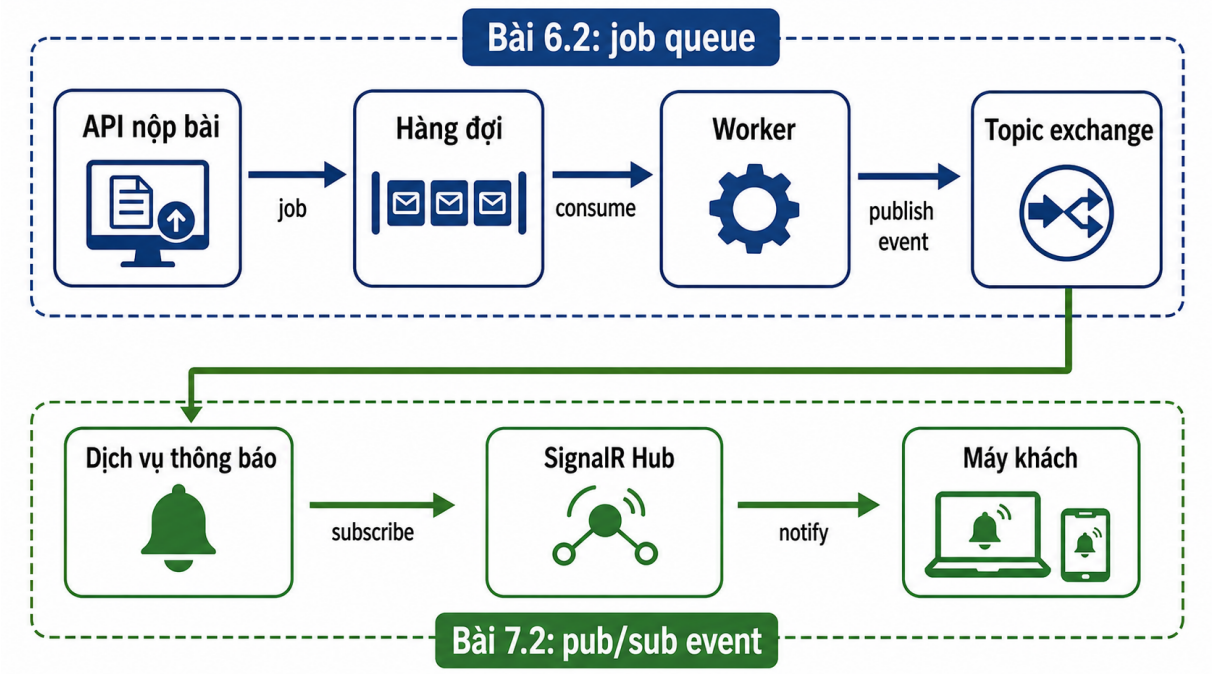
```
examples/chapter07/section02/
  src/
    EventContracts/
    EventPublisher/
    EventSubscriber/
  README.md
```

Mục 7.3 sẽ ghép toàn bộ kỹ thuật này vào hệ thống EduSubmit hoàn chỉnh: worker chấm điểm phát hành sự kiện `SubmissionGraded`, dịch vụ thông báo đăng ký nhận sự kiện, và SignalR đẩy kết quả thời gian thực đến máy khách.

7.3 [Ứng dụng] Áp dụng cơ chế truyền thông điệp pub/sub vào tương tác dịch vụ–máy khách

Bài thực hành này nối trực tiếp với bài 6.3. Ở bài 6.3, `Submission.Api` nhận bài nộp, gửi một `GradingJobMessage` vào hàng đợi `edusubmit.grading.jobs`, rồi trả về trạng thái `Processing`. `Grading.Worker` lấy job ra khỏi hàng đợi, chấm bài và in kết quả ra màn hình. Luồng đó đã tách API khỏi thời gian chấm bài, nhưng kết quả xử lý nền vẫn chưa quay lại được các thành phần khác trong hệ thống.

Trong bài này, ta giữ lại phần hàng đợi của chương 6 và bổ sung một tầng sự kiện. Sau khi chấm xong, `Grading.Worker` không gọi trực tiếp dịch vụ thông báo. Worker chỉ phát sự kiện `SubmissionGraded` lên một topic exchange của RabbitMQ. `Notification.Api` đăng ký nhận sự kiện đó, lưu kết quả tối giản và dùng SignalR để đẩy thông báo đến máy khách đang theo dõi bài nộp tương ứng.



Hình 7.5: Luồng ứng dụng 7.3: từ hàng đợi xử lý job sang sự kiện pub/sub và thông báo SignalR

Lát cắt mới của EduSubmit gồm năm project:

```
examples/chapter07/section03/
  EventDrivenNotificationSystem.slnx
  src/
    EduSubmit.Contracts/
    Submission.Api/
    Grading.Worker/
    Notification.Api/
    Submission.RealtimeClient/
```

Ba project đầu tiên kế thừa ý tưởng của bài 6.3. Hai project mới là `Notification.Api` và `Submission.RealtimeClient`. Điểm cần chú ý là `Notification.Api` không được `Grading.Worker` gọi bằng HTTP hoặc gRPC. Nó chỉ tự đăng ký với RabbitMQ để nhận

loại sự kiện mà nó quan tâm. Đây chính là khác biệt thực hành giữa hàng đợi xử lý công việc và publish–subscribe: hàng đợi chấm bài vẫn là một job cho một worker, còn sự kiện chấm xong có thể được nhiều dịch vụ cùng nhận.

7.3.1 Dịch vụ xử lý nền phát sự kiện khi xử lý xong

Trước hết cần mở rộng contract dùng chung. Ngoài tên hàng đợi từ bài 6.3, project `EduSubmit.Contracts` bổ sung tên exchange, routing key và kiểu dữ liệu mô tả sự kiện `SubmissionGradedEvent`. Khi viết file contract, nên đi theo thứ tự từ tên kênh truyền thông, thông điệp job, sự kiện kết quả, rồi cuối cùng mới đến các DTO phản hồi.

Hình 7.6: Quy trình viết contract dùng chung cho luồng job và sự kiện

Mã nguồn 7.8: Contract dùng chung cho job chấm bài và sự kiện chấm xong

```
namespace EduSubmit.Contracts;

public static class MessagingNames {
    public const string GradingQueue = "edusubmit.grading.jobs";
    public const string SubmissionEventsExchange = "edusubmit.submission.events";
    public const string SubmissionGradedRoutingKey = "submission.graded";
    public const string NotificationQueue =
    ↪  "edusubmit.notifications.submission-graded";
}

public sealed record SubmitAssignmentRequest(
    int AssignmentId,
    string StudentId,
    string AnswerText);

public sealed record GradingJobMessage(
    int SubmissionId,
    int AssignmentId,
    string StudentId,
    string AnswerText,
```

```
    DateTimeOffset SubmittedAt);

public sealed record SubmissionGradedEvent(
    int SubmissionId,
    int AssignmentId,
    string StudentId,
    int Score,
    string Status,
    DateTimeOffset GradedAt);

public sealed record SubmissionAcceptedResponse(
    int SubmissionId,
    string Status,
    string StatusUrl);

public sealed record SubmissionStatusResponse(
    int SubmissionId,
    string Status);
```

Trong listing này, hằng `MessagingNames.GradingQueue` vẫn giữ vai trò cũ của bài 6.3: đó là hàng đợi công việc chấm bài. Phần mới gồm ba khái niệm: exchange phát sự kiện, routing key `submission.graded` và record `SubmissionGradedEvent`. Exchange là điểm phát logic; routing key mô tả loại sự kiện; còn record sự kiện chứa những dữ liệu mà các bên nhận cần biết để phản ứng: bài nộp nào đã chấm xong, sinh viên nào, điểm bao nhiêu và thời điểm chấm.

`Submission.Api` trong bài này gần như giống bài 6.3: nhận yêu cầu HTTP, tạo `GradingJobMessage`, gửi job vào hàng đợi và trả `Processing`. Ta không đưa lại toàn bộ listing ở đây để tránh lặp; điểm quan trọng là API vẫn không biết `Notification.Api`. API chỉ biết hàng đợi công việc chấm bài, giống như ở chương 6.

Phần thay đổi chính nằm ở `Grading.Worker`. Worker vẫn tiêu thụ `edusubmit.grading.jobs`, nhưng sau khi tính điểm xong, nó tạo một `SubmissionGradedEvent` và publish sự kiện này vào topic exchange.

Mã nguồn 7.9: Worker nhận job chấm bài rồi phát sự kiện SubmissionGraded

```
await channel.QueueDeclareAsync(
    queue: MessagingNames.GradingQueue,
    durable: true,
    exclusive: false,
    autoDelete: false,
    arguments: null);

await channel.ExchangeDeclareAsync(
    exchange: MessagingNames.SubmissionEventsExchange,
    type: ExchangeType.Topic,
    durable: true,
    autoDelete: false);

var consumer = new AsyncEventingBasicConsumer(channel);

consumer.ReceivedAsync += async (_, eventArgs) => {
    var json = Encoding.UTF8.GetString(eventArgs.Body.Span);
    var job = JsonSerializer.Deserialize<GradingJobMessage>(json);
```

```
    if (job is null) {
        await channel.BasicNackAsync(
            eventArgs.DeliveryTag,
            multiple: false,
            requeue: false);
        return;
    }

    var score = Grade(job.AnswerText);
    await Task.Delay(TimeSpan.FromSeconds(2));

    var gradedEvent = new SubmissionGradedEvent(
        job.SubmissionId,
        job.AssignmentId,
        job.StudentId,
        score,
        "Graded",
        DateTimeOffset.UtcNow);

    await PublishGradedEventAsync(channel, gradedEvent);

    Console.WriteLine(
        $"Da cham submission {job.SubmissionId}: score={score}. Da phat event
        ↪  SubmissionGraded.");

    await channel.BasicAckAsync(
        eventArgs.DeliveryTag,
        multiple: false);
};

await channel.BasicConsumeAsync(
    queue: MessagingNames.GradingQueue,
    autoAck: false,
    consumer: consumer);

Console.WriteLine("Grading.Worker dang nghe job va se phat event submission.graded.");
Console.WriteLine("Nhan Ctrl+C de dung worker.");

await Task.Delay(Timeout.InfiniteTimeSpan);

static async Task PublishGradedEventAsync(
    IChannel channel,
    SubmissionGradedEvent gradedEvent) {
    var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(gradedEvent));
    var properties = new BasicProperties {
        ContentType = "application/json",
        MessageId = gradedEvent.SubmissionId.ToString(CultureInfo.InvariantCulture),
        Type = nameof(SubmissionGradedEvent),
        Persistent = true
    };

    await channel.BasicPublishAsync(
        exchange: MessagingNames.SubmissionEventsExchange,
        routingKey: MessagingNames.SubmissionGradedRoutingKey,
        mandatory: false,
        basicProperties: properties,
        body: body);
}
```

```
static int Grade(string answerText) {
    var normalized = answerText.Trim();
```

Có hai vùng code cần đọc kỹ. Vùng đầu tiên là đoạn khai báo `QueueDeclareAsync` và `ExchangeDeclareAsync`. Worker vẫn khai báo hàng đợi job để nhận công việc như bài 6.3, đồng thời khai báo thêm `SubmissionEventsExchange` kiểu `Topic`. Với topic exchange, worker không cần biết dịch vụ nào sẽ nhận sự kiện; nó chỉ publish với routing key `submission.graded`.

Vùng thứ hai là hàm publish event. Hàm này chuyển event chấm xong thành JSON và publish vào exchange thay vì gửi vào một hàng đợi cụ thể. Thuộc tính `Type` giúp người đọc nhật ký hoặc công cụ quan sát nhận ra loại thông điệp; `Persistent=true` cho RabbitMQ biết thông điệp nên được ghi bền vững khi cấu hình broker hỗ trợ.

Một chi tiết thiết kế đáng chú ý là worker chỉ gọi `BasicAckAsync` sau khi đã publish sự kiện thành công. Nếu worker xác nhận job trước rồi mới publish event, hệ thống có thể rơi vào trạng thái "đã chấm xong nhưng không ai nhận được sự kiện" khi tiến trình gặp lỗi ở giữa. Ví dụ này chưa triển khai đầy đủ transactional outbox, nhưng thứ tự xử lý đã giúp sinh viên nhìn thấy vấn đề nhất quán giữa xử lý job và phát sự kiện.

> Bài 6.3 dùng RabbitMQ như một hàng đợi công việc: một thông điệp chấm bài được một worker xử lý. Bài 7.3 dùng RabbitMQ thêm như một event bus: một sự kiện chấm xong có thể được nhiều queue khác nhau bind vào cùng exchange để nhiều dịch vụ phản ứng độc lập.

7.3.2 Dịch vụ nhận sự kiện và xử lý

Bên nhận sự kiện trong bài này là `Notification.Api`. Đây là một API ASP.NET Core có hai vai trò. Vai trò thứ nhất là chạy một `BackgroundService` để lắng nghe RabbitMQ. Vai trò thứ hai là mở SignalR hub để máy khách có thể đăng ký nhận thông báo theo từng bài nộp.

Đoạn khởi tạo ứng dụng khá ngắn:

Mã nguồn 7.10: Notification.Api đăng ký SignalR, kho thông báo và tiến trình nhận sự kiện

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSignalR();
builder.Services.AddSingleton<NotificationStore>();
builder.Services.AddHostedService<RabbitMqGradedEventSubscriber>();

var app = builder.Build();

app.MapGet("/", () => Results.Ok(new {
    service = "Notification.Api",
    status = "running"
}));

app.MapGet("/api/submissions/{submissionId:int}/notifications", (
    int submissionId,
    NotificationStore store) => {
        var message = store.Get(submissionId);
```

```
        return message is null
            ? Results.NotFound()
            : Results.Ok(message);
    });

app.MapHub<SubmissionHub>("/hubs/submissions");

app.Run();

public sealed class SubmissionHub : Hub {
```

`AddSignalR` thêm hạ tầng đẩy thông báo thời gian thực. `NotificationStore` là kho trong bộ nhớ để ví dụ có thể đọc lại kết quả đã nhận qua HTTP. Dòng đăng ký hosted service tạo tiến trình nền chuyên nhận sự kiện từ RabbitMQ. Trong ứng dụng thực tế, kho thông báo nên là cơ sở dữ liệu hoặc cache dùng chung; ở đây ta giữ trong bộ nhớ để tập trung vào luồng pub/sub.

Phần nhận sự kiện nằm trong `RabbitMqGradedEventSubscriber`. Dịch vụ này khai báo cùng exchange với worker, tạo một hàng đợi riêng cho dịch vụ thông báo, rồi bind hàng đợi đó với routing key `submission.graded`.

Mã nguồn 7.11: Dịch vụ thông báo subscribe sự kiện chấm xong từ topic exchange

```
    protected override async Task ExecuteAsync(CancellationToken stoppingToken) {
        var factory = new ConnectionFactory { HostName = _hostName };

        await using var connection =
            await factory.CreateConnectionAsync(stoppingToken);
        await using var channel =
            await connection.CreateChannelAsync(cancellationToken: stoppingToken);

        await channel.ExchangeDeclareAsync(
            exchange: MessagingNames.SubmissionEventsExchange,
            type: ExchangeType.Topic,
            durable: true,
            autoDelete: false,
            cancellationToken: stoppingToken);

        await channel.QueueDeclareAsync(
            queue: MessagingNames.NotificationQueue,
            durable: true,
            exclusive: false,
            autoDelete: false,
            arguments: null,
            cancellationToken: stoppingToken);

        await channel.QueueBindAsync(
            queue: MessagingNames.NotificationQueue,
            exchange: MessagingNames.SubmissionEventsExchange,
            routingKey: MessagingNames.SubmissionGradedRoutingKey,
            cancellationToken: stoppingToken);

        var consumer = new AsyncEventingBasicConsumer(channel);
        consumer.ReceivedAsync += async (_, eventArgs) => {
            var json = Encoding.UTF8.GetString(eventArgs.Body.Span);
```

```
            var gradedEvent = JsonSerializer.Deserialize<SubmissionGradedEvent>(json);

            if (gradedEvent is null) {
                await channel.BasicNackAsync(
                    eventArgs.DeliveryTag,
                    multiple: false,
                    requeue: false,
                    cancellationToken: stoppingToken);
                return;
            }

            store.Save(gradedEvent);

            var notification = new SubmissionNotification(
                gradedEvent.SubmissionId,
                gradedEvent.StudentId,
                gradedEvent.Score,
                gradedEvent.Status,
                gradedEvent.GradedAt);

            await hubContext.Clients
                .Group(SubmissionGroups.ForSubmission(gradedEvent.SubmissionId))
                .SendAsync("SubmissionGraded", notification, stoppingToken);

            logger.LogInformation(
                "Da gui thong bao SignalR cho submission {SubmissionId}",
                gradedEvent.SubmissionId);

            await channel.BasicAckAsync(
                eventArgs.DeliveryTag,
                multiple: false,
                cancellationToken: stoppingToken);
        };

        await channel.BasicConsumeAsync(
            queue: MessagingNames.NotificationQueue,
            autoAck: false,
            consumer: consumer,
            cancellationToken: stoppingToken);

        logger.LogInformation(
            "Notification.Api dang nghe event {RoutingKey}",
            MessagingNames.SubmissionGradedRoutingKey);

        await Task.Delay(Timeout.InfiniteTimeSpan, stoppingToken);
    }
}

public sealed record SubmissionNotification(
    int SubmissionId,
    string StudentId,
    int Score,
    string Status,
    DateTimeOffset GradedAt);
```

Đây là khác biệt quan trọng so với bài 6.3. Trong mô hình hàng đợi công việc, nhiều worker cùng đọc một queue để chia nhau xử lý job; mỗi job chỉ nên được một worker

xử lý. Trong mô hình pub/sub, mỗi dịch vụ quan tâm đến sự kiện nên có queue riêng. Nếu sau này thêm `Analytics.Api`, dịch vụ đó có thể tạo queue `edusubmit.analytics.submission-graded` và bind cùng routing key `submission.graded`. Khi worker phát một sự kiện, RabbitMQ sẽ sao chép sự kiện đó đến cả queue thông báo và queue phân tích.

Sau khi giải mã event chấm xong, `Notification.Api` lưu một bản thông báo, gửi thông báo đến nhóm SignalR tương ứng và cuối cùng mới gọi `BasicAckAsync`. Nhóm SignalR dùng khóa theo mẫu `submission:\{submissionId\}` để bảo đảm máy khách chỉ nhận thông báo của bài nộp mà nó đang theo dõi, thay vì nhận mọi kết quả chấm bài trong hệ thống.

7.3.3 Kết hợp SignalR để thông báo cho máy khách

SignalR trong bài này không thay thế RabbitMQ. Hai cơ chế phục vụ hai ranh giới khác nhau. RabbitMQ nối các tiến trình phía máy chủ với nhau: worker phát sự kiện, dịch vụ thông báo nhận sự kiện. SignalR nối dịch vụ phía máy chủ với máy khách đang mở kết nối: `Notification.Api` đẩy kết quả chấm bài về console client.

Hub của `Notification.Api` chỉ cần một phương thức để máy khách đăng ký theo dõi bài nộp:

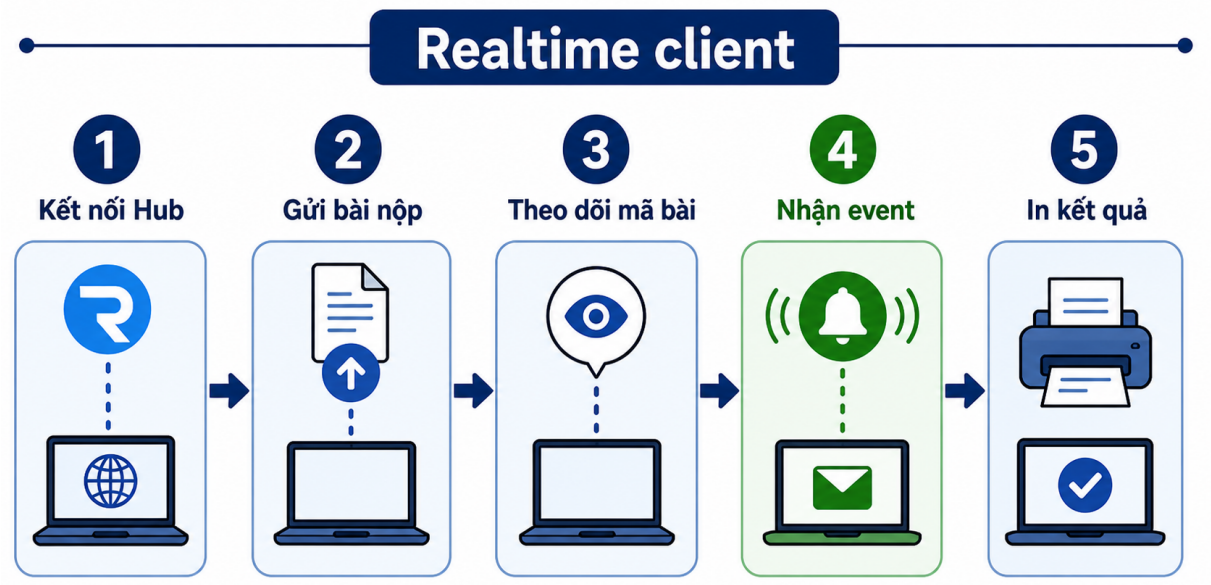
Mã nguồn 7.12: SignalR hub nhóm kết nối theo mã bài nộp

```
        return Groups.AddToGroupAsync(
            Context.ConnectionId,
            SubmissionGroups.ForSubmission(submissionId));
    }
}

public static class SubmissionGroups {
    public static string ForSubmission(int submissionId) =>
        ForSubmission(submissionId.ToString());

    public static string ForSubmission(string submissionId) =>
        $"submission:{submissionId}";
}

public sealed class NotificationStore {
    private readonly ConcurrentDictionary<int, SubmissionNotification> _items = new();

    public void Save(SubmissionGradedEvent gradedEvent) {
```

Máy khách trong ví dụ là một console app. Nó kết nối đến SignalR hub của dịch vụ thông báo, gửi bài nộp đến API nộp bài, lấy mã bài nộp, rồi gọi `WatchSubmission` để tham gia nhóm theo dõi. Khi dịch vụ thông báo nhận được event `SubmissionGraded`, client sẽ nhận được callback cùng tên.

Mã nguồn 7.13: Console client gửi bài nộp và chờ thông báo SignalR

```
using System.Net;
using System.Net.Http.Json;
using EduSubmit.Contracts;
using Microsoft.AspNetCore.SignalR.Client;

var submissionApiUrl = args.Length > 0
```



Hình 7.7: Quy trình viết console client gửi bài nộp và chờ thông báo SignalR

```
    ? args[0].TrimEnd('/')
    : "http://localhost:5200";

var notificationApiUrl = args.Length > 1
    ? args[1].TrimEnd('/')
    : "http://localhost:5300";

using var submissionHttp = new HttpClient {
    BaseAddress = new Uri(submissionApiUrl)
};

await using var connection = new HubConnectionBuilder()
    .WithUrl($"{notificationApiUrl}/hubs/submissions")
    .WithAutomaticReconnect()
    .Build();

var gradedNotification = new TaskCompletionSource();

connection.On<SubmissionNotification>("SubmissionGraded", notification => {
    Console.WriteLine(
        $"SignalR: submission {notification.SubmissionId} da cham xong,
        ↪  score={notification.Score}");
    gradedNotification.TrySetResult();
});

await connection.StartAsync();
Console.WriteLine("Da ket noi Notification.Api qua SignalR.");

var createResponse = await submissionHttp.PostAsJsonAsync(
    "/api/submissions",
    new SubmitAssignmentRequest(
        AssignmentId: 1,
        StudentId: "SV001",
        AnswerText: "Pub/sub giup worker phat su kien va nhieu dich vu co the phan ung
        ↪  doc lap."));

if (createResponse.StatusCode != HttpStatusCode.Accepted) {
```

```
    Console.WriteLine($"Khong tao duoc submission: {(int)createResponse.StatusCode}");
    return;
}

var accepted = await
↪  createResponse.Content.ReadFromJsonAsync<SubmissionAcceptedResponse>();
if (accepted is null) {
    Console.WriteLine("Khong doc duoc phan hoi tu Submission.Api.");
    return;
}

Console.WriteLine(
    $"Submission.Api tra ve: {accepted.SubmissionId} - {accepted.Status}");

await connection.InvokeAsync("WatchSubmission", accepted.SubmissionId.ToString());
Console.WriteLine("Dang cho event SubmissionGraded qua Notification.Api...");

var completed = await Task.WhenAny(
    gradedNotification.Task,
    Task.Delay(TimeSpan.FromSeconds(15)));

if (completed != gradedNotification.Task) {
    Console.WriteLine("Qua thoi gian cho event SubmissionGraded.");
}

public sealed record SubmissionNotification(
    int SubmissionId,
    string StudentId,
    int Score,
    string Status,
    DateTimeOffset GradedAt);
```

Trong listing này, `submissionApiUrl` và `notificationApiUrl` được tách riêng để nhấn mạnh hệ thống đang có nhiều tiến trình. Máy khách gửi HTTP request đến API nộp bài, nhưng kết quả cuối cùng lại đến từ dịch vụ thông báo qua SignalR. Đây là hình ảnh rất thực tế của hệ thống hướng sự kiện: đường đi của yêu cầu ban đầu và đường đi của thông báo kết quả không nhất thiết quay về cùng một dịch vụ.

Biên dịch toàn bộ solution bằng lệnh:

```
dotnet build examples/chapter07/section03/EventDrivenNotificationSystem.slnx
```

Để chạy thử, trước hết khởi động RabbitMQ:

```
docker run --name edusubmit-rabbitmq --rm -p 5672:5672 -p 15672:15672
↪  rabbitmq:3-management
```

Sau đó mở bốn cửa sổ dòng lệnh. Cửa sổ thứ nhất chạy API nhận bài nộp:

```
dotnet run --project examples/chapter07/section03/src/Submission.Api --urls
↪  http://localhost:5200
```

Cửa sổ thứ hai chạy dịch vụ thông báo:

```
dotnet run --project examples/chapter07/section03/src/Notification.Api --urls
↪  http://localhost:5300
```

Cửa sổ thứ ba chạy worker chấm bài:

```
dotnet run --project examples/chapter07/section03/src/Grading.Worker
```

Cửa sổ thứ tư chạy console client:

```
dotnet run --project examples/chapter07/section03/src/Submission.RealtimeClient --
↪  http://localhost:5200 http://localhost:5300
```

Khi chạy thành công, client sẽ in luồng tương tự:

```
Da ket noi Notification.Api qua SignalR.
Submission.Api tra ve: 1001 - Processing
Dang cho event SubmissionGraded qua Notification.Api...
SignalR: submission 1001 da cham xong, score=8
```

Quan sát này khép kín mạch từ bài 6.3 đến bài 7.3. `Processing` vẫn là phản hồi tức thời của API khi job đã vào hàng đợi. Điểm mới là kết quả `Graded` không còn bị kẹt trong console của worker; nó trở thành một sự kiện mà dịch vụ khác có thể nhận và chuyển tiếp cho máy khách. Nếu thêm dịch vụ phân tích, gửi email hoặc ghi lịch sử hoạt động, ta không cần sửa `Grading.Worker`; chỉ cần thêm subscriber mới cho sự kiện `submission.graded`.

Trong hệ thống thật, cần bổ sung thêm các cơ chế như lưu trạng thái bền vững, xử lý thông điệp lỗi, idempotency cho subscriber và truy vết phân tán. Tuy vậy, lát cắt thực hành này đã thể hiện đúng lõi của kiến trúc hướng sự kiện: thành phần xử lý nền phát hiện một sự thật đã xảy ra, phát sự kiện ra event bus, còn các thành phần khác tự phản ứng theo trách nhiệm riêng.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **MEP phát hành–đăng ký (pub/sub)**: Publisher công bố thông điệp lên một chủ đề (topic), tất cả subscriber đã đăng ký quan tâm đến chủ đề đó đều nhận được. Khác với hàng đợi điểm–điểm (một thông điệp – một consumer), pub/sub cho phép giao tiếp một–nhiều (một sự kiện – nhiều subscriber).
- **Bốn mô hình phối hợp**: Phân loại theo hai chiều – thời gian (đồng thời/không đồng thời) và tham chiếu (biết/không biết định danh). Bốn mô hình: Trực tiếp (REST, gRPC), Hộp thư (hàng đợi, worker pattern), Dựa trên sự kiện (pub/sub, SignalR), Không gian dữ liệu chung (Linda tuple spaces).
- **Hai biến thể pub/sub**: (1) Dựa trên chủ đề (topic-based) – subscriber đăng ký theo tên hoặc mẫu chủ đề, được RabbitMQ topic exchange hỗ trợ; (2) Dựa trên nội dung (content-based) – subscriber đăng ký theo điều kiện lọc trên nội dung thông điệp, mạnh hơn nhưng phức tạp hơn.

- **Sự kiện khác yêu cầu**: Yêu cầu (request) mang tính mệnh lệnh, có định hướng, điểm–điểm, đồng bộ, bên gửi biết rõ bên nhận. Sự kiện (event) mang tính khai báo, không định hướng, một–nhiều, bất đồng bộ, bên phát không biết ai sẽ nhận. Hệ thống thực tế thường dùng cả hai: yêu cầu cho thao tác đọc và tương tác cần phản hồi ngay; sự kiện cho thông báo thay đổi trạng thái và phản ứng dây chuyền.
- **Kiến trúc hướng sự kiện (EDA)**: Các dịch vụ không gọi nhau trực tiếp mà giao tiếp qua event bus. Mỗi dịch vụ phát hành sự kiện khi trạng thái thay đổi và đăng ký lắng nghe sự kiện mình quan tâm. Lợi ích cốt lõi: thêm dịch vụ mới (vd: phát hiện đạo văn) chỉ cần đăng ký lắng nghe sự kiện có sẵn, không cần sửa publisher hay subscriber hiện có.
- **Phổ giảm phụ thuộc**: Bốn mức từ liên kết chặt đến lỏng – REST (biết URL, đồng bộ) → gRPC (biết contract, đồng bộ) → Hàng đợi + Worker (biết tên queue, bất đồng bộ) → Pub/Sub + Sự kiện (không biết ai nhận, bất đồng bộ). Giảm phụ thuộc mang lại khả năng phát triển độc lập, chịu lỗi tốt hơn và mở rộng chức năng dễ dàng, nhưng đánh đổi bằng tính trong suốt giảm, nhất quán cuối cùng và độ phức tạp vận hành.
- **Event bus**: Cơ chế trung gian tiếp nhận sự kiện từ publisher và chuyển đến tất cả subscriber. Ở mức đơn giản: RabbitMQ topic exchange với nhiều queue cùng bind vào một exchange. Ở quy mô lớn: Apache Kafka, Azure Event Grid, AWS EventBridge (có thêm lưu trữ lâu dài, phát lại sự kiện, xử lý luồng).
- **Thực hành luồng hướng sự kiện**: `Grading.Worker` sau khi chấm xong phát sự kiện `SubmissionGradedEvent` vào topic exchange `edusubmit.submission.events` (thay vì gọi trực tiếp dịch vụ thông báo). `Notification.Api` tạo queue riêng, bind vào cùng exchange với routing key `submission.graded`, nhận sự kiện và đẩy kết quả đến máy khách qua SignalR. Mỗi subscriber có queue riêng – khác với competing consumers trong hàng đợi công việc.
- **SignalR bổ trợ, không thay thế**: RabbitMQ nối các tiến trình phía máy chủ (server-to-server), SignalR nối máy chủ với máy khách (server-to-client). Hai cơ chế phục vụ hai ranh giới khác nhau trong cùng một luồng xử lý.

Câu hỏi ôn tập

1. Nêu định nghĩa mẫu phát hành–đăng ký (pub/sub). Phân biệt pub/sub với mẫu hàng đợi điểm–điểm đã học ở bài 6 về: số lượng bên nhận, cơ chế phân phối thông điệp, và tình huống sử dụng.
2. Trình bày bốn mô hình phối hợp trong hệ thống phân tán dựa trên hai chiều: thời gian (có cần đồng thời?) và tham chiếu (có cần biết định danh?). Xếp REST, hàng đợi RabbitMQ, pub/sub và SignalR vào từng mô hình tương ứng.
3. Phân biệt hai biến thể của pub/sub: topic-based và content-based. Cho ví dụ cụ thể trong EduSubmit cho mỗi biến thể. Biến thể nào được RabbitMQ hỗ trợ trực tiếp?
4. Lập bảng so sánh yêu cầu (request) và sự kiện (event) theo ít nhất 5 tiêu chí: hướng giao tiếp, tính chủ động, đồng bộ/bất đồng bộ, số bên nhận, và thời điểm xử lý. Với mỗi tiêu chí, cho một ví dụ từ EduSubmit.
5. Mô tả kiến trúc hướng sự kiện (EDA) trong quy trình xử lý bài nộp của EduSubmit: từ lúc sinh viên nộp bài đến khi nhận được thông báo kết quả. Chỉ rõ: những sự kiện nào được phát hành, ai phát hành, ai nhận, và event bus đóng vai trò gì.

6. Giải thích lợi ích "khả năng mở rộng chức năng mà không sửa đổi hệ thống hiện có" của EDA. Nếu EduSubmit cần thêm dịch vụ phát hiện đạo văn, cần thay đổi những gì trong `Submission.Api` và `Grading.Worker`?
7. Trình bày bốn mức độ giảm phụ thuộc (coupling) từ REST đến pub/sub. Với mỗi mức, nêu: bên gửi cần biết gì về bên nhận, hai bên có cần chạy đồng thời không, và một ví dụ công nghệ. Mức độ phụ thuộc thấp hơn có luôn tốt hơn không? Vì sao?
8. Nêu và giải thích ba lợi ích và ba chi phí/đánh đổi của việc giảm phụ thuộc trong kiến trúc hướng sự kiện. Từ đó phát biểu nguyên tắc chỉ đạo khi quyết định mức độ giảm phụ thuộc cho một tương tác cụ thể.
9. Trong ví dụ thực hành, `Grading.Worker` gọi `BasicAckAsync` **sau khi** đã publish sự kiện `SubmissionGraded`. Giải thích lý do của thứ tự này. Điều gì có thể xảy ra nếu worker xác nhận job trước rồi mới publish sự kiện?
10. Tại sao mỗi subscriber trong pub/sub cần có hàng đợi riêng thay vì cùng đọc chung một hàng đợi như mô hình competing consumers? Nếu `Notification.Api` và `Analytics.Api` cùng bind vào exchange với routing key `submission.graded`, RabbitMQ sẽ phân phối sự kiện như thế nào?

Bài tập thực hành

1. **Thêm subscriber mới – Analytics.Api**. Tạo một project `Analytics.Api` mới, đăng ký nhận cả hai sự kiện `SubmissionReceived` và `SubmissionGraded` từ topic exchange. Dịch vụ này chỉ cần ghi nhật ký (log) mỗi khi nhận được sự kiện kèm thời gian. Gợi ý: tạo queue riêng `edusubmit.analytics.submission-events` và bind với hai routing key `submission.received` và `submission.graded`. Chạy toàn bộ hệ thống và xác nhận Analytics.Api nhận được cả hai loại sự kiện.
2. **Mô phỏng subscriber ngừng hoạt động**. Chạy đầy đủ hệ thống (Submission.Api, Grading.Worker, Notification.Api, RealtimeClient). Gửi vài bài nộp và xác nhận client nhận được thông báo. Dừng `Notification.Api`, gửi thêm bài nộp, chờ worker chấm xong, rồi khởi động lại `Notification.Api`. Quan sát: các sự kiện được phát hành trong lúc Notification.Api ngừng có được nhận sau khi khởi động lại không? Giải thích dựa trên cơ chế durable queue của RabbitMQ.
3. **Tích hợp thông báo qua nhiều kênh**. Mở rộng `Notification.Api` để khi nhận sự kiện `SubmissionGraded`, ngoài việc đẩy SignalR, dịch vụ còn ghi kết quả vào một file JSON trên đĩa (mô phỏng gửi email hoặc lưu lịch sử). Chạy thử và kiểm tra file JSON được tạo. Thảo luận: nếu ghi file thất bại (vd: hết dung lượng đĩa), có nên gọi `BasicNackAsync` và đưa thông điệp trở lại hàng đợi không? Vì sao?
4. **So sánh hai cách tích hợp**. Giữ nguyên luồng pub/sub ở bài này, viết thêm một phiên bản thay thế trong đó `Grading.Worker` gọi trực tiếp REST API của `Notification.Api` sau khi chấm xong (thay vì publish sự kiện). Đo và so sánh: số dòng code thay đổi khi thêm subscriber thứ ba (`Analytics.Api`), khả năng hoạt động khi `Notification.Api` tạm dừng, và mức độ phụ thuộc giữa worker và các dịch vụ nhận kết quả.

Câu hỏi thảo luận

1. Trong kiến trúc hướng sự kiện, một sự kiện có thể kích hoạt chuỗi phản ứng dây chuyền: `SubmissionReceived` → chấm điểm → `SubmissionGraded` → thông báo

→ `NotificationSent`. Nếu một mắt xích trong chuỗi thất bại, làm thế nào để đảm bảo toàn bộ quy trình không bị đình trệ? Thảo luận các chiến lược: dead letter queue, retry with backoff, saga pattern, và event sourcing. Chiến lược nào phù hợp nhất cho bài toán chấm điểm tự động?

2. Hệ thống EduSubmit hiện tại dùng RabbitMQ vừa làm hàng đợi công việc (cho chấm điểm) vừa làm event bus (cho pub/sub). Khi hệ thống mở rộng lên hàng trăm nghìn sinh viên, liệu một broker duy nhất có đáp ứng được không? Thảo luận: khi nào nên tách riêng queue cho job xử lý và event bus, khi nào nên chuyển từ RabbitMQ sang Apache Kafka, và sự khác biệt giữa message queue và event stream trong bối cảnh này.
3. Nhìn lại hành trình từ bài 1 đến bài 7: từ một API đơn giản giao tiếp REST với client, hệ thống EduSubmit đã phát triển thành một kiến trúc đa dịch vụ với REST, gRPC, hàng đợi công việc và pub/sub hướng sự kiện. Với mỗi ranh giới giao tiếp trong hệ thống hiện tại (client–API, API–worker, worker–notification, notification–client), hãy đánh giá xem lựa chọn MEP và công nghệ hiện tại đã phù hợp chưa? Có ranh giới nào nên được thiết kế lại không? Vì sao?

CHƯƠNG 8

VẬN HÀNH HỆ THỐNG PHÂN TÁN: ĐA TIẾN TRÌNH, CONTAINER, QUAN SÁT VÀ XỬ LÝ LỖI

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Phân biệt ứng dụng đơn khối và hệ thống phân tán qua ba khác biệt cốt lõi: ranh giới giao tiếp & độ trễ, lỗi một phần (partial failure), và đồng bộ thời gian.
- So sánh giao tiếp đồng bộ và bất đồng bộ giữa các tiến trình, và giải thích sự khác biệt giữa gọi hàm cục bộ với giao tiếp mạng (IPC).
- Giải thích khái niệm Docker Image, Docker Container, và quy trình đóng gói ứng dụng .NET thành container bằng Dockerfile.
- Phân loại năm dạng lỗi trong hệ thống phân tán theo Tanenbaum, và áp dụng cơ chế timeout, retry (exponential backoff + jitter), và idempotency key để xử lý lỗi giao tiếp.
- Đóng gói và triển khai hệ thống EduSubmit nhiều tiến trình bằng Docker Compose, mô phỏng lỗi một phần (dừng worker, dừng RabbitMQ), và quan sát phản ứng của hệ thống.

8.1 Các khái niệm cơ bản liên quan đến vận hành hệ thống phân tán

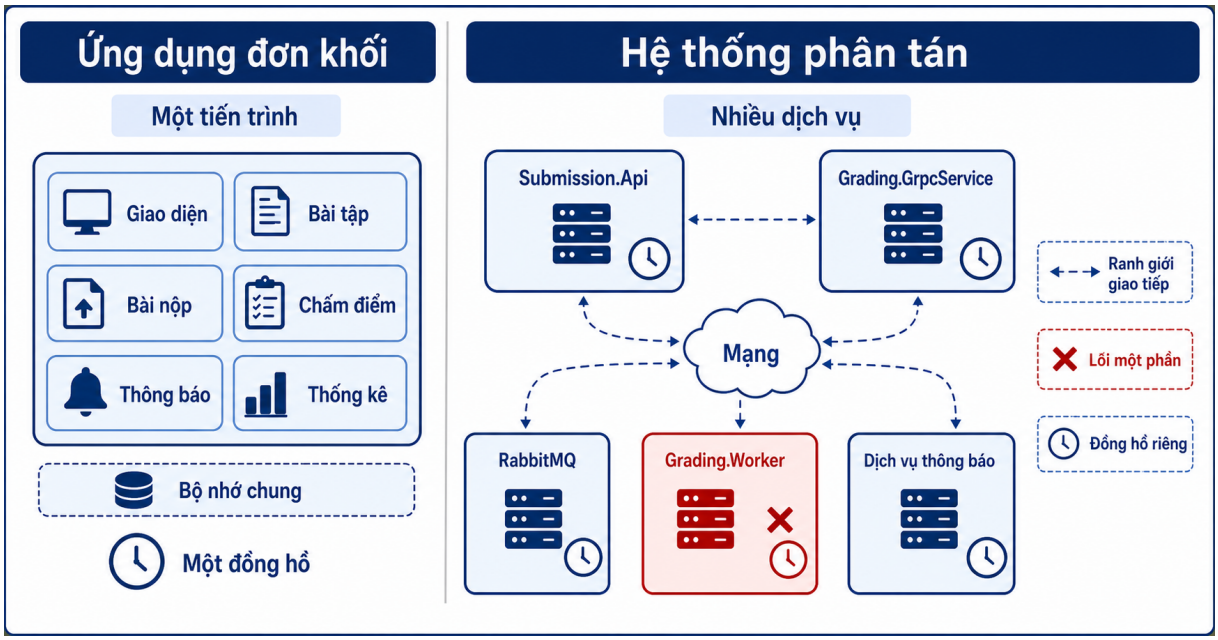
Việc chuyển từ phát triển ứng dụng đơn lẻ chạy trên môi trường cục bộ sang vận hành một hệ thống phân tán chạy trên nhiều máy chủ đặt ra nhiều thách thức mới cho người thiết kế và quản trị hệ thống. Để đảm bảo hệ thống phân tán hoạt động ổn định, tin cậy và có khả năng mở rộng, người phát triển phần mềm cần nắm vững các khái niệm cơ bản liên quan đến ranh giới vận hành, giao tiếp đa tiến trình, công nghệ đóng gói, các dạng lỗi đặc thù và cơ chế xử lý lỗi ở cấp độ giao thức.

8.1.1 Hệ thống và ứng dụng

Trong phát triển phần mềm truyền thống, lập trình viên thường làm việc với các ứng dụng đơn khối (monolithic applications) chạy trong phạm vi một tiến trình duy nhất trên một máy tính cục bộ. Khi đó, ranh giới giao tiếp chỉ giới hạn trong bộ nhớ vật lý của hệ điều hành, nơi các thành phần mã nguồn tương tác qua các lệnh gọi hàm trực tiếp.

Tuy nhiên, trong một hệ thống phân tán (distributed system), ranh giới giao tiếp này bị phá vỡ hoàn toàn. Hệ thống giờ đây là một mạng lưới gồm nhiều nút phần cứng và tiến trình phần mềm độc lập, phối hợp với nhau qua mạng để thực hiện một mục tiêu chung. Việc phân biệt giữa một "ứng dụng" đơn lẻ và một "hệ thống" phân tán được thể hiện qua các đặc trưng cốt lõi sau:

- **Ranh giới giao tiếp và độ trễ:** Gọi hàm cục bộ diễn ra trong vài nano giây đến micro giây với độ tin cậy tuyệt đối của phần cứng bộ nhớ. Ngược lại, giao tiếp mạng trong hệ thống phân tán phải trải qua các tầng giao thức kết nối mạng, chịu ảnh hưởng bởi độ trễ không thể dự đoán trước (từ vài mili giây đến hàng trăm mili giây) và nguy cơ mất gói tin hoặc mất kết nối vật lý.
- **Trạng thái độc lập và lỗi một phần (partial failure):** Trong ứng dụng đơn lẻ, nếu tiến trình chính bị sập (crash), toàn bộ ứng dụng sẽ ngừng hoạt động. Trong hệ thống phân tán, các nút hoạt động độc lập. Sự cố sập nguồn ở một máy chủ dịch vụ chấm bài không được phép làm ngắt quãng hoạt động của dịch vụ quản lý bài tập hay dịch vụ thông báo. Hệ thống phải đối mặt với tình trạng một số thành phần lỗi trong khi phần còn lại vẫn hoạt động bình thường.
- **Đồng bộ thời gian:** Các luồng xử lý cục bộ chia sẻ chung một đồng hồ vật lý duy nhất của CPU. Trong hệ thống phân tán, mỗi máy chủ sở hữu một đồng hồ tinh thể thạch anh độc lập với tốc độ trôi khác nhau, dẫn đến việc không thể xác định chính xác thứ tự thời gian tuyệt đối của các sự kiện xảy ra trên các máy chủ khác nhau nếu không dùng các giao thức đồng bộ thời gian chuyên biệt.



Hình 8.1: So sánh ranh giới vận hành giữa ứng dụng đơn khối và hệ thống phân tán

8.1.2 Đa tiến trình và giao tiếp giữa các dịch vụ

Tiến trình (process) là một chương trình đang thực thi, được hệ điều hành cấp phát một không gian địa chỉ bộ nhớ cô lập để bảo vệ an toàn hệ thống. Trong kiến trúc hệ thống phân tán, thay vì đặt mọi chức năng vào một tiến trình, hệ thống được cấu thành từ nhiều dịch vụ (multi-service) chạy đồng thời trong các tiến trình độc lập.

Khác với giao tiếp liên tiến trình (Inter-Process Communication – IPC) trên cùng một máy dùng bộ nhớ dùng chung hoặc đường ống (pipe), giao tiếp qua mạng đòi hỏi dữ liệu phải được tuần tự hóa và truyền qua socket. Chính cơ chế giao tiếp đa tiến trình qua mạng này là xương sống để kết nối các thành phần cô lập trong hệ thống phân tán:

- **Giao tiếp đồng bộ (Synchronous IPC):** Các dịch vụ gọi trực tiếp lẫn nhau qua

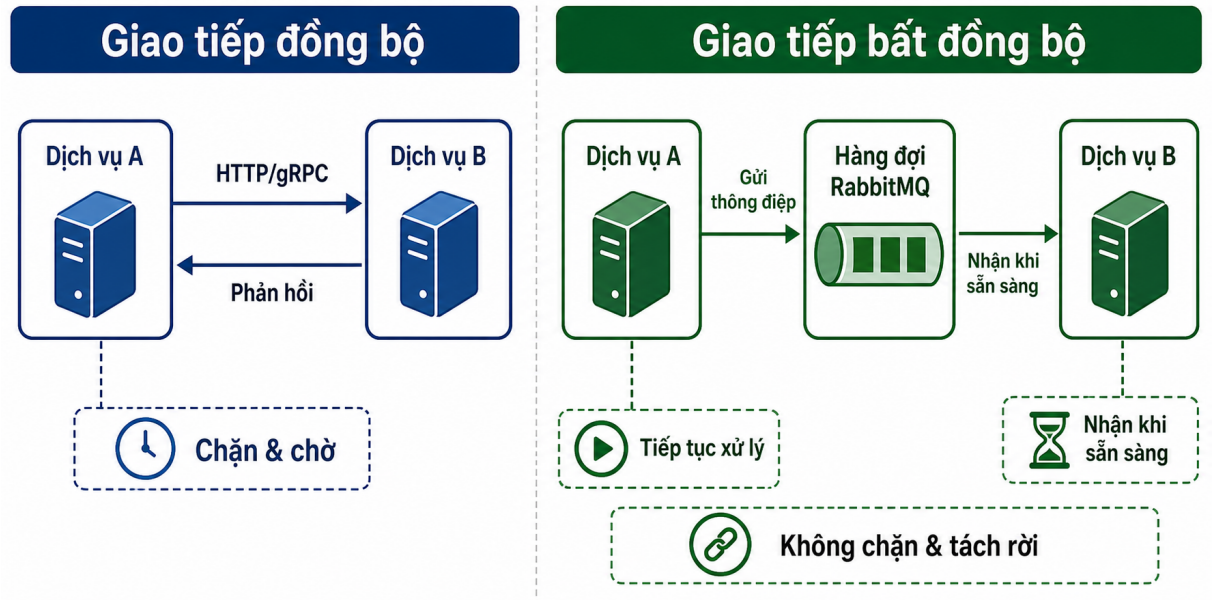
giao thức HTTP (RESTful API) hoặc RPC (gRPC). Tiến trình gọi sẽ chặn (block) luồng xử lý hiện tại để chờ kết quả phản hồi từ tiến trình nhận trước khi tiếp tục. Cơ chế này đơn giản nhưng dễ tạo ra sự phụ thuộc chặt chẽ và hiện tượng nghẽn dây chuyền (cascading failure).

- **Giao tiếp bất đồng bộ (Asynchronous IPC):** Các tiến trình giao tiếp gián tiếp qua một hệ thống trung gian hàng đợi thông điệp (Message Queue) như RabbitMQ. Dịch vụ gửi phát thông điệp vào hàng đợi và tiếp tục công việc của mình ngay lập tức mà không cần biết khi nào dịch vụ nhận sẽ xử lý xong thông điệp đó. Điều này giúp tách rời (decouple) các dịch vụ về mặt thời gian và không gian.

Bảng 8.1 so sánh sự khác biệt cơ bản giữa việc gọi hàm cục bộ trong một tiến trình và giao tiếp đa tiến trình qua mạng:

Bảng 8.1: So sánh giữa gọi hàm cục bộ và giao tiếp đa tiến trình qua mạng

Đặc tính	Gọi hàm cục bộ	Giao tiếp mạng (IPC)
Cơ chế truyền dữ liệu	Tham chiếu bộ nhớ (con trỏ)	Tuần tự hóa dữ liệu thành byte qua socket mạng
Tốc độ phản hồi	Cực kỳ nhanh (cỡ nano giây)	Chậm (cỡ mili giây), phụ thuộc băng thông mạng
Khả năng xảy ra lỗi	Gần như bằng không	Rất cao (mất mạng, quá tải, lỗi cấu hình tường lửa)
Trạng thái lỗi	Sập toàn bộ tiến trình	Lỗi một phần, cần xử lý biệt lập từng dịch vụ



Hình 8.2: So sánh giao tiếp đồng bộ và bất đồng bộ giữa các dịch vụ

8.1.3 Triển khai cơ bản với Docker

Khi vận hành một hệ thống phân tán đa tiến trình, việc cài đặt thủ công từng dịch vụ, thư viện phụ thuộc (runtime), và các dịch vụ trung gian (như RabbitMQ, hệ quản trị

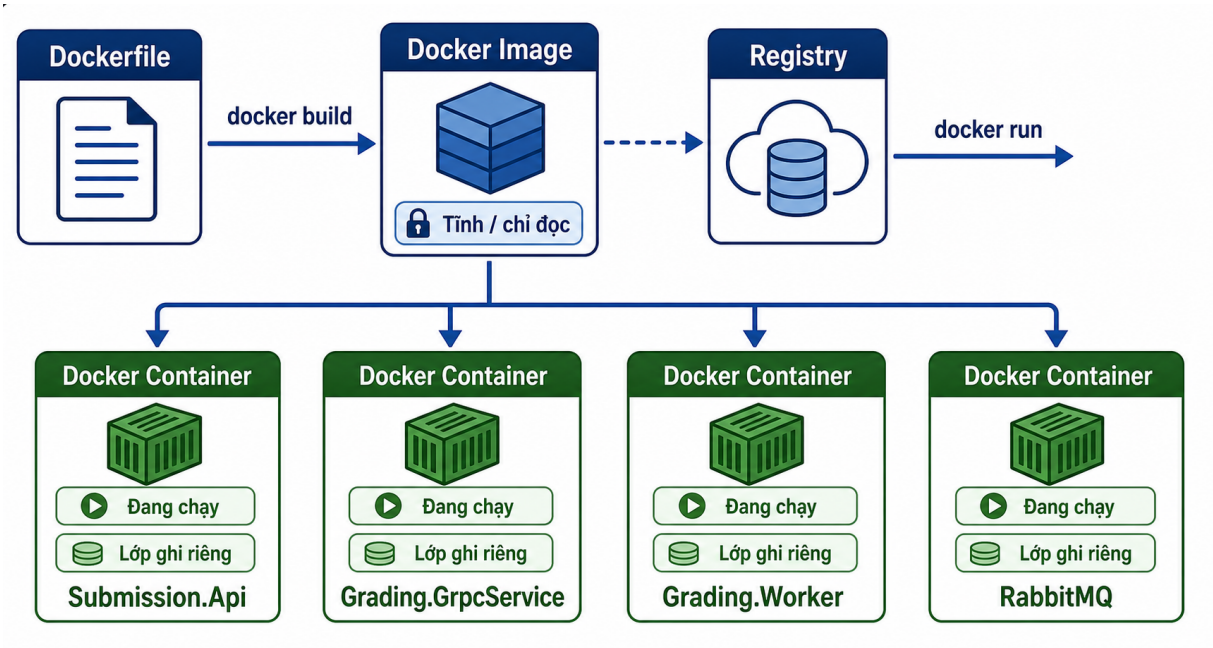
cơ sở dữ liệu) lên các hệ điều hành khác nhau rất dễ dẫn đến lỗi không đồng nhất môi trường làm việc ("mã chạy tốt ở máy tôi nhưng lỗi ở máy chủ").

Công nghệ đóng gói (containerization) ra đời để giải quyết triệt để bài toán này, trong đó Docker là công nghệ phổ biến nhất. Đóng gói cho phép gom toàn bộ mã nguồn, thư viện runtime, công cụ hệ thống và các tệp cấu hình cần thiết của một dịch vụ vào một đơn vị đóng gói duy nhất, hoạt động biệt lập với hệ điều hành máy chủ vật lý.

Trong Docker, người phát triển cần phân biệt hai khái niệm cốt lõi:

- **Docker Image (ảnh Docker):** Là một tệp tin tĩnh chứa bản thiết kế (blueprint) chỉ đọc của bộ chứa. Image chứa mã nguồn, hệ điều hành tối giản, các thư viện phụ thuộc và cấu hình định nghĩa cách khởi chạy ứng dụng. Image được xây dựng thông qua tệp cấu hình `Dockerfile` và được lưu trữ trên các kho chứa trực tuyến (Docker Registry).
- **Docker Container (bộ chứa Docker):** Là một thực thể sống chạy độc lập được tạo ra từ một image cụ thể. Mỗi bộ chứa hoạt động như một tiến trình được cô lập trên hệ điều hành máy chủ vật lý, sở hữu hệ thống tệp tin, bộ nhớ và cổng mạng riêng được ánh xạ từ máy chủ.

Lợi ích của Docker trong vận hành hệ thống phân tán bao gồm khả năng cô lập phụ thuộc tuyệt đối giữa các dịch vụ, đảm bảo sự đồng nhất môi trường từ khâu phát triển đến kiểm thử và chạy thật, đồng thời hỗ trợ việc nhân bản nhanh chóng (scaling) các dịch vụ xử lý nền khi tải hệ thống tăng cao.



Hình 8.3: Mối quan hệ giữa Dockerfile, Docker image và các container đang chạy

Trong hệ thống EduSubmit, mỗi thành phần sẽ được đóng gói vào một bộ chứa riêng: `Submission.Api` (REST/GraphQL), `Grading.GrpcService` (gRPC), `Grading.Worker` (dịch vụ xử lý nền) và `RabbitMQ` (hàng đợi thông điệp). Docker đảm bảo các bộ chứa này khởi chạy đồng nhất trên máy phát triển của từng sinh viên lẫn máy chủ kiểm thử, loại bỏ triệt để tình trạng "mã chạy tốt ở máy tôi nhưng lỗi ở máy bạn".

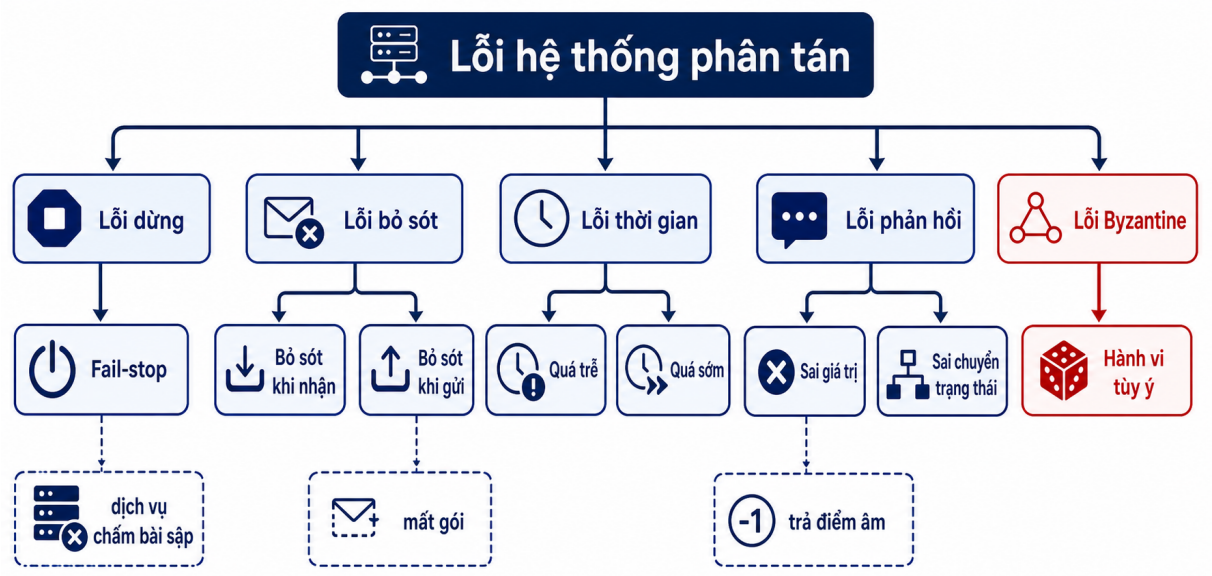
8.1.4 Lỗi trong hệ thống phân tán

Trong hệ thống phân tán, do các nút phần cứng kết nối qua một môi trường mạng không đáng tin cậy, các loại lỗi phát sinh phức tạp hơn nhiều so với hệ thống đơn máy chủ. Andrew S. Tanenbaum đã phân loại các dạng lỗi của máy chủ dựa trên hành vi bên ngoài như sau:

1. **Lỗi dừng hoạt động đột ngột (Crash failure):** Máy chủ đột ngột ngừng hoạt động nhưng trước đó vẫn xử lý hoàn toàn chính xác. Sau khi sập nguồn, máy chủ dừng phản hồi hoàn toàn. Khi các nút khác trong hệ thống có thể phát hiện chắc chắn nút này đã sập (ví dụ thông qua tín hiệu heartbeat định kỳ), trường hợp đặc biệt này được gọi là **lỗi dừng-phát-hiện-được (fail-stop failure)**. Cách duy nhất để phục hồi từ crash failure là khởi động lại tiến trình hoặc máy chủ vật lý.
2. **Lỗi bỏ sót (Omission failure):** Máy chủ không phản hồi các yêu cầu đầu vào. Dạng lỗi này chia thành hai loại:
    - *Bỏ sót khi nhận (Receive-omission failure):* Máy chủ hoàn toàn không nhận được thông điệp yêu cầu từ máy khách (do mất gói tin trên đường truyền mạng hoặc hàng đợi của socket bị tràn). Trạng thái bên trong của máy chủ không bị ảnh hưởng vì nó không biết về sự tồn tại của thông điệp yêu cầu. *Bỏ sót khi gửi (Send-omission failure):* Máy chủ nhận được yêu cầu, đã xử lý thành công và cập nhật trạng thái bên trong, nhưng gặp sự cố khi gửi thông điệp phản hồi ngược lại cho máy khách (do bộ đệm gửi bị đầy hoặc đứt kết nối mạng ngay sau khi xử lý). Điều này dẫn đến nguy cơ máy khách gửi lại yêu cầu đã được thực thi trước đó.
3. **Lỗi thời gian (Timing failure / Performance failure):** Máy chủ phản hồi đúng nội dung nhưng thời gian phản hồi nằm ngoài khoảng giới hạn yêu cầu (quá trễ do máy chủ bị quá tải CPU/RAM, hoặc quá sớm trong các ứng dụng truyền phát dữ liệu đa phương tiện gây tràn bộ đệm máy nhận).
4. **Lỗi phản hồi (Response failure):** Máy chủ phản hồi đúng thời gian nhưng nội dung thông điệp phản hồi không chính xác. Gồm hai loại phụ:
    - *Lỗi giá trị (Value failure):* Dịch vụ trả về dữ liệu bị sai lệch nội dung toán học hoặc cấu trúc thông tin (ví dụ: yêu cầu tính điểm trung bình của bài tập nhưng trả về kết quả âm).
    - *Lỗi chuyển trạng thái (State-transition failure):* Máy chủ đi chệch khỏi luồng xử lý mặc định do nhận phải các yêu cầu không mong muốn mà lập trình viên không dự phòng mã nguồn xử lý (ví dụ: máy chủ thực hiện nhầm lẫn các lệnh mặc định nguy hiểm).
5. **Lỗi tùy ý / Lỗi Byzantine (Arbitrary / Byzantine failure):** Dạng lỗi nghiêm trọng và khó xử lý nhất. Máy chủ bị lỗi này có thể gửi các phản hồi sai lệch ngẫu nhiên ở những thời điểm bất kỳ, hoặc giả mạo thông tin giao tiếp nhằm mục đích phá hoại hệ thống một cách có chủ ý. Hệ thống cần các thuật toán đồng thuận Byzantine đặc thù để cô lập nút lỗi này.

8.1.5 Gửi lại và quá thời gian chờ

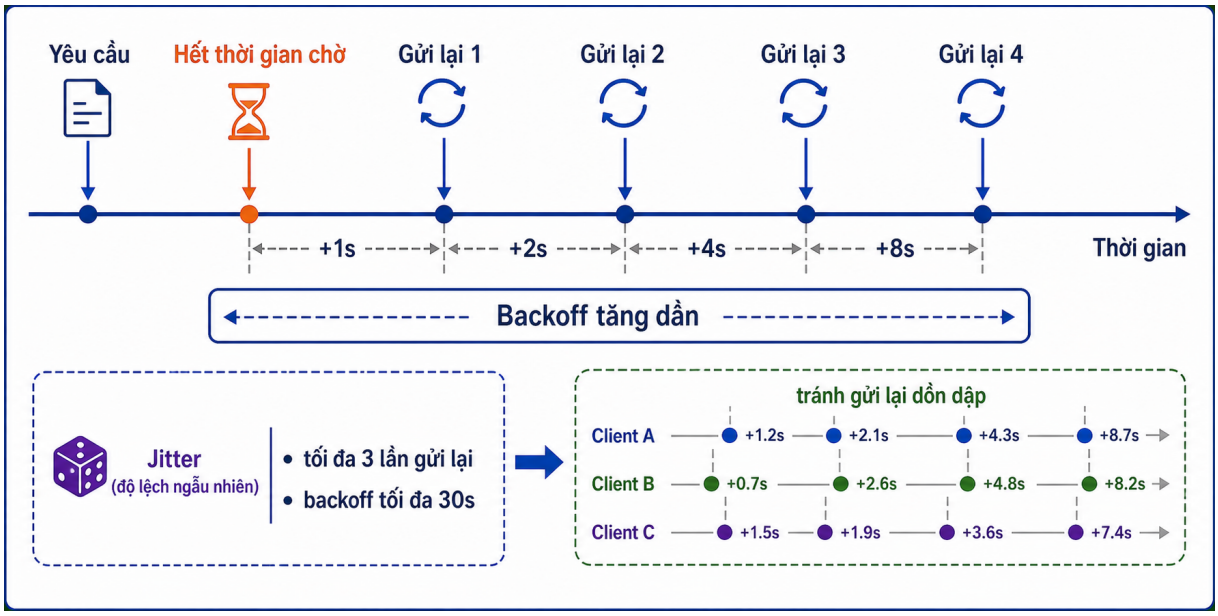
Để giảm thiểu tác động của lỗi mạng và lỗi sập nguồn tạm thời của máy chủ, máy khách tương tác trong hệ thống phân tán cần áp dụng các cơ chế xử lý lỗi chủ động, bao



Hình 8.4: Phân loại các dạng lỗi trong hệ thống phân tán theo Tanenbaum

gồm quá thời gian chờ (timeout) và gửi lại (retry).

- **Quá thời gian chờ (Timeout):** Khi gửi yêu cầu qua mạng, máy khách bắt đầu một đồng hồ đếm ngược. Nếu khoảng thời gian này trôi qua mà máy khách vẫn chưa nhận được thông điệp phản hồi (hoặc thông điệp xác nhận thu nhận), máy khách sẽ chủ động đóng kết nối hiện tại và coi thao tác đó đã gặp sự cố.
- **Gửi lại (Retry):** Khi phát hiện lỗi kết nối hoặc xảy ra sự kiện quá thời gian chờ, máy khách có thể thực hiện gửi lại yêu cầu đó thêm một số lần quy định. Khoảng thời gian giãn cách giữa các lần gửi lại nên được thiết kế tăng dần theo cấp số nhân (exponential backoff) kèm theo nhiễu ngẫu nhiên (jitter) để tránh hiện tượng dồn dập gửi yêu cầu làm tê liệt hoàn toàn máy chủ đang gặp quá tải.



Hình 8.5: Chiến lược gửi lại với backoff tăng dần và jitter

Tuy nhiên, việc gửi lại yêu cầu tiềm ẩn rủi ro chạy lặp lại các thao tác đã xử lý nếu lỗi xảy ra là lỗi bỏ sót khi gửi (send-omission failure) ở máy chủ. Tùy theo cách thiết kế, giao tiếp gọi thủ tục từ xa (Remote Procedure Call - RPC) cung cấp các đặc tính ngữ nghĩa (semantics) xử lý lỗi khác nhau:

- **Ngữ nghĩa "ít nhất một lần" (At-least-once semantics):** Máy khách sẽ gửi lại liên tục yêu cầu cho đến khi nhận được phản hồi thành công từ máy chủ. Ngữ nghĩa này đảm bảo công việc chắc chắn được thực hiện, nhưng có thể bị chạy nhiều lần. Thích hợp cho các thao tác an toàn hoặc không làm thay đổi trạng thái (như đọc dữ liệu, cập nhật thông tin tĩnh).
- **Ngữ nghĩa "nhiều nhất một lần" (At-most-once semantics):** Khi xảy ra sự kiện quá thời gian chờ, máy khách từ bỏ việc gửi lại ngay lập tức và báo lỗi cho người dùng. Thao tác chỉ được thực hiện tối đa một lần hoặc không lần nào. Thích hợp cho các thao tác nhạy cảm, không thể chạy lặp lại (như thanh toán tiền, trừ số dư tài khoản).
- **Ngữ nghĩa "chính xác một lần" (Exactly-once semantics):** Đảm bảo yêu cầu được xử lý duy nhất một lần trên máy chủ dưới mọi kịch bản lỗi mạng và lỗi sập nguồn. Về mặt lý thuyết, không có giao thức giao tiếp thuần túy nào đảm bảo được ngữ nghĩa này nếu không có sự trợ giúp từ tầng ứng dụng. Trong thực tế, exactly-once đòi hỏi phối hợp giữa cơ chế gửi lại, tính lũy đẳng và giao tác phân tán (distributed transaction) để khôi phục trạng thái nhất quán sau lỗi.

Để áp dụng an toàn cơ chế gửi lại với ngữ nghĩa "ít nhất một lần", các điểm cuối của máy chủ bắt buộc phải đảm bảo **tính lũy đẳng (idempotency)**. Một thao tác được coi là lũy đẳng nếu việc thực thi nó một lần hay nhiều lần liên tiếp đều mang lại cùng một kết quả hệ thống như nhau và không gây ra tác dụng phụ ngoài ý muốn.

Tính lũy đẳng trong thiết kế API
Trong thực tế, HTTP method `GET`, `PUT` (cập nhật toàn bộ) và `DELETE` vốn đã lũy đẳng theo đặc tả. Riêng `POST` (tạo mới tài nguyên) là thao tác không lũy đẳng – mỗi lần gọi `POST` đều tạo ra một tài nguyên mới. Đây chính là tình huống rủi ro nhất khi áp dụng retry: nếu máy khách gửi lại `POST` sau khi gặp timeout, máy chủ có thể tạo trùng lặp tài nguyên. Kỹ thuật Idempotency Key được thiết kế riêng để giải quyết bài toán này.

Để triển khai tính lũy đẳng cho các thao tác không lũy đẳng (như tạo mới một bài nộp trong EduSubmit), máy khách cần đính kèm một mã định danh duy nhất (Idempotency Key) trong tiêu đề yêu cầu; máy chủ sẽ kiểm tra mã này trong cơ sở dữ liệu trước khi xử lý để bỏ qua nếu phát hiện đó là yêu cầu gửi lại trùng lặp.

8.2 Kỹ thuật vận hành cơ bản với .NET và Docker

Phần 8.1 đã giới thiệu các khái niệm nền tảng về vận hành hệ thống phân tán: ranh giới giao tiếp, đa tiến trình, Docker, phân loại lỗi và cơ chế xử lý lỗi. Phần này đi vào các thao tác cụ thể: từ chạy nhiều tiến trình .NET với cấu hình khác nhau, đọc cấu hình từ file và biến môi trường, đóng gói service bằng Dockerfile, đến khởi chạy toàn bộ hệ thống bằng Docker Compose và thiết lập health check, retry, logging. Mỗi mục tập trung vào một kỹ thuật với đoạn mã tối giản, dùng ví dụ hai dịch vụ nhỏ: `WebApi` (API có health check, gọi backend qua HTTP với retry) và `BackendWorker` (dịch vụ nền xử lý task định

kỳ, graceful shutdown). Mã nguồn đầy đủ nằm trong `content/examples/chapter08/section02/`.

8.2.1 Chạy nhiều tiến trình .NET với cổng và cấu hình khác nhau

Trong hệ thống phân tán, mỗi dịch vụ chạy trong một tiến trình riêng với cổng mạng và cấu hình độc lập. Với .NET, việc này được thực hiện đơn giản qua đối số dòng lệnh `--urls` và biến môi trường:

Mã nguồn 8.1: Chạy hai tiến trình .NET trên hai cổng khác nhau

```
# Cua so 1: WebApi
dotnet run --project src/WebApi --urls http://localhost:5000

# Cua so 2: BackendWorker
dotnet run --project src/BackendWorker
```

WebApi lắng nghe trên cổng 5000, BackendWorker chạy nền không cần cổng HTTP. Khi chạy bằng Docker Compose (mục 8.2.5), mỗi container có không gian cổng riêng và giao tiếp với nhau qua tên dịch vụ thay vì `localhost`.

8.2.2 Đọc cấu hình từ `appsettings.json` và biến môi trường

.NET cung cấp hệ thống cấu hình phân lớp (configuration layering): biến môi trường ghi đè file `appsettings.json`, đối số dòng lệnh ghi đè biến môi trường. Cơ chế này cho phép cùng một mã nguồn hoạt động trên máy phát triển, máy kiểm thử và máy chủ sản xuất với cấu hình khác nhau.

Mã nguồn 8.2: Đọc cấu hình: file JSON làm nền, biến môi trường ghi đè

```
var builder = WebApplication.CreateBuilder(args);

// appsettings.json duoc nap tu dong
// Bien moi truong co tien to DEMO_ ghi de cau hinh
builder.Configuration.AddEnvironmentVariables("DEMO_");

// Su dung cau hinh
var backendUrl = builder.Configuration["BackendUrl"]
    ?? "http://localhost:5100";
```

File `appsettings.json` cung cấp giá trị mặc định:

Mã nguồn 8.3: Cấu hình mặc định trong appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "BackendUrl": "http://localhost:5100"
}
```

Với cơ chế này, khi chạy trong Docker Compose, biến môi trường `BackendUrl=http://backendworker:8080` sẽ tự động thay thế `http://localhost:5100` trong file JSON. Sinh viên không cần sửa code khi chuyển giữa môi trường chạy cục bộ và môi trường container.

Thứ tự ưu tiên của các nguồn cấu hình trong .NET (từ thấp đến cao):

1. `appsettings.json` – giá trị mặc định
2. `appsettings.\{Environment\}.json` – ghi đè theo môi trường (Development, Production)
3. Biến môi trường – ghi đè khi triển khai
4. Đối số dòng lệnh – ưu tiên cao nhất, dùng khi chạy thủ công

8.2.3 *Đóng gói service bằng Dockerfile nhiều giai đoạn*

Dockerfile là bản thiết kế để đóng gói ứng dụng thành Docker image. Kỹ thuật **nhiều giai đoạn (multi-stage build)** tách biệt giai đoạn biên dịch (cần SDK đầy đủ) khỏi giai đoạn chạy (chỉ cần runtime tối thiểu), giúp image cuối cùng nhỏ hơn đáng kể.

Mã nguồn 8.4: Dockerfile nhiều giai đoạn cho WebApi

```
# WebApi Dockerfile – multi-stage build
FROM mcr.microsoft.com/dotnet/sdk:10.0-noble AS build
WORKDIR /src
COPY src/WebApi/WebApi.csproj .
RUN dotnet restore
COPY src/WebApi/ .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:10.0-noble-chiseled
WORKDIR /app
COPY --from=build /app .
ENV ASPNETCORE_URLS=http://+:8080
HEALTHCHECK --interval=10s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
ENTRYPOINT ["./WebApi"]
```

Ba điểm cần chú ý trong Dockerfile trên:

- **Giai đoạn `build`**: Dùng image `dotnet/sdk` (đầy đủ công cụ biên dịch). Sao chép file `.csproj` trước, chạy `dotnetrestore`, rồi mới sao chép toàn bộ mã nguồn. Cách làm này tận dụng Docker layer cache: nếu chỉ thay đổi code mà không thêm package, bước restore được dùng lại từ cache.
- **Giai đoạn cuối**: Dùng image `dotnet/aspnet` (chỉ chứa runtime, không có SDK). Sao chép toàn bộ thư mục `/app` từ giai đoạn build. Image cuối cùng chỉ chứa những gì cần để chạy, giảm kích thước và bề mặt tấn công.
- `HEALTHCHECK`: Hướng dẫn Docker định kỳ gọi endpoint `/health` để kiểm tra container còn hoạt động. Nếu ba lần liên tiếp thất bại, container được đánh dấu `unhealthy`. Chi tiết về health check ở mục 8.2.6.

8.2.4 *Kết nối container qua network và service name*

Khi chạy bằng Docker Compose, mỗi container được cấp một địa chỉ IP trong mạng nội bộ (bridge network). Docker Compose tự động thiết lập DNS nội bộ: tên dịch vụ

khai báo trong `compose.yaml` trở thành hostname mà các container khác có thể dùng để giao tiếp.

Mã nguồn 8.5: Kết nối giữa các service qua tên DNS trong compose.yaml (trích)

```
services:
  webapi:
    environment:
      - BackendUrl=http://backendworker:8080
  backendworker:
    # Khong can cau hinh giac -- chi can ton tai trong cung network
```

Điểm cần ghi nhớ: `http://backendworker:8080` không phải là `localhost`. Trong container, `localhost` trỏ đến chính container đó, không phải container khác. Tên dịch vụ (`backendworker`) được Docker DNS phân giải thành địa chỉ IP nội bộ của container tương ứng – đây là cầu nối thay thế hoàn toàn địa chỉ IP tĩnh trong môi trường container.

8.2.5 Khởi chạy hệ thống bằng Docker Compose

Docker Compose cho phép định nghĩa và khởi chạy toàn bộ hệ thống nhiều container bằng một lệnh duy nhất. Tệp `compose.yaml` mô tả tất cả dịch vụ, cổng, biến môi trường và quan hệ phụ thuộc:

Mã nguồn 8.6: compose.yaml định nghĩa toàn bộ hệ thống hai dịch vụ

```
services:
  webapi:
    build:
      context: .
      dockerfile: src/WebApi/Dockerfile
    ports:
      - "5000:8080"
    environment:
      - BackendUrl=http://backendworker:8080
      - DEMO_Logging__LogLevel__Default=Information
    depends_on:
      - backendworker

  backendworker:
    build:
      context: .
      dockerfile: src/BackendWorker/Dockerfile
    environment:
      - DEMO_Logging__LogLevel__Default=Information
```

Các lệnh Docker Compose cơ bản:

Mã nguồn 8.7: Các lệnh Docker Compose thường dùng

```
# Khoi dong toan bo he thong (build lai image neu co thay doi)
docker compose -f compose.yaml up --build

# Khoi dong trong che do nền (detached)
docker compose -f compose.yaml up -d
```

```
# Xem log cua tat ca container
docker compose -f compose.yaml logs -f

# Dung va xoa toan bo container
docker compose -f compose.yaml down
```

depends_on trong compose.yaml đảm bảo backendworker được khởi động trước webapi. Tuy nhiên, depends_on chỉ đợi container khởi động, không đợi ứng dụng bên trong sẵn sàng. Trong hệ thống sản xuất, cần kết hợp health check (mục 8.2.6) và cơ chế thử lại (mục 8.2.7) để xử lý khoảng thời gian khởi động.

8.2.6 Thiết lập health check và structured logging

Health check là cơ chế để hệ thống bên ngoài (Docker, load balancer, Kubernetes) biết container còn hoạt động. Có hai cấp độ:

- **Docker HEALTHCHECK**: Khai báo trong Dockerfile (dòng lệnh curl định kỳ). Docker dùng kết quả để hiển thị trạng thái healthy / unhealthy trong dockerps.
- **Application health check**: Endpoint HTTP trong ứng dụng (/health), có thể kiểm tra sâu hơn như trạng thái kết nối cơ sở dữ liệu, hàng đợi.

Mã nguồn 8.8: Thiết lập health check endpoint trong .NET

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddHealthChecks();
var app = builder.Build();
app.MapHealthChecks("/health");
```

Structured logging là kỹ thuật ghi log dưới dạng có cấu trúc (thường là JSON) thay vì văn bản thuần túy, giúp công cụ tập trung log như Elasticsearch, Seq, hoặc Azure Monitor dễ dàng tìm kiếm và phân tích.

Mã nguồn 8.9: Structured logging với ILogger trong .NET

```
public sealed class DemoWorkerService(ILogger<DemoWorkerService> logger)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        // Structured log: tham so la key-value, khong phai chuoi noi tiep
        logger.LogInformation("Worker bat dau. PID={PID}", Environment.ProcessId);

        // Log voi nhieu tham so
        logger.LogInformation("Xu ly task #{TaskCount} luc {Time}",
            _taskCount, DateTime.Now);
    }
}
```

Trong structured logging, tham số truyền dưới dạng key-value (PID=\{PID\}, TaskCount=\{TaskCount\}) thay vì nối chuỗi. Công cụ thu thập log có thể lọc theo PID=1234 hoặc vẽ biểu đồ theo TaskCount – điều không thể làm được với chuỗi log thuần túy.

8.2.7 Thiết lập timeout, retry có giới hạn và graceful shutdown

Ba cơ chế xử lý lỗi ở cấp độ vận hành mà mọi dịch vụ .NET nên có:

Timeout và retry cho HTTP client. Khi gọi dịch vụ khác qua mạng, mỗi yêu cầu cần có thời gian chờ tối đa và chiến lược thử lại khi thất bại. .NET cung cấp gói `Microsoft.Extensions.Http.Resilience` tích hợp sẵn:

Mã nguồn 8.10: Cấu hình retry với exponential backoff cho HttpClient

```
builder.Services.AddHttpClient("Backend", client =>
{
    client.BaseAddress = new Uri("http://backendworker:8080");
    client.Timeout = TimeSpan.FromSeconds(5);
})
.AddStandardResilienceHandler(options =>
{
    options.Retry.MaxRetryAttempts = 3;
    options.Retry.BackoffType = Polly.DelayBackoffType.Exponential;
});
```

Chiến lược `Exponential` tăng gấp đôi thời gian chờ giữa các lần thử: lần 1 chờ 2s, lần 2 chờ 4s, lần 3 chờ 8s. Điều này giúp giảm tải cho dịch vụ đang gặp sự cố, tránh hiện tượng "sập dây chuyền" do hàng loạt client đồng loạt thử lại.

Graceful shutdown. Khi container bị dừng (Docker gửi tín hiệu SIGTERM), ứng dụng cần hoàn thành các tác vụ đang xử lý trước khi thoát, thay vì đột ngột ngắt giữa chừng:

Mã nguồn 8.11: Graceful shutdown: hoàn tất tác vụ dang dở trước khi thoát

```
public override async Task StopAsync(CancellationToken cancellationToken)
{
    logger.LogInformation(
        "Worker dang dung... Da xu ly {TaskCount} task", _taskCount);
    await base.StopAsync(cancellationToken);
}
```

.NET tự động gọi `StopAsync` khi nhận được SIGTERM từ Docker. Ứng dụng có 30 giây (mặc định, có thể cấu hình) để hoàn tất cleanup trước khi Docker buộc dừng bằng SIGKILL.

8.2.8 Quan sát container, log và trạng thái dịch vụ

Khi toàn bộ hệ thống đã chạy trong Docker, việc quan sát trạng thái vận hành được thực hiện qua các lệnh Docker. Không cần mở từng cửa sổ dòng lệnh riêng cho từng dịch vụ.

Mã nguồn 8.12: Các lệnh quan sát hệ thống container đang chạy

```
# Xem danh sach container va trang thai health
docker ps

# Xem log cua mot container cu the
docker logs demo-webapi-1
```

```
# Xem log realtime cua toan bo he thong
docker compose -f compose.yaml logs -f

# Xem tai nguyen (CPU, RAM) cua tung container
docker stats

# Kiem tra endpoint health check
curl http://localhost:5000/health
curl http://localhost:5000/api/status
```

Ba mức độ quan sát cần phân biệt:

1. **Container level**: `dockerps` (trạng thái `Up`, `healthy`, `unhealthy`), `dockerstats` (tài nguyên).
2. **Application level**: Endpoint `/health` (trạng thái ứng dụng), structured log (chi tiết xử lý bên trong).
3. **System level**: Kết hợp log từ tất cả container để truy vết một yêu cầu xuyên suốt nhiều dịch vụ (distributed tracing – sẽ được đề cập sâu hơn ở các khóa học nâng cao).

Tổng kết

Tám kỹ thuật trên là bộ công cụ tối thiểu để vận hành hệ thống .NET phân tán với Docker. Để xem toàn bộ mã nguồn hoàn chỉnh, sinh viên tham khảo:

```
examples/chapter08/section02/
  compose.yaml
  src/
    WebApi/
    BackendWorker/
  README.md
```

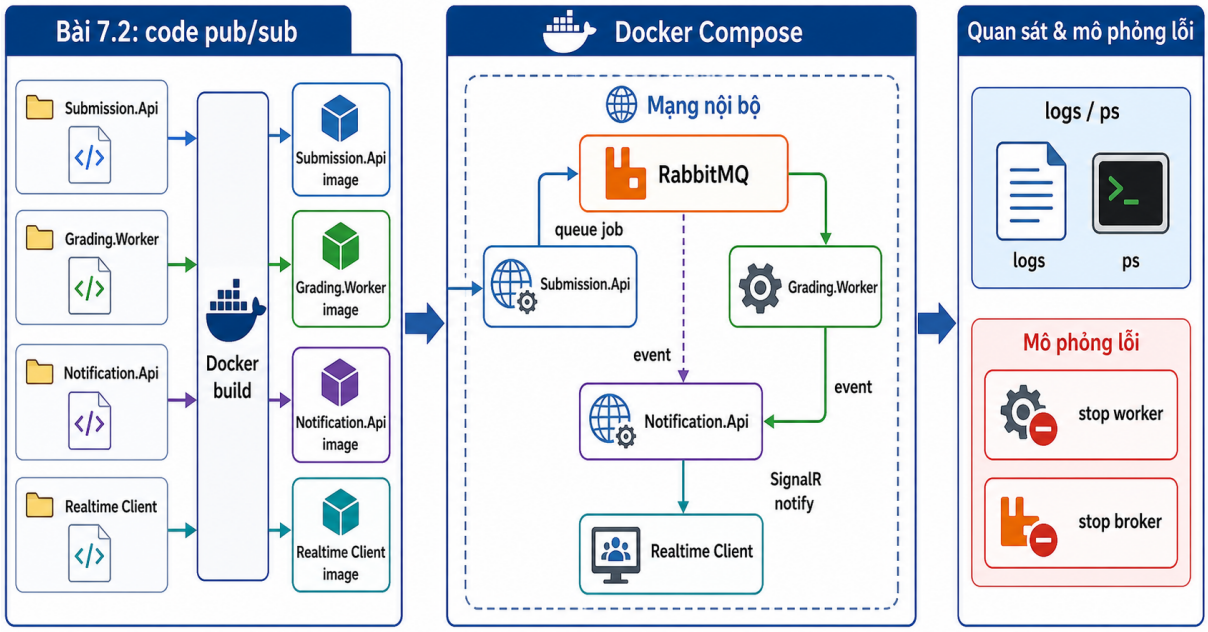
Mục 8.3 sẽ áp dụng toàn bộ kỹ thuật này vào hệ thống EduSubmit: đóng gói API, worker và dịch vụ thông báo thành các container riêng, khởi chạy bằng Docker Compose, mô phỏng lỗi một phần và quan sát phản ứng của hệ thống.

8.3 [Ứng dụng] Triển khai và kiểm thử khả năng phục hồi của EduSubmit

Bài thực hành này nối trực tiếp với bài 7.3. Ở bài 7.3, hệ thống EduSubmit đã có đủ luồng chức năng: `Submission.Api` nhận bài nộp, đưa job vào RabbitMQ, `Grading.Worker` xử lý job, phát sự kiện `SubmissionGraded`, `Notification.Api` nhận sự kiện và đẩy thông báo về `Submission.RealtimeClient` qua SignalR. Tuy nhiên, khi chạy thủ công bằng nhiều cửa sổ dòng lệnh, sinh viên mới nhìn thấy luồng giao tiếp ở mức lập trình, chưa nhìn thấy rõ ranh giới vận hành của một hệ thống phân tán.

Mục tiêu của bài 8.3 là chuyển lát cắt đó sang dạng có thể khởi động như một hệ thống nhiều tiến trình bằng Docker Compose. Ta không thay đổi trọng tâm nghiệp vụ của bài 7.3; thay vào đó, ta đóng gói từng tiến trình thành image riêng, đặt các container vào cùng một mạng nội bộ, cấu hình tên host RabbitMQ bằng biến môi trường, rồi mô phỏng một số lỗi đơn giản để quan sát hệ thống phản ứng ra sao.

Mã nguồn của bài này nằm trong thư mục:



Hình 8.6: Luồng ứng dụng 8.3: đóng gói lát cắt pub/sub của bài 7.3 và quan sát lỗi bằng Docker Compose

```
examples/chapter08/section03/
  EventDrivenNotificationSystem.slnx
  compose.yaml
  docker/
    Submission.Api.Dockerfile
    Notification.Api.Dockerfile
    Grading.Worker.Dockerfile
    Submission.RealtimeClient.Dockerfile
  src/
    EduSubmit.Contracts/
    Submission.Api/
    Grading.Worker/
    Notification.Api/
    Submission.RealtimeClient/
```

Cấu trúc này cố ý giữ gần giống bài 7.3 để sinh viên thấy ranh giới giữa "viết chức năng" và "vận hành chức năng". Phần `src/` vẫn là hệ thống pub/sub đã học. Phần mới của bài 8.3 là các `Dockerfile`, tệp `compose.yaml`, các lệnh quan sát trạng thái container và kịch bản mô phỏng lỗi.

8.3.1 Chạy API, dịch vụ xử lý nền, RabbitMQ

Trước khi đóng gói, cần bảo đảm code ứng dụng vẫn biên dịch được như một solution .NET bình thường. Lệnh kiểm tra đầu tiên là:

```
dotnet build examples/chapter08/section03/EventDrivenNotificationSystem.slnx
```

Nếu lệnh này lỗi, không nên chuyển sang Docker ngay. Docker không sửa lỗi biên

dịch; nó chỉ đặt cùng phần mềm đó vào một môi trường chạy có thể lặp lại. Khi build .NET đã thành công, bước tiếp theo là để Docker Compose xây dựng các image:

```
cd examples/chapter08/section02
docker compose build
```

Sau khi build image, khởi động các tiến trình máy chủ:

```
docker compose up -d rabbitmq submission-api notification-api grading-worker
docker compose ps
```

Lệnh `dockercomposeps` cho biết container nào đang chạy, cổng nào được ánh xạ ra máy host và container nào đã thoát. Đây là lệnh quan sát đầu tiên cần dùng khi hệ thống "không chạy như mong đợi". Với bài thực hành này, sinh viên cần thấy ít nhất bốn tiến trình máy chủ: `rabbitmq`, `submission-api`, `notification-api` và `grading-worker`. RabbitMQ cũng mở giao diện quản trị tại `http://localhost:15672` với tài khoản mặc định `guest`/`guest`.

Máy khách realtime được đặt thành một Compose profile riêng để không tự chạy liên tục cùng hệ thống máy chủ. Khi cần gửi một bài nộp thử và chờ thông báo SignalR, chạy:

```
docker compose run --rm realtime-client
```

Kết quả kỳ vọng tương tự:

```
Da ket noi Notification.Api qua SignalR.
Submission.Api tra ve: 1001 - Processing
Dang cho event SubmissionGraded qua Notification.Api...
SignalR: submission 1001 da cham xong, score=8
```

Ở bài 7.3, bốn cửa sổ dòng lệnh giúp ta thấy từng tiến trình riêng lẻ. Ở bài 8.3, Compose thay thế thao tác thủ công đó bằng một mô tả hệ thống. `realtime-client` gửi HTTP request đến `submission-api` qua tên service nội bộ, không cần biết địa chỉ IP của container. `submission-api`, `grading-worker` và `notification-api` đều dùng tên host `rabbitmq` để kết nối đến bộ môi giới thông điệp trong cùng mạng Compose.

8.3.2 Đóng gói hệ thống bằng Docker

Mỗi project chạy độc lập cần có một image riêng. Về mặt vận hành, `Submission.Api`, `Notification.Api`, `Grading.Worker` và `Submission.RealtimeClient` không còn là các thư mục code nằm cạnh nhau; chúng là các tiến trình có vòng đời riêng, nhật ký riêng và cấu hình môi trường riêng.

Listing sau là Dockerfile đại diện cho `Submission.Api`. Các Dockerfile còn lại dùng cùng cấu trúc: dùng .NET SDK để publish ở bước build, sau đó copy kết quả publish sang image runtime nhỏ hơn.

Mã nguồn 8.13: Dockerfile cho Submission.Api – đóng gói API thành image Docker

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

COPY EventDrivenNotificationSystem.slnx ./
COPY src ./src

RUN dotnet publish src/Submission.Api/Submission.Api.csproj \
    -c Release \
    -o /app/publish \
    /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app

ENV ASPNETCORE_URLS=http://+:8080

COPY --from=build /app/publish .

ENTRYPOINT ["dotnet", "Submission.Api.dll"]
```

Điểm quan trọng của Dockerfile này là tách hai giai đoạn. Giai đoạn `build` chứa SDK đầy đủ để khôi phục gói, biên dịch và publish ứng dụng. Giai đoạn cuối chỉ dùng `aspnet:10.0`, chứa runtime cần thiết để chạy API. Cách tách này giúp image chạy thật gọn hơn và tránh đưa công cụ biên dịch vào môi trường vận hành. Biến môi trường `ASPNETCORE_URLS=http://+:8080` khiến API lắng nghe trên cổng 8080 bên trong container; Compose sẽ quyết định ánh xạ cổng đó ra bên ngoài là 5200 hay 5300.

Tệp `compose.yaml` mô tả toàn bộ hệ thống nhiều tiến trình:

Mã nguồn 8.14: Docker Compose cho hệ thống EduSubmit nhiều tiến trình

```
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: edusubmit-rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 5s
      timeout: 5s
      retries: 12

  submission-api:
    build:
      context: .
      dockerfile: docker/Submission.Api.Dockerfile
    image: edusubmit/submission-api:chapter08-section02
    ports:
      - "5200:8080"
    environment:
      RabbitMq__HostName: rabbitmq
      ASPNETCORE_URLS: http://+:8080
    depends_on:
      rabbitmq:
```

```
      condition: service_healthy

  notification-api:
    build:
      context: .
      dockerfile: docker/Notification.Api.Dockerfile
    image: edusubmit/notification-api:chapter08-section02
    ports:
      - "5300:8080"
    environment:
      RabbitMq__HostName: rabbitmq
      ASPNETCORE_URLS: http://+:8080
    depends_on:
      rabbitmq:
        condition: service_healthy

  grading-worker:
    build:
      context: .
      dockerfile: docker/Grading.Worker.Dockerfile
    image: edusubmit/grading-worker:chapter08-section02
    environment:
      RabbitMq__HostName: rabbitmq
    depends_on:
      rabbitmq:
        condition: service_healthy

  realtime-client:
    build:
      context: .
      dockerfile: docker/Submission.RealtimeClient.Dockerfile
    image: edusubmit/realtime-client:chapter08-section02
    profiles: ["client"]
    command:
      - "http://submission-api:8080"
      - "http://notification-api:8080"
    depends_on:
      - submission-api
      - notification-api
      - grading-worker
```

Trong Compose, tên service cũng là tên DNS nội bộ. Vì vậy, khi `RabbitMq__HostName` được đặt là `rabbitmq`, các ứng dụng .NET sẽ kết nối đến đúng container RabbitMQ trong mạng Compose. Hai API được ánh xạ cổng ra máy host để người học có thể gọi thử từ trình duyệt hoặc công cụ HTTP: `Submission.Api` ở `localhost:5200`, `Notification.Api` ở `localhost:5300`. Ngược lại, `grading-worker` không cần mở cổng vì nó chỉ nhận job từ RabbitMQ và phát sự kiện ngược lại vào RabbitMQ.

Khối `depends_on` trong ví dụ này dùng điều kiện `service_healthy` với RabbitMQ. Điều đó giúp các service ứng dụng chờ RabbitMQ trả lời kiểm tra sức khỏe trước khi khởi động. Tuy nhiên, đây không phải là một cơ chế chịu lỗi đầy đủ. Nếu RabbitMQ đang chạy rồi bị dừng giữa chừng, các ứng dụng vẫn cần logic retry, reconnect và xử lý thông điệp chưa xác nhận. Phần này sẽ được mở rộng trong các nội dung về độ tin cậy; ở bài này, ta cố ý giữ ví dụ đơn giản để sinh viên quan sát được lỗi.

Khác biệt giữa build code và build image
`dotnetbuild` kiểm tra xem mã nguồn có biên dịch được hay không. `dockercomposebuild` kiểm tra thêm một lớp khác: ứng dụng có thể được đóng gói vào image, có đủ file publish và có đúng lệnh khởi động hay không. Một hệ thống phân tán cần vượt qua cả hai mức kiểm tra này trước khi được xem là có thể triển khai.

8.3.3 Mô phỏng lỗi và quan sát

Khi đã chạy trong Docker Compose, mỗi thành phần có thể dừng, khởi động lại hoặc ghi nhật ký độc lập. Đây là điều quan trọng nhất cần quan sát trong bài 8.3: lỗi trong hệ thống phân tán thường là lỗi một phần, không phải toàn bộ hệ thống cùng sập một lúc.

Trước hết, quan sát nhật ký của các tiến trình chính:

```
docker compose logs -f submission-api grading-worker notification-api
```

Sau đó dừng worker chấm bài:

```
docker compose stop grading-worker
docker compose run --rm realtime-client
```

Trong tình huống này, `Submission.Api` vẫn có thể nhận yêu cầu và trả về `Processing`, vì API chỉ cần đưa job vào hàng đợi. Tuy nhiên, không có worker nào lấy job ra xử lý, nên máy khách sẽ chờ quá thời gian nhận sự kiện `SubmissionGraded`. Khi khởi động worker lại:

```
docker compose start grading-worker
docker compose logs -f grading-worker notification-api
```

worker sẽ tiếp tục tiêu thụ job còn nằm trong RabbitMQ. Quan sát này nối trực tiếp với bài 6.3 và bài 7.3: hàng đợi giúp công việc không mất ngay khi worker tạm dừng, còn pub/sub giúp kết quả chấm xong được chuyển tiếp đến dịch vụ thông báo khi worker hoạt động trở lại.

Kịch bản thứ hai là dừng RabbitMQ:

```
docker compose stop rabbitmq
docker compose logs -f submission-api notification-api grading-worker
```

Lúc này bộ môi giới thông điệp là điểm phụ thuộc chung bị mất. `Submission.Api` không thể publish job mới, `Grading.Worker` không thể tiêu thụ job, và `Notification.Api` không thể nhận sự kiện. Đây là ví dụ rõ của lỗi hạ tầng trung gian: các service ứng dụng có thể vẫn là các tiến trình đang chạy, nhưng chức năng nghiệp vụ liên quan đến hàng đợi và sự kiện không còn hoàn chỉnh.

Sau khi thực hành xong, dọn toàn bộ container và mạng Compose:

Bảng 8.2: Một số thao tác quan sát trong bài ứng dụng 8.3

Thao tác	Điều cần quan sát	Ý nghĩa vận hành
`dockercomposeps`	Container đang chạy, cổng được ánh xạ, container đã thoát	Kiểm tra trạng thái tiến trình trước khi đọc sâu nhật ký
`dockercomposelogs-f`	Dòng nhật ký của API, worker và dịch vụ thông báo	Xác định thông điệp đang đi đến thành phần nào
Dừng `grading-worker`	API vẫn trả `Processing`, client quá thời gian chờ thông báo	Hàng đợi tách API khỏi worker nhưng kết quả bị trì hoãn
Dừng `rabbitmq`	Các service phụ thuộc RabbitMQ báo lỗi kết nối hoặc không xử lý được thông điệp	Bộ môi giới là phụ thuộc hạ tầng cần được giám sát và phục hồi

```
docker compose down
```

Kết thúc bài 8.3, sinh viên không chỉ biết cách chạy lại hệ thống của bài 7.3 bằng một lệnh Compose, mà còn nhìn thấy một bước chuyển quan trọng trong tư duy phát triển ứng dụng phân tán: từ việc "code đúng luồng" sang việc "vận hành được nhiều tiến trình, quan sát được trạng thái, và nhận diện được lỗi một phần". Đây là nền cho chương 9, nơi toàn bộ các lát cắt đã xây dựng sẽ được nhìn lại như một kiến trúc hệ thống phân tán hoàn chỉnh.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **Hệ thống và ứng dụng**: Ứng dụng đơn khối chạy trong một tiến trình, giao tiếp qua gọi hàm cục bộ (nano giây, tin cậy tuyệt đối). Hệ thống phân tán gồm nhiều nút độc lập giao tiếp qua mạng (mili giây, không thể dự đoán, nguy cơ mất gói tin). Ba khác biệt cốt lõi: ranh giới giao tiếp & độ trễ, trạng thái độc lập & lỗi một phần (partial failure), và đồng bộ thời gian (mỗi máy chủ có đồng hồ riêng).
- **Giao tiếp đa tiến trình (IPC)**: Đồng bộ (REST, gRPC) – tiến trình gọi chặn luồng chờ phản hồi, đơn giản nhưng dễ nghẽn dây chuyền. Bất đồng bộ (Message Queue như RabbitMQ) – tiến trình gửi thông điệp vào trung gian rồi tiếp tục, tách rời thời gian và không gian. So với gọi hàm cục bộ, IPC qua mạng khác biệt ở: cơ chế truyền dữ liệu (tuần tự hóa thay vì con trỏ), tốc độ (mili giây thay vì nano giây), và khả năng lỗi (cao thay vì gần như bằng không).
- **Docker và containerization**: Docker Image là bản thiết kế tĩnh (blueprint) chứa mã nguồn, runtime, thư viện và cấu hình. Docker Container là thực thể sống được tạo từ image, hoạt động cô lập với hệ thống tệp tin, bộ nhớ và cổng mạng riêng. Docker giải quyết bài toán "mã chạy tốt ở máy tôi nhưng lỗi ở máy bạn" bằng cách đảm bảo đồng nhất môi trường từ phát triển đến triển khai.
- **Phân loại lỗi trong hệ thống phân tán (theo Tanenbaum)**: (1) Crash failure – máy chủ ngừng đột ngột, trước đó xử lý chính xác; Fail-stop là trường hợp đặc biệt khi các nút khác phát hiện được sập. (2) Omission failure – bỏ sót khi nhận (không nhận được yêu cầu) hoặc bỏ sót khi gửi (đã xử lý nhưng không gửi được

phản hồi). (3) Timing failure – phản hồi đúng nội dung nhưng sai thời gian (quá trễ hoặc quá sớm). (4) Response failure – phản hồi sai nội dung: value failure (dữ liệu sai) hoặc state-transition failure (đi chệch luồng xử lý). (5) Byzantine failure – dạng lỗi nghiêm trọng nhất, máy chủ gửi phản hồi sai lệch ngẫu nhiên hoặc giả mạo có chủ ý.

- **Timeout và Retry**: Timeout – máy khách đặt đồng hồ đếm ngược, nếu hết thời gian chưa có phản hồi thì coi thao tác đã lỗi. Retry – gửi lại yêu cầu với độ trễ tăng dần theo cấp số nhân (exponential backoff) kèm nhiễu ngẫu nhiên (jitter) để tránh dồn dập làm tê liệt máy chủ.
- **Ba ngữ nghĩa RPC**: (1) At-least-once – gửi lại đến khi nhận phản hồi, đảm bảo công việc được thực hiện nhưng có thể trùng lặp; phù hợp cho thao tác đọc hoặc cập nhật tĩnh. (2) At-most-once – từ bỏ khi timeout, báo lỗi; phù hợp cho thao tác nhạy cảm (thanh toán). (3) Exactly-once – đảm bảo xử lý duy nhất một lần; không thể đạt được thuần túy bằng giao thức, cần phối hợp retry + idempotency + distributed transaction.
- **Tính lũy đẳng (Idempotency)**: Thao tác lũy đẳng nếu thực thi một hay nhiều lần đều cho cùng kết quả và không gây tác dụng phụ. GET, PUT, DELETE vốn lũy đẳng theo đặc tả HTTP. POST (tạo mới) là không lũy đẳng – đây là tình huống rủi ro nhất khi áp dụng retry. Kỹ thuật Idempotency Key: máy khách gửi mã định danh duy nhất trong tiêu đề; máy chủ kiểm tra mã này trước khi xử lý để bỏ qua yêu cầu trùng lặp.
- **Ứng dụng Docker Compose**: Chuyển hệ thống pub/sub từ bài 7.3 sang mô hình nhiều container: mỗi dịch vụ (`Submission.Api`, `Notification.Api`, `Grading.Worker`) được đóng gói thành image riêng qua Dockerfile hai giai đoạn (SDK build + runtime). Compose mô tả toàn bộ hệ thống trong `compose.yaml`: tên service là tên DNS nội bộ, biến môi trường cấu hình kết nối, `depends\_on` điều khiển thứ tự khởi động.
- **Quan sát lỗi một phần**: (1) Dừng worker – API vẫn nhận bài và trả `Processing`, nhưng client quá thời gian chờ thông báo vì không ai xử lý job; khi worker khởi động lại, job trong hàng đợi được tiêu thụ tiếp. (2) Dừng RabbitMQ – bộ môi giới là phụ thuộc chung, tất cả service mất khả năng giao tiếp qua hàng đợi và sự kiện. Hai kịch bản minh họa: lỗi trong hệ phân tán thường là lỗi một phần, không phải toàn bộ hệ thống cùng sập.

Câu hỏi ôn tập

1. Phân biệt ứng dụng đơn khối (monolithic) và hệ thống phân tán (distributed system) theo ba tiêu chí: ranh giới giao tiếp & độ trễ, trạng thái khi lỗi, và đồng bộ thời gian. Cho ví dụ minh họa từ EduSubmit cho mỗi tiêu chí.
2. Trình bày sự khác biệt giữa giao tiếp đồng bộ (synchronous IPC) và giao tiếp bất đồng bộ (asynchronous IPC) giữa các dịch vụ. Lập bảng so sánh gọi hàm cục bộ và giao tiếp mạng (IPC) theo bốn tiêu chí: cơ chế truyền dữ liệu, tốc độ phản hồi, khả năng xảy ra lỗi, và trạng thái lỗi.
3. Phân biệt Docker Image và Docker Container. Giải thích quy trình: từ `Dockerfile` → `dockerbuild` → Image → `dockerrun` → Container. Docker giải quyết vấn đề gì trong vận hành hệ thống phân tán?
4. Liệt kê và mô tả năm dạng lỗi trong hệ thống phân tán theo phân loại của Tanen-

baum: crash, omission, timing, response, và Byzantine. Với mỗi dạng lỗi, cho một ví dụ cụ thể có thể xảy ra trong hệ thống EduSubmit.

5. Phân biệt lỗi bỏ sót khi nhận (receive-omission) và lỗi bỏ sót khi gửi (send-omission). Tại sao send-omission lại nguy hiểm hơn khi áp dụng cơ chế retry? Cho ví dụ với thao tác nộp bài trong EduSubmit.
6. Giải thích cơ chế timeout và retry trong giao tiếp giữa các dịch vụ. Vẽ sơ đồ minh họa chiến lược exponential backoff với jitter. Tại sao cần thêm jitter thay vì chỉ dùng backoff cố định?
7. Trình bày ba ngữ nghĩa của RPC khi xử lý lỗi: at-least-once, at-most-once, và exactly-once. Với mỗi ngữ nghĩa, nêu: đảm bảo gì, đánh đổi gì, và tình huống áp dụng phù hợp trong EduSubmit. Tại sao exactly-once không thể đạt được thuần túy bằng giao thức?
8. Định nghĩa tính lũy đẳng (idempotency). Những HTTP method nào vốn đã lũy đẳng? Tại sao `POST` là thao tác rủi ro nhất khi áp dụng retry? Mô tả cách hoạt động của kỹ thuật Idempotency Key để giải quyết vấn đề này.
9. Trong bài thực hành Docker Compose, giải thích ý nghĩa của từng thành phần trong `compose.yaml`: tên service, biến môi trường `RabbitMq\_\_HostName=rabbitmq`, khối `depends\_on` với điều kiện `service\_healthy`, và ánh xạ cổng (port mapping). Tại sao `grading-worker` không cần mở cổng ra máy host?
10. Mô tả hai kịch bản mô phỏng lỗi trong bài thực hành: (a) dừng `grading-worker` và (b) dừng `rabbitmq`. Với mỗi kịch bản: thành phần nào vẫn hoạt động, thành phần nào bị ảnh hưởng, và người dùng cuối nhìn thấy gì? Từ đó giải thích khái niệm "lỗi một phần" (partial failure) trong hệ thống phân tán.

Bài tập thực hành

1. **Triển khai hệ thống bằng Docker Compose**. Sao chép mã nguồn từ `content/examples/chapter08/section02/`. Thực hiện đầy đủ quy trình: `dotnetbuild` → `dockercomposebuild` → `dockercomposeup-d` → `dockercomposerun--rmrealtime-client`. Chụp màn hình hoặc ghi lại nhật ký đầu ra của tất cả các bước. Giải thích từng dòng trong nhật ký.
2. **Mô phỏng và ghi nhận lỗi worker**. Sau khi hệ thống chạy ổn định, thực hiện: (a) dừng `grading-worker`, gửi 3 bài nộp, quan sát nhật ký; (b) khởi động lại worker, quan sát worker xử lý các job tồn đọng. Ghi lại thời gian từ lúc worker khởi động lại đến lúc tất cả job được xử lý. Giải thích: các job có được xử lý đúng thứ tự không? Có job nào bị mất không?
3. **Mô phỏng lỗi RabbitMQ và phục hồi**. Dừng RabbitMQ, gửi một bài nộp, quan sát nhật ký của `Submission.Api`. Khởi động lại RabbitMQ, quan sát các service có tự động kết nối lại không. Nếu không, giải thích tại sao và đề xuất cơ chế retry/reconnect nên được thêm vào code ứng dụng.
4. **Thêm Dockerfile cho dịch vụ mới**. Tạo một `Analytics.Worker` đơn giản (chỉ ghi nhật ký khi nhận sự kiện `SubmissionGraded`). Viết Dockerfile cho dịch vụ này, thêm vào `compose.yaml`, và chạy toàn bộ hệ thống. Xác nhận cả ba dịch vụ (`Submission.Api`, `Notification.Api`, `Analytics.Worker`) cùng nhận được sự kiện từ `Grading.Worker`.

Câu hỏi thảo luận

1. Một nhóm phát triển đề xuất dùng cơ chế retry với at-least-once semantics cho mọi thao tác trong EduSubmit, bao gồm cả `POST/api/submissions`. Hãy phân tích rủi ro của đề xuất này (trùng lặp bài nộp, lãng phí tài nguyên chấm điểm, sai lệch thống kê). Đề xuất cách kết hợp Idempotency Key với at-least-once để giảm thiểu rủi ro mà vẫn giữ được độ tin cậy.
2. Trong hệ thống phân tán, lỗi Byzantine (Byzantine failure) là dạng lỗi khó xử lý nhất. Mặc dù ít gặp trong hệ thống nội bộ của một tổ chức, hãy thảo luận: trong bối cảnh EduSubmit, những tình huống nào có thể dẫn đến hành vi tương tự Byzantine failure (vd: một worker bị tấn công và gửi điểm số giả mạo)? Những cơ chế nào có thể được dùng để phát hiện và giảm thiểu thiệt hại (vd: so sánh kết quả từ nhiều worker, chữ ký số cho sự kiện)?
3. Nhìn lại toàn bộ hành trình vận hành: từ việc chạy thủ công 4 cửa sổ dòng lệnh (bài 7.3) đến việc mô tả toàn bộ hệ thống trong một file `compose.yaml` (bài 8.3). Thảo luận: Docker Compose đã giải quyết được những vấn đề gì và chưa giải quyết được những vấn đề gì trong vận hành hệ thống phân tán? Khi nào cần chuyển từ Docker Compose sang các công cụ điều phối mạnh hơn như Kubernetes? Gợi ý: xét các khía cạnh – khởi động lại tự động, phân phối tải giữa các máy chủ vật lý, cập nhật phiên bản không gián đoạn (rolling update), và giám sát sức khỏe toàn cục.

CHƯƠNG 9

TỔNG QUAN KIẾN TRÚC HỆ THỐNG DỊCH VỤ PHÂN TÁN

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Mô tả kiến trúc tổng thể của hệ thống EduSubmit – xác định các thành phần chính, bộ nối giao tiếp, và luồng vận hành end-to-end từ lúc máy khách nộp bài đến khi nhận kết quả.
- Tổng kết và phân biệt bốn mô hình trao đổi thông điệp (MEP) đã học, ánh xạ từng MEP với công nghệ triển khai và vị trí áp dụng trong hệ thống.
- Phân tích sáu thành phần cơ bản của hệ thống phân tán và giải thích trách nhiệm, ranh giới mạng của từng thành phần.
- Vận dụng bốn nguyên tắc lựa chọn kiến trúc (Loose Coupling, Independent Scalability, Design for Failure, CAP) để giải thích các quyết định thiết kế trong EduSubmit.
- Đọc, chú giải và thảo luận sơ đồ kiến trúc phân tán – phản ứng với các kịch bản thay đổi và đề xuất điều chỉnh kiến trúc phù hợp.

9.1 Kiến trúc hệ thống phần mềm phân tán tổng thể gắn với dự án đã xây dựng

Chương này tổng kết toàn bộ kiến thức của học phần thông qua việc trực quan hóa và phân tích kiến trúc hệ thống phần mềm phân tán tổng thể. Chúng ta sẽ đặt các mảnh ghép công nghệ đã học trong các chương trước—bao gồm RESTful API, gRPC, hàng đợi thông điệp RabbitMQ và giao tiếp thời gian thực SignalR—vào một bức tranh kiến trúc thống nhất thông qua dự án mẫu EduSubmit (hệ thống nộp và chấm bài tập tự động).

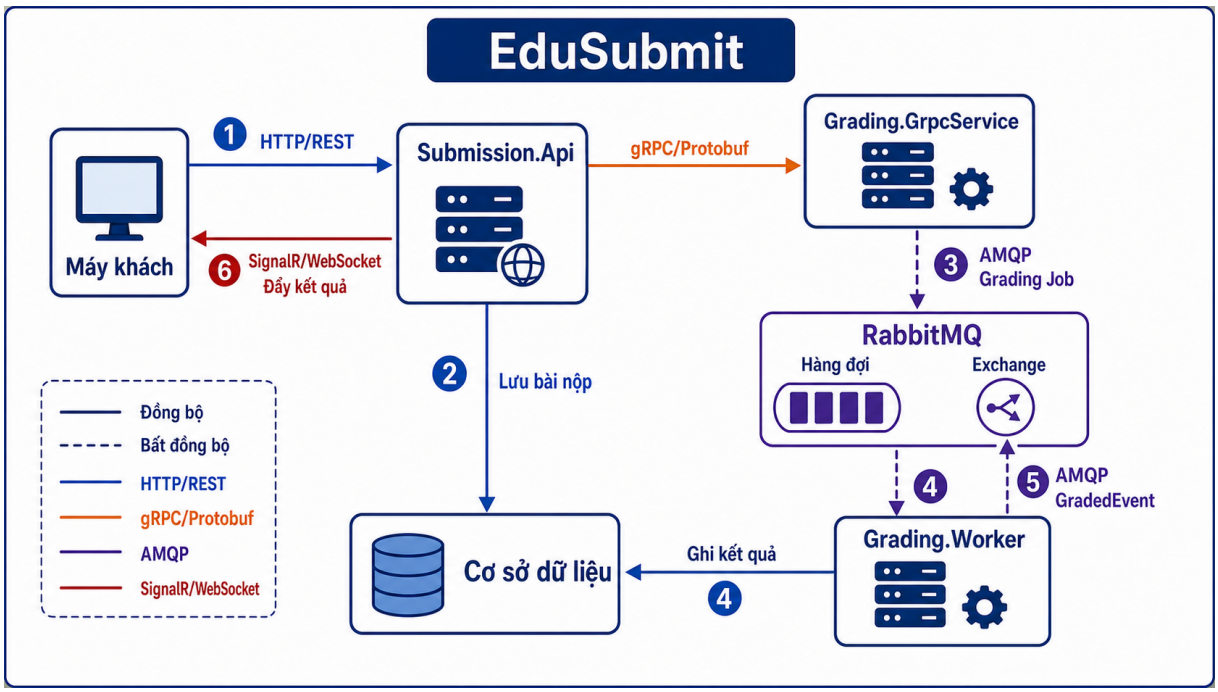
Từ thực hành đến kiến trúc tổng thể
Trong các chương trước, mỗi chương xây dựng một lát cắt giao tiếp riêng biệt của EduSubmit:

- Chương 5: `Submission.Api` ↔ `Grading.GrpcService` qua gRPC (gọi đồng bộ trực tiếp).
- Chương 6: `Submission.Api` → RabbitMQ → `Grading.Worker` (xử lý bất đồng bộ qua hàng đợi).
- Chương 7: `Grading.Worker` → RabbitMQ (sự kiện) → `Submission.Api` → SignalR → Máy khách (pub/sub + thời gian thực).

Chương này ghép cả ba lát cắt thành một kiến trúc tổng thể duy nhất. Trong thực tế, một hệ thống triển khai thật có thể chọn dùng một phần hoặc toàn bộ các đường giao tiếp này tùy theo yêu cầu nghiệp vụ. Sơ đồ dưới đây trình bày phiên bản đầy đủ nhất—nơi mọi MEP cùng hiện diện trong cùng một hệ thống.

9.1.1 Tổng thể kiến trúc hệ thống phân tán

Hệ thống EduSubmit được thiết kế theo mô hình kiến trúc đa dịch vụ (multi-service architecture) nhằm đảm bảo tính độc lập phát triển, khả năng mở rộng quy mô linh hoạt và tính kháng lỗi của từng thành phần. Sơ đồ kiến trúc tổng thể được biểu diễn dưới dạng các thành phần logic (components) kết nối với nhau qua các bộ nối giao thức (connectors) như Hình 9.1.



Hình 9.1: Kiến trúc tổng thể của hệ thống EduSubmit

Luồng vận hành của toàn bộ hệ thống phân tán diễn ra như sau:

1. Máy khách gửi yêu cầu nộp bài tập (HTTP POST) kèm theo tệp tin mã nguồn đến dịch vụ `Submission.Api`.
2. `Submission.Api` lưu trữ thông tin ban đầu của bài nộp vào cơ sở dữ liệu và gọi dịch vụ nội bộ `Grading.GrpcService` thông qua giao thức gRPC để yêu cầu khởi

tạo quy trình chấm bài. So với phiên bản đồng bộ ở chương 5 (gRPC trả kết quả ngay), ở đây `Grading.GrpcService` đóng vai trò điều phối (orchestrator): nhận yêu cầu, xác nhận hợp lệ, rồi chuyển tiếp công việc sang hàng đợi thay vì tự mình thực thi.

3. `Grading.GrpcService` tạo ra một công việc chấm bài (Grading Job) và đẩy thông điệp này vào hàng đợi của `RabbitMQ`.
4. Dịch vụ nền `Grading.Worker` (Worker Service) liên tục lắng nghe hàng đợi, lấy công việc chấm bài ra để thực thi trong một môi trường cô lập, sau đó lưu kết quả chấm bài vào cơ sở dữ liệu riêng của dịch vụ chấm bài.[1]
5. Sau khi chấm xong, `Grading.Worker` phát hành một sự kiện chấm bài hoàn tất (`GradedEvent`) lên một bộ chuyển đổi (Exchange) của `RabbitMQ`.
6. Dịch vụ `Submission.Api` đăng ký lắng nghe sự kiện này, cập nhật trạng thái bài nộp trong hệ thống và gọi `SignalRHub` để đẩy thông báo kết quả thời gian thực về thiết bị của máy khách.

9.1.2 Tổng kết các MEP và cách áp dụng

Mỗi luồng tương tác trong hệ thống EduSubmit được ánh xạ với một mô hình trao đổi thông điệp (Message Exchange Pattern - MEP) cụ thể nhằm giải quyết tốt nhất bài toán tại thời điểm đó:

- **Mô hình Yêu cầu–Phản hồi đồng bộ (Synchronous Request–Response):**
  - *Ứng dụng:* Sử dụng RESTful API (hoặc GraphQL) cho tương tác giữa Máy khách và `Submission.Api` khi máy khách cần truy vấn danh sách bài tập hoặc xem lịch sử nộp bài.
  - *Ứng dụng nội bộ:* Sử dụng gRPC cho giao tiếp dịch vụ–dịch vụ giữa `Submission.Api` và `Grading.GrpcService`. Lợi thế là tốc độ truyền tải cao nhờ tuần tự hóa nhị phân (Protocol Buffers) và hợp đồng giao tiếp rõ ràng.
- **Mô hình Hàng đợi bất đồng bộ (Asynchronous Queue-based):**
  - *Ứng dụng:* Đẩy các công việc chấm bài nặng vào hàng đợi `RabbitMQ` để `Grading.Worker` xử lý tuần tự. Việc này giúp giải phóng tài nguyên của API Web ngay lập tức, trả về trạng thái "Đang xử lý" (`Processing`) cho máy khách mà không bắt luồng xử lý web phải chờ đợi trong vài giây hoặc vài phút.
- **Mô hình Phát hành–Đăng ký hướng sự kiện (Publish–Subscribe):**
  - *Ứng dụng:* Khi `Grading.Worker` hoàn tất việc chấm điểm, sự kiện được phát đi rộng rãi lên Exchange của RabbitMQ. Bất kỳ dịch vụ nào đăng ký quan tâm (ví dụ: dịch vụ gửi email báo điểm cho sinh viên, dịch vụ cập nhật bảng vàng thành tích) đều có thể độc lập nhận thông điệp này để xử lý mà không làm ảnh hưởng đến dịch vụ chấm bài chính.
- **Mô hình Đẩy thông tin thời gian thực (Push/Streaming):**
  - *Ứng dụng:* Sử dụng `SignalR` (WebSocket) để thiết lập kết nối hai chiều lâu dài. Khi điểm số đã được cập nhật thành công, dịch vụ chủ động đẩy thông

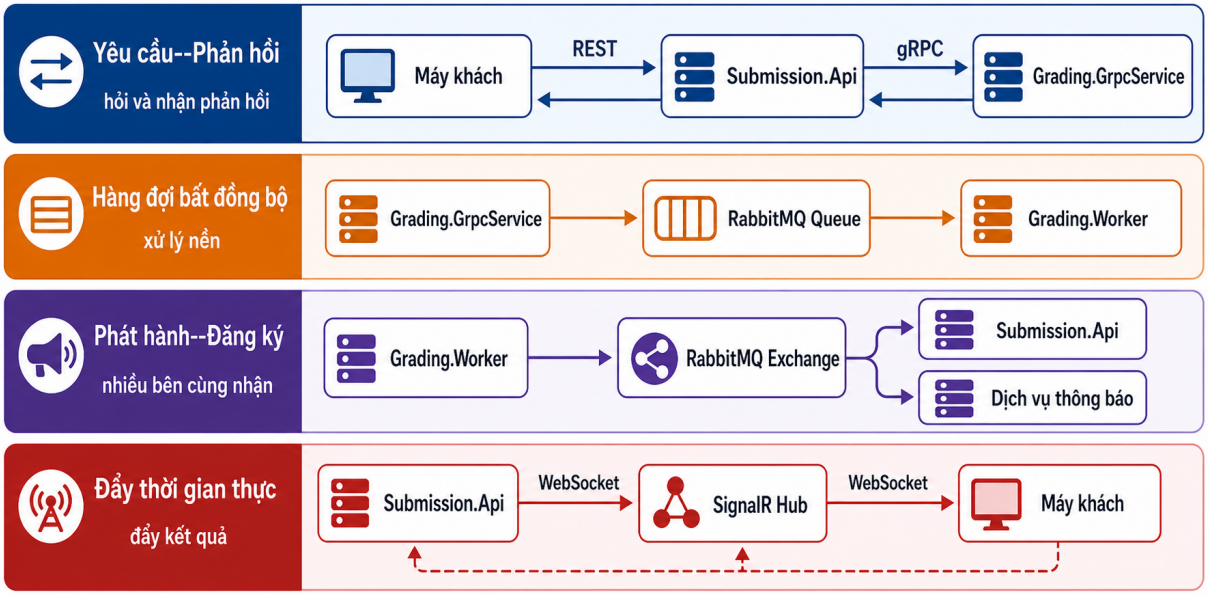
[1] Trong hệ thống triển khai thật, mỗi dịch vụ vi mô (microservice) nên sở hữu cơ sở dữ liệu riêng (database per service) để tránh phụ thuộc ngầm qua dữ liệu. Giáo trình này dùng chung một cơ sở dữ liệu để đơn giản hóa việc học, giúp sinh viên tập trung vào cơ chế giao tiếp.

báo kết quả về giao diện người dùng mà máy khách không cần phải liên tục gửi yêu cầu thăm dò (polling) gây lãng phí băng thông mạng.

Bảng 9.1 tổng kết mối liên hệ giữa bốn MEP, vị trí trong chương trình học, công nghệ triển khai và vai trò cụ thể trong EduSubmit:

Bảng 9.1: Tổng kết MEP – Chương – Công nghệ – Vai trò trong EduSubmit

MEP	Chương	Công nghệ	Vai trò trong EduSubmit
Yêu cầu–Phản hồi (Request–Response)	2, 4, 5	REST, GraphQL, gRPC	Máy khách truy vấn bài tập (REST/GraphQL); API gọi dịch vụ chấm bài nội bộ (gRPC).
Hàng đợi bất đồng bộ (Queue-based)	6	RabbitMQ, Worker Service	Đẩy công việc chấm bài nặng vào hàng đợi, giải phóng API Web, trả trạng thái `Processing`.
Phát hành–Đăng ký (Publish–Subscribe)	7	RabbitMQ (Exchange)	Worker phát sự kiện `GradedEvent`; nhiều dịch vụ độc lập cùng đăng ký nhận để xử lý.
Đẩy thời gian thực (Push/Streaming)	3, 7	SignalR, WebSocket	Đẩy thông báo điểm số về máy khách ngay khi chấm xong, không cần thăm dò định kỳ.

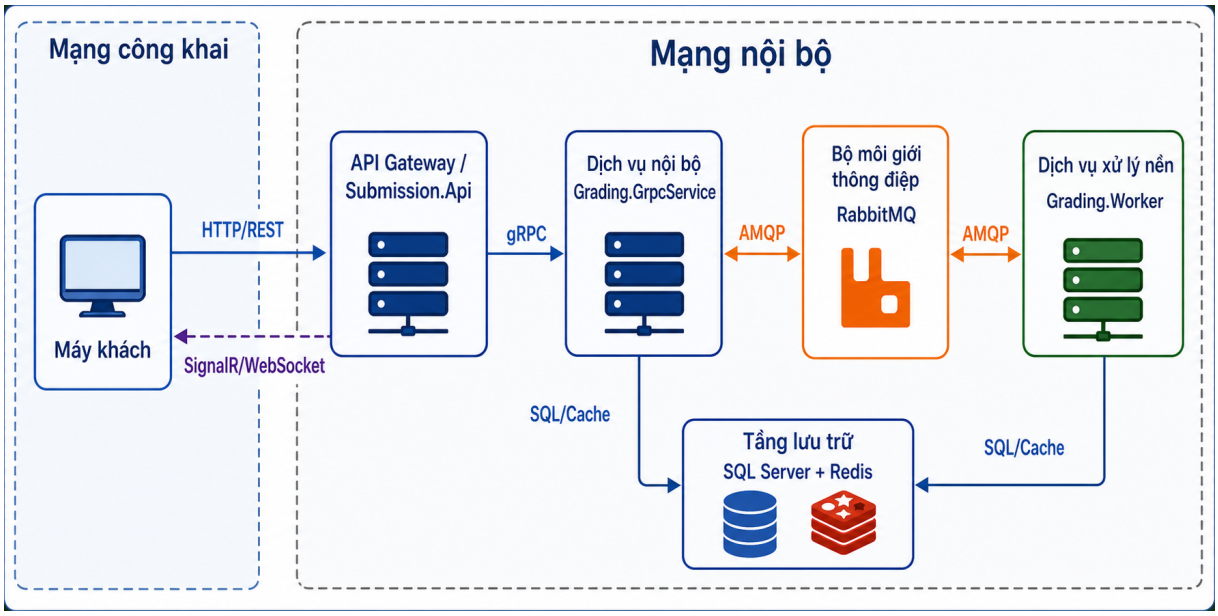


Hình 9.2: Bốn mô hình trao đổi thông điệp cùng hiện diện trong EduSubmit

9.1.3 Các thành phần của hệ thống phân tán

Để xây dựng một hệ thống phân tán hoàn chỉnh, chúng ta cần phân định rõ vai trò và trách nhiệm của từng thành phần phần cứng và phần mềm trong kiến trúc:

- **Máy khách (Client):** Đóng vai trò là điểm tương tác trực tiếp với người dùng cuối (giao diện dòng lệnh CLI, ứng dụng di động hoặc ứng dụng Web SPA). Nhiệm vụ của máy khách là thu thập thông tin đầu vào, hiển thị dữ liệu và duy trì kết nối thời gian thực với máy chủ.



Hình 9.3: Ranh giới mạng và vai trò của các thành phần trong EduSubmit

- **Điểm truy cập API (API Gateway / API Entrypoint):** Là cổng ngõ duy nhất tiếp nhận mọi yêu cầu từ máy khách. API Gateway chịu trách nhiệm định tuyến yêu cầu, xác thực người dùng, kiểm soát tần suất gọi tin (rate limiting) và chuyển đổi giao thức truyền thông trước khi chuyển yêu cầu đến các dịch vụ nội bộ.
- **Dịch vụ nội bộ (Internal Services):** Các tiến trình nghiệp vụ chuyên biệt (như dịch vụ chấm bài, dịch vụ quản lý tài khoản) hoạt động ẩn sau tường lửa, chỉ giao tiếp trực tiếp với Gateway hoặc các dịch vụ nội bộ khác thông qua giao thức hiệu năng cao (như gRPC).
- **Bộ chuyển thông điệp (Message Broker):** Hệ thống trung gian (như RabbitMQ) chịu trách nhiệm nhận, lưu trữ tạm thời và phân phối các thông điệp giữa các tiến trình độc lập, hỗ trợ giao tiếp hướng sự kiện.
- **Dịch vụ xử lý nền (Worker Services):** Các chương trình chạy ngầm, không có giao diện người dùng, nhiệm vụ chính là tiêu thụ (consume) các thông điệp từ hàng đợi và thực thi các tác vụ tính toán nặng.
- **Hệ thống lưu trữ (Storage Layer):** Gồm các cơ sở dữ liệu quan hệ (SQL Server, PostgreSQL) đảm bảo tính nhất quán dữ liệu giao dịch và các cơ sở dữ liệu phi quan hệ hoặc bộ nhớ đệm (Redis) để tối ưu tốc độ đọc dữ liệu tĩnh.

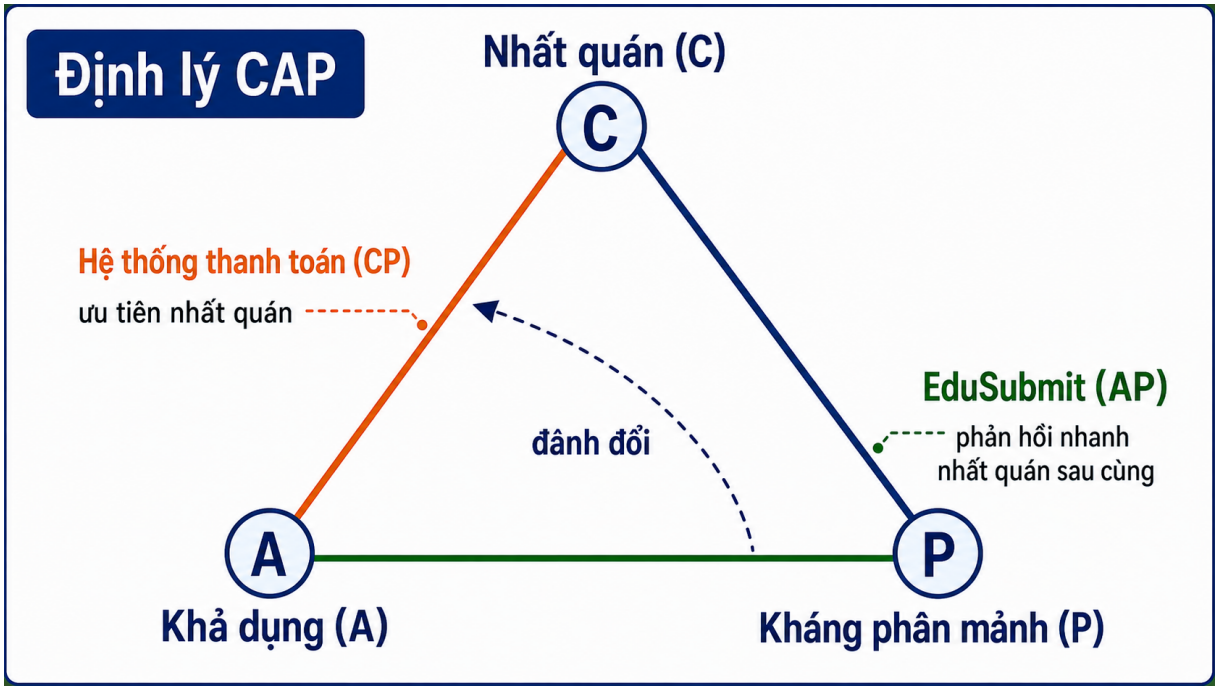
9.1.4 Nguyên tắc lựa chọn kiến trúc

Không có một kiến trúc hệ thống phân tán nào là tối ưu cho mọi bài toán. Người thiết kế hệ thống cần cân nhắc các nguyên tắc đánh đổi kỹ thuật (trade-offs) để đưa ra quyết định phù hợp nhất:

- **Nguyên tắc tách rời lỏng (Loose Coupling):** Đây là mục tiêu quan trọng nhất của kiến trúc hướng sự kiện. Các dịch vụ không nên gọi trực tiếp lẫn nhau nếu không bắt buộc. Việc tương tác thông qua thông điệp gián tiếp giúp một dịch vụ có thể được nâng cấp, bảo trì hoặc sập nguồn hoàn toàn mà không làm gián đoạn trực tiếp đến hoạt động của các dịch vụ khác.
- **Khả năng mở rộng độc lập (Independent Scalability):** Thay vì phải tăng năng

lực phần cứng cho toàn bộ hệ thống đơn khối, kiến trúc phân tán cho phép chúng ta tăng quy mô (scale-out) riêng biệt cho từng thành phần. Ví dụ: khi số lượng bài tập nộp tăng đột biến, chúng ta chỉ cần chạy thêm các container của dịch vụ nền `Grading.Worker` mà không cần nhân bản các API Web.

- **Thiết kế kháng lỗi (Design for Failure):** Hệ thống phân tán luôn hoạt động dưới giả định rằng mạng sẽ bị ngắt kết nối và các nút sẽ bị sập tại một thời điểm nào đó. Người thiết kế cần tích hợp các cơ chế xử lý lỗi đã học ở chương 8 (quá thời gian chờ, gửi lại với độ trễ lũy tiến, tính lũy đẳng) vào mọi điểm giao tiếp giữa các dịch vụ. Ở quy mô triển khai thật, hệ thống còn cần các mẫu thiết kế nâng cao hơn như ngắt mạch (Circuit Breaker)—tự động cắt đường gọi đến dịch vụ đang lỗi liên tục để tránh nghẽn dây chuyền—và hàng đợi thư chết (Dead Letter Queue) để cách ly các thông điệp không thể xử lý sau nhiều lần gửi lại.



Hình 9.4: Đánh đổi CAP và lựa chọn AP trong luồng nộp bài của EduSubmit

- **Định lý CAP và tính nhất quán dữ liệu:** Định lý CAP (còn gọi là định lý Brewer) phát biểu rằng một hệ thống phân tán chỉ có thể đảm bảo tối đa hai trong ba yếu tố tại cùng một thời điểm:
  - **Tính nhất quán (Consistency):** Mọi nút trong hệ thống đều nhìn thấy cùng một dữ liệu tại cùng một thời điểm. Một yêu cầu đọc dữ liệu từ bất kỳ nút nào cũng trả về kết quả giống hệt nhau.
  - **Tính khả dụng (Availability):** Mọi yêu cầu gửi đến một nút đang hoạt động đều nhận được phản hồi (thành công hoặc thất bại), không có yêu cầu nào bị treo vô thời hạn.
  - **Kháng phân mảnh mạng (Partition Tolerance):** Hệ thống vẫn tiếp tục hoạt động bình thường khi mạng giữa các nút bị đứt đoạn hoặc trễ bất thường.

  Trong thực tế, không một hệ thống phân tán nào có thể loại bỏ hoàn toàn nguy cơ phân mảnh mạng, vì vậy P là yếu tố bắt buộc. Người thiết kế buộc phải lựa chọn

giữa CP (nhất quán + kháng phân mảnh, hy sinh khả dụng) và AP (khả dụng + kháng phân mảnh, chấp nhận nhất quán sau cùng). EduSubmit chọn hướng AP cho luồng nộp bài: máy khách nhận phản hồi `202Accepted` ngay khi bài nộp được tiếp nhận thành công, còn kết quả chấm điểm sẽ hiển thị sau khi `Grading.Worker` hoàn tất và sự kiện được lan truyền qua hệ thống.

Ví dụ về đánh đổi CAP trong EduSubmit
Khi sinh viên gửi bài nộp qua `POST/api/submissions`:

- Phiên bản **CP** (nhất quán trước): `Submission.Api` chờ `Grading.Worker` chấm xong, lưu điểm vào cơ sở dữ liệu, rồi mới trả kết quả cho máy khách. Sinh viên nhìn thấy điểm ngay trong phản hồi, nhưng thời gian chờ có thể lên tới vài giây hoặc lâu hơn nếu hàng đợi đầy.
- Phiên bản **AP** (khả dụng trước): `Submission.Api` nhận bài, lưu trạng thái "Đang xử lý", trả về ngay `202Accepted`. Khi `Grading.Worker` chấm xong, sự kiện được phát đi và `SignalR` đẩy thông báo điểm số về máy khách. Sinh viên thấy trạng thái chuyển từ "Đang chấm" sang "Hoàn tất" trong thời gian thực.

EduSubmit chọn AP vì mục tiêu của hệ thống là phản hồi nhanh và không làm mất bài nộp của sinh viên, chấp nhận độ trễ vài giây để hiển thị điểm số.

9.2 [Thực hành] Trực quan hóa kiến trúc hệ thống phân tán đã xây dựng

Ở các chương trước, sinh viên đã lần lượt xây dựng nhiều lát cắt của EduSubmit: API nhận yêu cầu từ máy khách, giao tiếp thời gian thực qua SignalR, truy vấn dữ liệu qua GraphQL, gọi dịch vụ nội bộ bằng gRPC, xử lý nền qua RabbitMQ, và phát sự kiện khi chấm bài hoàn tất. Bài thực hành cuối cùng không thêm một công nghệ mới, mà yêu cầu người học nhìn lại toàn bộ hệ thống như một kiến trúc phân tán hoàn chỉnh.

Mục tiêu của phần thực hành này là chuyển từ tư duy "viết từng đoạn code chạy được" sang tư duy "đọc, mô tả và bảo vệ một thiết kế hệ thống". Khi làm đúng, sinh viên phải trả lời được bốn câu hỏi:

1. Hệ thống có những thành phần nào, mỗi thành phần chịu trách nhiệm gì?
2. Dữ liệu và thông điệp đi qua các thành phần theo thứ tự nào?
3. Mỗi kết nối đang dùng mô hình trao đổi thông điệp nào?
4. Nếu yêu cầu thay đổi, kiến trúc cần được điều chỉnh ở đâu và đánh đổi nào sẽ xuất hiện?

Sản phẩm cuối cùng của bài thực hành là một bản phân tích kiến trúc ngắn, gồm phần chú giải sơ đồ tổng thể, đoạn thuyết minh thiết kế, và phần thảo luận các kịch bản. Sinh viên không cần vẽ lại sơ đồ đã có ở mục 9.1. Nhiệm vụ chính là đọc đúng sơ đồ, giải thích được các quyết định thiết kế, và đề xuất biến thể kiến trúc cho ít nhất một kịch bản cụ thể.

9.2.1 Phân tích và chú giải kiến trúc hệ thống

Bài thực hành này dùng sơ đồ kiến trúc tổng thể ở Hình 9.1 và bảng tổng kết MEP ở Bảng 9.1 (mục 9.1) làm tài liệu tham chiếu chính. Sinh viên không cần vẽ lại sơ đồ. Nhiệm vụ là **đọc và chú giải** kiến trúc đã có: xác định thành phần, gắn nhãn giao thức,

phân biệt đồng bộ / bất đồng bộ, và giải thích lý do thiết kế.

Quy trình chú giải đề xuất gồm sáu bước:

1. **Khoanh vùng hệ thống**: xác định phần nào nằm ngoài hệ thống (máy khách, người dùng), phần nào nằm trong mạng nội bộ hoặc môi trường triển khai.
2. **Đối chiếu thành phần**: đối chiếu từng khối trong Hình 9.1 với danh sách thành phần đã nêu ở mục 9.1.3 (Máy khách, API Gateway, Internal Services, Message Broker, Worker Services, Storage Layer).
3. **Diễn giải luồng chính**: mô tả luồng nộp bài và chấm bài từ đầu đến cuối, bám theo thứ tự các bước đã đánh số trong mục 9.1.1.
4. **Gắn nhãn giao thức**: mỗi mũi tên cần được gán nhãn (`HTTP/REST`, `gRPC`, `AMQP`, `SignalR/WebSocket`, `SQL`), đối chiếu với Bảng 9.1 để kiểm tra ánh xạ MEP–công nghệ.
5. **Phân biệt đồng bộ và bất đồng bộ**: dùng đường liền cho lời gọi chờ phản hồi (REST, gRPC), đường đứt nét cho thông điệp hàng đợi hoặc sự kiện (RabbitMQ queue, exchange).
6. **Kiểm tra điểm lỗi**: đánh dấu ít nhất ba vị trí có thể lỗi, ví dụ API quá tải, RabbitMQ không khả dụng, worker xử lý chậm hoặc kết nối SignalR bị ngắt.

Một bản chú giải tốt cần làm rõ điều mà hình chưa nói hết.
Hình 9.1 đã cung cấp các khối và mũi tên. Phần thực hành yêu cầu người học giải thích **ý nghĩa** của các mũi tên đó: đâu là lời gọi đồng bộ, đâu là thông điệp bất đồng bộ, đâu là nơi có thể mở rộng độc lập, và đâu là điểm cần giám sát khi vận hành.

Sau khi hoàn thành phần chú giải, sinh viên cần viết một đoạn thuyết minh khoảng 300–500 từ. Đoạn thuyết minh không lặp lại toàn bộ nhãn trong hình, mà giải thích **lý do thiết kế**: vì sao API không trực tiếp chấm bài, vì sao công việc chấm đi qua hàng đợi, vì sao cần SignalR thay vì để máy khách liên tục thăm dò, và dữ liệu nào được lưu ở từng giai đoạn. Có thể dùng nguyên tắc CAP và Loose Coupling (mục 9.1.4) để lập luận cho các quyết định thiết kế.

9.2.2 Thảo luận các kịch bản thiết kế

Một kiến trúc phân tán không chỉ được đánh giá bằng việc "có đủ công nghệ". Kiến trúc tốt phải phản ứng hợp lý trước các kịch bản thay đổi. Trong phần này, sinh viên thảo luận theo nhóm nhỏ hoặc viết cá nhân, dựa trên sơ đồ đã phân tích ở mục trước. Nếu một kịch bản làm thay đổi đáng kể kiến trúc, nhóm có thể vẽ thêm một sơ đồ biến thể, nhưng sơ đồ đó phải thể hiện phần thay đổi chứ không lặp lại toàn bộ hình tổng thể.

Kịch bản đầu tiên là **số lượng bài nộp tăng đột biến trước hạn chót**. Nếu toàn bộ quy trình chấm bài chạy đồng bộ trong `Submission.Api`, API sẽ giữ kết nối HTTP quá lâu và dễ làm máy khách gặp hết thời gian chờ. Với kiến trúc hiện tại, API chỉ cần nhận bài và đẩy công việc vào RabbitMQ; phần có thể mở rộng độc lập là `Grading.Worker`. Khi tải tăng, có thể chạy thêm nhiều worker để tiêu thụ hàng đợi nhanh hơn mà không cần nhân bản toàn bộ API.

Kịch bản thứ hai là **dịch vụ chấm bài tạm thời gặp lỗi**. Nếu `Grading.Worker` dừng hoạt động, các bài nộp mới không nên bị mất. Hàng đợi đóng vai trò bộ đệm, giữ lại công việc cho đến khi worker khôi phục. Tuy nhiên, trạng thái hiển thị cho người dùng

phải được thiết kế cẩn thận: bài nộp có thể ở trạng thái `Processing` lâu hơn bình thường, và hệ thống nên có cơ chế cảnh báo vận hành khi độ dài hàng đợi vượt ngưỡng.

Kịch bản thứ ba là **máy khách mất kết nối thời gian thực**. SignalR giúp đẩy kết quả nhanh, nhưng không thể giả định kết nối WebSocket luôn tồn tại. Vì vậy, thông báo thời gian thực chỉ nên là một kênh cập nhật thuận tiện; trạng thái thật vẫn phải được lưu trong cơ sở dữ liệu. Khi máy khách kết nối lại, nó cần truy vấn lại trạng thái bài nộp qua API thay vì chỉ dựa vào thông báo đã bỏ lỡ.

Kịch bản thứ tư là **thêm dịch vụ mới mà không sửa luồng cũ**. Ví dụ, sau khi có điểm số, hệ thống muốn gửi email cho sinh viên hoặc cập nhật bảng thống kê của lớp. Nếu `Grading.Worker` gọi trực tiếp từng dịch vụ mới, worker sẽ ngày càng phụ thuộc vào nhiều thành phần. Cách phù hợp hơn là để worker phát sự kiện `GradedEvent`; các dịch vụ mới chỉ cần đăng ký nhận sự kiện từ exchange. Đây là nơi mô hình phát hành–đăng ký phát huy tác dụng.

Bảng 9.3 tóm tắt các kịch bản cần thảo luận và quyết định thiết kế tương ứng.

Bảng 9.2: Kịch bản thiết kế và phản ứng kiến trúc trong EduSubmit

Kịch bản	Thành phần chịu ảnh hưởng	Quyết định kiến trúc cần giải thích
Nhiều bài nộp cùng lúc	`Submission.Api`, RabbitMQ, `Grading.Worker`	Tách nhận yêu cầu khỏi xử lý nặng; mở rộng số worker
Worker tạm dừng hoặc xử lý chậm	RabbitMQ, `Grading.Worker`	Dùng hàng đợi làm bộ đệm; theo dõi độ dài hàng đợi
Máy khách mất kết nối SignalR	Máy khách, `Submission.Api`, cơ sở dữ liệu	Lưu trạng thái bền vững; cho phép truy vấn lại qua API
Thêm dịch vụ gửi email hoặc thống kê	RabbitMQ exchange, dịch vụ mới	Dùng pub/sub để thêm subscriber mà không sửa publisher
Cần điểm số nhất quán ngay lập tức	API, worker, cơ sở dữ liệu	Cân nhắc chuyển một phần luồng từ AP sang CP, chấp nhận tăng độ trễ

Khi thảo luận, không có một đáp án duy nhất cho mọi nhóm. Điều quan trọng là sinh viên nêu được đánh đổi. Ví dụ, tăng số worker giúp giảm thời gian chờ trong hàng đợi, nhưng có thể làm tăng tải lên cơ sở dữ liệu hoặc môi trường chấm bài. Dùng pub/sub giúp thêm chức năng dễ hơn, nhưng làm luồng xử lý khó theo dõi hơn nếu không có nhật ký tương quan và truy vết phân tán. Đẩy thông báo bằng SignalR cải thiện trải nghiệm người dùng, nhưng hệ thống vẫn cần API đọc trạng thái để khôi phục sau mất kết nối.

Tiêu chí tự đánh giá bài nộp thực hành
Một bài nộp đạt yêu cầu tối thiểu khi phần chú giải xác định đúng thành phần, đúng mũi tên giao tiếp chính, phân biệt được đồng bộ/bất đồng bộ và có thuyết minh luồng nộp bài. Bài nộp tốt hơn khi giải thích được lý do chọn từng MEP, chỉ ra điểm lỗi có thể xảy ra, và đề xuất được cách mở rộng hoặc khôi phục tương ứng. Sơ đồ biến thể là điểm cộng khi nó làm rõ một quyết định thiết kế mới.

Cuối bài, mỗi nhóm nên trả lời ngắn gọn ba câu hỏi phản tư:

1. Nếu chỉ được giữ lại một cơ chế bất đồng bộ trong hệ thống, nhóm sẽ giữ hàng đợi công việc hay pub/sub sự kiện? Vì sao?
2. Nếu RabbitMQ không khả dụng trong 10 phút, người dùng cuối sẽ nhìn thấy điều gì và dữ liệu nào cần được bảo toàn?
3. Nếu hệ thống cần chuyển từ "chấp nhận nhất quán sau cùng" sang "cần điểm ngay trong phản hồi", những thành phần nào phải thay đổi?

Ba câu hỏi này giúp nối lại toàn bộ mạch học phần: từ giao tiếp client–server, request–response, streaming, GraphQL, gRPC, hàng đợi, pub/sub, cho đến vận hành và xử lý lỗi. Khi sinh viên có thể trả lời chúng bằng chính kiến trúc EduSubmit, mục tiêu của dự án xuyên suốt đã hoàn thành.

Ôn tập cuối chương

Tóm tắt kiến thức cần nhớ

- **Kiến trúc tổng thể EduSubmit**: Hệ thống đa dịch vụ gồm 6 thành phần chính – Máy khách, `Submission.Api` (API Gateway), `Grading.GrpcService` (điều phối chấm bài), RabbitMQ (message broker), `Grading.Worker` (xử lý nền), và SignalR Hub (đẩy thông báo thời gian thực). Các thành phần giao tiếp qua 4 loại bộ nối: HTTP/REST, gRPC, AMQP, và WebSocket/SignalR.
- **Luồng vận hành end-to-end**: Máy khách → REST → `Submission.Api` → gRPC → `Grading.GrpcService` → AMQP (hàng đợi) → `Grading.Worker` (chấm bài) → AMQP (exchange, sự kiện `GradedEvent`) → `Submission.Api` → SignalR → Máy khách. API không trực tiếp chấm bài; worker không trực tiếp gọi API trả kết quả; cả hai giao tiếp qua trung gian RabbitMQ.
- **Bốn MEP trong cùng một hệ thống**: (1) Yêu cầu–Phản hồi (REST/GraphQL cho client–server, gRPC cho server–server); (2) Hàng đợi bất đồng bộ (RabbitMQ queue – tách API khỏi thời gian chấm bài); (3) Phát hành–Đăng ký (RabbitMQ exchange – một sự kiện, nhiều subscriber độc lập); (4) Đẩy thời gian thực (SignalR/WebSocket – kết quả đến máy khách không cần polling).
- **Sáu thành phần của hệ thống phân tán**: (1) Máy khách – tương tác người dùng; (2) API Gateway – cổng vào duy nhất, định tuyến, xác thực; (3) Dịch vụ nội bộ – nghiệp vụ chuyên biệt, ẩn sau tường lửa; (4) Message Broker – trung gian nhận, lưu, phân phối thông điệp; (5) Worker Services – tiến trình nền xử lý tác vụ nặng; (6) Storage Layer – cơ sở dữ liệu quan hệ và phi quan hệ.
- **Bốn nguyên tắc lựa chọn kiến trúc**: (1) Tách rời lỏng (Loose Coupling) – dịch vụ không gọi trực tiếp nếu không bắt buộc, tương tác qua thông điệp gián tiếp; (2) Khả năng mở rộng độc lập (Independent Scalability) – scale-out riêng từng thành phần; (3) Thiết kế kháng lỗi (Design for Failure) – giả định mạng và nút sẽ lỗi, tích hợp retry, timeout, circuit breaker, dead letter queue; (4) Định lý CAP – chỉ chọn 2 trong 3 (Consistency, Availability, Partition Tolerance), EduSubmit chọn AP cho luồng nộp bài.
- **Định lý CAP**: Hệ phân tán không thể đồng thời đảm bảo cả ba: Nhất quán (mọi nút thấy cùng dữ liệu), Khả dụng (mọi yêu cầu được phản hồi), và Kháng phân mảnh mạng (hoạt động khi mạng đứt đoạn). P là bắt buộc; lựa chọn giữa CP (hy sinh khả dụng, dữ liệu luôn đúng) và AP (chấp nhận nhất quán sau cùng, hệ thống luôn phản hồi). EduSubmit chọn AP: trả `202Accepted` ngay, điểm số hiển thị sau

vài giây qua SignalR.

- **Thực hành trực quan hóa kiến trúc**: Chuyển từ tư duy "viết code" sang tư duy "đọc, mô tả và bảo vệ thiết kế". Quy trình 6 bước: khoanh vùng hệ thống → đối chiếu thành phần → diễn giải luồng chính → gắn nhãn giao thức → phân biệt đồng bộ/bất đồng bộ → kiểm tra điểm lỗi. Sản phẩm: chú giải sơ đồ, bảng ánh xạ thành phần–giao thức–MEP, và thảo luận kịch bản.
- **Bốn kịch bản thiết kế**: (1) Tải đột biến – thêm worker, không cần nhân bản API; (2) Worker gặp lỗi – hàng đợi giữ công việc, trạng thái `Processing` kéo dài, cần cảnh báo độ dài hàng đợi; (3) Máy khách mất kết nối SignalR – trạng thái lưu trong DB, truy vấn lại qua API; (4) Thêm dịch vụ mới – pub/sub cho phép subscriber mới không cần sửa publisher.
- **Bài học xuyên suốt**: Từ chương 1 đến chương 9, EduSubmit phát triển từ một API đơn giản thành kiến trúc đa dịch vụ với đủ bốn MEP. Hành trình minh họa nguyên tắc cốt lõi của hệ phân tán: không có giải pháp vạn năng, mỗi ranh giới giao tiếp cần MEP và công nghệ phù hợp, và mọi lựa chọn đều đi kèm đánh đổi.

Câu hỏi ôn tập

1. Vẽ sơ đồ kiến trúc tổng thể của EduSubmit (tối thiểu 6 thành phần chính). Chú thích đầy đủ các bộ nối giao tiếp: giao thức, hướng giao tiếp, và đồng bộ/bất đồng bộ. Mô tả luồng nộp bài và chấm bài từ đầu đến cuối.
2. Liệt kê và mô tả bốn mô hình trao đổi thông điệp (MEP) đã học trong toàn bộ học phần. Với mỗi MEP, nêu: chương đã học, công nghệ triển khai, và ít nhất một vị trí áp dụng cụ thể trong EduSubmit.
3. Phân biệt vai trò của `Submission.Api`, `Grading.GrpcService`, và `Grading.Worker` trong kiến trúc tổng thể. Tại sao `Submission.Api` không trực tiếp chấm bài? Tại sao cần thêm `Grading.GrpcService` làm tầng điều phối thay vì để API gửi thẳng job vào RabbitMQ?
4. Trình bày sáu thành phần cơ bản của một hệ thống phân tán. Với mỗi thành phần, cho biết: trách nhiệm chính, vị trí trong mạng (bên ngoài/nội bộ), và ví dụ cụ thể từ EduSubmit.
5. Phát biểu định lý CAP. Giải thích ý nghĩa của từng yếu tố: Consistency, Availability, Partition Tolerance. Vẽ sơ đồ minh họa sự đánh đổi. EduSubmit chọn hướng nào cho luồng nộp bài? Giải thích lý do.
6. So sánh hai phiên bản xử lý bài nộp: CP (chờ chấm xong mới trả kết quả) và AP (trả `Accepted` ngay, điểm đến sau). Lập bảng so sánh về: thời gian phản hồi, trải nghiệm người dùng, rủi ro mất dữ liệu, và độ phức tạp triển khai.
7. Trình bày bốn nguyên tắc lựa chọn kiến trúc đã học: Loose Coupling, Independent Scalability, Design for Failure, và CAP. Với mỗi nguyên tắc, nêu một quyết định thiết kế cụ thể trong EduSubmit thể hiện nguyên tắc đó.
8. Mô tả quy trình 6 bước để phân tích và chú giải một sơ đồ kiến trúc phân tán. Khi chú giải Hình 9.1, cần làm rõ những thông tin gì mà hình chưa thể hiện hết?
9. Phân tích kịch bản: "Số lượng bài nộp tăng đột biến gấp 10 lần trước hạn chót." Những thành phần nào trong EduSubmit chịu ảnh hưởng? Cần thay đổi gì về cấu hình hoặc số lượng thể hiện? Thành phần nào **không** cần thay đổi? Vì sao?
10. Phân tích kịch bản: "Cần thêm dịch vụ gửi email thông báo điểm cho sinh viên." Mô tả các bước để tích hợp dịch vụ này vào kiến trúc hiện tại. Những thành phần

hiện có nào cần sửa đổi? Những thành phần nào không cần sửa đổi? Mô hình MEP nào được sử dụng cho tích hợp này?

Bài tập thực hành

1. **Chú giải sơ đồ kiến trúc tổng thể**. In hoặc mở Hình 9.1 trong chương. Thực hiện chú giải đầy đủ theo quy trình 6 bước. Viết đoạn thuyết minh 300–500 từ giải thích: vì sao API không trực tiếp chấm bài, vì sao cần cả hàng đợi lẫn exchange, vì sao cần SignalR thay vì polling, và dữ liệu được lưu ở đâu trong từng giai đoạn.
2. **Lập bảng ánh xạ thành phần–giao thức–MEP**. Từ sơ đồ kiến trúc, lập một bảng đầy đủ các bộ nối giao tiếp (dựa trên Bảng 9.1 trong mục 9.1.2, mở rộng thêm cột thành phần nguồn–đích cụ thể). Với mỗi bộ nối, bổ sung thêm cột: "Nếu bộ nối này bị đứt, điều gì xảy ra?" và "Cơ chế khôi phục".
3. **Vẽ sơ đồ biến thể cho kịch bản mới**. EduSubmit muốn thêm tính năng "bảng xếp hạng sinh viên theo điểm số" cập nhật theo thời gian thực. Vẽ một sơ đồ biến thể (không cần vẽ lại toàn bộ kiến trúc) thể hiện: dịch vụ mới `Leaderboard.Api`, cách dịch vụ này nhận dữ liệu, và cách máy khách xem bảng xếp hạng. Giải thích lựa chọn MEP cho từng kết nối mới.
4. **Phân tích đánh đổi CAP cho một use case**. Chọn một use case trong EduSubmit (vd: truy vấn danh sách bài tập, hoặc cập nhật hồ sơ sinh viên). Phân tích xem use case đó nên chọn CP hay AP. Viết một đoạn giải thích ngắn (200–300 từ) nêu lý do và đánh đổi.

Câu hỏi thảo luận

1. Nhìn lại toàn bộ hành trình từ chương 1 đến chương 9: EduSubmit đã phát triển từ một cặp client–server REST đơn giản thành một kiến trúc đa dịch vụ với đủ bốn MEP. Nếu được thiết kế lại từ đầu với kiến thức hiện tại, nhóm bạn sẽ giữ nguyên kiến trúc này hay chọn một cách tổ chức khác? Những ranh giới nào có thể được đơn giản hóa? Những công nghệ nào có thể được thay thế (vd: thay RabbitMQ bằng Kafka, thay gRPC bằng REST nội bộ)?
2. Định lý CAP thường bị hiểu sai thành "chỉ được chọn 2 trong 3" một cách tuyệt đối. Trong thực tế, nhiều hệ thống chọn giải pháp lai: một phần dữ liệu xử lý theo CP, phần khác theo AP. Với EduSubmit, hãy đề xuất một thiết kế lai: dữ liệu nào nên được xử lý theo CP (cần nhất quán tức thời), dữ liệu nào nên theo AP (chấp nhận nhất quán sau cùng)? Giải thích lý do cho từng loại dữ liệu.
3. Khi hệ thống EduSubmit mở rộng lên quy mô hàng chục nghìn sinh viên và hàng trăm nghìn bài nộp mỗi ngày, những điểm nghẽn (bottleneck) nào có thể xuất hiện trong kiến trúc hiện tại? Xét từng thành phần: `Submission.Api`, RabbitMQ, `Grading.Worker`, cơ sở dữ liệu, và SignalR. Với mỗi điểm nghẽn tiềm năng, đề xuất ít nhất một chiến lược giải quyết (phân mảnh dữ liệu, chia hàng đợi theo mức ưu tiên, mở rộng theo chiều ngang, v.v.).

KẾT LUẬN VÀ ĐỊNH HƯỚNG MỞ RỘNG

Học phần này bắt đầu từ một câu hỏi tưởng như đơn giản: làm thế nào để một máy khách gửi yêu cầu đến máy chủ và nhận lại kết quả? Từ câu hỏi đó, chúng ta đã từng bước mở rộng EduSubmit từ một API nhỏ thành một hệ thống nhiều tiến trình, có nhiều kiểu giao tiếp, có xử lý nền, có thông báo thời gian thực, có hàng đợi thông điệp, có sự kiện, và có môi trường triển khai bằng container. Hành trình này phản ánh đúng bản chất của phát triển ứng dụng phân tán: hệ thống không lớn lên chỉ bằng cách thêm code, mà bằng cách xác định đúng ranh giới giao tiếp giữa các thành phần.

Điểm quan trọng nhất cần giữ lại sau toàn khóa học không phải là tên của một công nghệ cụ thể. REST, GraphQL, SignalR, gRPC, RabbitMQ hay Docker đều có thể thay đổi theo thời gian; có hệ thống dùng công nghệ khác để giải quyết cùng một nhóm bài toán. Điều bền hơn là năng lực đặt câu hỏi thiết kế: thành phần nào cần nói chuyện với thành phần nào, theo kiểu đồng bộ hay bất đồng bộ, dữ liệu nào cần phản hồi ngay, công việc nào có thể xử lý sau, lỗi ở đâu có thể xảy ra, và người dùng cuối sẽ nhìn thấy điều gì khi một phần hệ thống gặp sự cố.

Nhìn lại hành trình xây dựng EduSubmit

Trong các chương đầu, EduSubmit được xây dựng như một hệ thống client–server cơ bản. Máy khách gọi API, API trả phản hồi, và luồng xử lý còn khá tuyến tính. Đây là điểm xuất phát cần thiết vì mọi hệ thống phân tán đều phải có một ranh giới giao tiếp đầu tiên: nơi yêu cầu đi từ bên ngoài vào hệ thống. REST giúp sinh viên hiểu tài nguyên, phương thức HTTP, mã trạng thái và hợp đồng dữ liệu giữa máy khách và máy chủ.

Sau đó, học phần mở rộng ranh giới giao tiếp sang thời gian thực và truy vấn linh hoạt. SignalR cho thấy không phải mọi tương tác đều phù hợp với mô hình máy khách hỏi, máy chủ trả lời. Có những tình huống máy chủ biết thông tin mới trước và cần chủ động đẩy về máy khách. GraphQL lại cho thấy một hướng khác: khi dữ liệu có nhiều trường và nhu cầu hiển thị khác nhau, máy khách có thể cần quyền mô tả chính xác dữ liệu muốn lấy.

Đến phần giao tiếp nội bộ giữa các dịch vụ, gRPC đưa vào khái niệm hợp đồng chặt chẽ hơn, hiệu quả hơn cho service-to-service. Từ đó, EduSubmit không còn là một API đơn lẻ mà bắt đầu có các dịch vụ nội bộ với trách nhiệm riêng. Tuy nhiên, khi tác vụ chấm bài trở nên nặng và chậm, lời gọi đồng bộ không còn là lựa chọn tốt. RabbitMQ và worker service giúp tách thời gian tiếp nhận yêu cầu khỏi thời gian xử lý công việc.

Ở các chương sau, hệ thống tiếp tục tiến thêm một bước: không chỉ đưa job vào hàng đợi, mà còn phát sự kiện khi một sự thật nghiệp vụ đã xảy ra. Khi `Grading.Worker` hoàn tất chấm bài, nó không cần biết những dịch vụ nào quan tâm đến kết quả. Nó chỉ phát sự kiện; các dịch vụ khác tự đăng ký nhận và phản ứng. Đây là chuyển đổi quan trọng từ tư duy "gọi ai để làm việc tiếp theo" sang tư duy "công bố điều đã xảy ra để hệ thống tự mở rộng phản ứng".

Cuối cùng, Docker và phần trực quan hóa kiến trúc giúp chuyển từ câu hỏi lập trình

sang câu hỏi vận hành. Một hệ thống phân tán không chỉ cần build được; nó cần chạy được nhiều tiến trình, quan sát được trạng thái, chịu được lỗi một phần, và có thể được giải thích bằng một sơ đồ kiến trúc rõ ràng. Chương 9 ghép lại các lát cắt đó thành bức tranh tổng thể.

Bản đồ các mô hình trao đổi thông điệp

Nếu chỉ nhớ một bảng sau khóa học, hãy nhớ Bảng 9.3. Bảng này không nhằm liệt kê công nghệ, mà nhằm nối bài toán giao tiếp với mô hình trao đổi thông điệp phù hợp.

Bảng 9.3: Bản đồ các mô hình trao đổi thông điệp trong học phần

Mô hình	Khi nên dùng	Công nghệ đã thực hành	Ví dụ trong EduSubmit
Yêu cầu–phản hồi	Máy khách hoặc dịch vụ cần câu trả lời ngay	REST, GraphQL, gRPC	Gửi bài nộp, truy vấn bài tập, gọi dịch vụ chấm bài nội bộ
Đẩy thời gian thực	Máy chủ cần chủ động báo thay đổi cho máy khách	SignalR, WebSocket	Báo kết quả chấm bài khi worker hoàn tất
Hàng đợi công việc	Tác vụ chậm, nặng, cần tách khỏi luồng API	RabbitMQ queue, worker service	Đưa job chấm bài vào hàng đợi và trả `Processing`
Phát hành–đăng ký	Một sự kiện có thể được nhiều dịch vụ quan tâm	RabbitMQ exchange, routing key	Phát sự kiện `SubmissionGraded` cho notification, email, analytics
Đóng gói và vận hành	Nhiều tiến trình cần chạy cùng nhau, có cấu hình rõ ràng	Docker, Docker Compose	Chạy `Submission . Api`, worker, RabbitMQ và client trong cùng môi trường

Không có mô hình nào thay thế hoàn toàn mô hình còn lại. REST không "thua" gRPC; gRPC không "hiện đại hơn" RabbitMQ; SignalR không thay thế hàng đợi. Mỗi mô hình giải quyết một dạng căng thẳng khác nhau trong hệ thống. Request–response tối ưu cho thao tác cần phản hồi tức thời. Queue-based tối ưu cho công việc cần xử lý sau. Publish–subscribe tối ưu cho mở rộng phản ứng mà không buộc publisher biết subscriber. Streaming/push tối ưu cho trải nghiệm người dùng cần cập nhật tức thời.

Khi thiết kế một hệ thống mới, hãy bắt đầu từ câu hỏi: "thành phần gọi có cần chờ kết quả không?" Nếu có, request–response thường là điểm khởi đầu. Nếu không, hãy hỏi tiếp: "đây là một công việc cần một consumer xử lý, hay là một sự kiện mà nhiều bên có thể quan tâm?" Câu trả lời sẽ dẫn đến hàng đợi công việc hoặc publish–subscribe. Nếu người dùng cần biết kết quả ngay khi có thay đổi, thêm một kênh push như SignalR ở ranh giới máy chủ–máy khách.

Từ code chạy được đến hệ thống vận hành được

Một lỗi phổ biến khi mới học hệ thống phân tán là xem "chạy được trên máy tôi" như đích đến. Trong một chương trình đơn tiến trình, điều đó có thể tạm đủ cho bài thực hành nhỏ. Trong một hệ thống phân tán, "chạy được" chỉ là bước đầu. Ta còn phải biết các tiến trình phụ thuộc vào nhau thế nào, cổng nào được mở ra ngoài, tên host nào được dùng trong mạng nội bộ, nhật ký nằm ở đâu, và thành phần nào có thể lỗi mà phần còn lại vẫn tiếp tục chạy.

Docker Compose trong chương 8 không được đưa vào chỉ để tiện khởi động hệ thống. Nó giúp người học nhìn thấy hệ thống như một tập hợp đơn vị vận hành: API, worker, broker, dịch vụ thông báo và máy khách thử nghiệm. Mỗi đơn vị có image, container, biến môi trường, cổng mạng và vòng đời riêng. Khi dừng worker, API vẫn có thể nhận yêu cầu và job vẫn nằm trong hàng đợi. Khi dừng RabbitMQ, nhiều thành phần cùng bị ảnh hưởng vì broker là phụ thuộc chung. Những quan sát này giúp sinh viên hiểu lỗi một phần bằng trải nghiệm thực tế, không chỉ bằng định nghĩa.

Từ góc nhìn vận hành, một hệ thống tốt cần trả lời được ít nhất bốn câu hỏi:

1. **Biết hệ thống có sống không**: service nào đang chạy, service nào đã thoát, health check nào đang thất bại?
2. **Biết thông điệp đang kẹt ở đâu**: request đã đến API chưa, job đã vào hàng đợi chưa, worker đã tiêu thụ chưa, event đã được subscriber nhận chưa?
3. **Biết lỗi có lan rộng không**: một worker lỗi có làm API dừng không, một broker lỗi có làm toàn bộ luồng bất đồng bộ ngưng không?
4. **Biết cách khôi phục tối thiểu**: khởi động lại tiến trình nào, đọc nhật ký nào, kiểm tra hàng đợi nào, và dữ liệu nào cần bảo toàn?

Những câu hỏi này là cầu nối từ học phần này sang các chủ đề nâng cao hơn như giám sát, truy vết phân tán, kiểm thử chịu lỗi, tự động triển khai và điều phối container.

Nguyên tắc chọn công nghệ

Trong thực tế, công nghệ không nên được chọn vì tên gọi hấp dẫn hoặc vì đang phổ biến. Công nghệ nên được chọn vì nó làm cho ranh giới giao tiếp trở nên rõ hơn, ổn định hơn hoặc dễ vận hành hơn. Có thể tóm tắt tư duy lựa chọn trong bốn nguyên tắc sau.

Thứ nhất, bắt đầu từ use case, không bắt đầu từ framework. Nếu use case là máy khách gửi bài và cần biết hệ thống đã nhận hay chưa, REST với `202Accepted` là đủ rõ. Nếu use case là lấy dữ liệu theo nhiều hình dạng khác nhau, GraphQL có thể hữu ích. Nếu use case là lời gọi nội bộ hiệu năng cao với hợp đồng chặt chẽ, gRPC là ứng viên tốt.

Thứ hai, tách thao tác nhanh khỏi thao tác chậm. API web nên phản hồi nhanh và không giữ kết nối lâu cho công việc nặng. Những tác vụ như chấm bài, xử lý tệp, gửi email hoặc tổng hợp thống kê thường phù hợp với hàng đợi và worker.

Thứ ba, phát sự kiện khi muốn mở rộng phản ứng. Nếu một kết quả có thể kéo theo nhiều hành động độc lập, như lưu lịch sử, gửi email, cập nhật bảng xếp hạng hoặc ghi thống kê, publish–subscribe giúp tránh việc publisher phụ thuộc vào tất cả consumer.

Thứ tư, thiết kế cho lỗi ngay từ đầu. Trong hệ thống phân tán, lỗi mạng, timeout, trùng thông điệp, mất kết nối realtime và tiến trình khởi động chậm là chuyện bình thường. Vì vậy, timeout, retry, idempotency, health check, nhật ký và cơ chế quan sát không phải phần phụ; chúng là một phần của thiết kế.

Những hướng mở rộng sau học phần

EduSubmit trong giáo trình là một mini system có chủ đích sư phạm. Nó đủ lớn để thể hiện các mô hình giao tiếp quan trọng, nhưng vẫn được giữ gọn để người học có thể đọc, chạy và sửa. Khi chuyển từ giáo trình sang đồ án hoặc hệ thống thật, có nhiều hướng mở rộng tự nhiên.

- **Xác thực và phân quyền**: thêm đăng nhập, token, vai trò sinh viên/giảng viên/quản trị, và kiểm tra quyền ở API gateway.
- **Lưu trữ bền vững**: thay các kho trong bộ nhớ bằng cơ sở dữ liệu thật, thiết kế schema cho bài tập, bài nộp, trạng thái chấm và lịch sử thông báo.
- **Transactional outbox**: bảo đảm việc cập nhật dữ liệu và phát sự kiện không bị lệch nhau khi tiến trình lỗi ở giữa.
- **Idempotency toàn hệ thống**: xử lý an toàn khi máy khách gửi lại request, broker phát lại thông điệp, hoặc subscriber nhận cùng một event nhiều lần.
- **Quan sát hệ thống**: bổ sung log có correlation id, metric cho hàng đợi, tracing qua nhiều service và cảnh báo khi độ trễ vượt ngưỡng.
- **Kiểm thử phân tán**: viết test cho hợp đồng API, test tích hợp với RabbitMQ, test kịch bản worker dừng, broker khởi động lại và client mất kết nối.
- **Triển khai nâng cao**: chuyển từ Docker Compose sang Kubernetes, thêm readiness/liveness probe, cấu hình secret, autoscaling và rolling update.
- **An toàn khi chạy code người học**: cô lập môi trường chấm bài bằng container hoặc sandbox, giới hạn CPU/RAM/thời gian chạy và kiểm soát truy cập mạng.

Những hướng này không làm thay đổi bài học cốt lõi của học phần. Chúng mở rộng cùng một tư duy: xác định ranh giới, chọn mô hình giao tiếp, bảo toàn dữ liệu quan trọng, quan sát được hệ thống và chấp nhận rằng lỗi là một phần thường trực của vận hành.

Lời kết

Phát triển ứng dụng phân tán không phải là việc ghép thật nhiều công nghệ vào một sơ đồ lớn. Đó là năng lực chia một bài toán thành các thành phần có trách nhiệm rõ ràng, nối chúng bằng các mô hình giao tiếp phù hợp, và hiểu những đánh đổi xuất hiện ở mỗi ranh giới. Một thiết kế tốt không chỉ chạy trong trường hợp lý tưởng; nó còn giải thích được chuyện gì xảy ra khi một phần hệ thống chậm, lỗi, mất kết nối hoặc cần mở rộng.

Nếu sau học phần này người học có thể nhìn một yêu cầu mới và tự hỏi: "luồng này cần phản hồi ngay hay có thể xử lý sau?", "đây là job hay event?", "ai cần biết sự kiện này?", "dữ liệu nào phải nhất quán ngay và dữ liệu nào có thể nhất quán sau?", "nếu thành phần này dừng thì người dùng thấy gì?" thì mục tiêu quan trọng nhất đã đạt được. Công nghệ sẽ tiếp tục thay đổi, nhưng những câu hỏi đó sẽ còn đi theo người phát triển phần mềm trong mọi hệ thống phân tán sau này.

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Anh

[1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2021.

[2] B. Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2024.

[3] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Boston, MA, USA: Addison-Wesley, 2011.

[4] W. Fokkink, *Distributed Algorithms: An Intuitive Approach*. Cambridge, MA, USA: MIT Press, 2013.

[5] A. Giretti, *Beginning gRPC with ASP.NET Core 6: Build Applications Using ASP.NET Core Razor Pages, Angular, and Best Practices in .NET 6*. Berkeley, CA, USA: Apress, 2022, doi: 10.1007/978-1-4842-8008-9.

[6] U. Joshi, *Patterns of Distributed Systems*. Boston, MA, USA: Addison-Wesley Professional, 2023.

[7] S. K. Rahimi and F. S. Haug, *Distributed Database Management Systems: A Practical Approach*. Hoboken, NJ, USA: Wiley-IEEE Press, 2010, doi: 10.1002/9780470602379.

[8] D. Tornow, *Think Distributed Systems: Mental Models of Reliable and Scalable Software*. Shelter Island, NY, USA: Manning Publications, 2025.

[9] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed. [Online]. distributed-systems.net, 2017. Available: https://www.distributed-systems.net/index.php/books/ds3/. [Accessed: Jul. 1, 2026].

Tài liệu tiếng Nga

[10] К. Индрасири and Д. Курупу, *gRPC: запуск и эксплуатация облачных приложений. Go и Java для Docker и Kubernetes*. Санкт-Петербург, Россия: Питер, 2021.